Robot Operating System 1 et 2

Introduction

Olivier Stasse

Copyright © 2026Olivier Stasse

PUBLISHED BY LAAS-CNRS

HTTPS ://HOMEPAGES.LAAS.FR/ OSTASSE

Licensed under the Creative Commons Attribution-NonCommercial 3.0 Unported License (the “License”). You may not use this file except in compliance with the License. You may obtain a copy of the License at `http://creativecommons.org/licenses/by-nc/3.0`. Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an “AS IS” BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

Edition, 21 septembre 2026

Table des matières

1 Introduction 13

1.1 Motivations 13

1.1.1 Complexité des systèmes robotiques 13

1.1.2 Application Robotique 14

1.2 Concepts de ROS 14

1.2.1 Plomberie 14

1.2.2 Outils 15

1.2.3 Capacités 15

1.2.4 Exemples 16

1.3 Ecosystème 16

1.3.1 Historique 16

1.3.2 Fonctionnalités 16

1.3.3 Systèmes supportés 18

1.4 Exemple : Asservissement visuel sur TurtleBot 2 19

I Introduction à ROS-1

2 Les paquets ROS 25

2.1 Configuration de l'environnement 25

2.1.1 Initialiser les variables d'environnement 25

2.1.2 Navigation dans le système de paquets 26

2.2 Structure générale des paquets ROS 26

2.3 Création d'un paquet 26

2.4 Description des paquets : le fichier package.xml 28

2.5 Compilation des paquets melodic 29

2.6 Chaînage de workspace 30

3 Graphe d'application avec ROS-1 . . . 31
3.1 Introduction aux concepts de ROS 31
3.1.1 Définition d'un node . . . 31
3.1.2 Le système des name services . . . 31
3.1.3 Lancer des nodes . . . 32
3.1.4 Topic . . . 33
3.2 rqt_console 35
3.3 roslaunch 35
3.3.1 Exemple . . . 35
3.4 rosbag 39
3.5 rosservice 39
3.5.1 rosservice list . . . 39
3.5.2 rosservice type (service) . . . 40
3.5.3 rosservice call . . . 40
3.6 rosparam 40
3.6.1 rosparam list . . . 41
3.6.2 rosparam dump et rosparam load . . . 42
3.7 Création de messages et de services 42
3.7.1 Introduction . . . 42
3.7.2 Création d'un msg . . . 43
3.7.3 Création d'un srv . . . 44
3.7.4 Modifications CMakeLists.txt communes à msg et srv . . . 45
4 Ecrire des nodes ROS-1 . . . 47
4.1 Topics 47
4.1.1 Emetteur . . . 47
4.1.2 Souscripteur . . . 51
4.1.3 Lancer les nodes . . . 53
4.2 Services 54
4.2.1 Serveur . . . 54
4.2.2 Client . . . 55
4.2.3 Lancer les nodes . . . 57
II Travaux Pratiques
5 TurtleBot3 . . . 61
5.1 Démarrage 61
5.1.1 Introduction Modèle du TurtleBot3 . . . 61
5.1.2 Installation . . . 61
5.2 Gazebo 61
5.2.1 Téléopérer le turtlebot . . . 62
5.3 Construction de cartes et navigation 62
5.3.1 Construction de cartes . . . 62
5.3.2 Navigation . . . 63
5.3.3 RVIZ . . . 63

6 Asservissement visuel . . . 65
6.1 Introduction aux tâches 65
6.1.1 Contrôleur en vitesse . . . 66
6.1.2 Projection d'un point dans une image . . . 66
6.1.3 Calcul de la matrice d'interaction . . . 67
6.1.4 Génération de la commande . . . 68
6.2 Traitement d'image pour le TurtleBot 2 69
6.2.1 Extraction d'une image . . . 69
6.2.2 Extraction de la couleur . . . 70
6.2.3 Calcul de la commande . . . 70
6.3 OpenCV - Code de démarrage 70
6.4 Travail à faire 73
6.4.1 Turtlebot 3 . . . 73

III Modèles et Simulation

7 Universal Robot Description Format (URDF) . . . 77
7.1 Capteurs - Sensors 77
7.1.1 Norme courante . . . 77
7.1.2 Limitations . . . 79
7.2 Corps - Link 79
7.2.1 Attributs . . . 79
7.2.2 Elements . . . 79
7.2.3 Résolution recommandée pour les mailles . . . 80
7.2.4 Elements multiples des corps pour la collision . . . 80
7.3 Transmission 81
7.3.1 Attributs de **transmission** . . . 81
7.3.2 Eléments de **transmission** . . . 81
7.3.3 Notes de développement . . . 82
7.4 Joint 82
7.4.1 Elément **<joint>** . . . 82
7.4.2 Attributs . . . 82
7.4.3 Elements . . . 82
7.5 Gazebo 83
7.5.1 Eléments pour les corps/links . . . 83
7.5.2 Eléments pour les joints . . . 83
7.6 model_state 84
7.6.1 Elément <model_state> . . . 84
7.6.2 Model State . . . 84
7.7 model 84

8 Simulation . . . 85
8.1 Introduction 85
8.2 Algorithme général d'un simulateur système 86
8.3 Algorithme général d'un moteur dynamique 86
8.3.1 Problème 1 : Realité . . . 86
8.3.2 Problème 2 : Flexibilité logicielle . . . 86

IV Introduction à ROS-2

9 Les paquets ROS 2 . . . 89
9.1 Configuration de l'environnement 89
9.1.1 Initialiser les variables d'environnement . . . 89
9.2 Structure générale des paquets ROS 90
9.3 Ligne de commande pour inspecter les paquets 90
9.4 Création d'un paquet 90
9.5 Description des paquets : le fichier package.xml 92
9.6 Compilation des paquets humble 93
9.7 Chaînage de workspace 93

10 Graphe d'application avec ROS-2 . . . 95
10.1 La commande ros2 95
10.2 Turtlesim et rqt 96
10.2.1 Utilisation de turtlesim . . . 96
10.2.2 Installer rqt . . . 97
10.2.3 Utiliser rqt . . . 97
10.2.4 Essayer le service spawn . . . 98
10.2.5 Essayer le service **set_pen** . . . 99
10.2.6 Redirection . . . 100
10.2.7 Arrêt de turtlesim . . . 100
10.3 Node 100
10.3.1 Définition d'un node . . . 100
10.3.2 ros2 run . . . 100
10.3.3 ros2 node list . . . 101
10.3.4 Redirection . . . 101
10.3.5 ros2 node info . . . 101
10.3.6 rqt_graph . . . 102
10.4 Topic 102
10.4.1 ros2 topic list . . . 102
10.4.2 ros2 topic bw . . . 104
10.4.3 ros2 topic echo . . . 104
10.4.4 ros2 topic hz . . . 105
10.4.5 ros2 topic info . . . 105
10.5 Service 106
10.5.1 ros2 service list . . . 107
10.5.2 ros2 service type (service) . . . 107
10.5.3 ros2 service list -t . . . 107
10.5.4 ros2 interface show . . . 108
10.5.5 ros2 service call . . . 108
10.6 Paramètres 108
10.6.1 ros2 param list . . . 109
10.6.2 ros2 param dump . . . 110
10.6.3 ros2 param load . . . 110
10.7 Actions 110
10.7.1 Introduction . . . 110
10.7.2 Préparation . . . 111
10.7.3 Utilisation des actions . . . 111
10.7.4 ros2 node info . . . 111

10.7.5 ros2 action list . . . 112
10.7.6 ros2 action info . . . 113
10.7.7 ros2 interface show . . . 113
10.7.8 ros2 action send_goal . . . 113
10.7.9 Résumé . . . 114
10.8 rqt_console 114
10.8.1 rqt . . . 114
10.8.2 rqt_console . . . 115
10.8.3 Niveaux d'enregistrement . . . 115
10.9 Bag : fichier de données 116
10.10 Launch : démarrer une application 116
10.10.1 Introduction . . . 116
10.10.2 Un exemple . . . 117
10.10.3 Préparer son package . . . 118
10.11 Création de messages et de services 119
10.11.1 Introduction . . . 119
10.11.2 Création d'un msg . . . 120
10.11.3 Création d'un srv . . . 120
10.11.4 CMakeList.txt . . . 120
10.11.5 package.xml . . . 120
10.11.6 Génération des codes . . . 121
10.11.7 Vérifier la création du fichier msg et srv . . . 121

11 Ecrire des nodes ROS-2 . . . 123
11.1 Topics 123
11.1.1 Création d'un paquet . . . 123
11.1.2 Emetteur . . . 124
11.1.3 Souscripteur . . . 129
11.1.4 Lancer les nodes . . . 132
11.2 Services 133
11.2.1 Création d'un paquet . . . 133
11.2.2 Serveur . . . 134
11.2.3 Client . . . 136
11.2.4 Lancer les nodes . . . 139

V Tutoriaux sur TIAGo

12 TIAGo . . . 143
12.1 Installation 143

13 Contrôle . . . 145
13.1 Téléopérer la base mobile avec le clavier 145
13.1.1 But . . . 145
13.1.2 Lancer la simulation . . . 145
13.1.3 Faire bouger le robot . . . 145
13.2 Faire bouger la base à travers des commandes en vitesse 145
13.2.1 But . . . 145
13.2.2 Lancer la simulation . . . 146
13.2.3 Faire bouger le robot en avant et en arrière . . . 146

13.3 Contrôleur des trajectoires articulaires 146
13.3.1 But du tutoriel . . . 147
13.3.2 Contrôleur du torse . . . 147
13.3.3 Contrôleur de la tête . . . 147
13.3.4 Contrôleur du bras . . . 147
13.3.5 Contrôleur du grippeur Hey5 . . . 147
13.3.6 Contrôleur du grippeur . . . 147
13.3.7 Lancer la simulation . . . 147
13.3.8 Lancer le node . . . 148
13.4 Bouger des articulations individuellement 149
13.4.1 Lancer la simulation . . . 149
13.4.2 Bouger les articulations individuellement . . . 150
13.4.3 Bouger les articulations à partir de la ligne de commande . . . 150
13.5 Contrôle de la tête 150
13.5.1 But du tutoriel . . . 150
13.5.2 Lancer la simulation . . . 151
13.5.3 Lancer l'exemple d'un client d'action en C++ . . . 151
13.6 Rejouer des mouvements du haut du corps prédéfinis 151
13.6.1 Lancer la simulation . . . 151
13.6.2 Jouer un mouvement via une interface graphique . . . 153
13.6.3 Jouer un mouvement en ligne de commande . . . 154

14 TIAGo Navigation . . . 155
14.1 Créer une carte avec gmapping 155
14.1.1 But . . . 155
14.1.2 Execution . . . 155
14.2 Localisation et planification de mouvement 157
14.2.1 But . . . 157
14.2.2 Localization . . . 157
14.2.3 Navigation autonome with rviz . . . 158

15 Planification de mouvements . . . 161
15.1 Planifier dans l'espace de configuration 161
15.1.1 But . . . 161
15.1.2 Lancer la simulation . . . 161
15.1.3 Lancer les nodes . . . 161
15.1.4 Inspection du code . . . 162
15.2 Planifier dans l'espace cartésien 163
15.2.1 But . . . 164
15.2.2 Lancer la simulation . . . 164
15.2.3 Lancer les nodes . . . 164
15.2.4 Inspection du code . . . 164
15.3 Planification avec Octomap 167
15.3.1 Goal . . . 167
15.3.2 Lancer la simulation . . . 167
15.3.3 Lancer les nodes . . . 168
15.3.4 Démarrer la démonstration . . . 168
15.3.5 Le plugin Rviz de MoveIt ! . . . 170

15.4 Démonstration d'une prise et d'un replacement d'un objet 170
15.4.1 But . . . 170
15.4.2 Lancer la simulation . . . 171
15.4.3 Lancer les nodes . . . 171
15.4.4 Démarrer la démonstration . . . 171

VI Navigation

16 Navigation : Nav2 . . . 177
16.1 Introduction 177
16.2 Concepts de Navigation 178
16.2.1 Construction de cartes . . . 178
16.2.2 Serveurs d'action . . . 179
16.2.3 Les nodes avec cycles de vie . . . 179
16.2.4 Arbre de Comportement - Behavior Trees . . . 179
16.3 Serveurs de navigation 180
16.3.1 Planificateurs, Contrôleurs, Lisseurs et serveurs de récupération . . . 180
16.3.2 Planificateurs . . . 180
16.3.3 Contrôleurs . . . 181
16.3.4 Comportements . . . 181
16.3.5 Lisseurs . . . 181
16.3.6 Suiveur de point intermédiaires . . . 181
16.4 Estimation d'état 182
16.4.1 Standards . . . 182
16.4.2 Localisation globale et SLAM . . . 182
16.4.3 Odométrie . . . 183
16.5 Représentation de l'environnement 183
16.5.1 Les cartes de coût et les couches . . . 183
16.5.2 Filtres de cartes de coût . . . 183
16.6 REP-105 184
16.6.1 Spécifications . . . 184
16.6.2 Relation entre repères . . . 184

17 Plugins pour la navigation . . . 185
17.1 Introduction 185
17.1.1 Pré requis . . . 185
17.2 Ecrire un nouveau plugin Costmap2D 185
17.3 Exporter et construire le plugin GradientLayer 187
17.4 Utiliser le plugin dans Costmap2D 188
17.5 Lancer le plugin GradientLayer 189

18 Plugin de planification . . . 191
18.1 Introduction 191
18.2 Création d'un nouveu plugin de planification de mouvements 191
18.3 Exporter le plugin de planification 193
18.4 Passer le nom du plugin par le fichier de paramètres 194
18.5 Lancer le plugin StraightLine 194

19 Ecrire un nouveau plugin de contrôleur 195
19.1 Introduction 195
19.2 Créer un nouveau plugin contrôleur 195
19.3 Exporter le plugin du contrôleur 198
19.3.1 Passer le nom du plugin à travers le fichier de paramètres 199
19.3.2 Lancer le plugin de contrôleur en poursuite pure 199

VII ROS2 Control

20 ros2_control : Introduction 203
20.1 Concepts 203
20.1.1 ControllerManager 205
20.1.2 Contrôleurs 205
20.2 Exemples d'interfaces matériel 205
20.2.1 Le champ ros2_control dans le fichier URDF 205
20.2.2 Accés à un micro-contrôleur par Ethernet 208
20.2.3 Ethercat 209
20.2.4 Les robots industriels 210
20.3 Exemple de contrôleur : diff_drive_controller 211
20.3.1 Chaînage des contrôleurs 213
20.3.2 Vue générale de ros2_control avec MoveIt 2 213

21 Les démonstrations de ros2_control 215
21.1 Exemple 1 : RRBot 215
21.1.1 Démarrer l'exemple 215
21.1.2 Vérification du matériel 215
21.1.3 Vérifier que les contrôleurs fonctionnent 216
21.1.4 Envoyer des commandes 216
21.1.5 Utilisation d'un autre contrôleur 217
21.1.6 Pour aller plus loin 217
21.2 Exemple 3 : Un robot avec plusieurs interfaces 217
21.2.1 Démarrage de RRBot 218
21.2.2 Vérification du matériel 218
21.2.3 Vérification des contrôleurs 218
21.2.4 Envoi d'une commande au contrôleur 218
21.2.5 Test des configurations illégales de contrôleur 219
21.3 Exemple 12 : Le chaînage de contrôleurs 219
21.3.1 Démarrage de l'exemple 220
21.3.2 Vérification des contrôleurs 220
21.3.3 Liste des interfaces matérielles 220
21.3.4 Lancement de la chaîne complète de contrôleurs 220
21.3.5 Vérification des contrôleurs 221
21.3.6 Vérification des interfaces 221
21.3.7 Envoi de commandes 221

22 ROS-2 control pour les robots de l'ODRI 223
22.1 Objet SystemOdriInterface pour la master board 223
22.1.1 En-tête de l'interface matérielle 226
22.1.2 Code de l'objet SystemInterface 227

22.2 Création du fichier URDF **235**
22.2.1 Source de la description géométrique du robot . . . 235
22.2.2 Fichier d'entrée . . . 235
22.2.3 Description de l'arbre cinématique . . . 236
22.2.4 Description de la partie ros2_control . . . 238
22.3 Lancement du robot **240**

VIII **Divers**

23 **Docker and ROS 2** . . . 243
23.1 Introduction **243**
23.2 Exemples simples **243**
23.2.1 Lancer deux nodes dans un seul container docker . . . 243
23.2.2 Lancer deux nodes dans deux dockers séparés . . . 244
23.2.3 docker compose . . . 244
23.3 Exemple de docker avec accélération graphique **244**
23.4 Construire son propre fichier Docker pour ROS **245**
23.4.1 Créer un fichier Dockerfile pour installer des paquets ROS . . . 245
23.4.2 Créer un fichier Dockerfile pour construire des paquets ROS . . . 245
23.5 Exemple d'utilisation pour le déploiement **246**
23.6 Scénarios de déploiements **247**
23.6.1 Volumes . . . 247
23.6.2 Matériel . . . 247
23.6.3 Réseaux . . . 247
23.7 Exemple de déploiement **248**
23.7.1 Docker Compose . . . 248
23.7.2 ROS 1 Bridge . . . 248
23.8 Connection réseau avec l'hôte **249**

IX **Appendices**

24 **Rappels sur le bash** . . . 253
24.1 Lien symbolique **253**
24.1.1 Voir les liens symboliques . . . 253
24.1.2 Créer un lien symbolique . . . 253
24.1.3 Enlever un lien symbolique . . . 253
24.2 Gestion des variables d'environnement **254**
24.3 Fichier .bashrc **254**

Bibliography . . . 255
Books **255**
Articles **255**
Chapitre de livres **255**
Autres **255**

1. Introduction

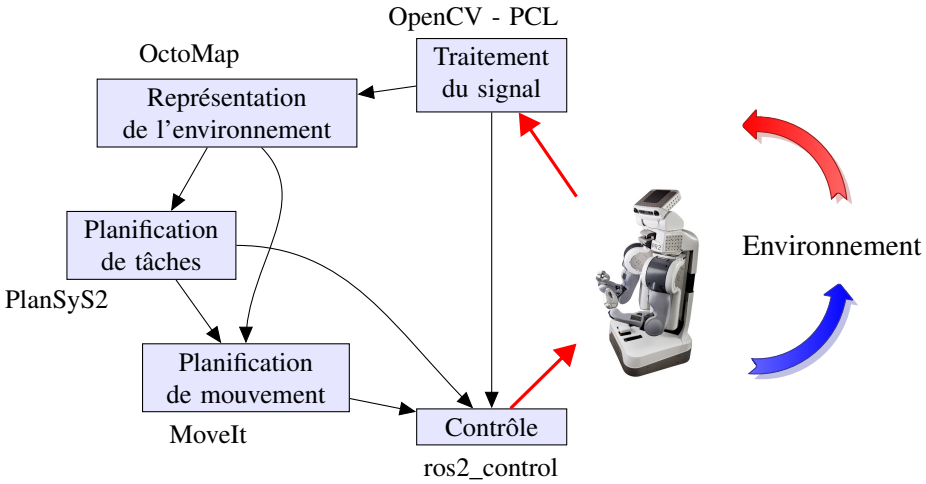


FIGURE 1.1 – Structure d'une application robotique

1.1 Motivations

1.1.1 Complexité des systèmes robotiques

Les systèmes robotiques font appel à de nombreuses compétences telles que la mécanique, l'électrotechnique, le contrôle, la vision par ordinateur, et de nombreux pans de l'informatique comme l'informatique temps-réel, le parallélisme, et le réseau. Souvent les avancées en robotique dépendent de verrous technologiques résolus dans ces champs scientifiques. Un système robotique peut donc être complexe et faire intervenir de nombreuses personnes. Afin de pouvoir être efficace dans la conception de son système robotique il est donc impératif de pouvoir réutiliser au maximum des outils existants et de pouvoir collaborer efficacement avec d'autres équipes.

En physique, des systèmes à grande échelle comme le *Large Hadron Collider* ont vu le jour grâce à des collaborations internationales à grande échelle. En informatique des projets comme le système d'exploitation Linux n'existe qu'à travers une collaboration internationale. Celle-ci peut impliquer des individus, mais on trouve également le support de sociétés qui ne souhaitent pas s'engager dans le développement complet

d'un système d'exploitation mais qui souhaiteraient de nouvelles fonctionnalités. En effet cette approche est souvent moins coûteuse car le développement d'un système d'exploitation est évaluer à 100 années pour un seul homme.

Le système ROS pour Robotics Operating System est le premier projet collaboratif robotique à grande échelle qui fournit un ensemble d'outils informatique permettant de gagner du temps dans la mise au point d'un robot, ou d'un système robotique.

1.1.2 Application Robotique

Une application robotique suit en général la structure représentée dans la figure 1. Elle consiste à générer une commande afin qu'un robot puisse agir sur un environnement à partir de sa perception de celui-ci. Il existe plusieurs boucles de *perception-action*. Elles peuvent être :

$40\ Khz$ très rapide par exemple pour le contrôle du courant,

$1\ Khz - 200\ Hz$ rapide, par exemple pour le contrôle du corps complet d'un robot humanoïde,

$1\ s - 5\ mn$ lente, par exemple dans le cadre de la planification de mouvements réactifs dans des environnements complexes.

$2 - 72h$ trés lente, par exemple dans le cadre de planification à grande échelle, ou sur des systèmes avec de trés grands nombre de degrés de libertés.

Dans tous les cas, il est nécessaire de transporter l'information entre des composants logiciels largement hétérogènes et faisant appel à des compétences très différentes. Chacun de ces composants logiciels a une structure spécifique qui dépend du problème à résoudre et de la façon dont il est résolu. Chacun est encore très actif du point de vue de la recherche scientifique, ou demande des niveaux de technicité relativement avancés. La réprésentation de l'environnement par exemple a connu récémment beaucoup de changements, et on trouve maintenant des prototypes industriels permettant de construire en temps réel des cartes denses de l'environnement sur des systèmes embarqués [7]. Il est donc nécessaire de pouvoir facilement tester des structures logiciels implémentant des algorithmes différents pour une même fonctionalité sans pour autant changer l'ensemble de la structure de l'application. Par exemple on veut pouvoir changer la boîte "Représentation de l'environnement" sans avoir à modifier les autres boîtes. Ceci est réalisé en utilisant des messages standards et les mêmes appels de service pour accéder aux logiciels correspondants à ces boîtes. Enfin, on souhaiterait pouvoir analyser le comportement du robot à différents niveaux pour investiguer les problèmes potentiels ou mesurer les performances du système. Dans le paragraphe suivant, les concepts de ROS permettant de répondre à ces challenges sont introduits.

1.2 Concepts de ROS

La clef de ROS est la mise en place de 4 grands types de mécanismes permettant de construire une application robotique comme celle décrite dans le paragraphe précédent. Ces mécanismes, décrits dans la figure. 1.2.1, sont :

- la *Plomberie*, c'est à dire la connection des composants logiciels entre eux quelque soit la répartition des noeuds de calcul sur un réseau,
- les *Outils*, c'est à dire un ensemble de logiciels permettant d'analyser, d'afficher et de débugger une application répartie,
- les *Capacités*, c'est à dire des librairies qui implémentent les fonctionnalités telles que la planification de tâches (SMACH), de mouvements (OMPL), la construction d'un modèle de l'environnement (Octomap),
- l'*Ecosystème*, c'est à dire un nombre suffisant d'utilisateurs tels que ceux-ci ont plus intérêt à collaborer plutôt qu'à reconstruire les mêmes outils.

1.2.1 Plomberie

La plomberie, représentée dans la figure 1.2.1, est implémentée grâce à un middleware. Un middleware fournit un bus logiciel qui permet à des objets localisés sur différents ordinateurs d'interagir. L'interaction s'effectue en transmettant des données sous une forme normalisée (les *messages*) via ce que pourrait voir comme des post-its (les *topics*). Un noeud de calcul (un *node*) produisant des données peut ainsi inscrire des données qu'il produit sur un post-it en utilisant le nom de celui-ci. Un autre noeud lui lira les données sur le post-it sans savoir quel noeud a produit ces données. La normalisation des données est réalisée grâce à un langage de description d'interface. Des programmes permettent de générer automatiquement la transcription



FIGURE 1.2 – Concepts généraux sous-jacents à ROS

entre ce langage de description et des langages de programmation comme le C++, python ou java. Afin de pouvoir communiquer entre les objets, le système fournit un annuaire sur lequel viennent s'enregistrer chacun des noeuds de calcul. Enfin un noeud peut demander un service à un autre noeud en suivant un mécanisme de d'appel de service à distance. Ces mécanismes très généraux existent aussi chez d'autre middlewares comme Corba, ou YARP un autre middleware robotique. En général un middleware fournit les mécanismes d'appels pour différents langagues et différents systèmes d'exploitation. On peut également stockés des paramètres qui correspondent à des données spécifiques à une application, à un robot ou des données qui d'une manière générale n'évoluent pas au cours de l'exécution d'une application.

1.2.2 Outils

Les outils sont une des raisons du succés de ROS. Nous trouvons ainsi :

— *rviz* : Une interface graphique permettant d'afficher les modèles des robots, des cartes de navigation reconstruites par des algorithmes de SLAM, d'interagir avec le robot, d'afficher des images, des points 3D fournis par des caméras 3D.

— *rqt_graph* : Une interface graphique permettant d'analyser le graphe d'applications et les transferts de données via les topics.

— *rosbag* : Un programme permettant d'enregistrer et de rejouer des séquences topics. La fréquence, la durée les topics à enregistrer peuvent être spécifiés.

— *rqt* : Une interface de contrôle incrémentale. Elle se base sur le système de plugins de la bibliothèque de GUI Qt.

— *catkin* : Un système de gestion de paquets, de génération de code automatique et compilation.

Chacun de ces outils a également ses propres limites, et de nombreux alternatives existent dans les paquets fournis sur ROS.

1.2.3 Capacités

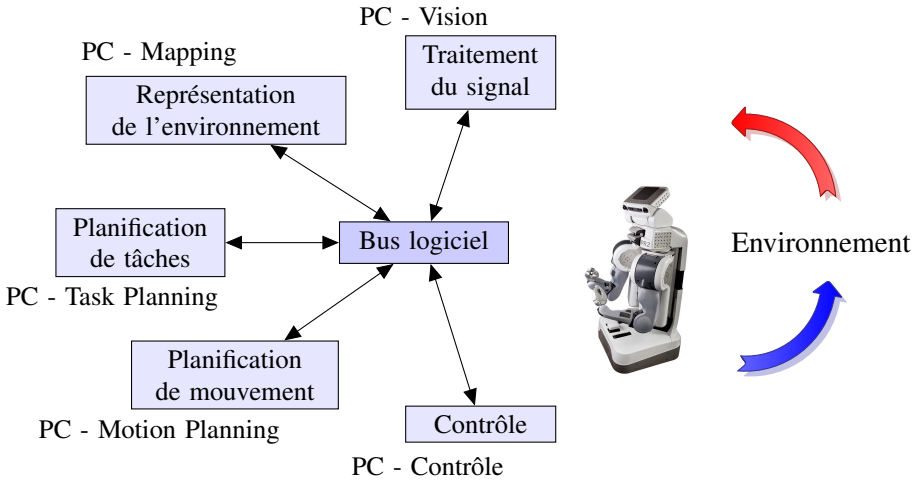


FIGURE 1.3 – Structure d'une application robotique

Les différents fonctionnalités décrites dans la figure 1 sont implémentés par des librairies spécialisés décrites dans la figure 1.2.3. Chacune est issue de plusieurs décennies de recherche en robotique et souvent encore l'objet de nombreuses avancées. Parmi celles-ci on peut trouver :

— Pour la vision :

  - La point cloud library
  - La librairie de vision par ordinateur.
- Pour la reconstruction d'environnements :
  - En 2D : La pile de navigation pour ROS-1 et la https ://navigation.ros.org/pile de navigation ROS-2
  - En 3D : ICP Mapping.
- Pour la planification de mouvements :
  - MoveIt !
- Pour la planification de mission :
  - SysPlan2
  - SMACH avec de la documentation disponible ici
  - Mission Planner pour les drônes
- Pour le contrôle :
  - iTask
  - ControlIt
  - Stack-Of-Tasks

1.2.4 Exemples

ROS-1 a été utilisé comme support pour des challenges robotiques comme le DARPA Robotics Challence et le challenge robotique de la NASA. Dans le cade du DARPA Robotics Challenge qui a pris place en Juin 2015, de nombreuses équipes ont du faire face à des défis d'intégration. ROS-1 a été une des briques fondamentales pour permettre aux équipes utilisant ATLAS de pouvoir fonctionner.

1.3 Ecosystème

1.3.1 Historique

ROS-1 a été écrit à partir de 2008 par la société Willow Garage fondée par Larry Page, également fondateur de Google. En plus de ROS-1, cette société a construit le PR-2, un robot très sophistiqué équippé de deux bras, d'une base mobile et avec des moyens de calcul importants. Les implémentations effectués par cette société ont été nombreux et ont permis d'atteindre une masse critique d'utilisateurs sans commune mesure avec les actions concertées précédentes. Pour cette raison on retrouve dans les documentations des paquets historiques souvent la référence à Willow Garage. Cette société a cependant été fermée en 2013 et a fait place à l'Open Source Robotics Foundation financé par plusieurs société. Un des faits récents les plus notables est le don de 50 millions de dollars effectués par Toyota, suivi d'un autre investissement de 50 millions de dollars pour construire une structure privée adossée à l'OSRF.

L'historique des releases de ROS et ROS-2 sont résumé dans les tableaux 1.1 et 1.2. Si on note au démarrage un manque de régularité, la politique actuelle consiste à caler les releases de ROS sur celles d'Ubuntu en Long Time Support (LTS) et celles plus courtes des années impaires. D'une manière générale on constate un décalage d'au moins une année entre la release de ROS et celle de la mise à jour des applications robotiques. C'est en général le temps qu'il faut pour résoudre les erreurs des releases en LTS et porter le code des applications robotiques.

1.3.2 Fonctionnalités

Il existe un certain nombre de logiciels très bien supportés par des projets et des communautés qui offre des fonctionnalités très intéressantes pour la robotique. Certains existent par ailleurs ROS et d'autres sont supportés directement par l'OSRF. En voici une liste non-exhaustive :

- **Définition de messages standards pour les robots**.
  ros_comm est le projet qui définit et implémente le mécanisme de communication initial et les types initiaux. De nombreux paquets viennent complèter les types nécessaires dans les applications robotiques.
- **Librairie pour la géométrie des robots**.
  La librarie KDL est utilisée pour implémenter le calcul de la position de tous les corps du robot en fonction de ces actionneurs et du modèle des robots.
- **Un language de description des robots**.

2008	Démarrage de ROS par Willow Garage
2010 - Janvier	ROS 1.0
2010 - Mars	Box Turtle
2010 - Aout	C Turtle
2011 - Mars	Diamondback
2011 - Aout	Electric Emys
2012 - Avril	Fuerte
2012 - Décembre	Groovy Galapagos
2013 - Février	Open Source Robotics Fundation poursuit la gestion de ROS
2013 - Aout	Willow Garage est absorbé par Suitable Technologies
2013 - Aout	Le support de PR-2 est repris par Clearpath Robotics
2013 - Septembre	Hydro Medusa (prévu)
2014 - Juillet	Indigo Igloo (EOL - Avril 2019)
2015 - Mai	Jade Turtle (EOL - Mai 2017)
2016 - Mai	Kinetic Kame (EOL - Mai 2021)
2017 - Mai	Lunar Loggerhead (EOL - Mai 2019)
2018 - Mai	Melodic Morenia (EOL - Mai 2023)
2020 - Mai	Noetic Ninjemys (EOL - Mai 2025)

TABLEAU 1.1 – Historique de ROS et des releases - REP 3

2017 - Dec	Ardent Apalone (EOL - Dec 2018)
2018 - June	Bouncy Bolson (EOL - June 2019)
2018 - Dec	Crystal Clemminys (EOL - Dec 2019)
2019 - Mai	Dashing Diademata (EOL - Mai 2021)
2019 - Nov	Eloquent Elusor (EOL - Mai 2020)
2020 - May	Foxy Fitzroy (EOL - Mai 2023)
2021 - May	Galactic Geochelone (EOL - Nov 2022)
2022 - May	Humble Hawksbill (EOL - Mai 2027)
2020 - June	Rolling Ridley (EOL - Ongoing)

TABLEAU 1.2 – Historique des releases de ROS 2 - REP 2000

URDF pour Universal Robot Description Format est un spécification en XML décrivant l'arbre cinématique d'un robot, ses caractéristiques dynamiques, sa géométrie et ses capteurs.

— **Un système d'appel de fonctions à distance interruptible**.

— **Un système de diagnostiques**.
Il s'agit ici d'un ensemble d'outils permettant de visualiser des données (rviz), d'afficher des messages (roslog/rosout), d'enregister et de rejouer des données (rosbag), et d'analyser la structure de l'application (rosgraph)

— **Des algorithmes d'estimation de pose**.
Des implémentations de filtre de Kalman, des systèmes de reconnaissance de pose d'objets basé sur la vision (ViSP).

— **Localisation - Cartographie - Navigation**.
Des implémentations d'algorithmes de SLAM (Self Localization and Map building) permettent de construire une représentation 2D de l'environnement et de localiser le robot dans cet environnement.

— **Une structure de contrôle**.
Afin de pouvoir normaliser l'interaction entre la partie haut-niveau d'une application robotique que sont la planification de mouvements, la planification de missions, avec partie matérielle des robots, le framework ros-control a été proposé.

1.3.3 Systèmes supportés

ROS-1 ne supporte officiellement que la distribution linux Ubuntu de la société Canonical. La raison principale est qu'il est très difficile de porter l'ensemble de ROS-1 sur tous les systèmes d'exploitation cela nécessiterait de la part de tous les contributeurs de faire l'effort de porter tous les codes sur plusieurs systèmes. Certains logiciels sont très spécifiques à des plateformes particulières et ont peu de sens en dehors de leur cas de figure. A l'inverse le coeur de ros constitués des paquets roscomm a été porté sur différents systèmes d'exploitations comme OS X, Windows, et des distributions embarquées de Linux (Angstrom, OpenEmbedded/Yocto).

ROS-2 était iniatlement prévu pour être supporté sur Ubuntu, Windows 10 et MacOSX, mais il a été décidé récemment que le support de MacOS allait être moins prioritaire du à des difficultés de portage et une faible utilisation.

Les releases de ROS sont spécifiées par la REP (ROS Enhancement Proposal) : `http://www.ros.org/reps/rep-0003.html`

Ceci donne pour les dernières releases :

— Noetic Ninjemys (Mai 2020 - Mai 2025)
  — Ubuntu Focal Fossa (20.04 LTS)
  — Debian Buster
  — Fedora 32
  — C++14, Boost 1.71/1.69/1.67, Lisp SBCL 1.4.16, Python 3.8, CMake 3.16.3/3.13.4/3.17
  — Ogre3D 1.9.x, Gazebo 9.0.0, PCL 1.10/1.9.1, OpenCV 4.2/3.2, Qt 5.13.2/5.12.5/5.11.3, PyQt5
— Melodic Melusa (Mai 2019 - Mai 2021)
  — Ubuntu Xenial (18.04 LTS)
  — Ubuntu Yakkety (17.10)
  — Debian Stretch
  — Fedora 28
  — C++14, Boost 1.62/1.65.1/1.66, Lisp SBCL 1.3.14, Python 2.7, CMake 3.7.2/3.9.1/3.10.2
  — Ogre3D 1.9.x, Gazebo 8.3.0/9.0.0, PCL 1.8.0/1.8.1, OpenCV 3.2, Qt 5.7.1/5.9.5/5.10.0, PyQt5
— Lunar Loggerhead (Mai 2017 - Mai 2019)
  — Ubuntu Xenial (16.04 LTS)
  — Ubuntu Yakkety (16.10)
  — Ubuntu Zesty (17.04)
  — C++11, Boost 1.58/1.61/1.62/, Lisp SBCL 1.2.4, Python 2.7, CMake 3.5.1/3.5.2/3.7.2
  — Ogre3D 1.9.x, Gazebo 7.0/7.3.1/7.5, PCL 1.7.2/1.8.0, OpenCV 3.2, Qt 5.5.1/5.6.1/5.7.1, PyQt5
— Kinetic Kame (Mai 2016 - Mai 2021)
  — Ubuntu Wily (15.10)
  — Ubuntu Xenial (16.04 LTS)
  — C++11, Boost 1.55, Lisp SBCL 1.2.4, Python 2.7, CMake 3.0.2
  — Ogre3D 1.9.x, Gazebo 7, PCL 1.7.x, OpenCV 3.1.x, Qt 5.3.x, PyQt5

1.4 Exemple : Asservissement visuel sur TurtleBot 2

Afin d'illustrer le gain à utiliser ROS considérons l'application illustrée dans la Fig.1.4. On souhaite utiliser un robot Turtlebot 2 équipé d'une Kinect et d'une base mobile Kobuki. Le but est de contrôler le robot de telle sorte à ce qu'il reconnaisse un objet par sa couleur et qu'il puisse génére une commande à la base mobile de telle sorte à ce que l'objet soit centré dans le plan image de la caméra du robot. Pour cela on doit extraire une image I du robot et générer un torseur cinématique $\begin{pmatrix} v \\ w \end{pmatrix}$ pour la base mobile.

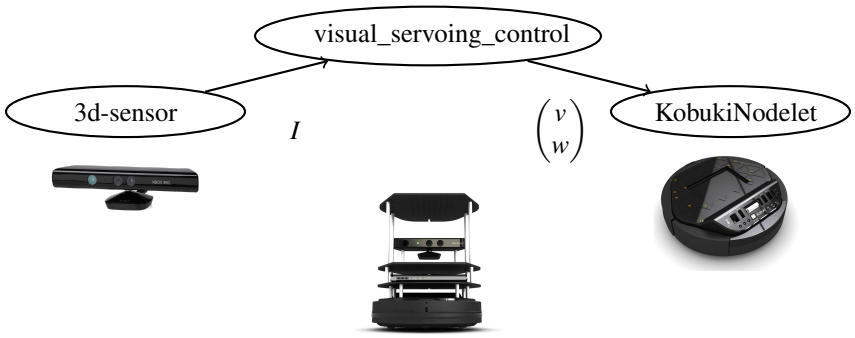


FIGURE 1.4 – Transfert d'une image I vers un node de contrôle qui calcule un torseur cinématique pour contrôler la base mobile du robot.

L'implémentation de cette structure logicielle s'effectue de la façon suivante sous ROS. En premier on utilise un noeud de calcul qui s'occupe de l'extraction de l'image d'une caméra Kinect. Dans la Fig.1.5 il est représenté par l'ellipse nommée **3d-sensor**. L'image I est alors publiée dans un topic appelé **/camera/image_color** qui peut-être vu comme un post-it. Celui-ci peut-être lu par un autre noeud de calcul appelé **visual_servoing_control** qui produit le torseur cinématique $\begin{pmatrix} v \\ w \end{pmatrix}$. Ce torseur est publié sur un autre topic appellé **/mobilebase/command/velocity**. Finalement celui-ci est lu par un troisième noeud de calcul appellé **KobukiNodelet**. Il est en charge de lire le torseur donné par rapport au centre de la base et le transformer en commande pour les moteurs afin de faire bouger la base mobile dans la direction indiquée.

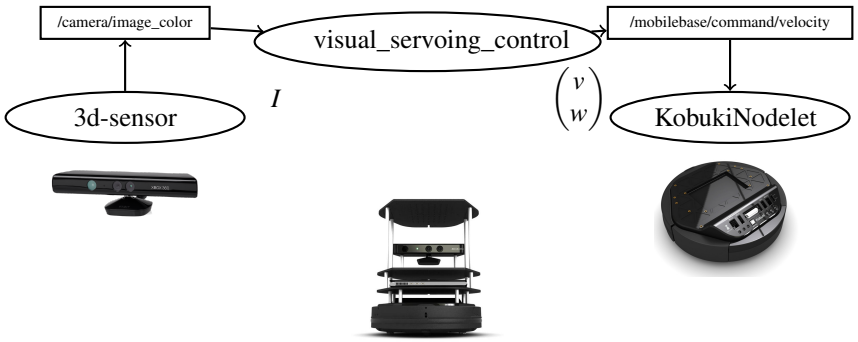


FIGURE 1.5 – Graphe d'une application ROS. Les ellipses représentent les noeuds de calcul ROS, et les rectangles les post-its où sont stockés des données disponibles pour tous les noeuds. Les flèches indiquent les flux d'informations échangées dans l'application.

Il est intéressant de noter que chacun des noeuds de calcul fournissent des fonctionnalités qui peuvent être indépendantes de l'application considérée. On peut par exemple utilise le noeud pour la Kinect sur un autre robot qui utiliserait le même capteur. Le programmateur de robot n'est ainsi pas obligé de réécrire à chaque fois un programme sensiblement identique. De même la base mobile peut-être bougée suivant une loi de commande différente de celle calculée par le noeud **visual_servoing_control**. Si un noeud est écrit de façon générique on peut donc le mutualiser à travers plusieurs applications robotiques, voir à travers plu-

sieurs robots. Enfin chacun de ces noeuds peut-être sur un ordinateur différent des autres noeuds. Cependant il est souvent nécessaire de pouvoir changer des paramètres du noeud afin de pouvoir l'adapter à l'application visée. Par exemple, on souhaiterait pouvoir contrôler les gains du noeud **visual_servoing_control**. Il est alors possible d'utiliser la notion de paramétres pour pouvoir changer les gains proportionnels K_p et dérivés K_d du contrôleur implémenté dans le noeud **visual_servoing_control** (cf Fig.1.6).

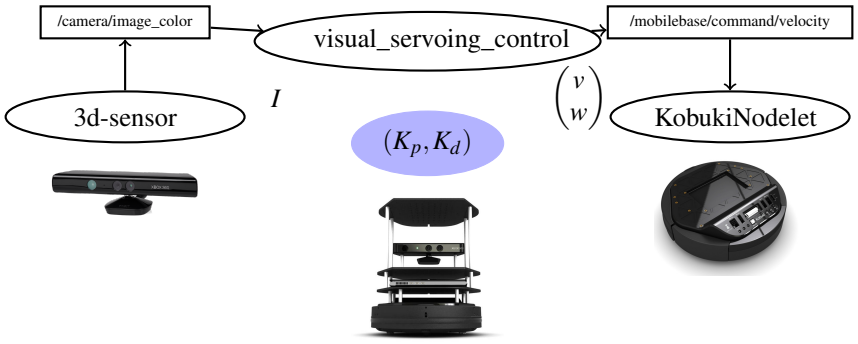


FIGURE 1.6 – Utilisation des paramètres pour moduler le comportement d'une loi de commande. Ici les paramètres K_p et K_d correspondent respectivement au gain proportionnel et au gain dérivé

Comme l'illustre la Fig.1.7 dans ROS, les programmes permettant à tous les noeuds d'interagir ensemble soit à travers des topics soit à travers des services sont **roscomm** (pour la communication) et **rosmaster** (pour les références entre noeuds). Un autre programme sert de serveur de paramètres **rosparam**. Le **rosmaster** enregistrent les nodes qui publient et qui souscrivent aux topics **rostopic**. Les publishers de topics sont informés de l'adresse des souscripteurs/subscribers par le **rosmaster**, et les communications entre les nodes par les topics s'effectuent directement. Une description technique détaillée est disponible à cette adresse : `https://wiki.ros.org/ROS/TechnicalOverview`.

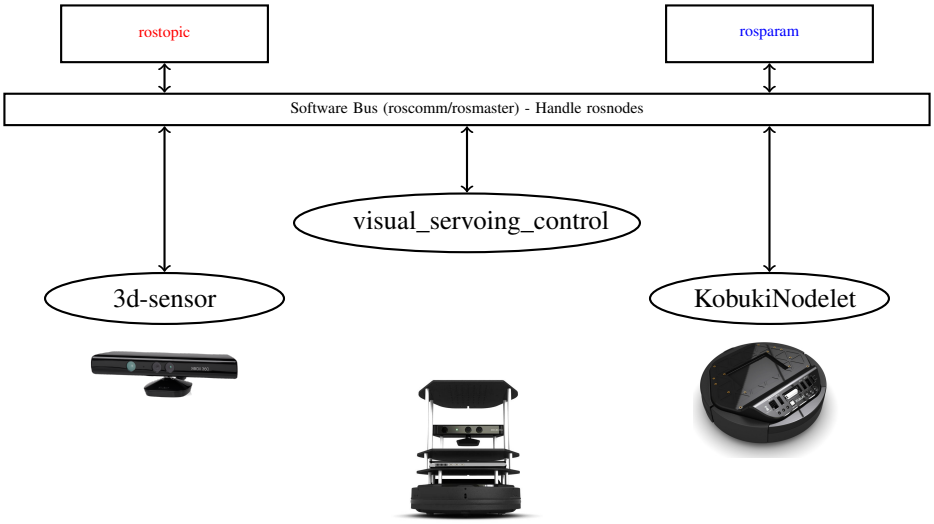


FIGURE 1.7 – Organisation interne de ROS pour implémenter les différents concepts

Une représentation plus générale du concept est donnée dans la figure Fig.1.8.

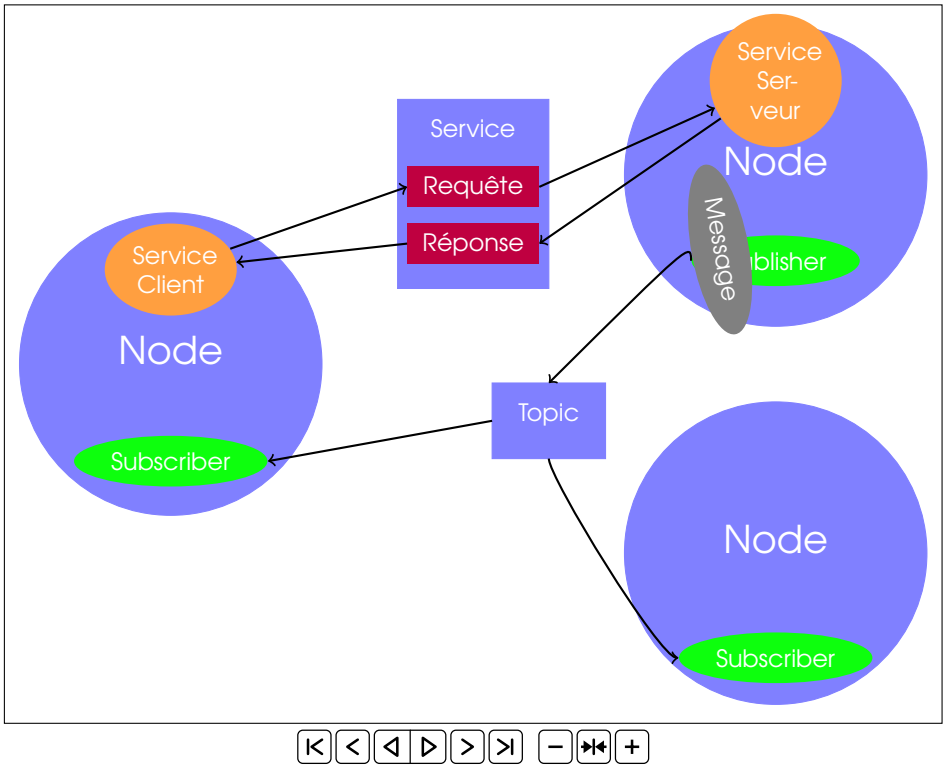


FIGURE 1.8 – Echange de données correspondant au mécanisme de service et de topic

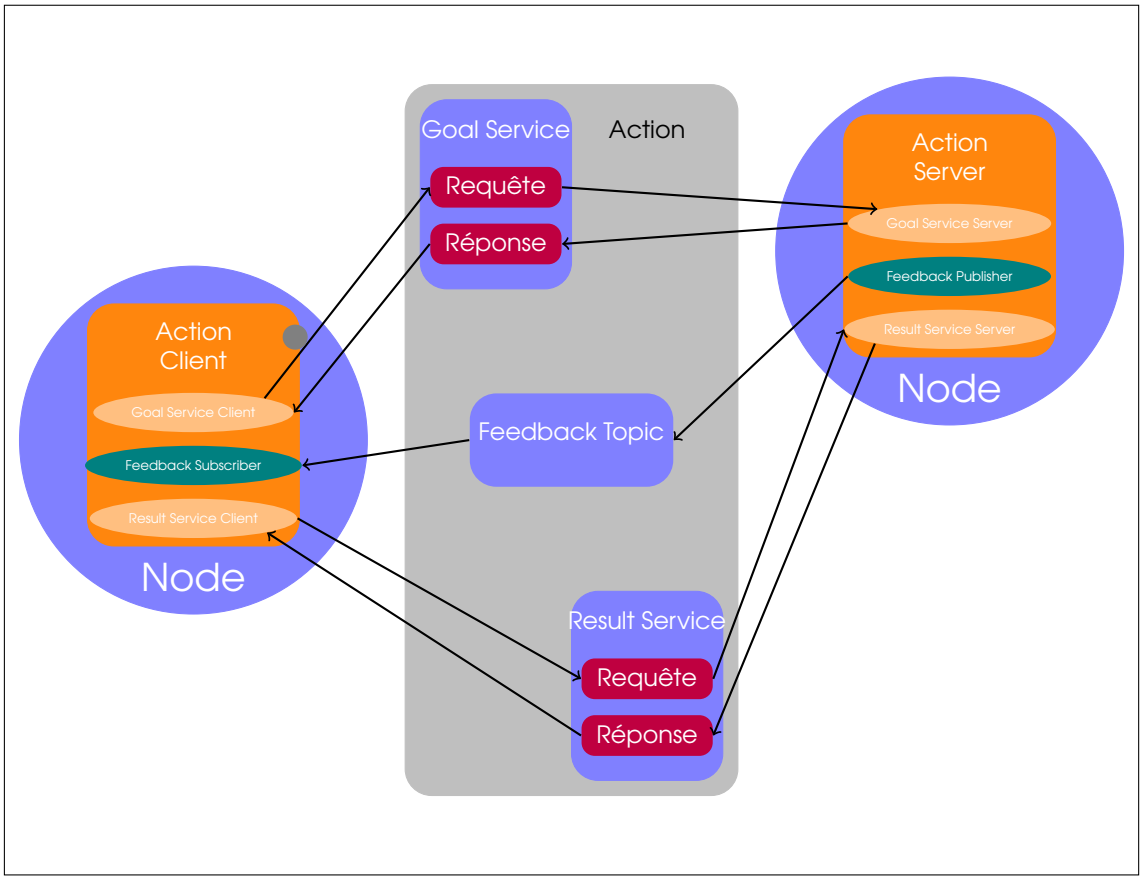


FIGURE 1.9 – Echange de données correspondant au mécanisme d'action

Introduction à ROS-1

2 Les paquets ROS 25

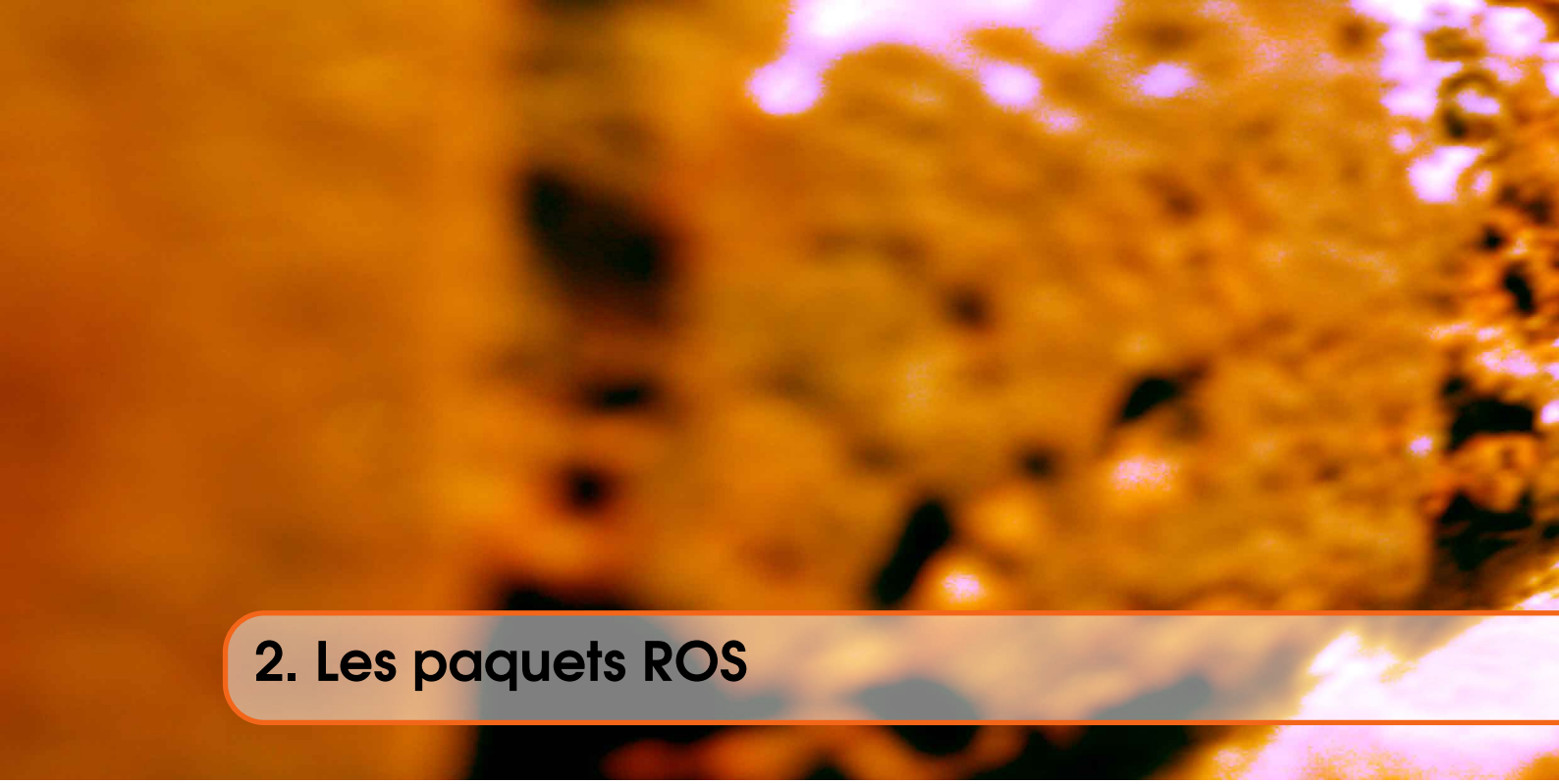
2.1 Configuration de l'environnement
2.2 Structure générale des paquets ROS
2.3 Création d'un paquet
2.4 Description des paquets : le fichier **package.xml**
2.5 Compilation des paquets melodic
2.6 Chaînage de workspace

3 Graphe d'application avec ROS-1 31

3.1 Introduction aux concepts de ROS
3.2 rqt_console
3.3 roslaunch
3.4 rosbag
3.5 rosservice
3.6 rosparam
3.7 Création de messages et de services

4 Ecrire des nodes ROS-1 47

4.1 Topics
4.2 Services



2. Les paquets ROS

Le système de gestion des paquets ROS est probablement un élèment clé de la réussite de ROS. Il est bien plus simple que le système de gestion des paquets par Debian. La contre-partie est que ce système ne fonctionne pas au-delà de l'ecosystème ROS.

2.1 Configuration de l'environnement

2.1.1 Initialiser les variables d'environnement

Afin de pouvoir utiliser ROS il est nécessaire de configurer son environnement shell. Sous linux, bash est le shell le plus utilisé. Il suffit donc d'ajouter la ligne suivante au fichier *.bashrc*

```
source /opt/ros/melodic/setup.bash
```

Pour créer son espace de travail il faut d'abord créer le répertoire associé

```
mkdir -p ~/catkin_ws/src
cd ~/catkin_ws/src
```

Puis ensuite initialiser l'espace de travail

```
catkin_init_workspace
```

Le tutoriel associé est : Tutorial Installing and Configuring Your ROS Environment : `http://wiki.ros.org/ROS/Tutorials/InstallingandConfiguringROSEnvironment`

Il est également recommandé de rajouter la ligne suivante dans le fichier *.bashrc* :

```
source $HOME/catkin ws/devel/setup.bash
```

Cependant le répertoire **devel** n'existe qu'une fois le script **catkin_make** utilisé (voir section9.6 pour plus de détails).

Une fois le fichier **setup.bash** sourcé on peut trouver deux variables d'environnement trés importantes :

```
env | grep ROS
ROS_MASTER_URI=http://localhost:11311
ROS_PACKAGE_PATH=/opt/ros/melodic/share
```

La variable **ROS_MASTER_URI** indique comment trouver l'annuaire des objets. Elle suit la structure suivante :

```
ROS_MASTER_URI=http://hostname:port_number
```

hostname indique l'ordinateur où **roscore** est lancé. **port_number** indique le port où **roscore** attend les connections.

La variable d'environnment *ROS_PACKAGE_PATH* indique les répertoires où chercher les paquets. Par défaut elle indique les paquets systèmes de la release ROS.

2.1.2 Navigation dans le système de paquets

On peut chercher un paquet avec le script **rospack**. Par exemple pour trouver le paquet **roscpp** :

```
rospack find roscpp
```

Comme il arrive souvent de devoir naviguer entre des paquets systèmes et des paquets de l'environnement de développement, il existe un équivalent de **cd** qui permet de naviguer directement entre les paquets :

```
roscd roscpp
roscd roscpp/cmake
```

Finalement il existe un répertoire particulier qui contient les logs de tous les nodes. On peut y accéder directement par :

```
roscd log
```

Le tutoriel associé est : Tutorial Navigation dans le système de fichiers
`http://wiki.ros.org/ROS/Tutorials/NavigatingTheFilesystem`

2.2 Structure générale des paquets ROS

Pour l'organisation des paquets ROS une bonne pratique consiste à les regrouper par cohérence fonctionnelle. Par exemple si un groupe de roboticiens travaille sur des algorithmes de contrôle, il vaut mieux les regrouper ensemble. Il est possible de faire cela sous forme d'un espace de travail (appelé également *workspace*). La figure Fig.22.2 représente une telle architecture. Dans un workspace, il faut impérativement créer un repertoire **src** dans lequel on peut créer un repertoire par paquet qui contient les fichiers d'un paquet.

Par exemple le paquet **package_1** contient au minimum deux fichiers :

— CMakeLists.txt : Le fichier indiquant comment compiler et installer le paquet
— package.xml : Le fichier ROS décrivant l'identité du paquet et ses dépendences.

Jusqu'à ROS groovy, il était possible de mettre ensemble des paquets dans des Stacks. Ceci a été simplifié et depuis l'introduction de **catkin** les Stacks ne sont plus supportées.4 Il suffit de mettre des paquets dans le même répertoire sous le répertoire **src** et sans aucun autre fichier pour faire des regroupements. Dans la figure Fig.22.2 on trouve par exemple le répertoire **meta_package_i** qui regroupe les paquets de **meta_package_i_sub_0** à **meta_package_i_sub_j**. On trouve par exemple des projets sous forme de dépôts qui regroupe de nombreux paquets légers ou qui contiennent essentiellement des fichiers de configuration. C'est par exemple le cas pour les dépots qui contiennent les paquets décrivant un robot. C'est le cas du robot TIAGO de la société PAL-Robotics https ://github.com/pal-robotics/tiago_robot.

Au même niveau que **src**, la plomberie de ROS va créer trois répertoires : **build**, **devel**, **install**.

Le répertoire **build** est utilisé pour la construction des paquets et à ce titre contient tous les fichiers objets. Le répertoire **devel** contient les exécutables et les librairies issus de la compilation des paquets et donc spécifiés comme cible dans le fichier **CMakeLists.txt**. Le répertoire **install** ne contient que les fichiers qui sont explicitement installés à travers les directives d'installation spécifiées dans le fichier **CMakeLists.txt** des paquets.

2.3 Création d'un paquet

Pour la création d'un paquet il faut donc se placer dans le répertoire **src** de l'espace de travail choisi. Dans la suite on supposera que l'espace de travail est le répertoire **catkin_ws**.

```
cd ~/catkin_ws/src
```

Pour créer un nouveau package et ses dépendences il faut utiliser la commande **catkin_create_pkg** en spécifiant le nom du paquet et chacune des dépendences.

```
catkin_create_pkg beginner_tutorials std_msgs rospy roscpp
catkin_create_pkg [package name] [depend1] [depend2] [depend3]
```

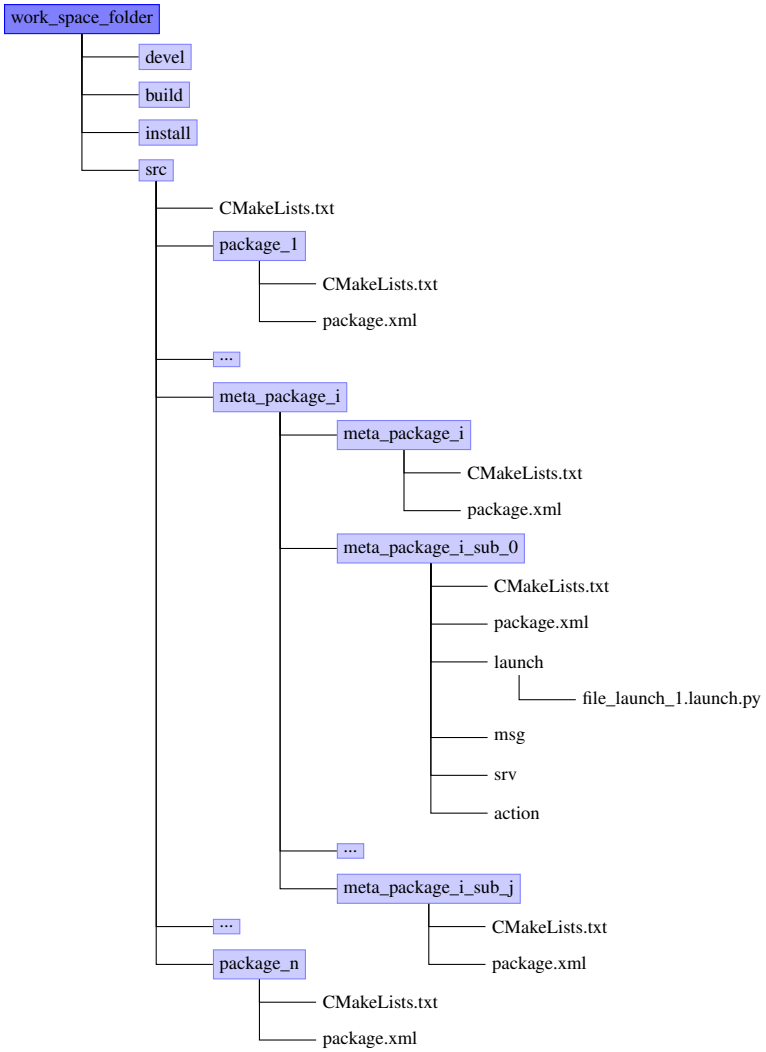


FIGURE 2.1 – Organisation de l'espace de travail : les paquets sources n'ont pas de structure hiérarchique (ou à un seul niveau lorsque les dépôts contiennent plusieurs paquets ROS).

On peut alors afficher les dépendances du premier ordre en utilisant la commande **rospack**.

```
rospack depends1 beginner_tutorials
std_msgs
rospy
roscpp
```

Les dépendances sont stockées dans le fichier **package.xml**.

```
roscd beginner_tutorials
cat package.xml
<package>
...
<buildtool_depend>catkin</buildtool_depend>
<build_depend>roscpp</build_depend>
...
</package>
```

2.4 Description des paquets : le fichier package.xml

Jusqu'à ROS groovy les paquets devait inclure un fichier de description appelé **manifest.xml**. Depuis l'introduction de **catkin** le fichier qui doit être inclus est **package.xml**.

Il existe deux versions du format spécifié dans le champ <package>. Pour les nouveaux paquets, le format recommandé est le format 2.

```
<package format=2>
</package>
```

Il y 5 champs nécessaires pour que le manifeste du paquet soit complet :

— <name> : Le nom du paquet.
— <version> : Le numéro du version du paquet.
— <description> : Une description du contenu du paquet.
— <maintainer> : Le nom de la personne qui s'occupe de la maintenance du paquet ROS.
— <license> : La license sous laquelle le code est publié.

La license utilisée le plus souvent sous ROS est BSD car elle permet d'utiliser ces codes également pour des applications commerciales.

Il est important de faire attention au contexte dans lequel le code peut-être licensié notamment dans un cadre professionnel.

```
<?xml version="1.0"?>
<package>
  <name>beginner_tutorials</name>
  <version>0.0.0</version>
  <description>The beginner_tutorials package</description>

  <!-- One maintainer tag required, multiple allowed, one person per tag -->
  <!-- Example:  -->
  <maintainer email="jane.doe@example.com">Jane Doe</maintainer>

  <!-- One license tag required, multiple allowed, one license per tag -->
  <!-- Commonly used license strings: -->
  <!--   BSD, MIT, Boost Software License, GPLv2, GPLv3, LGPLv2.1, LGPLv3 -->
  <license>TODO</license>
```

Le site web du paquet peut-être indiqué dans le champ <url>. C'est l'endroit où on trouve généralement la documentation.

```
  <!-- Url tags are optional, but mutiple are allowed, one per tag -->
  <!-- Optional attribute type can be: website, bugtracker, or repository -->
  <!-- Example: -->
  <!-- <url type="website">http://wiki.ros.org/beginner_tutorials</url> -->
```

Certains paquets servent à utiliser des logiciels tierces dans le cadre de ROS. Dans ce cas le paquet ROS encapsule ce logiciel tierce. C'est dans le champ <author> que l'on peut spécifier le nom de l'auteur de ce logiciel tierce. Il est possible de mettre plusieurs champs <author>.

```
  <!-- Author tags are optional, mutiple are allowed, one per tag -->
  <!-- Authors do not have to be maintianers, but could be -->
  <!-- Example: -->
  <!-- <author email="jane.doe@example.com">Jane Doe</author> -->
```

Les dépendences sont de 6 ordres :

— build dependencies (format 1 & 2) : spécifie les paquets nécessaires pour construire ce paquet. Il peut s'agir des paquets qui contiennet des en-têtes nécessaires à la compilation.
— build export dependencies (format 2) : spécifie les paquets nécessaires pour construire des librairies avec ce paquet. Notamment lorsque les en-têtes publiques de ce paquet font référence à ces paquets.
— execution dependencies (format 1 & 2) : spécifie les paquets nécessaires pour exécuter des programmes fournis par ce paquet. C'est le cas par exemple lorsque ce paquet dépend de librairies partagées.
— test dependencies (format 1 & 2) : spécifie uniquement les dépendences additionnelles pour les tests unitaires.
— build tool dependencies (format 1 & 2) : spécifie les outils de construction système nécessaires pour le construire. Typiquement le seul outil nécessaire est catkin. Dans le cas de la compilation croisée les outils sont ceux de l'architecture sur laquelle la compilation est effectuée.
— doc dependencies (format 2) : spécifie les outils pour générer la documentation.

Pour notre exemple voici les dépendences au format 1 avec ROS melodic exprimés dans le fichier **package.xml**.

```
  <!-- The *_depend tags are used to specify dependencies -->
  <!-- Dependencies can be catkin packages or system dependencies -->
  <!-- Examples: -->
  <!-- Use build_depend for packages you need at compile time: -->
  <!--   <build_depend>message_generation</build_depend> -->
  <!-- Use buildtool_depend for build tool packages: -->
  <!--   <buildtool_depend>catkin</buildtool_depend> -->
  <!-- Use run_depend for packages you need at runtime: -->
  <!--   <run_depend>message_runtime</run_depend> -->
  <!-- Use test_depend for packages you need only for testing: -->
  <!--   <test_depend>gtest</test_depend> -->
  <buildtool_depend>catkin</buildtool_depend>
  <build_depend>roscpp</build_depend>
  <build_depend>rospy</build_depend>
  <build_depend>std_msgs</build_depend>
  <run_depend>roscpp</run_depend>
  <run_depend>rospy</run_depend>
  <run_depend>std_msgs</run_depend>
```

Lorsque l'on souhaite groupé ensemble des paquets nous avons vu qu'il était possible de créer un **metapackage**. Ceci peut-être accompli en créant un paquet qui contient dans le tag suivant dans la section export :

```
  <export>
    <metapackage />
  </export>
```

Les metapackages ne peuvent avoir des dépendences d'exécution que sur les paquets qu'il regroupe.

```
  <!-- The export tag contains other, unspecified, tags -->
  <export>
    <!-- Other tools can request additional information be placed here -->

  </export>
</package>
```

2.5 Compilation des paquets melodic

La compilation de tous les paquets s'effectue avec une seule commande. Cette commande doit être exécutée dans le répertoire du workspace. C'est cette commande qui déclenche la création des répertoires **build** et **devel**.

```
cd ~/catkin_ws/
catkin_make
catkin_make install
```

Les répertoires **build** et **devel** étant créés automatiquement ils peuvent être effacés sans problème tant que le fichier **CMakeLists.txt** est cohérent avec l'environnement de programmation. Il est indispensable de le faire si le workspace a été déplacé sur le disque.

2.6 Chaînage de workspace

Lorsque l'une des commandes **catkin_make** ou **catkin build** est utilisée celle-ci crée un fichier nommé **setup.bash**. Ce fichier une fois sourcé permet ensuite de travailler dans le workspace en initialisant correctement toutes les variables d'environnements. Un point important à noter est que ces variable d'environnements peuvent déjà avoir une valeur lors de l'appel à **catkin_make** ou **catkin build**. Cette valeur est utilisée lors de la création du fichier **setup.bash**.

Ceci permet de faire du chaînage de workspace.

Par exemple dans le chapitre 12 il faut utiliser un workspace nommé **tiago_public_ws** permettant de simuler le robot Tiago. Avant de créer le répertoire **devel** il faut s'assurer que la variable d'environnement **ROS_PACKAGE_PATH** ne contient que la valeur

```
env | grep ROS
ROS_PACKAGE_PATH=/opt/ros/melodic/share
```

Une fois que la commande :

```
catkin build
```

a été utilisée, on peut sourcer le fichier **tiago_public_ws/devel/setup.bash**

On a alors :

```
env | grep ROS
ROS_PACKAGE_PATH=/home/etudiant/tiago_public_ws/devel/share...:/opt/ros/melodic/share
```

Tous les répertoires des paquets de **tiago_public_ws** se trouvent dans la variable **ROS_PACKAGE_PATH**.

Si on souhaite faire un workspace **app_for_tiago** qui utilise **tiago_public_ws** on peut alors après avoir sourcer le fichier **setup.bash** du workspace **tiago_public_ws** faire soit **catkin_make** ou **catkin build**. On a alors :

```
env | grep ROS
ROS_PACKAGE_PATH=/home/etudiant/app_for_tiago/devel/share/...:/home/etudiant/tiago_public_ws/devel/share...:/opt/
    ros/melodic/share
```

3. Graphe d'application avec ROS-1

Dans ce chapitre nous allons introduire les concepts de base permettant la construction d'une application robotique distribuée sur plusieurs ordinateurs. Notons que les aspects temps-réel ne sont pas directement adressés ici.

3.1 Introduction aux concepts de ROS

On retrouve dans ROS un certain nombre de concepts commun à beaucoup de middleware :

— Nodes : Un node est un exécutable qui utilise ROS pour communiquer avec d'autres nodes.
— Messages : Type de données ROS utilisés pour souscrire ou publier sur un topic.
— Topics : Les nodes peuvent *publier* des messages sur un topic aussi bien que *souscrire* à un topic pour recevoir des messages.
— Master : Nom du service pour ROS (i.e. aide les noeuds à se trouver mutuellement).
— Paramètres : Informations très peu dynamiques qui doivent être partagés dans l'application.
— rosout : Equivalent de stdout/stderr.
— roscore : Master+rosout+parameter server (serveur de paramètres).

3.1.1 Définition d'un node

On peut définir un node comme étant le processus associé à l'exécution d'un fichier exécutable dans un paquet ROS. Les noeuds ROS utilisent une librairie client pour communiquer avec les autres noeuds. Les noeuds peuvent publier ou souscrire à des topics. Les noeuds peuvent fournir ou utiliser un service. Les librairies client sont *rospy* pour python et *roscpp* pour C++.

3.1.2 Le système des name services

Pour pouvoir localiser et interagir entre les nodes il faut un **annuaire**. C'est le rôle de **roscore**. Il doit être lancé *systématiquement* mais une seule fois.

```
roscore
```

Si **roscore** ne se lance pas, il faut :

— Vérifier la connection réseau.
— Si roscore ne s'initialise pas et évoque des un problème de droits sur le répertoire .ros

En général on ne lance pas **roscore** directement ou à des fins de debuggage. Le lancement d'un graphe d'application s'effectue en général par **roslaunch** qui lance lui-même **roscore**. L'utilisation de **roslaunch**

nécessite un fichier XML décrivant le graphe d'application. Son utilisation est vue plus en détails dans le paragraphe 3.3.

Pour obtenir la liste des nodes actifs on peut utiliser la commande **rosnode** :

```
rosnode list
```

Pour obtenir des informations sur un node :

```
rosnode info /rosout
```

Tutoriel associé : Tutorial Understanding ROS Nodes : `http://wiki.ros.org/ROS/Tutorials/UnderstandingNodes`

3.1.3 Lancer des nodes

On peut lancer un fichier exécutable/node d'un paquet :

```
rosrun [package_name][node_name]
```

Par exemple pour lancer le node **turtlesim** :

```
rosrun turtlesim turtlesim_node
```

Une fenêtre bleue avec une tortue apparaît comme celle affichée dans la figure 3.1. La tortue peut-être différente car elles sont tirées aléatoirement dans un ensemble de tortues. Si vous souhaitez lancer le

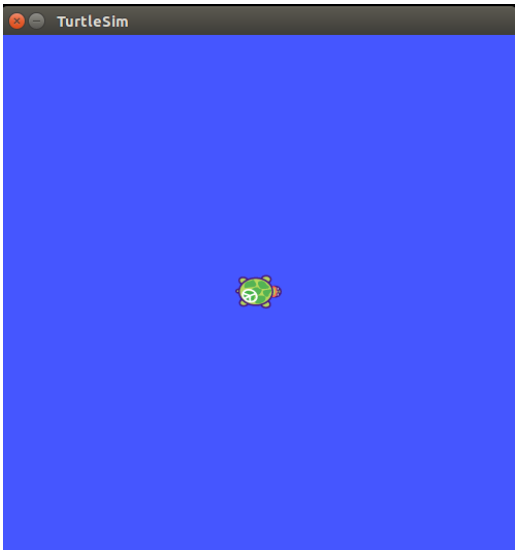


FIGURE 3.1 – Fenêtre résultat de rosrun turtlesim turtlesim_node

programme avec un nom différent il est possible de spécifier :

```
rosrun turtlesim turtlesim_node __name:=my_turtle
```

Pour vérifier si un node est actif on peut utiliser la commande **rosnode** :

```
rosnode ping my_turtle
```

Tutoriel associé : Tutorial Understanding ROS Nodes : `http://wiki.ros.org/ROS/Tutorials/UnderstandingNodes`

Afin de pouvoir commander la tortue dans la fenêtre il faut utiliser le node **turtle_teleop_key** qui permet d'envoyer des commandes avec le clavier :

```
rosrun turtlesim turtle_teleop_key
```

Pour visualiser le graphe de l'application on peut lancer la commande suivante :

```
rosrun rqt_graph rqt_graph
```

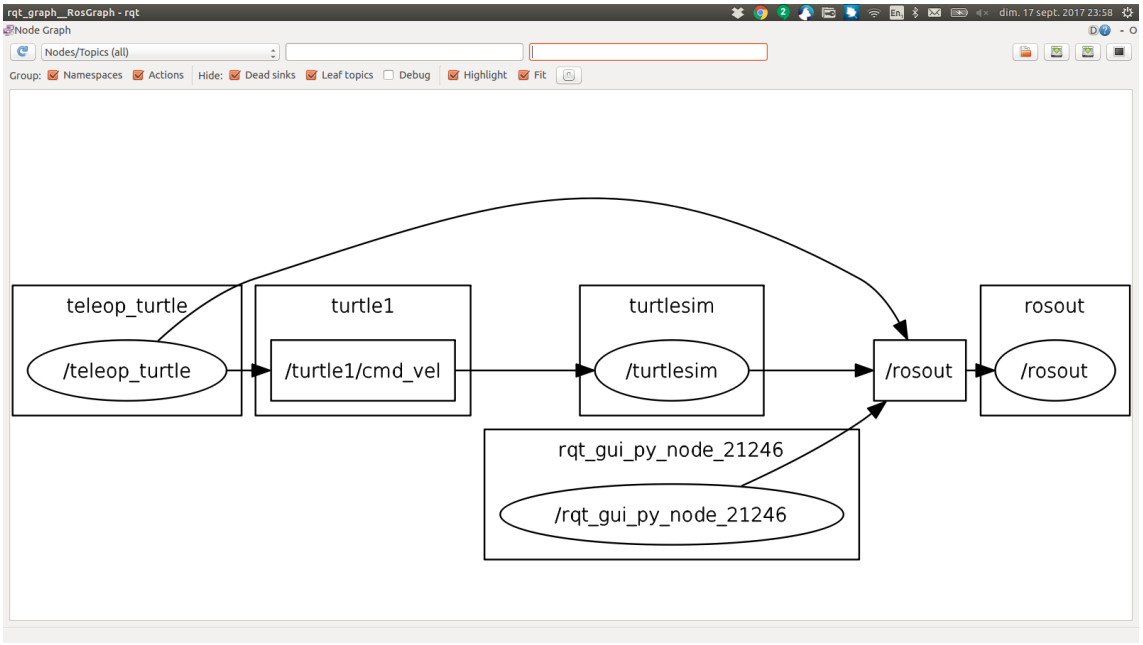


FIGURE 3.2 – Graphe de l'application **turtlesim** et **turtle_teleop_key**

Pour démarrer le graphe de l'affichage des topics :

```
rosrun rqt_plot rqt_plot
```

On obtient le graphe affiché dans la figure Fig.3.3.

3.1.4 Topic

Les topics sont des données publiées par des noeuds et auquelles les noeuds souscrivent. L'exécutable permettant d'avoir des informations sur les topics est **rostopic**. La liste des commandes en ROS-2 est similaire à celle de ROS-1 :

bw : Affiche la bande passante prise par le topic
echo : Affiche le contenu du topic en texte
hz : Affiche la fréquence de publication du topic
list : Affiche la liste des topics actifs
pub : Publie des données sur le topic
type : Affiche le type du topic

3.1.4.1 rostopic list

Cette commande affiche la liste des topics. Le résultat est le suivant :

```
/rosout
/rosout_agg
/statistics
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
```

3.1.4.2 rostopic bw

On obtient la bande passante utilisée par un topic en tapant :

```
rostopic bw /turtle1/pose
```

On obtient alors le résultat suivant :

```
average: 2.52KB/s
  mean: 0.02KB min: 0.02KB max: 0.02KB window: 100
```

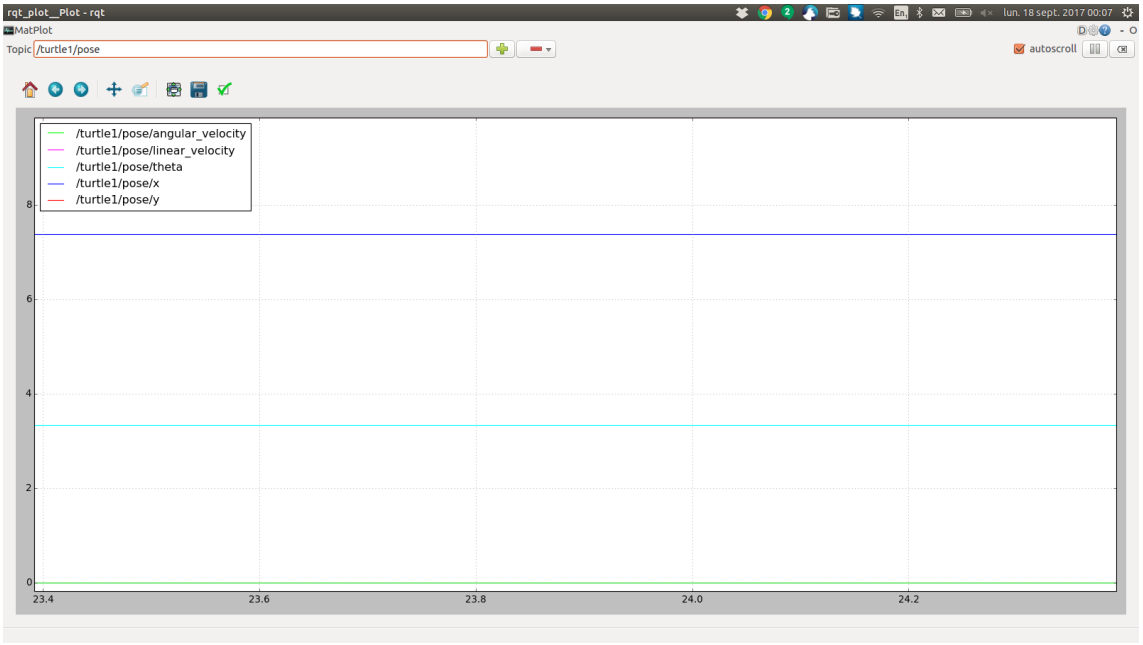


FIGURE 3.3 – Affichage de la position de la tortue dans la fenêtre

3.1.4.3 rostopic echo

Affiche le contenu d'un topic en fonction du message transmis. Par exemple :

```
rostopic echo /turtle1/pose
```

donne :

```
x: 5.544444561
y: 5.544444561
theta: 0.0
linear_velocity: 0.0
angular_velocity: 0.0
```

3.1.4.4 rostopic type

Cette commande affiche le message (structure de données au sens de ROS) d'un topic. Par exemple :

```
rostopic type /turtle1/pose
```

affiche :

```
turtlesim/Pose
```

3.1.4.5 rostopic pub

Cette commande permet de publier des données sur un topic. Elle suit la structure suivante :

```
rostopic pub /topic type
```

Par exemple si on revient à notre exemple on peut essayer :

```
rostopic pub /turtle1/cmd_vel geometry_msgs/Twist "linear:
  x: 0.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 1.0"
```

Cette commande demande à la tortue de tourner sur elle-même à la vitesse angulaire de 1 rad/s. Grâce à la complétion automatique il est possible de taper uniquement le nom du topic qui étend ensuite le type du topic et la structure de la donnée à transmettre. La durée de la prise en compte de l'information est limitée, on peut utiliser l'option **-r 1** pour répéter l'émission à une fréquence d'1 Hz.

3.2 rqt_console

rqt est une interface d'affichage non 3D qui se peuple avec des plugins. Elle permet de construire une interface de contrôle incrémentalement. L'exécutable permettant d'afficher les messages des noeuds de façon centralisé est **rqt_console**.

```
rosrun rqt_console rqt_console
rosrun rqt_logger_level rqt_logger_level
```

La deuxième commande permet de lancer le système d'affichage des messages d'alertes et d'erreur. Les deux commandes créent les deux fenêtres suivantes affichées dans Fig. 3.4. Pour illustrer cet

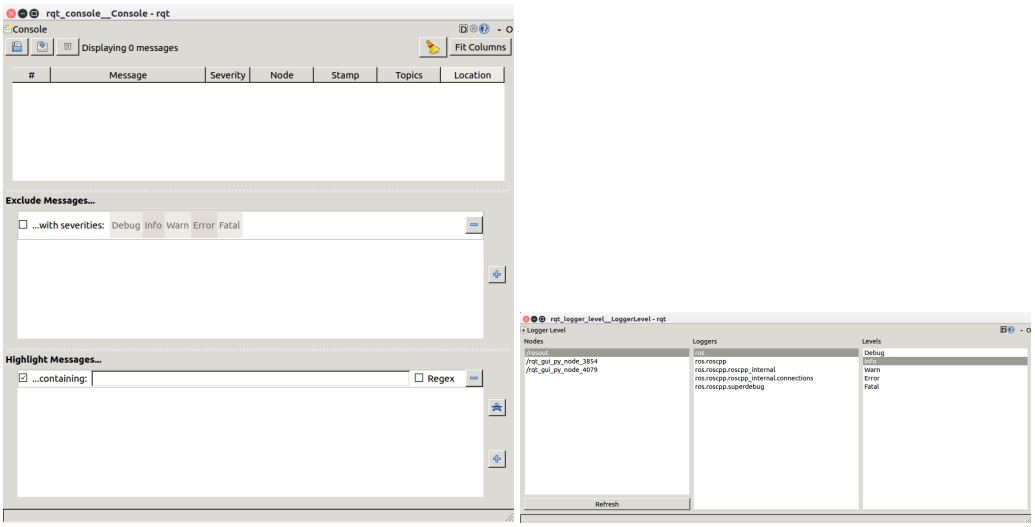


FIGURE 3.4 – Fenêtres correspondant à **rqt_console_console** et à **rqt_logger_level**

exemple on peut lancer le noeud **turtlesim** avec :

```
rosrun turtlesim turtlesim_node
```

La position de la tortue va alors s'afficher dans la console car il s'agit d'un message de niveau INFO. Il est possible de changer le niveau des messages affichés. En le mettant par exemple à WARN, et en envoyant la commande suivant à **turtlesim** :

```
rostopic pub /turtle1/cmd_vel geometry_msgs/Twist "linear:
  x: 2000.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 2000.0"
```

Elle pousse la tortue dans les coins de l'environnement et on peut voir un message affichant un warning dans la fenêtre du node **rqt_console_console** affichée dans la figure.3.5.

Tutoriel associé : Tutorial Using rqt console et roslaunch `http://wiki.ros.org/ROS/Tutorials/UsingRqtconsoleRoslaunch`;

3.3 roslaunch

roslaunch est une commande qui permet de lancer plusieurs noeuds. **roslaunch** lit un fichier xml qui contient tous les paramètres pour lancer une application distribuée ROS.

3.3.1 Exemple

Par exemple considérons le fichier launch suivant :

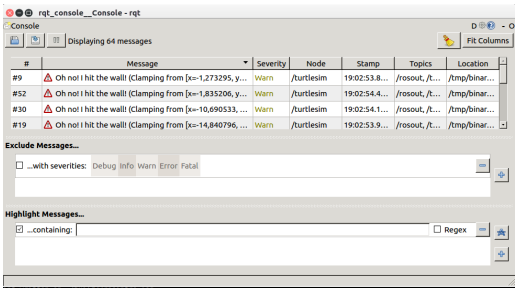


FIGURE 3.5 – Fenêtres correspondant à **rqt_console_console** lorsque la tortue s'écrase dans les murs

```
<launch>

  <group ns="turtlesim1">
    <node pkg="turtlesim" name="sim" type="turtlesim_node"/>
  </group>

  <group ns="turtlesim2">
    <node pkg="turtlesim" name="sim" type="turtlesim_node"/>
  </group>

  <node pkg="turtlesim" name="mimic" type="mimic">
    <remap from="input" to="turtlesim1/turtle1"/>
    <remap from="output" to="turtlesim2/turtle1"/>
  </node>

</launch>
```

Pour démarrer le graphe d'applications il suffit de lancer les commandes suivantes :

```
roslaunch [package] [filename.launch]
roscd beginner_tutorials
roslaunch beginner_tutorials turtlemimic.launch
```

On obtient alors deux fenêtres **turtlesim**.

Analysons maintenant le contenu du fichier **turtlemimic.launch**. La ligne suivante lance le node **sim** à partir de l'exécutable **turtlesim_node** qui se trouve dans le package **turtlesim** :

```
    <node pkg="turtlesim" name="sim" type="turtlesim_node"/>
```

Parce que cette ligne se trouve dans le bloc

```
  <group ns="turtlesim1">
  </group>
```

le node se trouve dans le namespace **turtlesim1** et se nommera :

```
/turtlesim1/sim
```

De même le deuxième groupe avec le namespace **turtlesim2** va créer un node qui se nommera :

```
/turtlesim2/sim
```

On peut le vérifier avec **rqt_graph** qui donne la figure Fig.3.6. On trouve également deux topics nommés **/turtlesim1/turtle1/pose** et **/turtlesim2/turtle1/cmd_vel**. Si on exécute :

```
rostopic list
```

On constate que l'on trouve deux ensembles de topics correspondants aux deux nodes de **turtlesim** et aux deux namespaces **turtlesim1** et **turtlesim2**.

Le bloc du fichier **turtlemimic.launch** :

```
  <node pkg="turtlesim" name="mimic" type="mimic">
    <remap from="input" to="turtlesim1/turtle1"/>
    <remap from="output" to="turtlesim2/turtle1"/>
  </node>
```

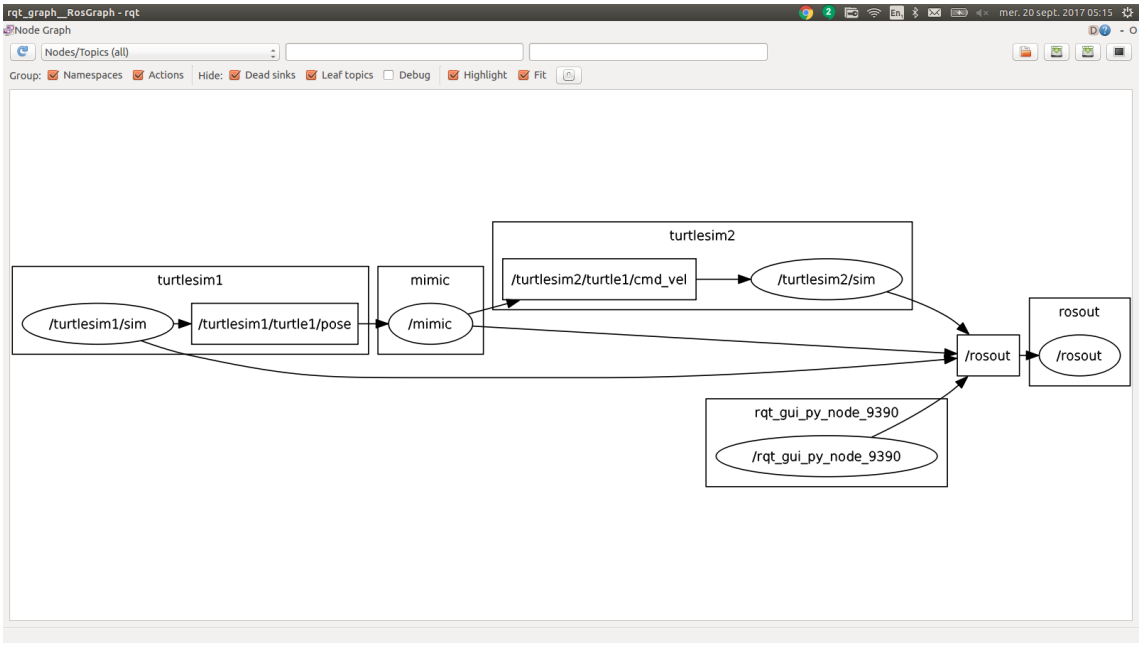


FIGURE 3.6 – Graphe correspondant au fichier turtlemimic.launch

remplit deux fonctions. La première est de créer le node **mimic** à partir de l'exécutable **mimic** dans le paquet **turtlesim**. La deuxième avec les deux lignes correspondant aux balises **remap** est de changer respectivement les namespaces **input** en **turtlesim1/turtle1**, et **output** en **turtlesim2/turtle1**. Ceci permet de pouvoir connecter deux nodes qui ne publiez pas et ne souscrivez pas sur le même topic.

Tutoriel associé : Tutorial Using rqt console et roslaunch `http://wiki.ros.org/ROS/Tutorials/UsingRqtconsoleRoslaunch`

3.3.1.1 Les fichiers XML launch

roslaunch évalue le fichier XML en une passe. Les tags includes sont traités suivant un ordre en profondeur d'abord. Les tags sont évalués en série et la derniére affectation est celle qui est utilisée.

L'utilisation de la surcharge peut être instable. Il n'y a pas de garantie que la surcharge soit garantie (i.e. si un nom de paramètre change dans un fichier inclus). Il est plutôt recommander de surcharger en utilisant les directives $(arg)/<arg>.

3.3.1.2 Substitutions

Les attributs des tags peuvent utiliser des arguments de substitutions que roslaunch va résoudre avant de lancer les nodes. Les arguments de substitutions supportés à l'heure actuelle sont :

— $(env ENVIRONMENT_VARIABLE)
Substitue la valeur d'une variable à partir de l'environnement. Le lancement échoue si la variable d'environnement n'est pas spécifiée. Cette valeur ne peut pas être surchargée par les tags <env>.

— $(optenv ENVIRONMENT_VARIABLE) $(optenv ENVIRONMENT_VARIABLE default_value)
Substitue la valeur d'une variable d'environnement s'il est spécifiée. Si le champ default_value est rempli il sera utilisé si la variable d'environnement n'est pas spécifiée. Si le champ default_value n'est pas fourni une chaîne de caractères vide sera utilisée. default_value peut-être constitué de multiples mots séparés par des espaces. Exemples :

```
<param name="foo" value="$(optenv NUM_CPUS 1)" />
<param name="foo" value="$(optenv CONFIG_PATH /home/marvin/ros_workspace)" />
<param name="foo" value="$(optenv VARIABLE ros rocks)" />
```

— $(find pkg)
i.e. $(find rospy)/manifest.xml. Spécifie un chemin relatif par rapport à un paquet. Le chemin dans le système de fichiers menant au paquet est substitué en ligne. L'utilisation de chemin relatifs à un

paquet est hautement encouragé car des chemins hard-codés empêche la portabilité du fichier launch. L'utilisation de slashes ou anti-slashes est résolu suivant la convention du système de fichiers.

— $(anon name)
i.e. $(anon rviz-1), génére un identifiant anonyme basé sur name. name lui même est un identifiant unique : des usages multiples de $(anon foo) va créer le même name anonymisé. C'est utilisé pour les attributs name des blocs <node> de façon à créer des nodes avec des noms anonymes, car ROS force les nodes à avoir des noms uniques. Par exemple :

```
<node name="$(anon foo)" pkg="rospy_tutorials" type="talker.py" />
<node name="$(anon foo)" pkg="rospy_tutorials" type="talker.py" />
```

va générer une erreur car il y a deux nodes avec le même nom.

— $(arg foo)
$(arg foo) est évalué à la valeur spécifiée par le tag <arg>. Il doit y avoir un tag correspondant à <arg> dans le même fichier de launch qui déclare arg. Par exemple :

```
<param name="foo" value="$(arg my_foo)" />
```

affecte l'argument my_foo au paramètre foo.
Voici un autre exemple :

```
<node name="add_two_ints_server" pkg="beginner_tutorials" type="add_two_ints_server" />
<node name="add_two_ints_client" pkg="beginner_tutorials" type="add_two_ints_client" args="$(arg a) $(arg b)
    " />
```

Celui-ci lance le serveur et le client de l'exemple add_two_ints et passe en paramétres les valeurs a et b. Le fichier peut être alors utilisé de la façon suivante :

```
roslaunch beginner_tutorials launch_file.launch a:=1 b:=5
```

— $(eval <expression>) A partir de Kinetic
$(eval <expression>) permet d'évaluer des expressions en python arbitrairement complexes. Par exemple :

```
<param name="circumference" value="$(eval 2.* 3.1415 * arg('radius'))"/>
```

Note : Les expressions utilisant eval doivent analyser la chaîne de caractéres compléte. Il n'est donc pas possible de mettre d'autres expressions utilisant eval dans la chaîne en argument. L'expression suivante ne fonctionnera donc pas :

```
<param name="foo" value="$(arg foo)$(eval 6*7)bar"/>
```

Pour pallier à cette limitation toutes les commandes de substitution sont disponibles comme des fonctions dans eval. Il est donc possible d'écrire :

```
"$(eval arg('foo') + env('PATH') + 'bar' + find('pkg'))"
```

A des fins de simplicité, les arguments peuvent aussi implicitement résolus. Les deux expressions suivantes sont identiques :

```
"$(eval arg('foo'))"
"$(eval foo)"
```

— $(dirname) A partir de Lunar
$(dirname) retourne le chemin absolu vers le répertoire du fichier launch où il est apparu. Ceci peut-être utilisé en conjonction avec eval et if/unless pour modifier des comportements basés sur le chemin d'installation ou simplement comme une façon de faire référence à des fichiers launch ou yaml relatifs au fichier courant plutôt qu'à la racine d'un paquet (comme avec $(find PKG)). Par exemple :

```
<include file="$(dirname)/other.launch" />
```

va chercher le fichier `other.launch` dans le même répertoire que le fichier launch dans lequel il est apparu.

Les arguments de substitution sont résolus sur la machine locale. Ce qui signifie que les variables d'environnement et les chemins des paquets ROS auront leurs valeurs déduites à partir de la machine locales même pour des processus lancés sur des machines distantes.

— if=value (optionel)
Si `value` est à vrai la balise et son contenu sont inclus.
— unless=value (optionel)
A moins que `value` soit à vrai la balise et son contenu sont inclus.

Par exemple :

```
<group if="$(arg foo)">
  <!-- Partie qui ne sera incluse que si foo est vrai -->
</group>

<param name="foo" value="bar" unless="$(arg foo)" />  <!-- Ce param\'etre ne sera pas affect\'e tant que "unless"
     est vrai -->
```

3.3.1.3 Les attributs if et unless

Toutes les balises comprennent les attributs `if` et `unless` qui inclus ou exclus une balise sur l'évaluation d'une valeur. "1" et "true" sont considérées comme des valeurs vraies, "0" et "false" sont considérées comme des valeurs fausses. D'autres valeurs sont considérées comme une erreur.

3.4 rosbag

Il est possible d'enregistrer et de rejouer des flux de données transmis par les topics. Cela s'effectue en utilisant la commande **rosbag**. Par exemple la ligne de commande suivante enregistre toutes les données qui passe par les topics à partir du moment où la commande est active (et si des évènements ou données sont génèrés sur les topics).

```
rosbag record -a
```

On peut par exemple lancer le node **turtlesim** et le noeud **teleop_key** pour envoyer des commandes.

Par défaut le nom du fichier **rosbag** est constitué de l'année, du mois et du jour et post fixé par **.bag**. Il est possible ensuite de n'enregistrer que certains topics, et de spécifier un nom de fichier. Par exemple la ligne de commande suivante n'enregistre que les topics **/turtle1/cmd_vel** et **/turtle1/pose** dans le fichier **subset.bag**.

```
rosbag record -O subset /turtle1/cmd_vel /turtle1/pose
```

3.5 rosservice

Les services sont une autre façon pour les noeuds de communiquer entre eux. Les services permettent aux noeuds d'envoyer des *requêtes* et et de recevoir des *réponses*. Dans la suite on supposera que le noeud **turtlesim** continue à fonctionner. Il va servir à illustrer les commandes vues dans ce paragraphe.

La commande **rosservice** permet d'interagir facilement avec le système client/serveur de ROS appelés *services*. **rosservice** a plusieurs commandes qui peuvent être utilisées sur les services comme le montre l'aide en ligne :

```
rosservice args print service arguments
rosservice call call the service with the provided args
rosservice find find services by service type
rosservice info print information about service
rosservice list list active services
rosservice type print service type
rosservice uri  print service ROSRPC uri
```

3.5.1 rosservice list

La commande

```
rosservice list
```

affiche la liste des services fournis par les nodes de l'application.

```
/clear
/kill
/reset
/rosout/get_loggers
```

```
/rosout/set_logger_level
/spawn
/turtle1/set_pen
/turtle1/teleport_absolute
/turtle1/teleport_relative
/turtlesim/get_loggers
/turtlesim/set_logger_level
```

On y trouve 9 services fournis par le node **turtlesim** : reset, clear, spawn, kill, turtle1/set_pen, /turtle1/teleport_absolute, /turtle1/teleport_relative, turtlesim/get_loggers, et turtlesim/set_logger_level. Il y a deux services relatifs au node **rosout** : /rosout/get_loggers and /rosout/set_logger_level.

3.5.2 rosservice type (service)

Il est possible d'avoir le type de service fourni en utilisant la commande :

```
rosservice type [service]
```

Ce qui donne pour le service clear :

```
rosservice type /clear
```

Le résultat est le suivant :

```
std_srvs/Empty
```

Ce service est donc vide car il n'y a pas d'arguments d'entrée et de sortie (i.e. aucune donnée n'est envoyée lorqu'une requête est effectuée et aucune donnée n'est transmise lors de la réponse).

3.5.3 rosservice call

Il est possible de faire une requête en utilisant la commande :

```
rosservice call [service] [args]
```

Ici aucun argument est nécessaire car le service est vide :

```
rosservice call /clear
```

Comme on peut s'y attendre le fond de la fenêtre du noeud **turtlesim** est effacé. Considérons maintenant un cas où le service a des arguments en examinant le service **spawn** :

```
rosservice type /spawn
```

Le résultat est la structure de la requête et de la réponse.

```
float32 x
float32 y
float32 theta
string name
---
string name
```

La requête est donc constituée de trois réels qui sont respectivement la position et l'orientation de la tortue. Le quatrième argument est le nom du node et est optionnel. Ce service nous permet de lancer une autre tortue, par exemple :

```
rosservice call /spawn 2 2 0.2 ""
```

On trouve maintenant une deuxième tortue dans la fenêtre.

3.6 rosparam

ROS embarque un serveur de paramètres qui permet de stocker des données et de les partager sur toute l'application. Les mécanismes de synchronisation des paramètres est complètement différent des topics qui implémente un système de flux de données. Ils servent notamment à stocker : le modèle du robot (**robot_description**), des trajectoires de mouvements, des gains, et toutes informations pertinentes. Les paramètres peuvent se charger et se sauvegarder grâce au format YAML (Yet Another Language). Ils peuvent également être définis en ligne de commande. Le serveur de paramètre peut stocker des entiers, des réels, des booléens, des dictionnaires et des listes. rosparam utilise le langage YAML pour la syntaxe de ces paramètres. Dans des cas simples, YAML paraît trés naturel.

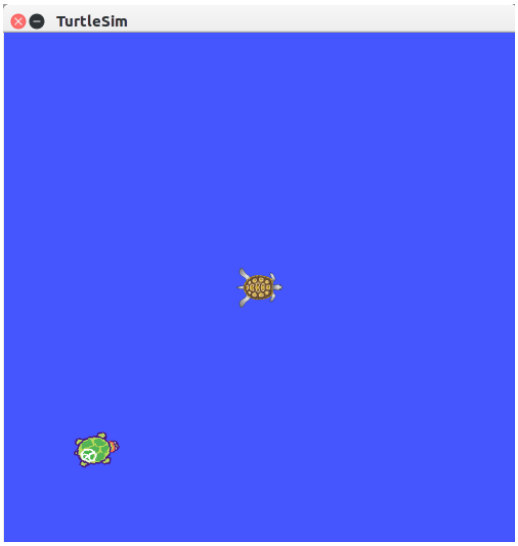


FIGURE 3.7 – Fenêtre résultat de rosrun call /spawn 2 2 0.2 ""

```
  rosparam set  set parameter
  rosparam get  get parameter
  rosparam load load parameters from file
  rosparam dump dump parameters to file
  rosparam delete delete parameter
  rosparam list list parameter names
```

3.6.1 rosparam list

Il est possible d'avoir la liste des paramètres en tapant la commande :

```
rosparam list
```

On obtient alors la liste suivante :

```
/background_b
/background_g
/background_r
/rosdistro
/roslaunch/uris/host_kinokoyama__38903
/rosversion
/run_id
```

Les 3 premiers paramètres permettent de contrôler la couleur du fond de la fenêtre du node **turtlesim**. La commande suivante change la valeur du canal rouge de la couleur de fond :

```
rosparam set /background_r 150
```

Pour que la couleur soit prise en compte il faut appeler le service **clear** de la façon suivante :

```
rosservice call /clear
```

Le résultat est représenté dans la figure 3.8.

On peut voir la valeur des autres paramètres dans le serveur de paramètres. Pour avoir la valeur du canal vert de la couleur de fond on peut utiliser :

```
rosparam get /background_g
```

On peut aussi utiliser la commande suivante pour voir toutes les valeurs du serveur de paramètres :

```
rosparam get /
```

Le résultat est le suivant :

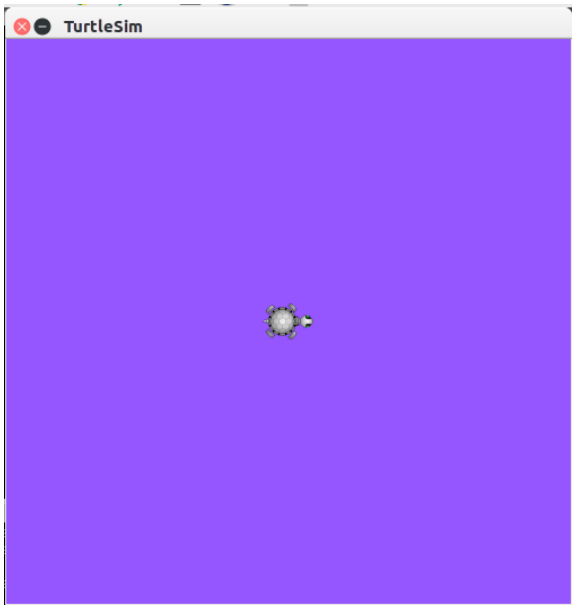


FIGURE 3.8 – Résultat de l'appel **rosparam set /background_r 150**

```
background_b: 255
background_g: 86
background_r: 150
rosdistro: 'kinetic

  '
roslaunch:
  uris: {host_kinokoyama__41913: 'http://kinokoyama:41913/'}
rosversion: '1.12.7

  '
run_id: 5250890c-ab42-11e7-9f88-104a7dbc5f2f
```

Il est possible de stocker ces informations dans un fichier qu'il est ensuite possible de recharger.

3.6.2 rosparam dump et rosparam load

La structure générale de la command est :

```
rosparam dump [file_name] [namespace]
rosparam load [file_name] [namespace]
```

Il est possible d'enregistrer tout les paramètres dans le fichier **params.yaml**

```
rosparam dump params.yaml
```

Il est ensuite possible de charger les fichiers dans de nouveaux espaces de noms par exemple copy :

```
rosparam load params.yaml copy
rosparam get /copy/background_b
255
```

3.7 Création de messages et de services

3.7.1 Introduction

Les messages sont définis par des structures appellées **msg**, tandis que les services le sont par des les structures **srv**.

— **msg** : msg files sont de simples fichiers textes qui donnent les champs d'un message ROS. Ils sont utilisés pour génèrer le code source des messages dans des languages différents.

— **srv** : un fichier srv décrit un service. Il est composé de deux parties : une requête et une réponse.

Les fichiers msg sont stockés dans le répertoire msg d'un paquet, et les fichiers srv sont stockés dans le répertoire srv.

Les fichiers msg sont de simples fichiers textes avec un type de champ et un nom de champ par ligne. Les types de champs possibles sont :

— int8, int16, int32, int64 (plus uint *)
— float32, float64
— string
— time, duration
— other msg files
— des tableaux à taille variables ([])
— des tableaux à taille fixes ([C], avec C la taille du tableau)

Il existe un type spécial dans ROS : **Header**, l'entête contient une trace temporelle (horodatage) et des informations sur les repères de réfèrence qui sont utilisés le plus couramment en ROS. Il est fréquent que la première ligne d'un message soit **Header header**.

Voici un exemple qui utilise un entête, une chaïne de caractères, et deux autres messages :

```
Header header
string child_frame_id
geometry_msgs/PoseWithCovariance pose
geometry_msgs/TwistWithCovariance twist
```

srv files sont comme des fichiers msg, à part qu'ils contiennet deux parties : une requête et une réponse. Les deux parties sont séparées par une ligne '—' Voici un exemple de fichier srv :

```
int64 A
int64 B
---
int64 Sum
```

3.7.2 Création d'un msg

3.7.2.1 Format du fichier

Créons maintenant un msg dans le paquet **beginnner_tutorials**

```
roscd beginner_tutorials
mkdir msg
echo "int64 num" > msg/Num.msg
```

Le message précèdent ne contient qu'une ligne. Il est possible de faire des fichiers plus compliqués en ajoutant plusieurs élèments (un par ligne) comme ceci :

```
string first_name
string last_name
uint8 age
uint32 score
```

3.7.2.2 Génération du code source

Afin de générer les codes sources (C++, Python) correspondant au fichier il est nécessaire d'effectuer un certain nombre de modifications sur les fichiers package.xml et CMakeLists.txt.

Dans le fichier package.xml il faut que les deux lignes suivantes sont décommentées :

```
<build_depend>message_generation</build_depend>
<run_depend>message_runtime</run_depend>
```

Durant la compilation du paquet seule la ligne build_depend (message_generation) est nécessaire, tandis qu'à l'execution du paquet c'est la ligne run_depend (message_runtime).

Il faut ensuite ouvrir le fichier CMakeLists.txt avec votre éditeur favori.

Il faut ajouter la dépendance au paquet message_generation à l'appel de find_package qui existe déjà dans le fichier de façon à pouvoir générer des messages. Il est possible de faire cela en ajoutant message_generation à la liste COMPONENTS de la façon suivante :

```
  # Do not just add this to your CMakeLists.txt, modify the existing text to add message_generation before the
      closing parenthesis
find_package(catkin REQUIRED COMPONENTS
   roscpp
   rospy
```

```
  std_msgs
  message_generation
)
```

Il arrive parfois que le paquet puisse être construit sans mettre toutes les dépendances dans l'appel à find_package. Ceci est du au fait que lorsqu'on combine plusieurs paquets ensemble, si un paquet appelle find_package, celui-ci considéré est configué avec les mêmes valeurs. Mais ne pas faire cet appel empêche la compilation du paquet lorsque celui est isolé.

Il faut également exporté la dépendence à message_runtime.

```
catkin_package(
...
CATKIN_DEPENDS message_runtime ...
...)
```

Il faut ensuite indiquer au fichier CMakeLists.txt quel fichier contient le nouveau message que le système doit traiter. Pour cela il faut modifier le bloc suivant :

```
# add_message_files(
#   FILES
#   Message1.msg
#   Message2.msg
# )
```

en

```
add_message_files(
  FILES
  Num.msg
)
```

En ajoutant les fichiers msg à la main dans le fichier CMakeLists.txt, cmake sait lorsqu'il doit reconfigurer le projet lorsque d'autres fichiers msg sont ajoutés.

Il faut ensuite effectuer les étapes décries dans le paragraphe 3.7.4.

3.7.2.3 Utilisation de rosmsg

On peut vérifier que ROS est capable de voir le fichier Num.msg en utilisant la commande rosmsg. L'utilisation de cette commande de façon générale est :

```
rosmsg show [type_message]
```

Par exemple :

```
rosmsg show [type_message]
```

On voit alors :

```
int64 num
```

Dans l'exemple précédent le type de message a deux parties :

- beginner_tutorials : le nom du paquet dans lequel le type est défini.
- Num : Le nom du msg Num

Si vous ne vous souvenez pas du paquet dans lequel le msg est défini, il est possible d'utiliser :

```
[beginner_tutorials/Num]:
int64 num
```

3.7.3 Création d'un srv

3.7.3.1 Format du fichier

Les fichiers srv se trouvent par convention dans le répertoire srv d'un paquet :

```
roscd beginner_tutorials
mkdir srv
```

Au lieu de créer à la main un fichier srv nous allons utiliser celui d'un paquet déjà existant : Pour cela l'utilisation de roscp est trés pratique pour copier un fichier d'un paquet :

```
roscp rospy_tutorials AddTwoInts.srv srv/AddTwoInts.srv
```

Le fichier *AddTwoInts.srv* a été copié du paquet *rospy_tutorials* vers le paquet *beginner_tutorials* et plus précisément dans le répertoire *srv*.

Il reste encore une étape qui comme précédemment consiste à transformer le fichier *srv* en fichier C++, Python et autres languages.

A moins de l'avoir fait dans l'étape précédente, il faut s'assurer que dans le fichier *package.xml* les deux lignes suivant aient été décommentées :

```
  <build_depend>message_generation</build_depend>
  <run_depend>message_runtime</run_depend>
```

Comme précédemment le système a besoin du paquet *message_generation* pendant la phase de compilation, et du paquet *message_runtime* pendant la phase d'execution.

De même si cela n'a pas été fait précédemment dans l'étape précédente, il faut ajouter la dépendence au paquet *message_generation* dans le fichier *CMakeLists.txt*.

```
# Do not just add this line to your CMakeLists.txt, modify the existing line
find_package(catkin REQUIRED COMPONENTS
  roscpp
  rospy
  std_msgs
  message_generation
  )
```

Malgré son nom *message_generation* fonctionne à la fois pour msg et srv.

Les mêmes changements que pour l'étape précédente doivent être faits pour le fichier *package.xml*.

Il faut ensuite enlever les caractères # pour décommenter les lignes suivantes :

```
# add_service_files(
#   FILES
#   Service1.srv
#   Service2.srv
# )
```

Il faut ensuite remplacer les fichiers finissant par srv par les fichiers srv présents *AddTwoInts.srv* en l'occurence :

```
add_service_files(
  FILES
  AddTwoInts.srv
)
```

Il est maintenant possible de générer les fichiers liés aux srv. On peut aller directement au paragraphe 3.7.4.

3.7.3.2

3.7.4 Modifications CMakeLists.txt communes à msg et srv

Si le bloc suivant est toujours commenté :

```
# generate_messages(
#   DEPENDENCIES
# #  std_msgs  # Or other packages containing msgs
# )
```

Il faut décommenter les lignes et ajouter tout paquets nécessaires aux fichiers .msg par exemple dans notre cas, il faut ajouter std_msgs qui donne :

```
generate_messages(
  DEPENDENCIES
  std_msgs
)
```

Une fois que les nouveaux messages ou services ont été ajoutés il faut donc :

```
# In your catkin workspace
$ roscd beginner_tutorials
$ cd ../..
$ catkin_make install
$ cd -
```

Chaque fichier msg dans le répertoire msg va être traduit sous forme de fichier source dans tous les languages supportés. Les fichiers d'entête C++ seront générés dans le répertoire /catkin_ws/devel/include/beginner_tutorials.

Le script python sera créé dans /catkin_ws/devel/lib/python2.7/dist-packages/beginner_tutorials/msg. The lisp file lui apparaît dans : /catkin_ws/devel/share/common-lisp/ros/beginner_tutorials/msg/.

De la même façon les fichiers .srv dans le répertoire srv seront générés dans les langages supportés. Pour le C++, les fichiers d'entête seront générés dans le même répertoire que pour les fichiers .msg.

4. Ecrire des nodes ROS-1

Dans ce chapitre nous allons examiner l'écriture en C++ et en python des *nodes* implémentant les concepts vus dans le chapitre précédent. Les *nodes* est le mot dans la communauté ROS pour désigner un executable connecté au middleware ROS. Nous allons commencer par les *topics* puis examiner l'utilisation des *services*.

4.1 Topics

Dans ce paragraphe nous allons examiner l'écriture d'un node qui émet constamment un message et d'un autre node qui reçoit le message.

4.1.1 Emetteur

Dans l'émetteur nous devons faire les choses suivantes :

1. Initialiser le système ROS.
2. Avertir que le node va publier des messages *std_msgs/String* sur le topic "chatter".
3. Boucler sur la publication de messages sur "chatter" 10 fois par seconde.

4.1.1.1 C++

Il faut tout d'abord créer un répertoire src dans le paquet **beginner_tutorials** :

```
mkdir src
```

Ce répertoire va contenir tous les fichiers sources C++ du paquet.

Il faut ensuite créer le fichier **talker.cpp** et copier le code suivant :

```
#include "ros/ros.h"
#include "std_msgs/String.h"

#include <sstream>

/**
 * This tutorial demonstrates simple sending of messages over the ROS system.
 */
int main(int argc, char **argv)
{
  /**
   * The ros::init() function needs to see argc and argv so that it can perform
   * any ROS arguments and name remapping that were provided at the command line.
   * For programmatic remappings you can use a different version of init() which takes
   * remappings directly, but for most command-line programs, passing argc and argv is
```

```
   * the easiest way to do it.  The third argument to init() is the name of the node.
   *
   * You must call one of the versions of ros::init() before using any other
   * part of the ROS system.
   */
  ros::init(argc, argv, "talker");

  /**
   * NodeHandle is the main access point to communications with the ROS system.
   * The first NodeHandle constructed will fully initialize this node, and the last
   * NodeHandle destructed will close down the node.
   */
  ros::NodeHandle n;

  /**
   * The advertise() function is how you tell ROS that you want to
   * publish on a given topic name. This invokes a call to the ROS
   * master node, which keeps a registry of who is publishing and who
   * is subscribing. After this advertise() call is made, the master
   * node will notify anyone who is trying to subscribe to this topic name,
   * and they will in turn negotiate a peer-to-peer connection with this
   * node.  advertise() returns a Publisher object which allows you to
   * publish messages on that topic through a call to publish().  Once
   * all copies of the returned Publisher object are destroyed, the topic
   * will be automatically unadvertised.
   *
   * The second parameter to advertise() is the size of the message queue
   * used for publishing messages.  If messages are published more quickly
   * than we can send them, the number here specifies how many messages to
   * buffer up before throwing some away.
   */
  ros::Publisher chatter_pub = n.advertise<std_msgs::String>("chatter", 1000);

  ros::Rate loop_rate(10);

  /**
   * A count of how many messages we have sent. This is used to create
   * a unique string for each message.
   */
  int count = 0;
  while (ros::ok())
  {
    /**
     * This is a message object. You stuff it with data, and then publish it.
     */
    std_msgs::String msg;

    std::stringstream ss;
    ss << "hello world " << count;
    msg.data = ss.str();

    ROS_INFO("%s", msg.data.c_str());

    /**
     * The publish() function is how you send messages. The parameter
     * is the message object. The type of this object must agree with the type
     * given as a template parameter to the advertise<>() call, as was done
     * in the constructor above.
     */
    chatter_pub.publish(msg);

    ros::spinOnce();

    loop_rate.sleep();
    ++count;
  }


  return 0;
}
```

L'entête ros.h

```
#include "ros/ros.h"
```

permet d'inclurer toutes les en-têtes ROS nécessaires d'une manière efficace et compact.

L'entête "std_msgs/String.h" :

```
#include "std_msgs/String.h"
```

permet d'accéder à la déclaration C++ des messages std_msgs/String. Cette entête est générée automatiquement à partir du fichier **std_msgs/String.msg** qui se trouve dans le paquet std_msgs.

```
21  ros::init(argc, argv, "talker");
```

initialise ROS. Ceci permet à ROS de faire des changements de noms à travers la ligne de commande. Cette fonctionnalité n'est pas utilisé ici, mais c'est ici que l'on spécifie le nom du node, et pour cet exemple le nom est *talker*. Il ne faut oublier que les noms sont uniques dans le graphe en cours d'exécution.

Le nom doit être un nom de base sans /.

```
28  ros::NodeHandle n;
```

créer un lien entre ROS et le node. Le premier objet NodeHandle instancié s'occupe de l'initialisation du node. Le dernier détruit s'occupe de nettoyer toutes les ressources que le node a utilisé.

```
47  ros::Publisher chatter_pub = n.advertise<std_msgs::String>("chatter", 1000);
```

Cette ligne dit au master (instancié par rosmaster) que le node va publier des messages de type std_msgs/String sur le topic "chatter". Cela permet de dire à tous les noeuds à l'écoute du topic que des données vont être publiées. Le second argument est la taille de la file d'attente au cas où le node publie plus données qu'elles ne peuvent être transmises. Après 1000 images, les données les plus anciennes sont abandonnées. La méthode `NodeHandle ::advertise()` renvoit un objet de type `ros ::Publisher`. Cet objet a deux fonctions : la première est de fournir une méthode `publish()` pour publier des données, et la deuxième est d'arrêter la publication des données lors que l'objet n'est plus dans le champ d'exécution. Note : Il faut être particulièrement attentif à la portée des variables notamment dans les classes.

```
49  ros::Rate loop_rate(10);
```

Un objet `ros ::Rate` permet de spécifier une fréquence à laquelle on souhaite créer une boucle. L'objet chronomètre le temps entre deux appels à `Rate ::sleep()` et faire dormir le thread courant le temps nécessaire pour réaliser la boucle (sauf si le retour sur `Rate ::sleep()` n'est pas assez rapide).

Dans le cas présent, le thread est réveillé à 10 Hz.

```
55  int count = 0;
56  while (ros::ok())
57  {
```

Par défaut roscpp établit une redirection des signaux d'interruptions comme comme SIGINT (Ctrl-C). Si ce signal est envoyé alors `ros ::ok()` retourne `false`.

Plus précisément `ros ::ok()` renvoit `false` si :

— un signal SIGINT est reçu (Ctrl-C).
— un autre node avec le même nom a sorti le node du graphe d'application.
— `ros ::shutdown()` a été appellé par une autre partie de l'application.
— tous les objets `ros ::NodeHandles()` ont été détruits.

Une fois que `ros ::ok()` retourne `false` tous les appels à ROS vont échouer.

```
61    std_msgs::String msg;
62
63    std::stringstream ss;
64    ss << "hello world " << count;
65    msg.data = ss.str();
```

Le node émet un message sur ROS en utilisant une classe adaptée aux messages, générée à partir d'un fichier *msg*. Des types de données plus compliqués sont possibles, mais pour l'instant l'exemple utilise un message de type "String" qui a un seul membre "data". Les lignes précédentes ne servent qu'à construire le message.

```
75  chatter_pub.publish(msg);
```

Cette ligne demande l'émission du message à tous les nodes connectés.

```
67  ROS_INFO("%s", msg.data.c_str());
```

Le contenu du message est affiché sur la console ROS du graphe d'application.

```
77    ros::spinOnce();
```

Appelé `ros ::spinOnce()` n'est pas nécessaire ici, car aucune information ne nécessitant un traitement n'est attendue. Cependant si une souscription à un topic est ajoutée, l'appel à `ros ::spinOnce()` est nécessaire pour les méthodes de callbacks soient appelées. Cet appel est donc rajouté pour éviter les oublis.

```
79     loop_rate.sleep();
```

C'est l'appel qui permet à l'objet `ros ::Rate` de suspendre le thread courant afin d'atteindre la fréquence d'émission à 10 Hz.

4.1.1.2 Python

Pour le code python, tous les fichiers sont mis dans le répertoire **scripts** :

```
1  mkdir scripts
2  cd scripts
```

Il est possible de télécharger le script exemple **talker.py** dans le répertoire `scripts` et de le rendre executable :

```
1 wget https://raw.github.com/ros/ros_tutorials/kinetic-devel/rospy_tutorials/001_talker_listener/talker.py
2 chmod +x talker.py
```

```
1  #!/usr/bin/env python
2  # license removed for brevity
3  import rospy
4  from std_msgs.msg import String
5
6  def talker():
7      pub = rospy.Publisher('chatter', String, queue_size=10)
8      rospy.init_node('talker', anonymous=True)
9      rate = rospy.Rate(10) # 10hz
10     while not rospy.is_shutdown():
11         hello_str = "hello world %s" % rospy.get_time()
12         rospy.loginfo(hello_str)
13         pub.publish(hello_str)
14         rate.sleep()
15
16 if __name__ == '__main__':
17     try:
18         talker()
19     except rospy.ROSInterruptException:
20     pass
```

```
1 #!/usr/bin/env python
```

Cette ligne permet d'exécuter le script directement sans avoir à l'appeller par python.

```
3 import rospy
4 from std_msgs.msg import String
```

Importer le module rospy est nécessaire si on écrit un node ROS. L'import du module permet d'utiliser les messages de type String.

```
7 pub = rospy.Publisher('chatter', String, queue_size=10)
8 rospy.init_node('talker', anonymous=True)
```

Cette section définit l'interface talker au reste de ROS. `pub = rospy.Publisher("chatter", String, queue_size=10)` déclare que votre node publie sur le topic `chatter` en utilisant le message `String`. `String` est la classe `std_msgs.msg.String`. La variable `queue_size` limite le nombre de messages dans la file d'attente si le souscripteur ne les traite pas assez vite.

La ligne suivante `rospy.init_node(NAME,...)` est trés importante car elle donne à `rospy` le nom du node. Jusqu'à ce que `rospy` ait cette information le node ne peut pas communiquer avec ROS Master. Dans ce cas, le node va avoir le nom `talker`. NOTE : le nom doit avoir une nom de base, il ne peut pas contenir de slashes "/". `anonymous=True` assure donc que votre node a un nom unique en ajoutant des nombres aléatoires à la fin de `NAME`. Le lien suivant Initialization and Shutdown - Initializing your ROS Node fournit plus d'informations pour l'initialisation des nodes.

```
9 rate = rospy.Rate(10) # 10hz
```

Cette ligne crée un objet `rate` de type `Rate`. Avec la méthode `sleep()`, il est possible de faire des boucles à la fréquence désirée. Avec un argument de 10, il est prévu que la boucle se fasse 10 fois par seconde. Il faut cependant que le temps de traitement soit inférieur à un dixième de seconde.

```
    while not rospy.is_shutdown():
        hello_str = "hello world %s" % rospy.get_time()
        rospy.loginfo(hello_str)
        pub.publish(hello_str)
        rate.sleep()
```

Cette boucle représente une utilisation standard de rospy : vérifier que le drapeau `rospy.is_shutdown()` et ensuite faire le travail. On doit vérifier que `is_shutdown()` pour vérifier que le programme doit sortir (par exemple s'il y a Ctrl-C ou autre). Dans ce cas, le travail est d'appeller `pub.publish(hello_str)` qui publie une chaîne de caractères sur le topic `chatter`. La boucle appelle `rate.sleep()` qui s'endort suffisamment longtemps pour maintenir la fréquence de la boucle.

Cette boucle appelle aussi `rospy.loginfo(str)` qui effectue trois tâches : les messages sont impriés sur l'écran, c'est écrit dans le fichier de log de Node, et c'est écrit dans `rosout`. `rosout` est utile pour le déboggage : il est possible d'analyser des messages en utilisant `rqt_console` au lieu de trouver la fenêtre d'affichage du node. `std_msgs.msg.String` est un message très simple, pour les types plus compliqués les arguments des constructeurs sont du même ordre que les fichiers msg. Il est également possible de ne pas passer des arguments et de passer des arguments directement.

```
msg = String()
msg.data = str
```

ou il est possible d'initialiser certains des champs et laisse le reste avec des valeurs par défaut :

```
String(data=str)
```

```
    try:
        talker()
    except rospy.ROSInterruptException:
        pass
```

En plus de la vérification standard de `__main__`, ce code attrape une exception `rospy.ROSInterruptException` qui peut être lancée par les méthodes `rospy.sleep()` et `rospy.Rate.sleep()` quand Ctrl-C est utilisé ou quand le node est arrêté. La raison pour laquelle cette exception est levée consiste à éviter de continuer à exécuter du code aprés l'appel à la méthode `sleep()`.

4.1.2 Souscripteur

Les étapes du souscripteur peuvent se résumer de la façon suivante :

— Initialiser le système ROS
— Souscrire au topic chatter
— Spin, attendre que les messages arrivent
— Quand le message arrive la fonction `chatterCallback()` est appellée.

4.1.2.1 C++

Pour le souscripteur il suffit de créer le fichier **./src/listener.cpp** dans le paquet **beginner_tutorials** et copier le code suivant : https ://raw.github.com/ros/ros_tutorials/kinetic-devel/roscpp_tutorials/listener/listener.cpp

```
#include "ros/ros.h"
#include "std_msgs/String.h"

/**
 * This tutorial demonstrates simple receipt of messages over the ROS system.
 */
void chatterCallback(const std_msgs::String::ConstPtr& msg)
{
  ROS_INFO("I heard: [%s]", msg->data.c_str());
}

int main(int argc, char **argv)
{
  /**
```

```
   * The ros::init() function needs to see argc and argv so that it can perform
   * any ROS arguments and name remapping that were provided at the command line.
   * For programmatic remappings you can use a different version of init() which takes
   * remappings directly, but for most command-line programs, passing argc and argv is
   * the easiest way to do it.  The third argument to init() is the name of the node.
   *
   * You must call one of the versions of ros::init() before using any other
   * part of the ROS system.
   */
  ros::init(argc, argv, "listener");

  /**
   * NodeHandle is the main access point to communications with the ROS system.
   * The first NodeHandle constructed will fully initialize this node, and the last
   * NodeHandle destructed will close down the node.
   */
  ros::NodeHandle n;

  /**
   * The subscribe() call is how you tell ROS that you want to receive messages
   * on a given topic.  This invokes a call to the ROS
   * master node, which keeps a registry of who is publishing and who
   * is subscribing.  Messages are passed to a callback function, here
   * called chatterCallback.  subscribe() returns a Subscriber object that you
   * must hold on to until you want to unsubscribe.  When all copies of the Subscriber
   * object go out of scope, this callback will automatically be unsubscribed from
   * this topic.
   *
   * The second parameter to the subscribe() function is the size of the message
   * queue.  If messages are arriving faster than they are being processed, this
   * is the number of messages that will be buffered up before beginning to throw
   * away the oldest ones.
   */
   ros::Subscriber sub = n.subscribe("chatter", 1000, chatterCallback,
                                     ros::TransportHints().tcpNoDelay() );

  /**
   * ros::spin() will enter a loop, pumping callbacks.  With this version, all
   * callbacks will be called from within this thread (the main one).  ros::spin()
   * will exit when Ctrl-C is pressed, or the node is shutdown by the master.
   */
  ros::spin();

  return 0;
}
```

En considérant uniquement les parties différentes avec l'émetteur, on trouve les lignes suivantes :

```
void chatterCallback(const std_msgs::String::ConstPtr& msg)
{
  ROS_INFO("I heard: [%s]", msg->data.c_str());
}
```

C'est la fonction qui sera appellée quand un nouveau message arrive sur le topic *chatter*. Le message est passé dans un `boost ::shared_ptr`, ce qui signifie qu'il est possible de stocker le message sans se préoccuper de l'effacer et sans copier les données.

```
  ros::Subscriber sub = n.subscribe("chatter", 1000, chatterCallback,
                                    ros::TransportHints().tcpNoDelay());
```

Cet appel souscrit au topic *chatter* auprès du master. ROS appelle la fonction `chatterCallback()` à chaque fois qu'un nouveau message arrive. Le second argument est la taille de la file d'attente. Si les messages ne sont pas traités suffisamment rapidement, le système stocke jusqu'à 1000 messages puis commence à oublier les messages les plus anciens.

`NodeHandle ::subscribe()` retourne un objet `ros ::Subscriber` qui doit être conservé tout le temps de la souscription. Quand l'objet `ros ::Subscriber` est détruit le node est automatiquement désabonné du topic *chatter*.

Il existe des versions de `NodeHandle ::subscribe()` qui permette de spécifier une méthode d'une classe, ou même tout object appellable par un objet `Boost.Function`.

```
  ros::spin();
```

`ros ::spin()` entre dans une boucle et appelle les callbacks méthodes aussi vite que possible. Cette méthode est implémenté de façon à prendre peu de temps CPU. `ros ::spin()` sort une fois que `ros ::ok()` retourne

`false` ce qui signifie que `ros ::shutdown()` a été appelé soit par le gestionnaire de Ctrl-C, le master signifiant le shutdown, ou par un appel manuel.

4.1.2.2 Python

Il faut télécharger le fichier `listener.py` dans votre répertoire :

```
#!/usr/bin/env python
import rospy
from std_msgs.msg import String

def callback(data):
    rospy.loginfo(rospy.get_caller_id() + "I heard %s", data.data)

def listener():

    # In ROS, nodes are uniquely named. If two nodes with the same
    # node are launched, the previous one is kicked off. The
    # anonymous=True flag means that rospy will choose a unique
    # name for our 'listener' node so that multiple listeners can
    # run simultaneously.
    rospy.init_node('listener', anonymous=True)

    rospy.Subscriber("chatter", String, callback)

    # spin() simply keeps python from exiting until this node is stopped
    rospy.spin()

if __name__ == '__main__':
    listener()
```

Il faut que le script soit exécutable :

```
chmod +x listener.py
```

Le code du script `listerner.py` est similaire à `talker.py` . On y a ajouté une méthode de callback pour souscrire aux messages.

```
rospy.init_node('listener', anonymous=True)

rospy.Subscriber("chatter", String, callback)

# spin() simply keeps python from exiting until this node is stopped
rospy.spin()
```

Ceci déclare que le node souscrit au topic `chatter` qui est de type `std_msgs.msg.String` . Quand de nouveaux messages arrivent, `callback` est appellé avec le message comme premier argument.

L'appel à `init_node` inclus `anonymous=True` . Cet argument génére un nombre aléatoire et l'ajoute au nom du noeud si celui existe déjà dans le graphe ROS. On peut ainsi lancer plusieurs fois le script `listener.py` .

`rospy.spin()` empêche simplement le node de sortir tant qu'on ne lui demande pas de s' arrêter. Contrairement à roscpp, `rospy.spin()` n'affecte pas les fonctions callbacks des souscripteurs car ils ont leurs propres threads.

Nous utilisons CMake comme notre système de construction, et il doit être utilisé même pour les nodes Pythons. En effet il est nécessaire de générer le code Python pour les messages et les services. Pour cela il faut aller dans le workspace catkin et lancer la commande catkin_make :

```
cd ~/catkin_ws
catkin_make
```

4.1.3 Lancer les nodes

Il suffit ensuite de lancer le listener dans un terminal :

```
rosrun beginner_tutorials listener.py
```

et le talker dans un autre terminal :

```
rosrun beginner_tutorials talker.py
```

4.2 Services

Dans cette partie nous allons écrire un serveur et un client pour le service **AddTwoInts.srv**. Le serveur va recevoir deux entiers et retourner la somme. Avant de commencer il faut s'assurer que le fichier **AddTwoInts.srv** existe bien dans le répertoire srv.

4.2.1 Serveur

4.2.1.1 C++

```
#include "ros/ros.h"
#include "beginner_tutorials/AddTwoInts.h"

bool add(beginner_tutorials::AddTwoInts::Request  &req,
         beginner_tutorials::AddTwoInts::Response &res)
{
  res.sum = req.a + req.b;
  ROS_INFO("request: x=%ld, y=%ld", (long int)req.a, (long int)req.b);
  ROS_INFO("sending back response: [%ld]", (long int)res.sum);
  return true;
}

int main(int argc, char **argv)
{
  ros::init(argc, argv, "add_two_ints_server");
  ros::NodeHandle n;

  ros::ServiceServer service = n.advertiseService("add_two_ints", add);
  ROS_INFO("Ready to add two ints.");
  ros::spin();

  return 0;
}
```

```
#include "ros/ros.h"
#include "beginner_tutorials/AddTwoInts.h"
```

La première ligne correspond à la déclaration de l'entête ROS. La deuxième ligne correspond à la déclaration du service *AddTwoInts*, et l'en-tête générée lors de l'appel à **cmake_install**.

```
#include "ros/ros.h"
#include "beginner_tutorials/AddTwoInts.h"
```

```
bool add(beginner_tutorials::AddTwoInts::Request  &req,
         beginner_tutorials::AddTwoInts::Response &res)
```

La fonction fournit le service pour ajouter deux entiers, il utilise les types *request* et *response* défini le fichier srv and retourne un booléen.

```
{
  res.sum = req.a + req.b;
  ROS_INFO("request: x=%ld, y=%ld", (long int)req.a, (long int)req.b);
  ROS_INFO("sending back response: [%ld]", (long int)res.sum);
  return true;
}
```

Ici les deux *ints* sont additionnés et stockés dans la réponse. Enfin certaines informations à propos de la requête et de la réponse sont loggées. Finalement le service retourne *true* quand la fonction est finie.

```
  ros::ServiceServer service = n.advertiseService("add_two_ints", add);
```

Ici le service est crée et publié sur ROS.

4.2.1.2 Python

Pour ce serveur il faut créer le fichier **scripts/add_two_ints_server.py** dans le paquet **beginner_tutorials** en copiant le code suivant :

```
#!/usr/bin/env python

from beginner_tutorials.srv import *
import rospy

def handle_add_two_ints(req):
    print "Returning [%s + %s = %s]"%(req.a, req.b, (req.a + req.b))
    return AddTwoIntsResponse(req.a + req.b)
```

```
9
10 def add_two_ints_server():
11     rospy.init_node('add_two_ints_server')
12     s = rospy.Service('add_two_ints', AddTwoInts, handle_add_two_ints)
13     print "Ready to add two ints."
14     rospy.spin()
15
16 if __name__ == "__main__":
17     add_two_ints_server()
```

La partie spécifique à ROS pour la déclaration de service est très petite elle est limitée à la ligne :

```
1   s = rospy.Service('add_two_ints', AddTwoInts, handle_add_two_ints)
```

4.2.2 Client

4.2.2.1 C++

Pour le client, il faut créer le fichier *src/add_two_ints_client.cpp* et copier la suite dedans :

```
1  #include "ros/ros.h"
2  #include "beginner_tutorials/AddTwoInts.h"
3  #include <cstdlib>
4
5  int main(int argc, char **argv)
6  {
7    ros::init(argc, argv, "add_two_ints_client");
8    if (argc != 3)
9    {
10     ROS_INFO("usage: add_two_ints_client X Y");
11     return 1;
12   }
13
14   ros::NodeHandle n;
15   ros::ServiceClient client = n.serviceClient<beginner_tutorials::AddTwoInts>("add_two_ints");
16   beginner_tutorials::AddTwoInts srv;
17   srv.request.a = atoll(argv[1]);
18   srv.request.b = atoll(argv[2]);
19   if (client.call(srv))
20   {
21     ROS_INFO("Sum: %ld", (long int)srv.response.sum);
22   }
23   else
24   {
25     ROS_ERROR("Failed to call service add_two_ints");
26     return 1;
27   }
28
29   return 0;
30 }
```

Le client s'architecture de la façon suivante :

```
1   ros::ServiceClient client = n.serviceClient<beginner_tutorials::AddTwoInts>("add_two_ints");
```

Cet appel crée un client pour le service *add_two_ints*. L'objet *client* de type `ros ::ServiceClient` retourné est utilisé plus tard pour appeller le service.

```
1   beginner_tutorials::AddTwoInts srv;
2   srv.request.a = atoll(argv[1]);
3   srv.request.b = atoll(argv[2]);
```

Ici l'objet *srv* instancie la classe de service et on assigne des valeurs aux membres du champ *request*. Une classe service contient deux membres, `request` et `response` qui correspondent aux classes `Request` and `Response` .

```
1   if (client.call(srv))
```

Cette ligne appelle effectivement le service. Comme les appels aux services sont bloquants, il va retourner une fois que l'appel est fait. Si l'appel service fonctionne, `call()` retourne `true` et la valeur `srv.response` sera valide. Si l'appel n'a pas fonctionné `call()` retourne `false` et la valeur dans `srv.response` ne sera pas valide.

4.2.2.2 Construire les nodes en C++

Pour construire les nodes il faut modifier le fichier **CMakeLists.txt** qui se trouve dans le paquet **beginner_tutorials** :

```
add_executable(add_two_ints_server src/add_two_ints_server.cpp)
target_link_libraries(add_two_ints_server ${catkin_LIBRARIES})
add_dependencies(add_two_ints_server beginner_tutorials_gencpp)

add_executable(add_two_ints_client src/add_two_ints_client.cpp)
target_link_libraries(add_two_ints_client ${catkin_LIBRARIES})
add_dependencies(add_two_ints_client beginner_tutorials_gencpp)
```

Ces lignes vont créer deux exécutables **add_two_ints_server** et **add_two_ints_client** qui vont aller par défaut dans le répertoire du paquet dans l'espace devel, localisé par défaut dans `/catkin_ws/devel/lib/<package_name>`. Il est possible de lancer les exécutables directement ou par rosrun. Ils ne sont pas placés dans '<prefix>/bin' car cela polluerait le chemin PATH lorsque l'on installe des paquets dans le système. Si l'on souhaite avoir l'exécutable dans le chemin à l'installation, il est possible de faire une cible d'installation.

Il faut maintenant lancer **catkin_make** :

```
cd ~/catkin_ws
catkin_make
```

4.2.2.3 Python

Pour ce client il faut créer le fichier **scripts/add_two_ints_client.py** dans le paquet **beginner_tutorials** en copiant le code suivant :

```
#!/usr/bin/env python

import sys
import rospy
from beginner_tutorials.srv import *

def add_two_ints_client(x, y):
    rospy.wait_for_service('add_two_ints')
    try:
        add_two_ints = rospy.ServiceProxy('add_two_ints', AddTwoInts)
        resp1 = add_two_ints(x, y)
        return resp1.sum
    except rospy.ServiceException, e:
        print "Service call failed: %s"%e

def usage():
    return "%s [x y]"%sys.argv[0]

if __name__ == "__main__":
    if len(sys.argv) == 3:
        x = int(sys.argv[1])
        y = int(sys.argv[2])
    else:
        print usage()
        sys.exit(1)
    print "Requesting %s+%s"%(x, y)
    print "%s + %s = %s"%(x, y, add_two_ints_client(x, y))
```

Il ne faut pas oublier de mettre les droits suivants pour faire du script un exécutable :

```
chmod a+x scripts/add_two_ints_client.py
```

Le code pour écrire un client de service est relativement simple. Pour les clients il n'est pas obligatoire d'appeller `init_node()`. Il faut d'abord attendre le service :

```
  rospy.wait_for_service('add_two_ints')
```

Cette méthode bloque l'exécution jusqu'à ce que le service *add_two_ints* soit disponible. Ensuite on crée un objet pour pouvoir y accéder.

```
  add_two_ints = rospy.ServiceProxy('add_two_ints', AddTwoInts)
```

On peut ensuite utiliser cet objet comme une fonction normale et l'utiliser ainsi :

```
  resp1 = add_two_ints(x, y)
  return resp1.sum
```

Parce que nous avons déclaré le service comme étant `AddTwoInts`, lorsque **catkin_make** est appelé deux classes sont créées : `AddTwoIntsRequest` et `AddTwoIntsResponse`. La valeur retournée est effectuée dans un objet `AddTwoIntsResponse`. Si l'appel échoue une exception de type `rospy.ServiceException` est lancée, et il faut donc mettre en place les blocs `try/except` correspondants.

4.2.2.4 Construire les nodes en python

Même pour les nodes python il est nécessaire de passer par le système de build. Cela permet de s'assurer que les codes pythons correspondants aux messages et services sont générés. Il faut donc aller dans son espace catkin et lancer **catkin_make** :

```
cd ~/catkin_ws
catkin_make
```

4.2.3 Lancer les nodes

Il suffit ensuite de lancer le client dans un terminal :

```
rosrun beginner_tutorials add_two_ints_client.py
```

et le server dans un autre terminal :

```
rosrun beginner_tutorials add_two_ints_server.py
```

Travaux Pratiques

5 TurtleBot3 61

5.1 Démarrage
5.2 Gazebo
5.3 Construction de cartes et navigation

6 Asservissement visuel 65

6.1 Introduction aux tâches
6.2 Traitement d'image pour le TurtleBot 2
6.3 OpenCV - Code de démarrage
6.4 Travail à faire

5. TurtleBot3

Ce chapitre est une version résumée des tutoriaux indiqués à chaque paragraphe.

5.1 Démarrage

5.1.1 Introduction Modèle du TurtleBot3

Dans ce contexte particulier, le TurtleBot3 sera simulé dans le container du docker.

Il existe plusieurs modèles de TurtleBot3 : burger, waffle, waffle_pi. Dans ce TP nous utiliserons le modèle waffle.

Il faut donc vérifier que votre variable d'environnement TURTLEBOT3_MODEL possède la bonne valeur. Cette variable d'environnement doit être initialisée de la façon suivante :

```
export TURTLEBOT3_MODEL=waffle
```

Pour avoir cette valeur à chaque fois, il faut inclure cette ligne dans votre fichier **.bashrc**.

5.1.2 Installation

Si vous ne parvenez pas à faire fonctionner gazebo, vous pouvez installer les paquets nécessaires avec :

```
sudo apt update
sudo apt install ros-foxy-turtlebot3-description ros-foxy-turtlebot3-simulations ros-foxy-turtlebot3-bringup ros-
    foxy-turtlebot3-navigation2 ros-foxy-turtlebot3 ros-foxy-xacro ros-foxy-turtlebot3-teleop ros-foxy-turtlebot3
    -cartographer
```

Si ROS ne trouve le package turtlebot3_bringup vous pouvez l'installer avec :

```
apt install ros-foxy-turtlebot3-bringup
```

5.2 Gazebo

Pour simuler le robot sous Gazebo comme représenté dans la figure Fig.5.1, il faut lancer :

```
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

Pour obtenir une maison (ATTENTION : il ne faut pas lancer deux fois gazebo) :

```
ros2 launch turtlebot3_gazebo turtlebot3_house.launch.py
```

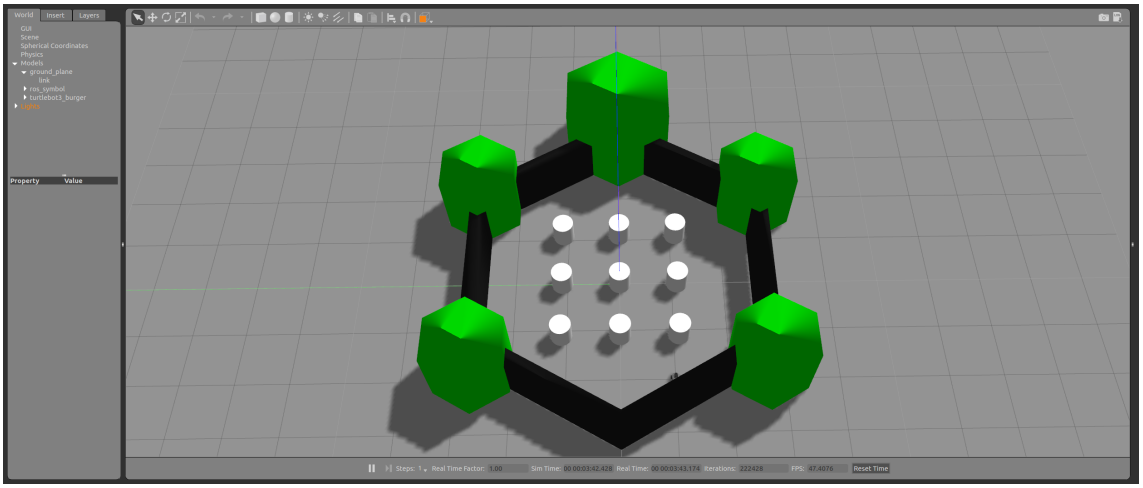


FIGURE 5.1 – Le turtlebot 3 dans une arène hexagonale avec des piliers verts

Notes :

— Si Gazebo est lancé en premier, il est possible que sa base de données des modèles se mette à jour. Cette opération peut prendre quelques minutes.

— Assurez vous que vos variables d'environnement soient bien à jour et notamment que le fichier **setup.bash** soit bien appellé dans le fichier **.bashrc** :

```
source /opt/ros/release_name/setup.bash
```

5.2.1 Téléopérer le turtlebot

Dans un deuxième terminal vous pouvez téléopérer le robot en tapant sur le PC (et non sur le turtlebot) :

```
ros2 run turtlebot3_teleop teleop_keyboard
```

Dans le terminal vous devez voir apparaitre les lignes suivantes :

```
Control Your Turtlebot!
---------------------------
Moving around:
        w
   a    s    d
        x

w/x : increase/decrease only linear speed by 10%
a/d : increase/decrease only angular speed by 10%
space key, s : force stop

CTRL-C to quit

currently:  speed 0.2 turn 1
```

Les touches *w* et *x* permettent d'aller en avant et en arrière. Les touches *a* et *d* permettent de tourner à droite et à gauche.

Le tutorial associé est disponible ici.

5.3 Construction de cartes et navigation

5.3.1 Construction de cartes

Dans cette partie, le robot utilise une ligne de l'image de profondeur fournie par le laser pour construire une carte de l'environnement. Le tutorial associé est disponible ici.

1. En simulation il faut lancer le fichier de launch **gmapping_demo** sur un autre terminal. Remarque importante, cette commande lance un node **rviz** :

```
# From turtlebot laptop
ros2 launch turtlebot3_cartographer cartographer.launch.py use_sim_time:=True
```

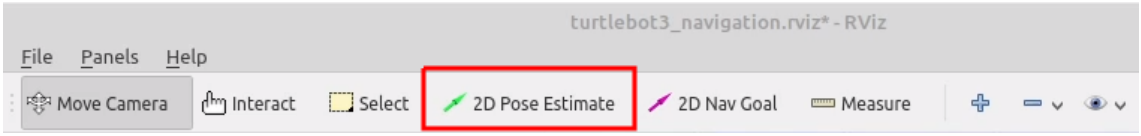


FIGURE 5.2 – Button pour l'estimation de la pose 2D

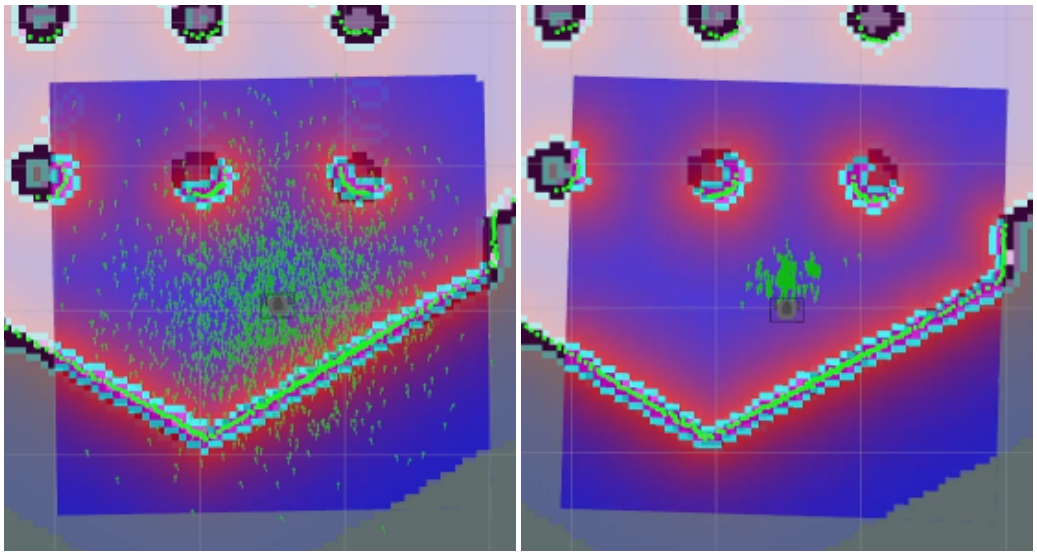


FIGURE 5.3 – AMCL partical

2. Il faut alors explorer l'environnement pour construire la carte avec les nodes de téléopération (voir paragraphe 5.2.1).
3. Une fois que la carte est suffisante, il faut la sauver en utilisant la commande suivante dans un nouveau terminal :

```
ros2 run nav2_map_server map_saver_cli -f ~/map
```

Note : Ne coupez aucun serveur pour ne pas perdre la carte !

5.3.2 Navigation

Dans cette partie, le robot utilise une carte de l'environnement. Le tutorial associé est disponible ici.

1. Sur le robot il faut lancer le fichier de launch en spécifiant la carte générée :

```
export TURTLEBOT_MAP_FILE=/tmp/my_map.yaml
ros2 launch turtlebot3_navigation2 navigation2.launch.py use_sim_time:=True map:=$HOME/map.yaml
```

5.3.3 RVIZ

5.3.3.1 Localisation du turtlebot

Lorsqu'il démarre le turtlebot ne sait pas où il est. Pour lui fournir sa localisation approximative sur la carte :

— Cliquer sur le bouton "2D Pose Estimate"

— Cliquer sur la carte où le TurtleBot se trouve approximativement et pointer la direction du TurtleBot.

Vous devriez voir une collection de flèches qui sont des hypothèses de positions du TurtleBot. Le scan du laser doit s'aligner approximativement avec les murs de la carte. Si l'algorithme ne converge pas, il est possible de répéter la procédure.

5.3.3.2 Teleoperation

Les opérations de téléopérations peuvent fonctionner en parallèle de la navigation. Elles sont toutefois prioritaires sur les comportements de navigation autonomes si une commande est envoyée. Souvent, il est

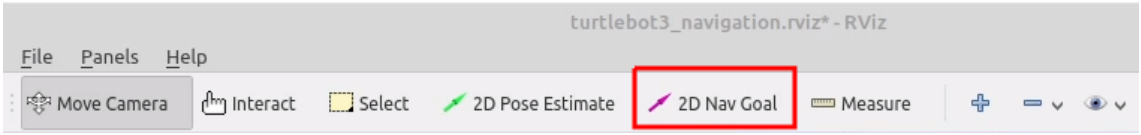


FIGURE 5.4 – Définir un but de navigation avec le bouton de RVIZ

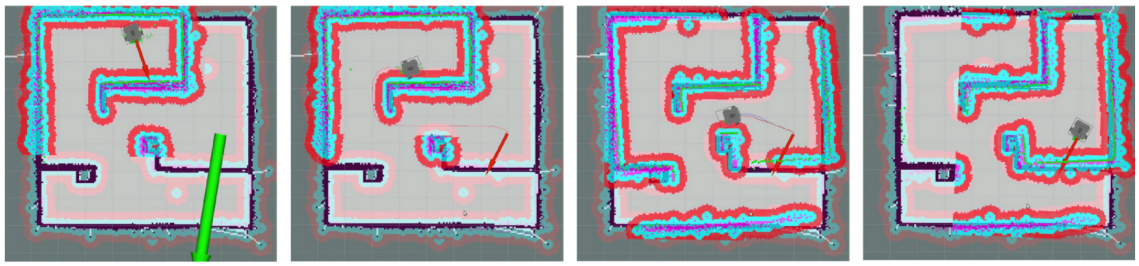


FIGURE 5.5 – Le système trouve un chemin à partir du but de navigation défini

utile de téléopérer le robot lorsque la localisation a été fournie au robot de façon à ce que l'algoritme converge vers une bonne estimation. En effet de nouvelles caractéristiques de l'environnement permettent de mieux discriminer les hypothèses.

5.3.3.3 Lancer un but de navigation

Une fois le robot localisé, il lui est possible de planifier un chemin dans l'environnement. Pour lui envoyer un but :

— Cliquer sur le boutton "2D Nav Goal" (voir Fig.5.4).

— Cliquer sur la carte où l'on souhaite voir aller le Turtlebot et pointer la direction que celui-ci doit avoir à la fin. Le système trouve par lui même le chemin à suivre. Il s'arrête si des obstacles se trouve sur son chemin. (voir Fig.5.5)

La planification peut ne pas fonctionner s'il y a des obstacles où si le but est bloqué.

6. Asservissement visuel

Dans ce chapitre on s'intéresse à la génération de mouvements pour deux robots : le robot Turtlebot2 et le robot Baxter illustré dans la figure Fig.6. Le but est générer un mouvement de façon à ce que l'objet soit centré dans l'image de la caméra du TurtleBot2 ou de la caméra d'un des bras du robot Baxter. Pour cela on commence par une courte introduction aux fonctions de tâches basée sur [4].

6.1 Introduction aux tâches

Une tâche du point de vue du contrôle permet de générer un loi de commande pour bouger un robot suivant un critère donné. La tâche consiste donc à réguler une erreur définie par la fonction suivante :

$$\mathbf{e}(t) = \mathbf{s}(t) - \mathbf{s}^*(t) \tag{6.1}$$

$\mathbf{s}(t)$ correspond à une quantité mesurable à partir des capteurs du robot. On notera en général $\mathbf{s}(t)$ son modèle en fonction de paramètres de l'environnement et d'un modèle, et $\hat{\mathbf{s}}(t)$ sa mesure.

Par exemple considérons le robot Baxter représenté dans la figure Fig.6, en utilisant les encodeurs du robot $\hat{\mathbf{q}} \in \mathbb{R}^{12}$ on peut calculer avec la position de son préhenseur avec le modèle géométrique. On obtient



FIGURE 6.1 – Le robot Baxter en rendu 3D utilisé dans le cadre de travaux pratiques. Le robot a trois caméra une au niveau de la tête, et deux dans chaque poignet.

alors :

$$\mathbf{s}_{pre}(t) : \mathbb{R}^{12} \to SE(3), \mathbf{q} \mapsto \begin{pmatrix} \mathbf{R}_{pre} & \mathbf{T}_{pre} \\ \mathbf{0}^\top & 1 \end{pmatrix} \tag{6.2}$$

avec $\mathbf{R}_{pre}$ l'orientation du préhenseur sous forme de matrice de rotation, et $\mathbf{T}_{pre}$ la position du préhenseur. Il peut s'agir de la position de son centre de masse :

$$\mathbf{s}_{com}(t) : \mathbb{R}^{12} \to \mathbb{R}^3, \mathbf{q} \mapsto \begin{pmatrix} x_{com} \\ y_{com} \\ z_{com} \end{pmatrix} \tag{6.3}$$

ou d'un point dans l'image de la caméra du robot.

6.1.1 Contrôleur en vitesse

Considérons le robot baxter regardant un objet de couleur unie avec sa caméra. En utilisant OpenCV il est possible de détecter sa couleur dans une image, et de calculer le centre de gravité (u_{cog}, v_{cog}) issu de la détection de la couleur. On peut relier la variation de la position de ce point dans l'image (en supposant que l'objet ne bouge pas) et la variation de la position de la caméra. Supposons la vitesse spatial de la caméra soit notée : $\mathbf{V}_c = \frac{\delta \mathbf{o}_c}{\delta t} = (\mathbf{v}_c, \mathbf{w}_c)$ alors on a :

$$\dot{\mathbf{s}} = \frac{\delta \mathbf{s}}{\delta \mathbf{o}_c} \frac{\delta \mathbf{o}_c}{\delta t} = \mathbf{L}_s \mathbf{V}_c \tag{6.4}$$

avec $\mathbf{L}_s \in \mathbb{R}^{3\times 6}$, appellée la *matrice d'interaction* liée à $\mathbf{s}$. En utilisant l'équation Eq.6.1, et en supposant que $\mathbf{s}^*(t)$ est constant on a donc :

$$\dot{\mathbf{e}} = \frac{\delta}{\delta t}(\mathbf{s}(t) - \mathbf{s}^*(t)) = \dot{\mathbf{s}} = \mathbf{L}_s \mathbf{V}_c \tag{6.5}$$

Supposons maintenant que nous souhaitions que l'erreur soit régulée suivant une descente exponentielle alors

$$\dot{\mathbf{e}} = -\lambda \mathbf{e} \tag{6.6}$$

En utilisant l'équation Eq.6.5 on a donc :

$$\mathbf{V}_c = -\lambda \mathbf{L}_s^+ \mathbf{e} \tag{6.7}$$

Notons que ce problème est équivalent au problème d'optimisation suivant :

$$\begin{cases} \min_{\mathbf{V}_c} & \| \mathbf{V}_c \| \\ \text{tel que} & \mathbf{e} = -\lambda \mathbf{L}_s \mathbf{V}_c \end{cases} \tag{6.8}$$

En l'occurence, $\mathbf{e} = \hat{\mathbf{s}}(t) - \mathbf{s}^*$ où $\hat{\mathbf{s}}(t)$ est la *mesure* de la caractéristique dans le plan image. Il nous faut maintenant calculer $\mathbf{L}_s$ et générer $\mathbf{V}_c$.

6.1.2 Projection d'un point dans une image

La figure Fig. 6.1.2 illustre la projection du point $\mathbf{M} = (X_w Y_w Z_w)^\top = (X_c Y_c Z_c)^\top$ dans le plan image I au point $m = (Z_c x, Z_c y, Z_c)$. Ils sont reliés par la relation suivante :

$$Z_c \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} f\alpha & 0 & c_x & 0 \\ 0 & f & c_y & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{pmatrix} \tag{6.9}$$

avec f la distance du plan image I au centre de la caméra aussi appellée la *focale*, α le rapport entre le taille des pixels dans la direction horizontale et la direction verticale, et (c_x, c_y) le centre de l'image dans les coordonnées de la caméra.

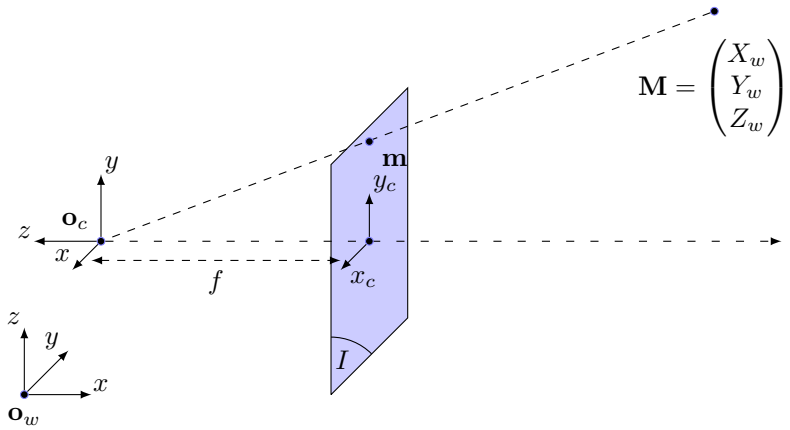


FIGURE 6.2 – o_c est l'origine du repère de la caméra, o_w l'origine du repère monde. La projection du point $\mathbf{M}$ dans le plan image I donne $\mathbf{m}$.

Si on remarque que :

$$\begin{pmatrix} f\alpha & 0 & c_x & 0 \\ 0 & f & c_y & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} = \begin{pmatrix} f\alpha & 0 & c_x \\ 0 & f & c_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \tag{6.10}$$

On peut poser :

$$K_f = \begin{pmatrix} f\alpha & 0 & c_x \\ 0 & f & c_y \\ 0 & 0 & 1 \end{pmatrix}, \prod = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \tag{6.11}$$

avec K_f une représentation canonique de la matrice des paramètres intrinséques de la caméra, et $\prod$ la matrice de projection canonique. En notant que :

$$\begin{pmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{pmatrix} = \begin{pmatrix} \mathbf{R}_c & \mathbf{T}_c \\ \mathbf{0}^\top & 1 \end{pmatrix} \begin{pmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{pmatrix} = \mathbf{M}_c\mathbf{M} \tag{6.12}$$

avec $\mathbf{R}_c$ l'orientation de la caméra dans le monde et $\mathbf{T}_c$ la position de la caméra dans le monde. Nous avons donc :

$$Z_c \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} f\alpha & 0 & c_x \\ 0 & f & c_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} \mathbf{R}_c & \mathbf{T}_c \\ \mathbf{0}^\top & 1 \end{pmatrix} \begin{pmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{pmatrix} = \mathbf{K}_f \prod \mathbf{M}_c\mathbf{M} \tag{6.13}$$

Finalement le modèle de la caractéristique visuelle mesurée est :

$$\mathbf{s} = \mathbf{m} = \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \frac{1}{Z_c} \mathbf{K}_f \prod \mathbf{M}_c\mathbf{M} \tag{6.14}$$

Notons que la caméra ne mesure pas $\mathbf{m}$ mais $\frac{\mathbf{m}}{Z_c}$.

6.1.3 Calcul de la matrice d'interaction

On cherche maintenant à relier la variation de la caractéristique visuelle avec celle de la caméra. En supposant que la focale de la caméra $f = 1$, que $\alpha = 1$ et que le centre de la caméra est à zéro $(c_x, c_y) = (0,0)$

alors on peut écrire :

$$\begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \frac{1}{Z_c} \begin{pmatrix} f\alpha & 0 & c_x \\ 0 & f & c_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{pmatrix} = (\frac{X_c}{Z_c} \ \frac{Y_c}{Z_c} \ 1)^\top \tag{6.15}$$

Donc :

$$\begin{pmatrix} \dot{x} \\ \dot{y} \end{pmatrix} = \begin{pmatrix} \dot{X}_c/Z + X_c\dot{Z}_c/Z_c^2 \\ \dot{Y}_c/Z + Y_c\dot{Z}_c/Z_c^2 \end{pmatrix} = \begin{pmatrix} (\dot{X}_c + x\dot{Z}_c)/Z_c \\ (\dot{Y}_c + y\dot{Z}_c)/Z_c \end{pmatrix} \tag{6.16}$$

On peut relier la vitesse du point $\mathbf{M}$ avec la vitesse spatiale de la caméra $\mathbf{V}_c = (\mathbf{v}_c, \omega_c) = (v_x, v_y, v_z, \omega_x, \omega_y, \omega_z)$ par :

$$\dot{\mathbf{M}} = -\mathbf{v}_c - \omega_c \times M \Leftrightarrow \begin{cases} \dot{X}_c = -v_x - \omega_y Z_c + \omega_z Y_c \\ \dot{Y}_c = -v_y - \omega_z X_c + \omega_x Z_c \\ \dot{Z}_c = -v_z - \omega_x Y_c + \omega_y X_c \end{cases} \tag{6.17}$$

Cette relation peut-être écrite de la façon suivante :

$$\begin{cases} \dot{x} = -v_x/Z_c + xv_z/Z_c + xy\omega_x - (1+x^2)\omega_y + y\omega_z \\ \dot{y} = -v_y/Z_c + yv_z/Z_c + (1+y^2)\omega_x - xy\omega_y - x\omega_z \end{cases} \tag{6.18}$$

Maintenant la matrice la relation peut s'écrire :

$$\dot{\mathbf{s}} = \mathbf{L_s}\mathbf{V}_c \tag{6.19}$$

avec

$$\mathbf{L_s} = \begin{pmatrix} -1/Z_c & 0 & x/Z_c & xy & -(1+x^2) & y \\ 0 & -1/Z_c & y/Z_c & 1+y^2 & -xy & -x \end{pmatrix} \tag{6.20}$$

On peut voir que dans la formulation de la matrice d'interaction se trouve la profondeur du point M par rapport au repère de la caméra. Par conséquent tout schéma de contrôle qui utilise cette matrice d'interaction doit utiliser une estimation de Z_c. Enfin les hypothèses concernant les valeurs du centre de la caméra et de la focale ne sont généralement pas vérifiées et il est nécessaire de prendre en compte les valeurs de la matrice des paramètres intrinséques.

6.1.4 Génération de la commande

En utilisant une paramétrisation des matrices homogènes (par exemple Davit-Hartenberg) correspondant à la position des corps du robot on peut écrire pour le robot Baxter la fonction reliant la position des moteurs du robot à la position de la caméra :

$$\mathbf{s}_{cam}(\mathbf{q}) = \begin{pmatrix} \mathbf{R}_{cam} & \mathbf{T}_{cam} \\ \mathbf{0}^\top & 1 \end{pmatrix} \tag{6.21}$$

Par dérivation on peut alors calculer :

$$\mathbf{V}_c = \frac{\delta \mathbf{o}_c}{\delta \mathbf{q}} \frac{\delta \mathbf{q}}{\delta s} \tag{6.22}$$

On a donc :

$$\mathbf{V}_c = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}} \tag{6.23}$$

Note : lorsque le robot n'est pas fixé au sol $\mathbf{q}$ et $\dot{\mathbf{q}}$ ne sont pas obligatoirement de même taille. En effet la position du robot est paramétrée par un repére flottant. Pour $\dot{\mathbf{q}}$ la vitesse angulaire est exprimée suivant les axes $(x, y, z) \in \mathbb{R}^3$ mais l'orientation dans $\mathbf{q}$ peut être paramétrée soit par une matrice de rotation de taille 9, ou un quaternion de taille 4, ou des angles d'Euler de taille 3. Dans tous les cas, pour un robot non

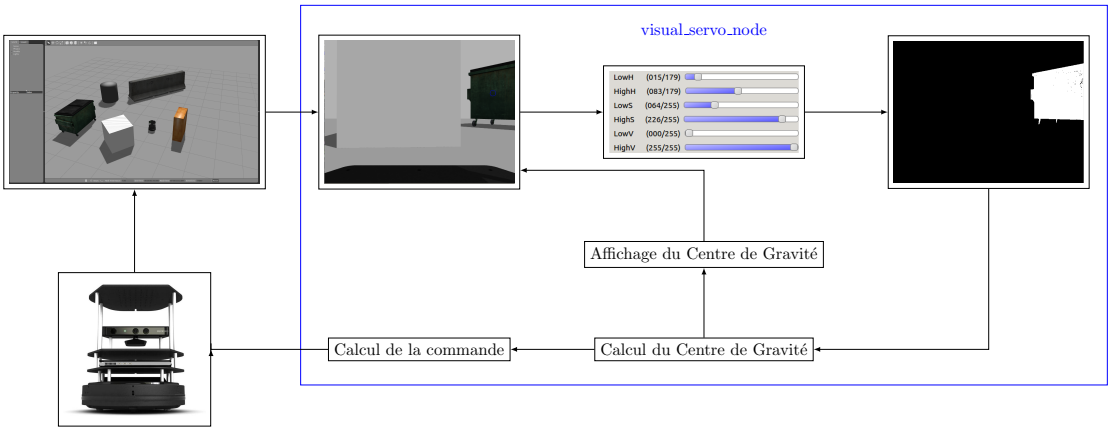


FIGURE 6.3 – Séquence d'opérations de base

fixé au sol, il n'est pas possible d'écrire Eq.6.22 mais on peut toujours avoir Eq.6.23 qui est l'application de l'espace tangent de la paramétrisation de la pose spatial du robot (et de ses moteurs) dans l'espace des vitesses. Par abus de language la matrice de l'équation Eq.6.23 est appellée Jacobienne articulaire.

Finalement en considérant le problème spécifié par l'équation Eq.6.8 on peut écrire :

$$\begin{cases} \min_{\dot{\mathbf{q}}} & \| \dot{\mathbf{q}} \| \\ \text{tel que} & \mathbf{e} = -\lambda \mathbf{L}_s \mathbf{J}(\mathbf{q}) \dot{\mathbf{q}} \end{cases} \tag{6.24}$$

ce qui peut s'écrire algèbriquement :

$$\dot{\mathbf{q}} = -\lambda (\mathbf{L}_s \mathbf{J}(\mathbf{q}))^{+} \mathbf{e} \tag{6.25}$$

Pour implémenter le contrôleur sur le robot Baxter il faut implémenter la matrice Jacobienne compléte et la matrice d'interaction. Pour le robot Turtlebot 2 on peut simplifier en considérant uniquement la vitesse angulaire ω_y et la matrice $\mathbf{J}(\mathbf{q})$ étant une matrice identité.

6.2 Traitement d'image pour le TurtleBot 2

Le but de ce programme est de générer une commande qui suit un objet en utilisant la couleur. La structure du programme est donc la suivante :

1. La première étape est d'obtenir une image couleur en souscrivant au topic approprié de la caméra du turtlebot.
2. La couleur est ensuite extraite de l'image en utilisant l'espace HSV et en testant l'appartenance d'un pixel à un intervalle.
3. En calculant la position du centre de gravité de l'objet dans l'image on calcule la commande à envoyer aux moteurs de la base mobile pour diminuer la différence entre le centre de l'image et le centre de gravité de l'objet. La commande est envoyée en publiant sur le topic approprié.

L'ensemble se basera sur la structure du programme fournie à la fin de ce sujet.

6.2.1 Extraction d'une image

La première étape est d'obtenir une image couleur à partir du robot. Il est donc nécessaire d'écrire la partie du code permettant de souscrire au topic fournissant cette image. On pourra pour cela s'inspirer de l'exemple du client souscripteur disponible à l'adresse suivante :
http ://wiki.ros.org/image_transport/Tutorials/SubscribingToImages.

Il faudra penser à adapter le contexte au fait que nous considérons ici du C++. La fonction de callback devrait être notamment traité de façon différente par rapport à l'exemple.

La deuxième étape consiste à extraire les informations de la structure fournie par ROS. Les couleurs doivent être placées de façon appropriée dans l'image **cv_ptr**.

6.2.2 Extraction de la couleur

L'image est ensuite convertie au format HSV et placée dans la structure **imgHSV** (ligne 73). La structure **imgThresholded** contient le résultat du traitement de l'image HSV (ligne 76 – 77). Les pixels qui sont dans les intervalles spécifiés (lignes 42 – 48) par les ascenceurs sont mis à 255 tandis que les autres sont mis à 0. A partir de ces pixels il est possible de calculer le centre de gravité (lignes 80 – 96).

6.2.3 Calcul de la commande

Pour le Turtlebot 2 la commande se calcule ici trés simplement en effectuant la différence entre la position du centre de gravité et le centre de l'image. Cette différence est ensuite multipliée par un gain. Le but de cette partie est de reconstruire le message à envoyer sur le topic utilisé pour la commande du robot. La rotation va permettre au robot de centrer l'objet dans l'image.

Le publisher s'écrit en utilisant le code disponible à l'adresse suivante :
http ://wiki.ros.org/roscpp_tutorials/Tutorials/WritingPublisherSubscriber.

6.3 OpenCV - Code de démarrage

Le code est disponible sur github. Vous pouvez l'installer dans votre espace de travail de la façon suivante :

```
cd ~/catkin_ws/src
git clone https://github.com/olivier-stasse/tp_ros_visual_servo.git
```

Si vous êtes sur Noetic :

```
git checkout noetic
```

```
#include <iostream>
#include <unistd.h>
#include <cv_bridge/cv_bridge.h>
#include <ros/ros.h>

#include "opencv2/highgui/highgui.hpp"
#include "opencv2/imgproc/imgproc.hpp"
#include <image_transport/image_transport.h>

#include <sensor_msgs/Image.h>
#include <sensor_msgs/image_encodings.h>
#include <geometry_msgs/Twist.h>

using namespace cv;
using namespace std;

static const std::string OPENCV_WINDOW = "Image window";

class ImageConverter
{
  ros::NodeHandle nh_;
  image_transport::ImageTransport it_;
  image_transport::Subscriber image_sub_;
  ros::Publisher cmd_vel_pub_;

  int iLowH_,iHighH_;
  int iLowS_,iHighS_;
  int iLowV_,iHighV_;

  // Commandes parameters are made with a tuple =
  // [Parameter name, current value of the parameter, default value of the parameter]
  typedef std::tuple<std::string, double, double> param_ltype;
  // Dictionnary of the parameters
  std::map<std::string, param_ltype> list_params;

  // Center of gravity.
  double cx_, cy_;
  unsigned int nb_pts_;

public:
  ImageConverter()
    : it_(nh_),iLowH_(15), iHighH_(83),
      iLowS_(190), iHighS_(226),
      iLowV_(0), iHighV_(255),
      cx_(0.0),cy_(0.0), nb_pts_(0)
  {
    // Creates subscriber to get the input video feed
```

```
48
49
50      // Window to show the image
51      cv::namedWindow(OPENCV_WINDOW);
52      // Window named "Control" to display sliders
53      cv::namedWindow("Control", WINDOW_AUTOSIZE);
54
55      // Create trackbars in "Control" window
56      cv::createTrackbar("LowH", "Control", &iLowH_, 179); //Hue (0 - 179)
57      cv::createTrackbar("HighH", "Control", &iHighH_, 179);
58
59      cv::createTrackbar("LowS", "Control", &iLowS_, 255); //Saturation (0 - 255)
60      cv::createTrackbar("HighS", "Control", &iHighS_, 255);
61      cv::createTrackbar("LowV", "Control", &iLowV_, 255); //Value (0 - 255)
62      cv::createTrackbar("HighV", "Control", &iHighV_, 255);
63
64      // Create publisher to send to control
65
66
67      list_params["Kp"]=std::make_tuple(std::string("/visual_servo/Kp"),0.001,0.001);
68      list_params["dx_clamp"]=std::make_tuple(std::string("/visual_servo/dx_clamp"),10.0,10.0);
69      list_params["wz_min"]=std::make_tuple(std::string("/visual_servo/wz_min"),0.01,0.01);
70      list_params["wz_max"]=std::make_tuple(std::string("/visual_servo/wz_max"),1.7,1.7);
71
72      initParameters();
73    }
74
75    void initParameters()
76    {
77      // For each parameters
78      for ( auto it = list_params.begin(); it != list_params.end(); ++it )
79        {
80    // If the parameter is already set
81    if (nh_.hasParam(std::get<0>(it->second)))
82      // Read the current value.
83      nh_.getParam(std::get<0>(it->second),std::get<1>(it->second));
84    else
85      // Set the default value.
86      nh_.setParam(std::get<0>(it->second),std::get<1>(it->second));
87        }
88    }
89
90    ~ImageConverter()
91    {
92      cv::destroyWindow(OPENCV_WINDOW);
93    }
94
95    double computeCmdZ(double DeltaX)
96    {
97      double Kp = std::get<1>(list_params["Kp"]);
98      double DeltaX_Clamp = std::get<1>(list_params["dx_clamp"]);
99      double wz_min = std::get<1>(list_params["wz_min"]);
100     double wz_max = std::get<1>(list_params["wz_max"]);
101
102     // Compute simple linear relationship.
103     double cmdz = -Kp*(DeltaX);
104
105     ROS_INFO_STREAM("Kp: " << Kp);
106     ROS_INFO_STREAM("DeltaX: " << DeltaX);
107     ROS_INFO_STREAM("cmdz: " << cmdz);
108     // Lowest saturation
109     double signe_cmd = cmdz > 0.0 ? 1.0 : -1.0;
110
111     // Control saturation
112
113     // Set control to zero if the robot is near the target.
114
115     // Minimal value if the robot is not near enough but needs to move
116
117     return cmdz;
118   }
119
120   void processImages(const sensor_msgs::ImageConstPtr& msg,
121              cv_bridge::CvImagePtr & cv_ptr,
122              Mat & imgThresholded)
123   {
124     // From ros image to opencv
125     try
126     {
127       cv_ptr = cv_bridge::toCvCopy(msg, sensor_msgs::image_encodings::RGB8);
128     }
129     catch (cv_bridge::Exception& e)
130     {
131       ROS_ERROR("cv_bridge exception: %s", e.what());
```

```
      return;
    }

    Mat imgHSV;
    // From RGB to HSV
    cvtColor(cv_ptr->image, imgHSV, COLOR_RGB2HSV);

    // Analysis object and provides a Thresholded image
    inRange(imgHSV, Scalar(iLowH_, iLowS_, iLowV_),
      Scalar(iHighH_, iHighS_, iHighV_), imgThresholded);
  }

  void displayImages(cv_bridge::CvImagePtr & cv_ptr,
         Mat & imgThresholded)
  {
    if (nb_pts_>0)
      {
        // Draw the CoG in the image
  cv::circle(cv_ptr->image, cv::Point((int)cx_,(int)cy_), 10, CV_RGB(0,0,255));
      }

    // Show the thresholded image
    imshow("Thresholded Image", imgThresholded);

    // Update GUI Window
    cv::imshow(OPENCV_WINDOW, cv_ptr->image);

  }

  void computeCoG(Mat & imgThresholded)
  {
    // Compute Center-Of-Gravity of the object.
    cx_=0.0,cy_=0.0;
    nb_pts_=0;
    unsigned long int idx=0;
    for(unsigned int j=0;j<imgThresholded.rows;j++)
      {
  for(unsigned int i=0;i<imgThresholded.cols;i++)
    {
      if (imgThresholded.data[idx]>124)
        {
    cx_+=(double)i;
    cy_+=(double)j;
    nb_pts_++;
        }
      idx+=1;
    }
      }
    if (nb_pts_>0)
      {
  cx_ = cx_/(double)nb_pts_;
  cy_ = cy_/(double)nb_pts_;
      }
  }

  void imageCb(const sensor_msgs::ImageConstPtr& imgMsg)
  {

    // 1 - Update each parameters by iterating over list_params.

    // 2- Image processing
    // Convert sensors_msg::Image to opencv image
    cv_bridge::CvImagePtr cv_ptr;
    // Result image
    Mat imgThresholded;
    processImages(imgMsg,cv_ptr,imgThresholded);

    // Compute the Center-of-Gravity.
    computeCoG(imgThresholded);

    // 3 - Compute Control for base

    // 4 - Send Control value to base

    displayImages(cv_ptr, imgThresholded);

    // 5 - Opencv update
    cv::waitKey(3);
  }
};


int main( int argc, char** argv )
{
```

```
216   ros::init(argc,argv,"tp_ros_visual_servo");
217   ImageConverter ic;
218   ros::spin();
219 }
```

6.4 Travail à faire

6.4.1 Turtlebot 3

6.4.1.1 Compilation

Compiler le code, rajouter les champs de classes manquants et compléter les fichiers **package.xml** et **CMakeLists.txt**.

Conseils :

1. Vérifiez le nom du code source. Utilisez le nom de l'exécutable pour changer un nom de fichier source si vous devez le faire.
2. Vérifiez les étapes nécessaires à la création d'un node en vous inspirant des souscripteurs et des publieurs sur des topics.
3. En cas de SEGFAULT pensez au fait qu'en C++ l'ordre de déclaration des variables est important.
4. Si vous avez un problème avec l'édition des liens n'oubliez pas que vous pouvez spécifier des dépendences vers d'autres paquets en modifiant les fichiers **package.xml** (blocks <build_depend> et <run_depend>) et **CMakeLists.txt**.

6.4.1.2 Souscription au topic fournissant l'image

Créer le souscripteur pour obtenir l'image et appeler la méthode **imageCb** lorsqu'une image est présente sur le topic. Conseils :

1. Cherchez quel est le nom du topic auquel vous souhaitez souscrire. Utilisez par exemple la commande rostopic.
2. Le nom du topic est relié au fait que l'on cherche à retrouver une image.
3. Utilisez les tutoriels permettant d'écrire des souscripteurs et des publieurs sur des topics.
4. Utilisez également les tutoriels qui font des callbacks sur des classes C++.
5. Recherchez des exemples sur Internet utilisant la classe **ImageTransport**.

6.4.1.3 Test du pipeline

Tester le pipeline de traitement d'image avec un gilet jaune.

Conseils :

1. Utilisez les paramètres ROS pour pouvoir publier les valeurs $H_{min}, H_{max}, S_{min}, S_{max}, V_{min}, V_{max}$. Regardez la documentation qui se trouve ici : http ://wiki.ros.org/roscpp/Overview/Parameter%20Server
2. Dans le constructeur, testez si les paramètres existent si non alors les mettre dans l'arbre des paramètres. Pour cela utiliser un namespace. Par exemple :

```
1     if (!nh_.hasParam("/visual_servo_initiales/Hmin"))
2        nh_.getParam("/visual_servo_initiales/Hmin",Hmin_);
3
```

6.4.1.4 Commande du robot

Etapes :

1. Trouver le nom du topic qui prend un message **<geometry_msgs ::Twist>** et qui fait bouger la base mobile du Tiago.
2. Créer le publisheur permettant d'envoyer une commande en vitesse à la base mobile.
3. Calculer $|\Delta_x|$ la différence entre le milieu de l'image ci_x égale à 320 et les coordonnées suivant l'axe x du centre de gravité.
4. Calculer la commande à envoyer sur le robot. Pour le moment, on ne fait pas de mouvement linéaire $\mathbf{v} = [v_x, v_y, v_z]^\top = [0.0, 0.0, 0.0]^\top$. Les vitesses angulaires autour de l'axe ω_x et de l'axe ω_y sont à zéros. La vitesse angulaire ω_z est celle qui nous intéresse. La figure Fig.6.4.1.4 illustre la forme de la commande qui dépend de la variation en pixels. Lorsque $|\Delta_x|$ est inférieure à une certain valeur la commande est nulle. Sinon on cherche à ce que la commande soit linéairement proportionnelle à Δ_x grâce au gain K_p. Lorsque Δ_x est positif le robot doit tourner vers la droite et donc ω_z doit être négatif.

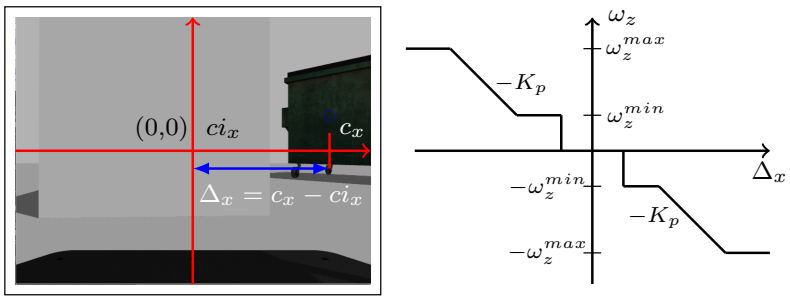


FIGURE 6.4 – Commande en vitesse à partir de la variation en pixels.

Enfin si la commande est trop grande elle doit être saturée soit par $-\omega_z^{max}$ soit par ω_z^{max}. Si elle est positive mais trop petite alors il faut une commande minimale $-\omega_z^{min}$ soit par ω_z^{min} pour pouvoir vaincre la friction.

5. Construire le message qui est envoyé à la base mobile.
6. Implémenter un accés aux paramètres de façon à pouvoir changer les gains en ligne.
7. Tester le contrôleur sur gazebo.
8. Tester le contrôleur sur le robot.
9. Déplacer le contrôleur sur le robot.
10. Ecrire un fichier de launch qui initialise par défaut les gains du contrôleur soit sur le robot soit sur gazebo.

6.4.1.5 Utilisation de la kinect

Etapes :

1. Trouver le nom du topic qui fournit un message **<sensor_msgs ::Image>** issue de la caméra de profondeur.
2. Ecrire le prototype de la méthode capable de récupérer l'image de profondeur.
3. Créer le souscripteur permettant de récuperer l'image de profondeur.
4. Développer la méthode permettant d'extraire l'information de distance de l'image de profondeur.

II Modèles et Simulation

7 Universal Robot Description Format (URDF) 77

7.1 Capteurs - Sensors
7.2 Corps - Link
7.3 Transmission
7.4 Joint
7.5 Gazebo
7.6 model_state
7.7 model

8 Simulation 85

8.1 Introduction
8.2 Algorithme général d'un simulateur système
8.3 Algorithme général d'un moteur dynamique

7. Universal Robot Description Format (URDF)

Le format URDF décrit au format XML le modèle d'un robot. Le paquet urdfm implémente le parser qui analyse ce modèle. Ce chapitre se base largement sur la page ros suivante http ://wiki.ros.org/urdf/XML.

7.1 Capteurs - Sensors

7.1.1 Norme courante

L'élément **<sensor>** décrit les propriétés basiques d'un capteur visuel (caméra/capteur de rayons) Voici un exemple d'un élément décrivant une caméra :

```
   <sensor name="my_camera_sensor" update_rate="20">
   <parent link="optical_frame_link_name"/>
   <origin xyz="0 0 0" rpy="0 0 0"/>
   <camera>
     <image width="640" height="480" hfov="1.5708" format="RGB8" near="0.01" far="50.0"/>
   </camera>
 </sensor>
```

Et voici un exemple d'un élément représentant un laser :

```
<sensor name="my_ray_sensor" update_rate="20">
   <parent link="optical_frame_link_name"/>
   <origin xyz="0 0 0" rpy="0 0 0"/>
   <ray>
     <horizontal samples="100" resolution="1" min_angle="-1.5708" max_angle="1.5708"/>
     <vertical samples="1" resolution="1" min_angle="0" max_angle="0"/>
   </ray>
</sensor>
```

Cet élément a les attributs suivants :

— **name** : *(requis) (string)*
Le nom du capteur

— **update_rate** : *(optional) (float) (Hz)*
La fréquence a laquelle le capteur produit les données. Si cette valeur n'est pas spécifiée les données sont générées à chaque cycle.

— **type** : *(nouveau) (required) (string)*
Le type du capteur peut être **camera**,**ray**,**imu**,**magnetometer**,**gps**, **force_torque**,**contact**,**sonar**,**rfidtag**,**rfid**.

Les éléments cet élément sont les suivants :

— **<parent>** : *(required)*
link*(required) (string)*
Le nom du corps auquel le capteur est attaché.
— **<origin>** : *(optional : à defaut l'identité si ce n'est pas spécifié*
Il s'agit de la position de l'origine du capteur optique relative au repère de référence du corps parent. Le repère du capteur optique adopte la convention suivante : z-vers le devant, x à droite, y en bas.
  — **xyz** : *(optionel : par défaut à 0*
  Représente la position suivant les axes x, y, z.
  — **rpy** : *(optionel : par défaut à l'identité*
  Représente les axes fixes roll, pitch et yaw en radians.
— **<camera>** : *(optionel)*

  — **<image>** : *(requis)*

    — **width** : *(requis) (unsigned int) (pixels)*
    Largeur de l'image en pixels
    — **height** : *(requis) (unsigned int) (pixels)*
    Hauteur de l'image en pixels
    — **format** : *(requis)(string)*
    Format de l'image. Peut-être chacune des chaînes définies dans le fichier **image_encodings.h** dans le paquet **sensors_msgs**.
    — **hfov** : *((requis)(float)(radians)*
    Largeur du champ de vue de la caméra (*rad*).
    — **near** : *(requis)(float)(m)*
    Distance la plus proche de la perception de la caméra. (*m*)

    — **far** : *(requis)(float)(m)*
    Distance la plus lointaine de la perception de la caméra. (*m*)

— **<ray>** : *(optionel)*

  — **<horizontal>** : *(optionel)*

    — **samples** : *(optionel : default 1)(unsigned int)*
    Le nombre de rayons simulés pour générer un cycle complet de perception laser.
    — **resolution** : *(optionel : default 1)(float)*
    Ce nombre est multiplié par **samples** pour déterminer le nombre de données produite. Si la résolution est inférieure à 1 les données de profondeur sont interpolées, si la résolution est supérieure à 1 les données sont moyennées.
    — **min_angle** : *(optionel : default 0)(float)(radians)* :

    — **max_angle** : *(optionel : default 0)(float)(radians)* :
    Doit être supérieur ou égale à **min_angle**.
  — **<vertical>** : *(optionel)*

    — **samples** : *(optionel : default 1)(unsigned int)*
    Le nombre de rayons simulés pour générer un cycle complet de perception laser.
    — **resolution** : *(optionel : default 1)(float)*
    Ce nombre est multiplié par **samples** pour déterminer le nombre de données produite. Si la résolution est inférieure à 1 les données de profondeur sont interpolées, si la résolution est supérieure à 1 les données sont moyennées.
    — **min_angle** : *(optionel : default 0)(float)(radians)* :

    — **max_angle** : *(optionel : default 0)(float)(radians)* :
    Doit être supérieur ou égale à **min_angle**.

7.1.2 Limitations

Peu de capteur peuvent exprimés dans le format URDF. C'est une faiblesse particuliérement problématique pour des programmes qui en ont besoin. C'est le cas pour la simulation et le contrôle. Cela démontre également que dans la pratique la plupart des développements utilisent très peu les simulations des capteurs ou le font à travers d'autres formats. C'est notamment le cas pour gazebo qui utilise le format SDF.

7.2 Corps - Link

L'élément **link** décrit un corps rigide avec une inertie et des caractéristiques visuelles.

```
<link name="my_link">
  <inertial>
    <origin xyz="0 0 0.5" rpy="0 0 0"/>
    <mass value="1"/>
    <inertia ixx="100"  ixy="0"  ixz="0" iyy="100" iyz="0" izz="100" />
  </inertial>

  <visual>
    <origin xyz="0 0 0" rpy="0 0 0" />
    <geometry>
      <box size="1 1 1" />
    </geometry>
    <material name="Cyan">
      <color rgba="0 1.0 1.0 1.0"/>
    </material>
  </visual>

  <collision>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <geometry>
      <cylinder radius="1" length="0.5"/>
    </geometry>
  </collision>
</link>
```

7.2.1 Attributs

— **name** : *(requis) (string)*
Le nom du corps lui même.

7.2.2 Elements

— **<inertial>** *(optionel)*
Les propriétés inertielles du corps.
  — **<origin>** *(optionel : l'identité si rien n'est spécifié)*
  Il s'agit de la pose du repère de référence inertiel, relative au repère du corps de référence. L'origine du repère de référence inertiel doit être au centre de gravité. Les axes du repère de référence inertiel n'ont pas forcément besoin d'être alignés avec les axes principaux de l'inertie.
    — **xyz** *(optionel : par défaut à zéro*
    Représente le décalage selon les axes x, y, z
    — **rpy** *(optionel : par défaut égale à l'identité si non spécifié*
    Représente la rotation du corps selon les axes $(roll, pitch yaw)$ en radians.
  — **<mass>** La masse du corps est fixée par l'attribut **value** de cet élément
  — **<inertia>** La matrix d'inertie 3×3 représentée dans le repère inertiel. Parce que cette matrice d'inertie est symmétrique, seuls les 6 éléments diagonaux supérieurs sont donnés ici en utilisant les attributs **ixx**,**ixy**,**ixz**,**iyy**, **yz**,**izz**.
— **<visual>** *(optionel)* Les propriétés visuelles du corps. Cet élément spécifie la forme de l'objet (boite, cylindre, etc.) pour la visualisation. **Note** : il est possible d'avoir plusieurs blocks <visual> pour le même corps. Leur union définit la représentation visuelle du corps.
  — **name** *(optionel)* Spécifie un nom pour une partie de la géométrie d'un corps. Cela est particulièremen utile pour faire référence à une partie de la géométrie d'un corps.
  — **<origin>** Le repère de référence de l'élément visuel par rapport au repère de référence du corps.
    — **xyz** Représente la position de l'élément visuel dans le repère de référence du corps (en m).
    — **rpy** Représente l'orientation de l'élément visuel dans le repère de référence du corps (en *radians*).

- **<geometry>** *(requis)* La forme de l'objet visuel. Cela peut-être fait l'un des élèments suivants :
  - **box** L'attribut **size** contients la taille (longueur, larger, hauteur) de la boîte. L'origine de la boîte est le centre.
  - **cylinder** Spécifie le rayon (**<radius>**) et la hauteur (**length**) du cylindre. L'origine du cylindre est son centre.
  - **sphere** Spécifie le rayon (**<radius>**). L'origine de la sphère est son centre.
  - **mesh** Il s'agit d'un fichier décrivant une surface par des triangles. Il est spécifié par un nom de fichier (**<filename>**) et un facteur d'échelle (**<scale>**) aligné suivant l'axe de la boîte englobante de la surface. Le format recommandé est Collada (**.dae**) mais les fichiers STL (**.stl**) sont également supportés. Les fichiers de surfaces ne sont pas transférés entre les machines faisant référence au même modèle (/robot_description). Cela doit être un fichier local.
- **<material>** *(optionel)* Spécifie le matériel de l'élément visuel. Il est permis de spécifier un élément matériel à l'extérieur de l'élément **<link>** dans l'élément **<robot>**. Dans un élèment link il est possible de référencer le matériel par son nom.
  - **name** *(optionel)* Spécifie le nom du matériel.
  - **<color>** *(optionel)*
    rgba La couleur d'un matériel spécifié par un ensemble de quatre nombres représentant rouge/vert/blue/alpha, chacun dans l'intervalle $[0, 1]$.
  - **<texture>** *(optionel)*
    La texture d'un matériel est spécifié par un fichier nommé **filename**.
- **<collision>** (optionel) Les propriétés de collision d'un corps. Elles peuvent être différentes des informations visuelles. Par exemple, on utilise souvent des modèles simplifés pour la collision afin de réduire les temps de calcul. Il peut y avoir des instances multiples de **collision** pour un même corps. L'union des géométries que ces instances définissent forment la représentation pour la collision de ce corps.
  - **name** (optionel)
    Spécifie un nom pour une partie géométrique du corps. Ceci est utile pour faire référene à des parties spécifiques de la géométrie d'un corps.
  - **<origin>** (optionel : défaut à l'identité si n'est pas spécifié) Le référentiel de référence de l'élément collision exprimé dans référentiel de référence du corps.
    - **xyz** (optionel : par défaut à zéro) Représente la position de l'élément de collision.
    - **rpy** (optionel : par défaut à l'identité) Représente les axes fixes roulis, tangage et lacet exprimés en radians.
  - **<geometry>** Cette partie suit la même description que pour l'élément **visuel** décrit plus haut.

7.2.3 Résolution recommandée pour les mailles

Pour la détection de collision utilisant les paquets ROS de planification de mouvements le moins de face possible est recommandé pour les mailles de collision qui sont spécifiées dans l'URDF (idéalement moins que 1000). Si possible, l'approximation des éléments pour la collision avec des primitives géométriques est encouragée.

7.2.4 Elements multiples des corps pour la collision

Il a été décidé que les fichiers URDFs ne devait pas accepter des multiples groupes d'éléments de collision pour les corps, et cela même s'il y a des applications pour cela. En effet l'URDF a été conçu pour ne représenter que les propriétés actuelles du robot, et non pas pour des algorithmes extérieurs de détection de collision. Dans l'URDF les éléments **visual** doivent être le plus précis possible, les éléments de **collision** doivent être toujours une approximation la plus proche, mais avec bien moins de triangles dans les mailles.

Si des modèles de collision moins précis sont nécessaires pour le control ou la détection de collision il est possible de placer ces mailles/géométries dans des éléments XMLs spécialisés. Par exemple, si vos contrôleurs ont besoin de représentations particuliérement simplifiées, il est possible d'ajouter la balise **<collision_checking>** après l'élément **<collision>**.

Voici un exemple :

```
40  <link name="torso">
41     <visual>
42       <origin rpy="0 0 0" xyz="0 0 0"/>
```

```
      <geometry>
        <mesh filename="package://robot_description/meshes/base_link.DAE"/>
      </geometry>
    </visual>
    <collision>
      <origin rpy="0 0 0" xyz="-0.065 0 0.0"/>
      <geometry>
        <mesh filename="package://robot_description/meshes/base_link_simple.DAE"/>
      </geometry>
    </collision>
    <collision_checking>
      <origin rpy="0 0 0" xyz="-0.065 0 0.0"/>
      <geometry>
        <cylinder length="0.7" radius="0.27"/>
      </geometry>
    </collision_checking>
    <inertial>
      ...
    </inertial>
  </link>
```

Le parseur URDF ignorera ces éléments spécialisés et un programme particulier et spécialisé peut parser le XML pour obtenir cette information.

7.3 Transmission

Un élément de transmission est une extension du modèle URDF de description des robots qui est utilisé pour décrire la relation entre un actionneur et un joint. Cela permet de modéliser des engrenages ou des mécanismes parallèles. Une transmission transforme des variables d'efforts et de flots de telle sorte que leur produit (la puissance) reste constante. Plusieurs actionneurs peuvent être liés à de multiples joints à travers une transmission complexe.

Voici un exemple d'un élément de transmission :

```
<transmission name="simple_trans">
  <type>transmission_interface/SimpleTransmission</type>
  <joint name="foo_joint">
    <hardwareInterface>EffortJointInterface</hardwareInterface>
  </joint>
  <actuator name="foo_motor">
    <mechanicalReduction>50</mechanicalReduction>
    <hardwareInterface>EffortJointInterface</hardwareInterface>
  </actuator>
  </transmission>
```

7.3.1 Attributs de transmission

L'élément de transmission a un attribut :

— **name** (requis)
Spécifie le nom unique d'une transmission.

7.3.2 Eléments de transmission

La transmission a les éléments suivants :

— **<type>** (une occurence)
Spécifie le type de transmission.

— **<joint>** (une ou plusieurs occurences)
Un joint auquel la transmission est connectée. Le joint est spécifié par l'attribut **name**, et les sous-éléments suivants :

  — **<hardwareInterface>** (une ou plusieurs occurences)
  Spécifie une interface matérielle supportée (par exemple **EffortJointInterface** ou **PositionJointInterface**). Notons que la valeur de cet élément doit être **EffortJointInterface** quand cette transmission est chargée dans Gazebo et **hardware_interface/EffortJointInterface** quand cette transmission est chargée dans RobotHW.

— **<actuator>** (une ou plusieurs occurences)
Il s'agit de l'actionneur auquel la transmission est connectée. L'actionneur est spécifié son attribut **name** et les sous éléments suivants :

— **<mechanicalReduction>** (optionel)
Spécifie la réduction mécanique de la transmission joint/actionneur. Cette balise n'est pas nécessaire pour toutes les transmissions.
— **<hardwareInterface>** (optionel) (une ou plusieurs occurences) Spécifie une interface matérielle supportée. Notons que la balise **<hardwareInterface>** ne doit être spécifiée que pour des releases précédent Indigo. L'endroit pour utiliser cette balise est la balise **<joint>**.

7.3.3 Notes de développement

Pour le moment seul le projet **ros_control** utilise ces éléments de transmissions. Développer un format de transmission plus complet qui est extensible à tous les cas est quelque chose de difficile. Une discussion est disponible ici.

7.4 Joint

7.4.1 Elément <joint>

L'élément joint décrit la cinématique et la dynamique d'un joint et spécifie les limites de sécurité. Voici un exemple d'un élément joint :

```
<joint name="my_joint" type="floating">
  <origin xyz="0 0 1" rpy="0 0 3.1416"/>
  <parent link="link1"/>
  <child link="link2"/>

  <calibration rising="0.0"/>
  <dynamics damping="0.0" friction="0.0"/>
  <limit effort="30" velocity="1.0" lower="-2.2" upper="0.7" />
  <safety_controller k_velocity="10" k_position="15" soft_lower_limit="-2.0" soft_upper_limit="0.5" />
</joint>
```

7.4.2 Attributs

L'élément joint a deux attributs :
— **name** (requis)
Spécifie un nom unique pour le joint.
— **type** (requis)
Spécifie le type de joint qui peut prendre une des valeurs suivantes :
— *revolute* - une charnière qui tourne autour d'un axe spécifié par les attributs décrits ci-dessous et les limites inférieures et supérieures.
— *continuous* - une charnière qui tourne autour d'un axe spécifié sans aucune limite.
— *prismatic* - un joint coulissant le long d'un axe spécifié avec un intervalle d'utilisation limité par une valeur inférieure et une valeur supérieure.
— *fixed* - Ce n'est pas réellement un joint car il ne bouge pas. Tous les degrées de liberté sont fixés. Ce type de joint ne requiert pas d'axe et n'a pas d'intervalle d'utilisation spécifié par une valeur supérieure et inférieure.
— *floating* - Ce joint permet des mouvements suivant les 6 degrés de liberté.
— *planar* - Ce joint permet des mouvements dans un plan perpendiculaire à l'axe.

7.4.3 Elements

L'élément **joint** a les sous éléments suivants :
— **<origin>** (optionel : défaut à l'identité si non spécifié)
Il s'agit de la transformée du corps parent vers le corps enfant. Le joint est localisé à l'origine du référentiel enfant.
— **xyz** (optionel : par défaut à zéro)
Représente la position du joint.
— **rpy** (optionel : par défaut au vector zéro)
Représente la rotation autour un axe de rotation fixé : d'abord autour de l'axe de roulis (roll), puis autour de l'axe de tangage (pitch), puis celui de lacet (yaw). Tous les angles sont spécifiés en radians.

— **<parent>** *(requis)*
Le corps parent avec l'attribut nécessaire :
— **link** Le nom du corps qui est le parent du joint dans cet arbre.
— **<child>** *(requis)*
Le corps enfant avec l'attribut nécessaire :
— **link** Le nom du corps qui est l'enfant du joint dans cet arbre.

7.5 Gazebo

7.5.1 Eléments pour les corps/links

Nom	Type	Description
material	value	Le matériel de l'élément visuel
gravity	bool	Utilisation de la gravité
dampingFactor	double	La constante de décroissance de la descente exponentielle pour la vitesse d'un corps - Utilise la valeur et multiplie la vitesse précédente du corps pary $(1 - dampingFactor)$.
maxVel	double	Vitesse maximum contact d'un corps (corrigée par troncature).
minDepth	double	Profondeur minimum autorisée avant d'appliquer une impulsion de correction.
mu1,mu2 fdir1	double string	Coefficients de friction μ pour les directions principales de contact pour les surfaces de contacts comme définies par le moteur dynamique Open Dynamics Engine (ODE) (voir la description des paramètres dans le guide d'utilisateur d'ODE) 3-tuple spécifiant la direction de mu1 dans le repère de référence local des collisions.
kp,kd	double	Coefficient de rigidité k_p et d'amortissement k_d pour les contacts des corps rigides comme défini par ODE (ODE utilisent erp et cfm mais il existe un mapping entre la erp/cfm et la rigidité/amorti)
selfCollide	bool	Si ce paramètre est à vrai, le corps peut entrer en collision avec d'autres corps du modèle.
maxContacts	int	Le nombre maximum de contacts possibles entre deux entités. Cette valeur écrase la valeur spécifiée la valeur de l'élément $max_contacts$ définit dans la section **physics**.
laserRetro	double	Valeur d'intensité retournée par le capteur laser.

7.5.2 Eléments pour les joints

Nom	Type	Description
stopCfm,stopErp	double	Arrêt de la constraint force mixing (cfm) et error reduction parameter (erp) utilisée par ODE
provideFeedback	bool	Permet aux joints de publier leur information de tenseur (force-torque) à travers un plugin Gazebo
implicitSpringDamper, cfmDamping	bool	Si ce drapeau est à vrai, ODE va utiliser ERP and CFM pour simuler l'amorti. C'est une méthode numérique plus stable pour l'amortissement que la méthode par défaut. L'élément **cfmDamping** est obsolète et doit être changé en **implicitSpringDamper**.
fudgeFactor	double	Met à l'échelle l'excés du moteur du joint dans les limites du joint. Doit-être entre zéro et un.

7.6 model_state

Note : Cette partie est un travail en progrès qui n'est pas utilisé à l'heure actuelle. La représentation des configurations pour des groupes de joints peut-être également utilisée en utilisant le format srdf.

7.6.1 Elément <model_state>

Cet élément décrit un état basic du modèle URDF correspondant. Voici un exemple de l'élément state :

```
<model_state model="pr2" time_stamp="0.1">
 <joint_state joint="r_shoulder_pan_joint" position="0" velocity="0" effort="0"/>
 <joint_state joint="r_shoulder_lift_joint" position="0" velocity="0" effort="0"/>
</model_state>
```

7.6.2 Model State

— **<model_state>**

- **model** (requis : string)
  Le nom du modèle dans l'URDF correspondant.
- **timestamp** (optionnel : float, en secondes)
  Estampillage temporel de cet état en secondes.
- **<joint_state>** (optionnel : string).
  - **joint** (requis : string)
    Le nom du joint auquel cet état se réfère.
  - **position** (optionnel : float ou tableau de floats)
    Position pour chaque degré de liberté de ce joint.
  - **velocity** (optionnel : float ou tableau de floats)
    Vitesse pour chaque degré de liberté de ce joint.
  - **effort** (optionnel : float ou tableau de floats)
    Effort pour chaque degré de liberté de ce joint.

7.7 model

Le format de description unifié des robots (URDF pour Unified Robot Description Format) est une spécificiation XML pour décrire un robot. La spécification ne peut pas décrire tous les formats bien que celui-ci ait été conçu pour être le plus général possible. La limitation principale est que seule des structures en arbre peuvent être représentée ce qui ne permet pas de représenter des robots parallèles. De plus les specifications supposent que le robot est constitué de corps rigides connectés par des joints, les éléments flexibles ne sont pas supportés. La spécification couvre :

- La description cinématique et dynamique du robot
- La représentation visuelle du robot
- Modèle de collision du robot

La description d'un robot est constitué d'un ensemble de corps, et de joints connectant tous les corps ensembles. La description typique d'un robot ressemble donc à :

```
<robot name="pr2">
  <link> ... </link>
  <link> ... </link>
  <link> ... </link>

  <joint>  ....  </joint>
  <joint>  ....  </joint>
  <joint>  ....  </joint>
</robot>
```

Vous pouvez constater que la racine du format URDF est l'élément **<robot>**.

8. Simulation

8.1 Introduction

La simulation dans le contexte de la robotique permet en général de vérifier qu'un logiciel ou plus généralement une application distribuée, fait ce qui a été spécifié [10]. Le simulateur dans ce cas va remplacer le monde extérieur, et plus précisément il va calculer les actions des actionneurs et les signaux que devraient percevoir les capteurs. Il faut donc pour cela un simulateur de la dynamique du monde extérieur et un simulateur des phénomènes physique observés par les capteurs. La complexité des simulateurs vient du fait qu'il y a toujours une différence entre le monde réel et le monde simulé. Suivant le degré de réalité souhaité, il peut être nécessaire d'utiliser des algorithmes différents. Par exemple on ne simule pas de la même façon un robot qui évolue dans l'eau, un robot à roues, ou encore un drone.

Il existe cependant une confusion entre des architectures logicielles permettant de simuler

— une application logicielle complète. Elle permet d'utiliser le même logiciel en simulation et dans la réalité.
— la physique d'interaction entre le robot et son environnement

On appelle en général le premier un simulateur système tandis que le deuxième est appelé le moteur dynamique du simulateur. Gazebo et VREP rentre dans la première catégorie. ODE, Bullet et Newton sont dans la deuxième catégorie. Il arrive également que pour contrôler un robot il soit nécessaire de simuler la dynamique entre le robot et son environnement. Il s'agit encore d'une autre catégorie où le rôle essentiel du simulateur est de pouvoir fournir à un instant donné une commande à envoyer aux moteurs. Les formulations mathématiques peuvent être différentes car l'application est différente. Elle peuvent néanmoins partager un grand nombre de points communs.

On peut synthètiser la différence entre les simulateurs systèmes dont les buts sont :

— de simuler les capteurs, les actionneurs et l'environnement,
— pouvoir aller de la simulation au robot en minimisant les modifications,
— représenter de façon précise la réalité (grâce à un simulateur dynamique),
— maintenir la stabilité du calcul

tandis que les simulateurs de contrôle sont généralement prévus pour :

— le temps réel,
— capturer la dynamique principale du robot,
— être intégré dans la boucle de contrôle

Exemples de simulateurs systèmes : Gazebo, Stage, Morse, OpenHRP. Exemples de simulateurs de contrôle : MuJoCo, RBDL, Pinocchio, Robotran.

8.2 Algorithme général d'un simulateur système

Data : Modèle du robot, modèle du monde, état initial du robot et du monde
Result : Etat du monde, sortie des capteurs
Initialisation ;
while *La simulation n'est pas finie* **do**
  for *chaque contrôleur c* **do**
    Lire les entrées capteurs pour le contrôleur *c*. Calculer la commande du contrôleur *c*.
  end
  Calculer un pas de la dynamique. Récupérer l'état du monde. Afficher l'état du monde. **for** *chaque contrôleur c* **do**
    Appliqué la commande du contrôleur *c*.
  end
end

8.3 Algorithme général d'un moteur dynamique

Il existe de nombreuses façons de poser le problème [3] mais on trouve l'algorithme général suivant :
Data : Modèle du robot et du monde, état courant du robot et du monde, paire de collisions, Force externes
Result : Position du robot et du monde
En utilisant la commande du contrôleur ;
Calcul de la position des corps par intégration (par exemple en utilisant Runge-Kutta 4-5) de la dynamique des corps ;
Détection des collisions ;
Calcul des forces de contacts ;
(voir le rigid body pipeline 1 dans [6]).

Par exemple le simulateur système Gazebo et Morse utilisent tous les deux ODE pour la simulation. Mais Gazebo peut également utiliser 3 autres moteurs dynamiques : Bullet [6], DART [11] et Simbody [13].

8.3.1 Problème 1 : Realité

La simulation de l'impact est très souvent difficile à effectuer de façon précise. Par exemple dans [1] la simulation du franchissement d'obstacles d'un robot humanoïde avec OpenHRP 2.x a fourni un impact de 800 N. Cependant sur le robot réel l'impact était de 1200 N qui aurait pu mener à la perte des capteurs de force localisés dans les chevilles du robot. La version d'OpenHRP 3.x a permis une meilleur simulation dynamique permettant de mieux discriminer les mouvements menant à des impacts trop violents des mouvements admissibles. La précision de la simulation nécessite cependant plus de calcul et donc prend plus de temps.

D'une manière générale il est difficile d'avoir un haut niveau de fiabilité en simulation. La compliance des actionneurs et des structures mécaniques est souvent difficile à réaliser car elle s'effectue à des fréquences nécessitant une très grande précision de simulation. La simulation de la vision est également souvent nécessaire afin de pouvoir simuler des algorithmes de traitement d'images.

8.3.2 Problème 2 : Flexibilité logicielle

Du point de vue logiciel on souhaiterait pouvoir passer de la simulation système du robot sans aucune modification ou simplement à travers de fichiers de configuration. Il est également souhaitable de pouvoir changer des parties du simulateur par exemple le moteur dynamique, afin de pouvoir faire des simulations dans des modes différents. Pour cela il est nécessaire d'avoir un middelware suffisamment puissant et pouvoir simuler les capteurs.

IV Introduction à ROS-2

9 Les paquets ROS 2 89

9.1 Configuration de l'environnement
9.2 Structure générale des paquets ROS
9.3 Ligne de commande pour inspecter les paquets
9.4 Création d'un paquet
9.5 Description des paquets : le fichier **package.xml**
9.6 Compilation des paquets humble
9.7 Chaînage de workspace

10 Graphe d'application avec ROS-2 95

10.1 La commande ros2
10.2 Turtlesim et rqt
10.3 Node
10.4 Topic
10.5 Service
10.6 Paramètres
10.7 Actions
10.8 rqt_console
10.9 Bag : fichier de données
10.10 Launch : démarrer une application
10.11 Création de messages et de services

11 Ecrire des nodes ROS-2 123

11.1 Topics
11.2 Services

9. Les paquets ROS 2

Le système de gestion des paquets ROS est probablement un élèment clé de la réussite de ROS. Il est bien plus simple que le système de gestion des paquets par Debian. La contre-partie est que ce système ne fonctionne pas au-delà de l'ecosystème ROS.

9.1 Configuration de l'environnement

9.1.1 Initialiser les variables d'environnement

Afin de pouvoir utiliser ROS il est nécessaire de configurer son environnement shell. Sous linux, bash est le shell le plus utilisé. Il suffit donc d'ajouter la ligne suivante au fichier *.bashrc*

```
source /opt/ros/humble/setup.bash
```

Pour créer son espace de travail il faut d'abord créer le répertoire associé

```
mkdir -p ~/dev_ws/src
cd ~/dev_ws/src
```

Le tutoriel associé est : Tutorial Installing and Configuring Your ROS Environment : `https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Configuring-ROS2-Environment.html`

Une fois le fichier **setup.bash** sourcé on peut trouver deux variables d'environnement trés importantes :

```
  env | grep ROS
```

Le résultat est :

```
ROS_VERSION=2
ROS_PYTHON_VERSION=3
ROS_LOCALHOST_ONLY=0
ROS_DISTRO=humble
```

La variable d'environnment ROS_PACKAGE_PATH n'existe plus, elle est remplacée par AMENT_PREFIX_PATH.

Note 9.1 Il est impératif si vous souhaitez pouvoir travailler sur le même réseau que d'autres utilisateurs de ROS de changer les variables :
export ROS_DOMAIN_ID=11
export ROS_LOCALHOST_ONLY=1
Le mieux étant bien sûr de modifier le fichier *.bashrc* ■

9.2 Structure générale des paquets ROS

Pour l'organisation des paquets ROS une bonne pratique consiste à les regrouper par cohérence fonctionnelle. Par exemple si un groupe de roboticiens travaille sur des algorithmes de contrôle, il vaut mieux les regrouper ensemble. Il est possible de faire cela sous forme d'un espace de travail (appelé également *workspace*). La figure Fig.22.2 représente une telle architecture. Dans un workspace, il faut impérativement créer un repertoire **src** dans lequel on peut créer un repertoire par paquet qui contient les fichiers d'un paquet.

Par exemple le paquet **package_1** contient au minimum deux fichiers :

— CMakeLists.txt : Le fichier indiquant comment compiler et installer le paquet
— package.xml : Le fichier ROS décrivant l'identité du paquet et ses dépendences.

Il suffit de mettre des paquets dans le même répertoire sous le répertoire **src** et sans aucun autre fichier pour faire des regroupements. Dans la figure Fig.22.2 on trouve par exemple le répertoire **meta_package_i** qui regroupe les paquets de **meta_package_i_sub_0** à **meta_package_i_sub_j**. On trouve par exemple des projets sous forme de dépôts qui regroupe de nombreux paquets légers ou qui contiennent essentiellement des fichiers de configuration. C'est par exemple le cas pour les dépots qui contiennent les paquets décrivant un robot. C'est le cas du robot TIAGO de la société PAL-Robotics https ://github.com/pal-robotics/tiago_robot.

Au même niveau que **src**, la plomberie de ROS va créer trois répertoires : **build**, **log**, **install**.

Le répertoire **build** est utilisé pour la construction des paquets et à ce titre contient tous les fichiers objets. Le répertoire **install** ne contient que les fichiers qui sont explicitement installés à travers les directives d'installation spécifiées dans le fichier **CMakeLists.txt** des paquets. Le répertoire **log** contient les fichiers des sorties des opérations de compilation et d'installation.

9.3 Ligne de commande pour inspecter les paquets

On peut afficher la liste des paquets disponibles en utilisant la commande :

```
ros2 pkg list
```

Il est cependant possible d'obtenir la liste des exécutables d'un paquet avec :

```
ros2 pkg executables package_name
```

On peut également obtenir le préfixe d'un paquet avec :

```
ros2 pkg prefix package_name
```

On peut également obtenir le fichier package.xml d'un paquet avec :

```
ros2 pkg xml package_name
```

9.4 Création d'un paquet

Pour la création d'un paquet il faut donc se placer dans le répertoire **src** de l'espace de travail choisi. Dans la suite on supposera que l'espace de travail est le répertoire **dev_ws**.

```
cd ~/dev_ws/src
```

Pour créer un nouveau package et ses dépendences il faut utiliser la commande **dev_create_pkg** en spécifiant le nom du paquet et chacune des dépendences.

```
ros2 pkg create --build-type ament_cmake --license Apache-2.0 --node-name my_node beginner_tutorials
ros2 pkg create [package name] [depend1] [depend2] [depend3]
```

Les dépendences sont stockées dans le fichier **package.xml**.

```
cd beginner_tutorials
cat package.xml
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="h
ttp://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>beginner_tutorials</name>
  <version>0.0.0</version>
  <description>TODO: Package description</description>
  <maintainer email="ostasse@todo.todo">ostasse</maintainer>
```

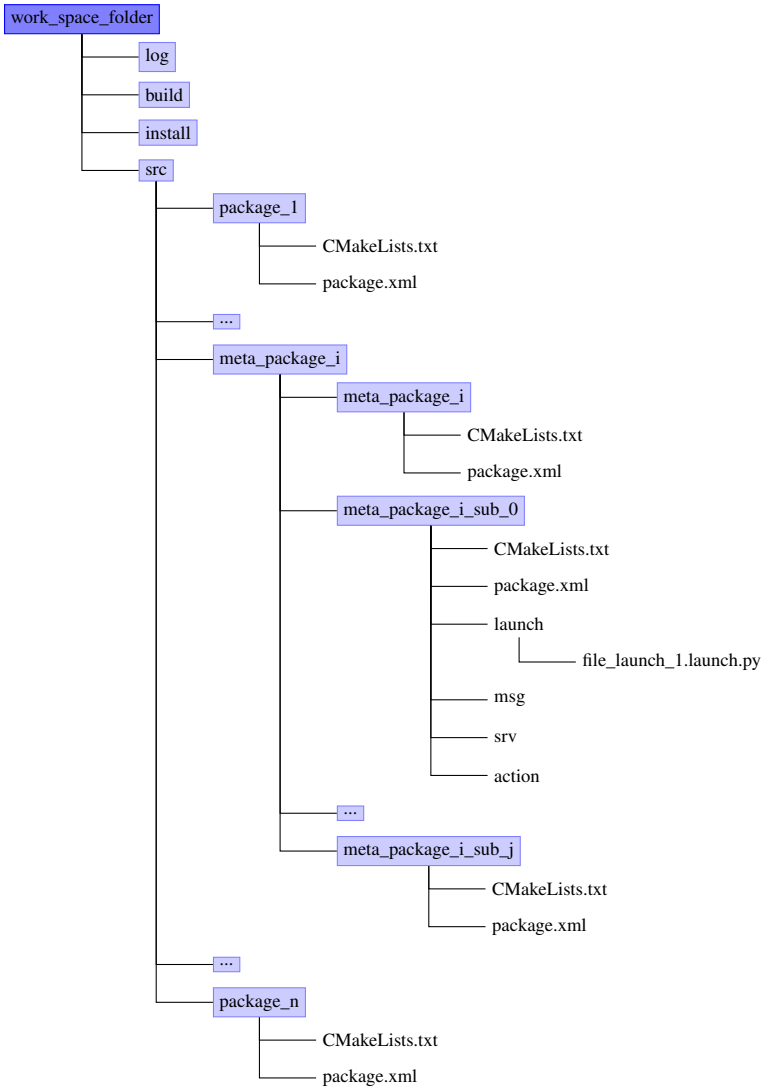


FIGURE 9.1 – Organisation de l'espace de travail : les paquets sources n'ont pas de structure hiérarchique (ou à un seul niveau lorsque les dépôts contiennent plusieurs paquets ROS).

```
  <license>Apache-2.0</license>

  <buildtool_depend>ament_cmake</buildtool_depend>

  <test_depend>ament_lint_auto</test_depend>
  <test_depend>ament_lint_common</test_depend>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

9.5 Description des paquets : le fichier package.xml

Jusqu'à ROS groovy les paquets devait inclure un fichier de description appelé **manifest.xml**. Depuis l'introduction de **catkin** le fichier qui doit être inclus est **package.xml**.

Il existe trois versions du format spécifié dans le champ <package>. Pour les nouveaux paquets, le format recommandé est le format 3.

> **Note 9.2** Le format 3 du fichier package.xm est défini plus en détail dans la REP 0149 Le format 2 est définit dans la REP 0140 Le format 1 est définit dans la REP 0127 ■

```
<package format="3">
</package>
```

Il y 5 champs nécessaires pour que le manifeste du paquet soit complet :

— <name> : Le nom du paquet.
— <version> : Le numéro du version du paquet.
— <description> : Une description du contenu du paquet.
— <maintainer> : Le nom de la personne qui s'occupe de la maintenance du paquet ROS.
— <license> : La license sous laquelle le code est publié.

Les licenses les plus utilisées sous ROS sont Apache-2.0 (ROS2) ou BSD (ROS1) car elles permettent d'utiliser ces codes également pour des applications commerciales.

Il est important de faire attention au contexte dans lequel le code peut-être licensié notamment dans un cadre professionnel.

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/
    XMLSchema"?>
<package format="3">
  <name>beginner_tutorials</name>
  <version>0.0.0</version>
  <description>TODO: Package description</description>
  <maintainer email="olivier.stasse@laas.fr">ostasse</maintainer>
  <license>TODO: License declaration</license>
```

Le site web du paquet peut-être indiqué dans le champ <url>. C'est l'endroit où on trouve généralement la documentation.

```
  <!-- Url tags are optional, but mutiple are allowed, one per tag -->
  <!-- Optional attribute type can be: website, bugtracker, or repository -->
  <!-- Example: -->
  <!-- <url type="website">http://wiki.ros.org/beginner_tutorials</url> -->
```

Certains paquets servent à utiliser des logiciels tierces dans le cadre de ROS. Dans ce cas le paquet ROS encapsule ce logiciel tierce. C'est dans le champ <author> que l'on peut spécifier le nom de l'auteur de ce logiciel tierce. Il est possible de mettre plusieurs champs <author>.

```
  <!-- Author tags are optional, mutiple are allowed, one per tag -->
  <!-- Authors do not have to be maintianers, but could be -->
  <!-- Example: -->
  <!-- <author email="jane.doe@example.com">Jane Doe</author> -->
```

Les dépendences sont de 6 ordres :

— build dependencies (format 1 & 2) : spécifie les paquets nécessaires pour construire ce paquet. Il peut s'agir des paquets qui contiennet des en-têtes nécessaires à la compilation.
— build export dependencies (format 2) : spécifie les paquets nécessaires pour construire des librairies avec ce paquet. Notamment lorsque les en-têtes publiques de ce paquet font référence à ces paquets.

— execution dependencies (format 1 & 2) : spécifie les paquets nécessaires pour exécuter des programmes fournis par ce paquet. C'est le cas par exemple lorsque ce paquet dépend de librairies partagées.
— test dependencies (format 1 & 2) : spécifie uniquement les dépendances additionnelles pour les tests unitaires.
— build tool dependencies (format 1 & 2) : spécifie les outils de construction système nécessaires pour le construire. Typiquement le seul outil nécessaire est catkin. Dans le cas de la compilation croisée les outils sont ceux de l'architecture sur laquelle la compilation est effectuée.
— doc dependencies (format 2) : spécifie les outils pour générer la documentation.

Pour notre exemple voici les dépendences au format 3 avec ROS humble exprimées dans le fichier **package.xml**.

```
  <buildtool_depend>ament_cmake</buildtool_depend>

  <test_depend>ament_lint_auto</test_depend>
  <test_depend>ament_lint_common</test_depend>
```

Il est possible également de spécifier des dépendences à la documentation, des conflits, et le fait que le paquet remplace un paquet plus ancien

```
  <doc_depend>doxygen</doc_depend>

  <conflict>alternative_implementation</conflict>

  <replace>my_old_package</replace>
```

Lorsque l'on souhaite groupé ensemble des paquets nous avons vu qu'il était possible de créer un **metapackage**. Ceci peut-être accompli en créant un paquet qui contient dans le tag suivant dans la section export :

```
  <export>
    <metapackage />
  </export>
```

Les metapackages ne peuvent avoir des dépendences d'exécution que sur les paquets qu'il regroupe.

Enfin pour exporter les objects construits par le paquet la dépendance suivante est indiquée :

```
  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

9.6 Compilation des paquets humble

La compilation de tous les paquets s'effectue avec une seule commande. Cette commande doit être exécutée dans le répertoire du workspace. C'est cette commande qui déclenche la création des répertoires **build** et **install**.

```
cd ~/dev_ws/
colcon build
```

Note 9.3 Contrairement à ROS1 **colcon build** effectue une installation automatiquement. ■

Les répertoires **build** et **install** étant créés automatiquement ils peuvent être effacés sans problème tant que le fichier **CMakeLists.txt** est cohérent avec l'environnement de programmation. Il est indispensable de le faire si le workspace a été déplacé sur le disque. En ROS 2 le répertoire **devel** n'existe plus.

Il est maintenant possible de

9.7 Chaînage de workspace

Lorsque la commande **colcon build** est utilisée celle-ci crée un fichier nommé **setup.bash** dans le répertoire **install**. Ce fichier une fois sourcé permet ensuite de travailler dans le workspace en initialisant

correctement toutes les variables d'environnements. Un point important à noter est que ces variable d'environnements peuvent déjà avoir une valeur lors de l'appel à **colcon build** . Cette valeur est utilisée lors de la création du fichier **setup.bash**.

Ceci permet de faire du chaînage de workspace.

Par exemple on souhaite créer un workspace nommé **ping_ws** qui dépend du workspkace **pong_ws**. Avant de créer le répertoire **install** il faut s'assurer que la variable d'environnement **AMENT_PREFIX_PATH** ne contient que la valeur

```
env | grep AMENT
AMENT_PREFIX_PATH=/opt/ros/humble
```

Une fois que la commande :

```
colcon build
```

a été utilisée, on peut sourcer le fichier **pong_ws/install/setup.bash**

On a alors :

```
env | grep ROS
AMENT_PREFIX_PATH=~/pong_ws/install/cpp_pubsub:/opt/ros/humble
```

Tous les répertoires des paquets de **pong_ws** se trouvent dans la variable **AMENT_PREFIX_PATH**.

On peut donc faire maintenant un workspace **ping_ws** qui utilise **pong_ws** on peut alors après avoir sourcer le fichier **setup.bash** du workspace **pong_ws** faire soit **colcon build**. On a alors :

```
env | grep ROS
AMENT_PREFIX_PATH=~/ping_ws/install/ping_package:~/pong_ws/install/cpp_pubsub:/opt/ros/melodic/share
```

10. Graphe d'application avec ROS-2

Dans ce chapitre nous allons introduire les changements de concepts liés à ROS-2. La différence essentielle du point de vue de l'utilisateur est la disparition du concept de master. Il n'y donc plus d'annuaire d'objets. Les noeuds se découvrent les uns les autres en suivant un protocole assez similaire à celui d'Internet.

On retrouve de nombreux concepts similaires à ROS-1 :

— les nodes
— les topics
— les services
— les actions
— les interfaces

Il y a toutefois des modifications dans la gestion des paramètres qui n'existe plus maintenant que comme des paramètres rattachés à un node.

D'une manière générale tous les concepts similaires à ROS-1 sont atteignables via l'utilisation de la commande *ros2*.

Les actions sont cependant bien mieux explicitées dans le monde ROS-2. On peut également noter que de manière plus cohérentes tous les types sont regroupés sous le nom d'interface. C'est cohérent avec l'usage d'IDL (Interface Description Language) pour définir les types, les objets dans les languages gérés par les différentes implémentations de DDS.

10.1 La commande ros2

Elle permet d'accéder à un certain nombre d'objets :

— action
— bag
— component
— control
— daemon
— doctor
— interface
— launch
— lifecycle
— multicast

— node
— param
— pkg
— run
— security
— service
— topic
— wtf

10.2 Turtlesim et rqt

Cette section correspond au didacticiel Introduction à Turtlesim. Turtlesim est un petit simulateur pour apprendre ROS-2. Il illustre les concepts de ROS-2 à un niveau très simple pour ensuite les utiliser sur un robot ou un simulateur plus complet.

rqt est une interface graphique pour ROS-2. Tout ce que fait rqt peut-être fait par des commandes en ligne. Mais il fournit un moyen d'utiliser les éléments ROS-2 plus facilement et plus amicale pour les utilisateurs.

Le didacticiel suppose que l'environnement est déjà configuré correctement.

10.2.1 Utilisation de turtlesim

10.2.1.1 Vérification de l'installation de turtlesim

On peut vérifier que le paquer **turtlesim** est installé en utilisant la commande suivante :

```
ros2 pkg executables turtlesim
```

Ce qui donne :

```
turtlesim draw_square
turtlesim mimic
turtlesim turtle_teleop_key
turtlesim turtlesim_node
```

L'installation du package turtlesim avec linux s'effectue avec :

```
sudo apt update

sudo apt install ros-humble-turtlesim
```

10.2.1.2 Lancer turtlesim

Pour lancer le node **turtlesim** il faut lancer la commande suivante :

```
ros2 run turtlesim turtlesim_node
```

Vous devriez voir la sortie suivante dans le terminal :

```
[INFO] [1608722845.948451886] [turtlesim]: Starting turtlesim with node name /turtlesim
[INFO] [1608722845.985347636] [turtlesim]: Spawning turtle [turtle1] at x=[5,544445], y=[5,544445], theta
    =[0,000000]
```

Ces informations indiquent le nom par défaut de la tortue, i.e. **turtle1**, et les coordonnées où elle apparaît.

Une fenêtre bleue avec une tortue apparaît comme celle affichée dans la figure 10.1. La tortue peut-être différente car elles sont tirées aléatoirement dans un ensemble de tortues.

10.2.1.3 Utiliser turtlesim

Pour démarrrer un node permettant de contrôler la tortue, il faut ouvrir un nouveau terminal avec l'environnement proprement configuré :

```
ros2 run turtlesim turtle_teleop_key
```

Vous devriez avoir maintenant 3 fenêtres ouvertes : un terminal où fonctionne **turtlesim_node**, un autre terminal avec **turtle_teleop_key** et la fenêtre turtlesim. Il est préférable d'arranger les fenêtres avec de façon à voir la fenêtre graphique turtlesim, mais il faut que le focus de votre souris soit sur le terminal de **turtle_teleop_key** de façon à pouvoir contrôler la tortue de **turtlesim**.

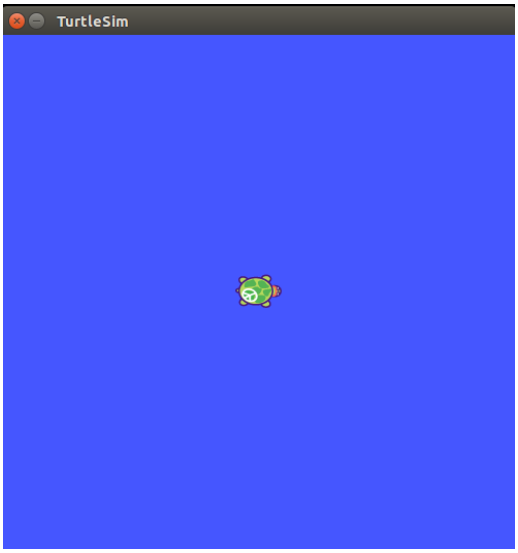


FIGURE 10.1 – Fenêtre résultat de ros2 run turtlesim turtlesim_node

Les flèches du clavier permettent de contrôler la tortue. Elle se déplace sur l'écran utilisant son crayon ("pen") pour tracer le chemin parcouru.

> **Note 10.1** Les flèches ne font bouger la tortue que sur une distance courte. Ce comportement est motivé par le fait que réalistement on ne souhaite pas continuer à émettre une commande si, par exemple, l'opérateur perd la connection avec le robot. ■

Vous pouvez voirs les nodes, les services associés, topics et actions grâce à la commande **list** :

```
ros2 node list
ros2 topic list
ros2 service list
ros2 action list
```

Ces concepts sont développés dans d'autres didacticiels. A ce stade on ne cherche à avoir qu'une vue très générale de turtlesim, la suite utilise rqt (une interface utilisateur graphique pour ROS 2) pour regarder les services.

10.2.2 Installer rqt

Il faut ouvrir un terminal pour installer **rqt** et ses plugins. Sous linux il suffit de faire :

```
sudo apt update
sudo apt install ros-humble-rqt*
```

Pour démarrer rqt :

```
rqt
```

10.2.3 Utiliser rqt

La première fois où cette commande est démarrer la fenêtre peut-être vide. Il suffit de sélectionner dans Plugins > Services > Service Caller de la barre de menu tout en haut.

> **Note 10.2** rqt peut prendre du temps à localiser tous les plugins. Si en cliquant sur **Plugins** vous ne voyez pas **Services**, il faut fermer rqt, et entrer la commande **rqt –force-discover** dans le terminal ■

On peut utiliser le bouton **Refresh** pour être sûr d'avoir la liste de tous les services du node turtlesim.

Il faut cliquer sur la liste déroulante **Service** pour voir tous les services de turtlesim, et sélectionner le service **/spawn**.

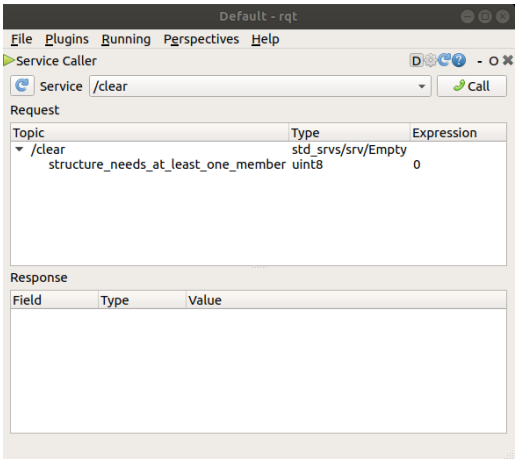


FIGURE 10.2 – Fenêtres correspondant à **rqt** avec Services

10.2.4 Essayer le service spawn

Nous allons utiliser le service **/spawn**. Il est possible de deviner qu'à partir du nom, le service **/spawn** va créer une autre tortue dans la fenêtre graphique.

Il faut donner à la nouvelle tortue un nom unique, comme **turtle2** en double cliquant entre les quotes simples dans la colonne **Expression**. Cette expression correspond à la valeur de la variable **name** du service qui est de type **string**.

Il faut également entrer des coordonnées pour la tortue à créer, comme **x = 1.0** et **y = 1.0**.

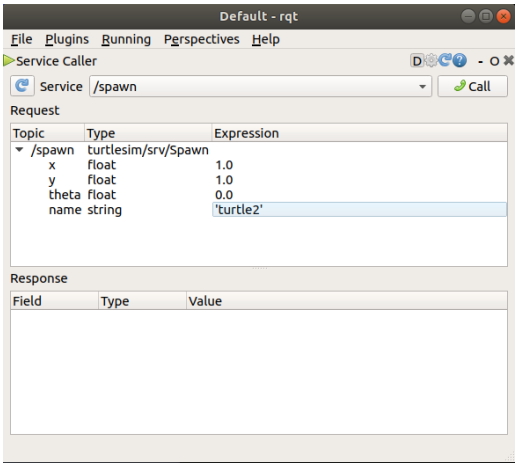


FIGURE 10.3 – Valeurs à spécifier pour l'appel au service **spawn** dans l'interface **rqt**

Note 10.3 Si vous essayez de créer une tortue avec le même nom qu'une tortue existante, par exemple le nom par défaut **turtle1**, vous obtiendrez un message d'erruer dans le terminal dans lequel est lancé **turtlesim_node** :

```
[ERROR] [turtlesim]: A turtle named [turtle1] already exists
```

Pour lancer la tortue turtle2, il faut appeller le service en cliquant sur le bouton **Call** en haut à droite de la fenêtre graphique rqt.

Vous verrez une nouvelle tortue (avec un choix aléatoire) aux coordonnées fixées en x et y.

Si vous rafraichissez la liste de services dans rqt, vous verrez qu'il y a maintenant des services liés aux nouvelles tortues **/turtle2/...** en plus de celles de **/turtle1...**.

10.2.5 Essayer le service set_pen

Essayons de changer la trace de la tortue en utilisant le service **set_pen**.

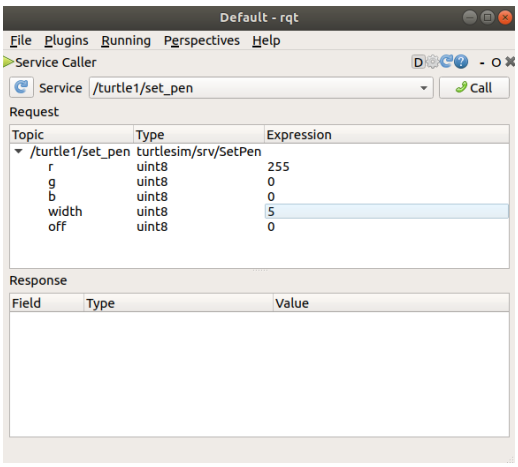


FIGURE 10.4 – Fenêtre correspondant au service **set_pen**

Les valeurs pour **r**,**g** et **b** sont comprises entre 0 et 255, définissent la couleur de la trace de la tortue turtle1 et **width** sa largeur.

Pour avoir la tortue turtle1 tracer une ligne rouge très visible, il faut changer la valeur **r** à 255 et la valeur **width** à 5. Il ne faut pas oublier d'appeller le service après avoir mise à jour les valeurs.

Il faut maintenant retourner au terminal où **turtle_teleop_node** est exécuté et appuyer sur les flèches, vous devriez voir que la trace de turtle1 a changé (cf. fig. 10.5)

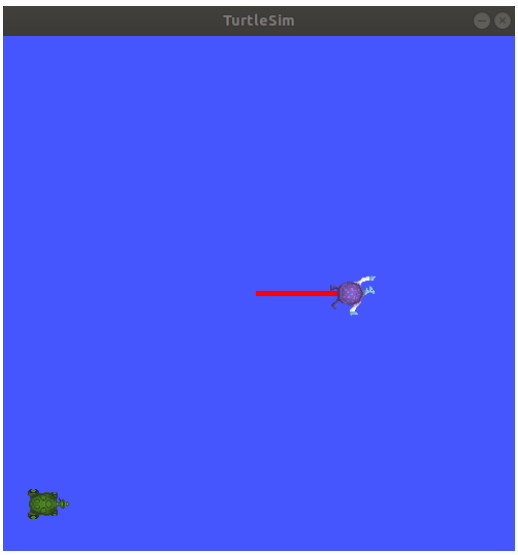


FIGURE 10.5 – Fenêtre montrant l'effet de l'appel au service **set_pen**

Vous avez probablement noté qu'il n'est pas possible de bouger turtle2. Ceci peut-être accompli en redirigeant le topic **cmd_vel** d'un node de type turtle_teleop_key vers la tortue turtle2.

10.2.6 Redirection

Dans un nouveau terminal, il faut lancer la commande suivante :

```
ros2 run turtlesim turtle_teleop_key --ros-args --remap turtle1/cmd_vel:=turtle2/cmd_vel
```

Il est maintenant possible de bouger la tortue turtle2 quand ce terminal a le focus de la souris, et la tortue turtle1 quand c'est l'autre terminal qui est actif.

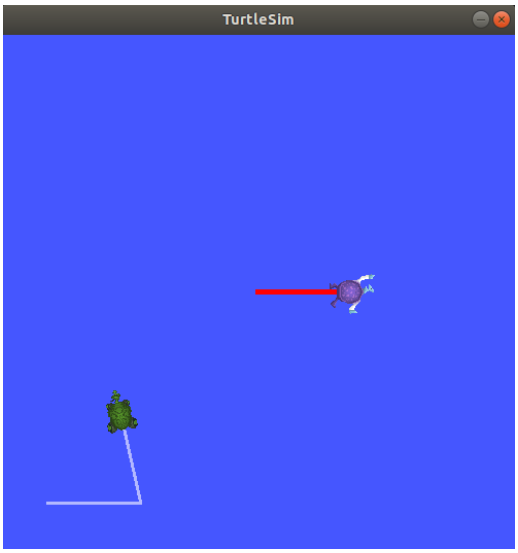


FIGURE 10.6 – Faire bouger la deuxième tortue grâche à la redirection

10.2.7 Arrêt de turtlesim

Pour arrêter la simulation, il suffit de faire la combinaison de touches **Ctrl + C** dans le terminal de **turtlesim_node** et utiliser la touche **q** dans le terminal teleop.

10.3 Node

10.3.1 Définition d'un node

Comme pour ROS-1 on peut définir un node comme étant un processsus associé à l'exécution d'un fichier exécutable dans un paquet ROS. Voyons un peu plus précisément les notions utilisées rapidement dans l'exercice précédent.

10.3.2 ros2 run

Pour lancer ce processus il faut utiliser la commande **ros2 run** qui a déjà été vu précédemment :

```
ros2 run <package_name> <executable_name>
```

Par exemple pour démarrer turtlesim, il a fallu ouvrir un terminal et entrer la commande suivante :

```
ros2 run turtlesim turtlesim_node
```

La fenêtre graphique s'ouvre comme vu dans le didacticiel précédent.
Ici le nom du package est **turtlesim** et le nom de l'éxecutable **turlesim_node**.
On peut donc voir les exécutables dans un paquet ROS, ici `turtlesim`, avec :

```
ros2 pkg executables turtlesim
```

Ce qui donne :

```
turtlesim draw_square
turtlesim mimic
turtlesim turtle_teleop_key
turtlesim turtlesim_node
```

Pour obtenir la liste des nodes actifs on peut utiliser la commande **ros2 node list**.

10.3.3 ros2 node list

La commande **ros2 node list** va vous montrer les noms de tous les nodes actifs. C'est utile lorsque vous souhaitez interagir avec un node, ou si vous souhaitez savoir quels sont les nodes dans un système qui en exécutent beaucoup.

En ouvrant un autre terminal et avec turtlesim toujours actif dans une autre fenêtre, il faut entrer la commande :

```
ros2 node list
```

Ce qui donne :

```
/turtlesim
```

Dans un autre terminal, taper :

```
ros2 run turtlesim turtle_teleop_key
```

Ici, le système cherche le paquet **turtlesim** et l'exécutable **turtle_teleop_key**.

En retournant dans le terminal où vous avez tapé **ros2 node list**, relancez la même commande. On voit maintenant le nom des deux nodes actifs :

```
/turtlesim
/teleop_turtle
```

10.3.4 Redirection

La redirection permet de changer des propriétés du node comme le nom d'un topic, le nom d'un service à des valeurs particuliéres. Dans le didacticiel précédent, la redirection a été utilisé pour que **turtle_teleop_key** puisse contrôler la tortue souhaitée.

Essayons maintenant de changer le nom de notre node **/turtlesim**. Dans un nouveau terminal, il faut lancer la commande suivante :

```
ros2 run turtlesim turtlesim_node --ros-args --remap __node:=my_turtle
```

Comme on exécute une deuxième fois le node turtlesim_node, une autre fenêtre graphique s'ouvre. Toutefois, en retournant sur le terminal où on a exécutée la commande **ros2 node list**, on peut voir les trois noms suivants :

```
  /my_turtle
  /turtlesim
  /teleop_turtle
```

10.3.5 ros2 node info

Maintenant que vous connaissez les noms de vos nodes, vous pouvez avoir accés à plus d'information à leur propos avec :

```
ros2 node info <node_name>
```

Pour obtenir des informations sur votre dernier node **my_turtle** :

```
ros2 node info /my_turtle
```

ce qui done :

```
There are 2 nodes in the graph with the exact name "/turtlesim". You are seeing information about only one of them.

/my_turtle
  Subscribers:
    /my_turtle/cmd_vel: geometry_msgs/msg/Twist
    /parameter_events: rcl_interfaces/msg/ParameterEvent
    /turtle1/cmd_vel: geometry_msgs/msg/Twist
  Publishers:
    /my_turtle/color_sensor: turtlesim/msg/Color
    /my_turtle/pose: turtlesim/msg/Pose
    /parameter_events: rcl_interfaces/msg/ParameterEvent
    /rosout: rcl_interfaces/msg/Log
```

```
    /turtle1/color_sensor: turtlesim/msg/Color
    /turtle1/pose: turtlesim/msg/Pose
  Service Servers:
    /clear: std_srvs/srv/Empty
    /kill: turtlesim/srv/Kill
    /my_turtle/set_pen: turtlesim/srv/SetPen
    /my_turtle/teleport_absolute: turtlesim/srv/TeleportAbsolute
    /my_turtle/teleport_relative: turtlesim/srv/TeleportRelative
    /reset: std_srvs/srv/Empty
    /spawn: turtlesim/srv/Spawn
    /turtle1/set_pen: turtlesim/srv/SetPen
    /turtle1/teleport_absolute: turtlesim/srv/TeleportAbsolute
    /turtle1/teleport_relative: turtlesim/srv/TeleportRelative
    /turtlesim/describe_parameters: rcl_interfaces/srv/DescribeParameters
    /turtlesim/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
    /turtlesim/get_parameters: rcl_interfaces/srv/GetParameters
    /turtlesim/list_parameters: rcl_interfaces/srv/ListParameters
    /turtlesim/set_parameters: rcl_interfaces/srv/SetParameters
    /turtlesim/set_parameters_atomically: rcl_interfaces/srv/SetParametersAtomically
  Service Clients:

  Action Servers:
    /my_turtle/rotate_absolute: turtlesim/action/RotateAbsolute
    /turtle1/rotate_absolute: turtlesim/action/RotateAbsolute
  Action Clients:
```

Tutoriel associé : Tutorial Understanding ROS Nodes : `https://docs.ros.org/en/humble/Tutorials/Understanding-ROS2-Nodes.html`

10.3.6 rqt_graph

Pour visualiser le graphe de l'application on peut lancer la commande suivante :

```
rqt_graph
```

Il est également possible d'ouvrir rqt_graph en ouvrant **rqt** et en sélectionnant **Plugins > Introspection > Node Graph**.

Il est possible de voir les nodes et les topics comme dans la figure fig. 10.7. On y voit deux actions à la périphérie du graphe. Lorsque vous passez votre souris sur les topics au centre, des couleurs apparaissent.

Le graphe montre comment le node **/turtlesim** et le node **teleop_turtle** communiquent ensemble à travers un topic. Le node **teleop_turtle** publie des données (les pressions sur le clavier que vous effectuez pour bouger la tortue) sur le topic **/turtle1/cmd_vel**, et le node **/turtlesim** a souscris à ce topic pour recevoir des données.

Les mécanismes de mise en évidence de rqt_graph sont très utiles pour examiner des systèmes plus complexes avec beaucoup de nodes et des topics connectés de façons très différentes.

Pour démarrer le graphe de l'affichage des topics :

```
ros2 run rqt_plot rqt_plot
```

On obtient le graphe affiché dans la figure Fig.10.8.

10.4 Topic

Les topics sont des données publiées par des noeuds et auquelles les noeuds souscrivent. L'exécutable permettant d'avoir des informations sur les topics est **ros2 topic**.

La liste des commandes en ROS-2 est similaire à celle de ROS-1 :

- bw : Affiche la bande passante prise par le topic
- echo : Affiche le contenu du topic en texte
- hz : Affiche la fréquence de publication du topic
- info : Fournit des informations sur un topic
- list : Affiche la liste des topics actifs
- pub : Publie des données sur le topic
- type : Affiche le type du topic

10.4.1 ros2 topic list

Cette commande affiche la liste des topics.

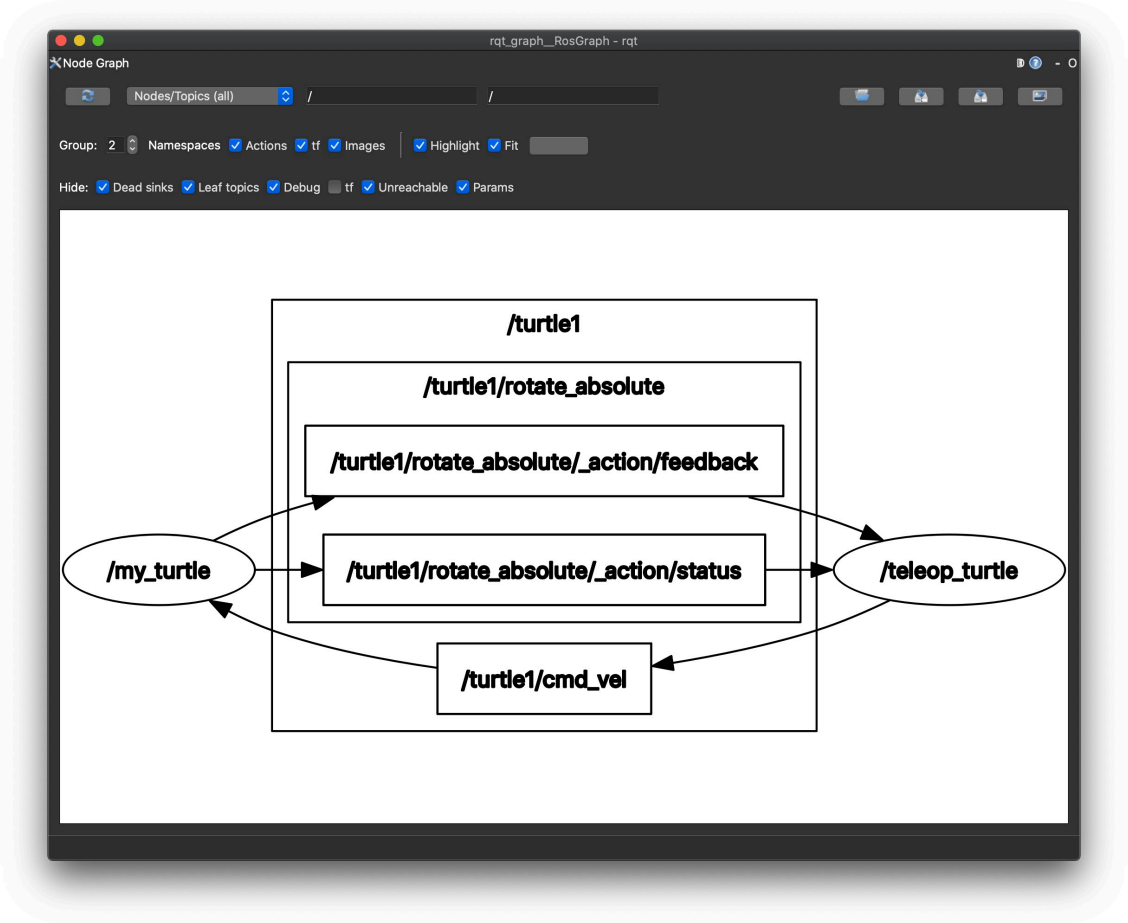


FIGURE 10.7 – Graphe de l'application **turtlesim** et **turtle_teleop_key**

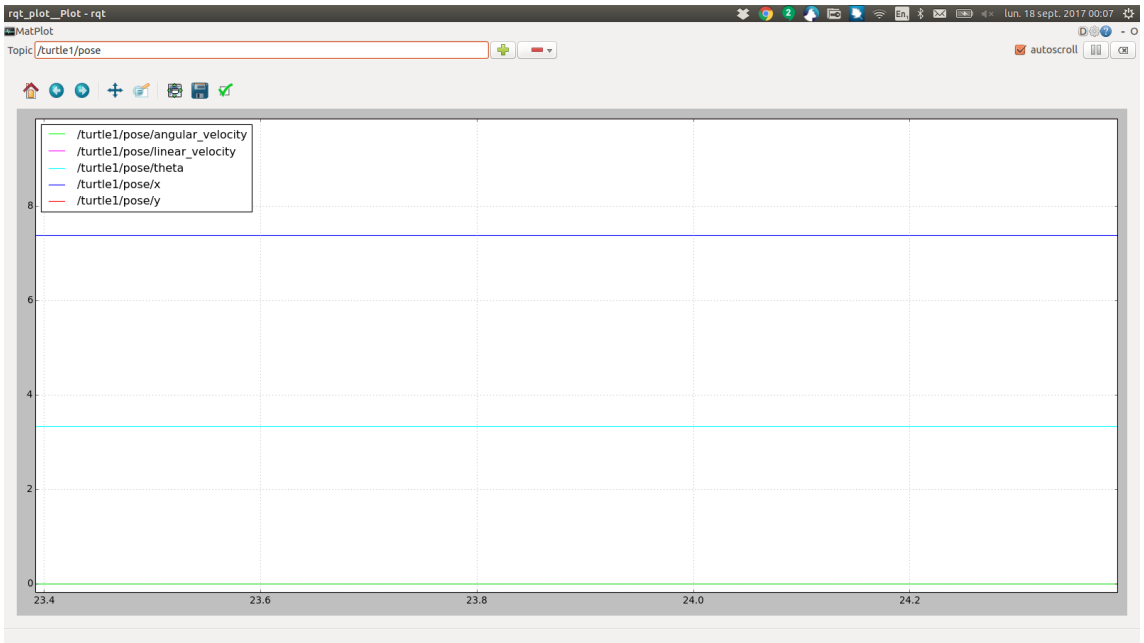


FIGURE 10.8 – Affichage de la position de la tortue dans la fenêtre

```
ros2 topic list
```

Le résultat est le suivant :

```
/parameter_events
/rosout
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
```

La commande suivante

```
ros2 topic list -t
```

va retourner la même liste de topics, mais cette fois avec le type de topic ajouter et entourer de crochets.

```
/parameter_events [rcl_interfaces/msg/ParameterEvent]
/rosout [rcl_interfaces/msg/Log]
/turtle1/cmd_vel [geometry_msgs/msg/Twist]
/turtle1/color_sensor [turtlesim/msg/Color]
/turtle1/pose [turtlesim/msg/Pose]
```

Ces attributs, et plus particulièrement les types, montrent comment les nodes savent qu'ils échanges la même information alors que celle-ci est transmise à travers les topics.

Si vous vous demandez où se trouvent tous ces topics dans rqt_graph, vous pouvez décocher toutes les boîtes sous Hide.

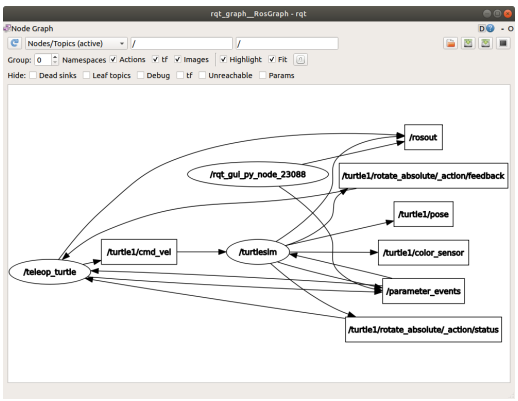


FIGURE 10.9 – Fenêtre de rqt_graph en décochant les boîtes à droite de l'option **Hide**

10.4.2 ros2 topic bw

On obtient la bande passante utilisée par un topic en tapant :

```
ros2 topic bw /turtle1/pose
```

On obtient alors le résultat suivant :

```
39 B/s from 52 messages
  Message size mean: 24 B min: 24 B max: 24 B
38 B/s from 52 messages
  Message size mean: 24 B min: 24 B max: 24 B
```

10.4.3 ros2 topic echo

Pour afficher le contenu d'un topic en fonction du message transmis on peut utiliser la commande :

```
ros2 topic echo <topic_name>
```

Par exemple, puisque nous savons que le topic **/teleop_turtle** publie des données pour **/turtlesim** sur le topic **/turtle1/cmd_vel**, utilisons la commande **echo** pour faire de l'introspection sur ce topic :

```
ros2 topic echo /turtle1/cmd_vel
```

Cette commande ne publiera rien au départ. C'est parce que le topic attend que **/teleop_turtle** publie quelque chose.

Il faut donc mettre le focus sur le terminal où le node **turtle_teleop_key** s'exécute et utiliser les flèches pour bouger la tortue autour. En regardant le terminal où la commande **echo** s'exécute, on peut voir les données être publiées pour chaque mouvement effectué :

```
linear:
  x: 2.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
---
```

On peut alors retourner sur la fenêtre rqt_graph et déselectionner la boîte **Debug**.

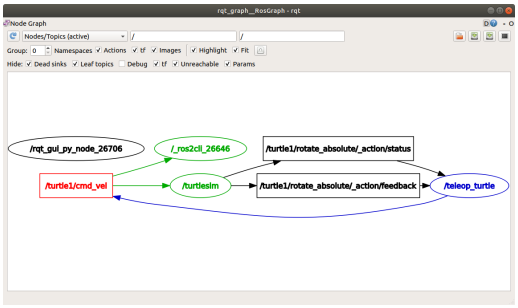


FIGURE 10.10 – Fenêtres correspondant à **rqt_graph** avec le mode Debug

Le node **/_ros2cli_26646** est le node créée par la commande **echo** (le nombre peut-être différent). On peut voir maintenant que le publisher publie des données sur le topic **/turtle1/cmd_vel**, et qu'il y a deux souscripteurs à ce topic.

10.4.4 ros2 topic hz

Cette commande permet d'afficher la fréquence à laquelle les données sont publiées :

```
ros2 topic hz /turtle1/pose
```

avec

```
average rate: 47.815
  min: 0.000s max: 2.621s std dev: 0.10933s window: 576
average rate: 49.036
  min: 0.000s max: 2.621s std dev: 0.10374s window: 640
```

10.4.5 ros2 topic info

Affiche des informations sur un topic c'est à dire le type, le nombre de publishers et le nombre de subscribers. Par exemple :

```
ros2 topic info /turtle1/pose
```

donne :

```
Type: turtlesim/msg/Pose
Publisher count: 2
Subscription count: 0
```

10.4.5.1 ros2 interface show

Les nodes envoient des données par les topics en utilisant les messages. Les publishers et les subscribers doivent envoyer et recevoir les même types de messages pour communiquer.

Les types de topic que nous avons vu précédemment après avoir exécuté **ros2 topic list -t** nous permettent de connaître le type de message a utilisé sur chaque topic. Ainsi le topic **cmd_vel** a le type :

```
geometry_msgs/msg/Twist
```

Ceci spécifie que dans le paquet **geometry_msgs** il y a un **msg** appelé **Twist**.

Il est possible d'exécuter **ros2 interface show <msg type>** sur ce type pour le connaître en détails, spécifiquement la structure des données que le message attend.

```
ros2 interface show geometry_msgs/msg/Twist
```

Le résultat est le suivant :

```
# This expresses velocity in free space broken into its linear and angular parts.
Vector3  linear
Vector3  angular
```

Cette information nous indique le node **/turtlesim** attend un message avec deux vecteurs, **linear** et **angular** de trois éléments chacun. Cela correspond aux données vues passant du node **/teleop_turtle** au node **/turtlesim** avec la commande **echo**.

10.4.5.2 ros2 topic type

Cette commande affiche le message (structure de données au sens de ROS) d'un topic. Par exemple :

```
ros2 topic type /turtle1/pose
```

affiche le type du topic (appelé message) :

```
turtlesim/msg/Pose
```

10.4.5.3 Afficher la structure d'un topic (message)

Pour afficher la structure d'un message il faut utiliser *ros2 interface show* :

```
ros2 interface show turtlesim/msg/Pose
```

ce qui affiche :

```
float32 x
float32 y
float32 theta

float32 linear_velocity
float32 angular_velocity
```

10.4.5.4 ros2 topic pub

Cette commande permet de publier des données sur un topic. Elle suit la structure suivante :

```
ros2 topic pub <topic_name> <msg_type> '<args>'
```

Par exemple si on revient à notre exemple on peut essayer :

```
ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0, y: 0.0, z: 0.0}, angular: {x:
    0.0, y: 0.0, z: 1.8}}"
```

Cette commande demande à la tortue de tourner sur elle-même à la vitesse angulaire de 1.8 rad/s. Grâce à la complétion automatique il est possible de taper uniquement le nom du topic qui étend ensuite le type du topic et la structure de la donnée à transmettre. La durée de la prise en compte de l'information est limitée, on peut utiliser l'option **-r 1** pour répéter l'émission à une fréquence d'1 Hz.

10.5 Service

Les services sont une autre façon pour les noeuds de communiquer entre eux. Les services permettent aux noeuds d'envoyer des *requêtes* et de recevoir des *réponses*. Tandis que les topics permettent aux nodes de souscrire aux flux de données

Dans la suite on supposera que le noeud **turtlesim** continue à fonctionner. Il va servir à illustrer les commandes vues dans ce paragraphe.

La commande **ros2 service** permet d'interagir facilement avec le système client/serveur de ROS appelés *services*. **ros2 service** a plusieurs commandes qui peuvent être utilisées sur les services comme le montre l'aide en ligne :

call : Appelle le service
find : Trouve une liste de services diponible pour un type spécifique
list : Liste des services actifs
type : Affiche le type d'un service

10.5.1 ros2 service list

La commande

```
ros2 service list
```

affiche la liste des services fournis par les nodes de l'application.

```
/clear
/kill
/reset
/spawn
/turtle1/set_pen
/turtle1/teleport_absolute
/turtle1/teleport_relative
/turtlesim/describe_parameters
/turtlesim/get_parameter_types
/turtlesim/get_parameters
/turtlesim/list_parameters
/turtlesim/set_parameters
/turtlesim/set_parameters_atomically
```

On y trouve 9 services fournis par le node **turtlesim** : clear, kill, reset, spawn, turtle1/set_pen, /turtle1/teleport_absolute, /turtle1/teleport_relative, turtlesim/describe_parameters, turtlesim/get_parameters,turtlesim/list_parameters, turtlesim/set_parameters, turtlesim/set_parameters_atomically. Les 6 services relatifs aux paramètres se retrouvent avec tous les nodes.

Les deux services relatifs au node **rosout** : /rosout/get_loggers and /rosout/set_logger_level n'existent plus en ROS-2.

10.5.2 ros2 service type (service)

Il est possible d'avoir le type de service fourni en utilisant la commande :

```
ros2 service type <service>
```

Ce qui donne pour le service clear :

```
ros2 service type /clear
```

Le résultat est le suivant :

```
std_srvs/srv/Empty
```

Ce service est donc vide car il n'y a pas d'arguments d'entrée et de sortie (i.e. aucune donnée n'est envoyée lorqu'une requête est effectuée et aucune donnée n'est transmise lors de la réponse).

10.5.3 ros2 service list -t

Pour voir tous les types des services actifs en même temps, on peut ajouter l'option **–show-types**, dont l'abbréviation est **-t** à la commande **list** :

```
ros2 service list -t
```

Le résultat est le suivant :

```
/clear [std_srvs/srv/Empty]
/kill [turtlesim/srv/Kill]
/reset [std_srvs/srv/Empty]
/spawn [turtlesim/srv/Spawn]
...
/turtle1/set_pen [turtlesim/srv/SetPen]
/turtle1/teleport_absolute [turtlesim/srv/TeleportAbsolute]
/turtle1/teleport_relative [turtlesim/srv/TeleportRelative]
```

10.5.4 ros2 interface show

Pour obtenir la structure de la requête et de la réponse on peut utiliser la commande **interface**. Si on souhaite afficher cette structure pour le service **turtlesim/srv/Spawn** :

```
ros2 interface show turtlesim/srv/Spawn
```

le rérsultat est :

```
float32 x
float32 y
float32 theta
string name
---
string name
```

La requête est donc constituée de trois réels qui sont respectivement la position et l'orientation de la tortue. Le quatrième argument est le nom du node et est optionnel. Le type de la réponse est spécifié après le séparateur définit par "—".

Voyons maintenant comment faire appel à un service.

10.5.5 ros2 service call

Il est possible de faire une requête en utilisant la commande :

```
ros2 service call [service] [args]
```

Pour effacer la fenêtre graphique on va utiliser le service **/clear**. On a vu précédemment que le type est **Empty**. Pour voir sa structure on peut faire :

```
ros2 interface show std_srvs/srv/Empty
```

Le résultat est le suivant :

```
---
```

Comme nous aurions pu le deviner grâce au nom, ceci signifie que le service n'a ni argument d'entrée ni argument de sortie.

On peut donc l'appeller de la manière suivante :

```
ros2 service call /clear
```

Comme on peut s'y attendre les traces de la fenêtre du noeud **turtlesim** est effacé. Considérons maintenant un cas où le service a des arguments en examinant le service **spawn** :

```
ros2 service type /spawn
```

Le résultat est :

```
turtlesim/srv/Spawn
```

Ce service nous permet de lancer une autre tortue, par exemple :

```
ros2 service call /spawn turtlesim/srv/Spawn "{x:0.0, y:0.0, theta: 0.0, name: "my_turtle"}"
```

On trouve maintenant une deuxième tortue dans la fenêtre comme représenter dans la figure Fig.10.11.

10.6 Paramètres

ROS-2 gère les paramètres qui permet de stocker des données. Contrairement à ROS-1 les paramètres ne sont pas centralisés mais répartis sur chacun des nodes.

Les mécanismes de synchronisation des paramètres est complètement différent des topics qui implémente un système de flux de données. Ils servent notamment à stocker : le modèle du robot (**robot_description**), des trajectoires de mouvements, des gains, et toutes informations pertinentes. Les paramètres peuvent se charger et se sauvegarder grâce au format YAML (Yet Another Language). Ils peuvent également être définis en ligne de commande. Le serveur de paramêtre peut stocker des entiers, des réels, des booléens, des dictionnaires et des listes. Les paramètres utilisent le langage YAML pour la syntaxe de ces paramètres. Dans des cas simples, YAML paraît trés naturel.

Les commandes suivantes permettent de gérer les paramètres :

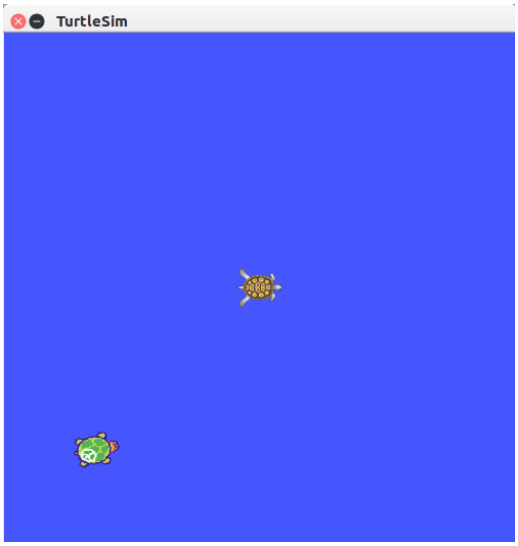


FIGURE 10.11 – Fenêtre résultat de ros2 run call /spawn turtlesim/srv/Spawn "x :0.0, y :0.0, theta : 0.0, name : 'my_turtle'"

delete : Efface un paramètre
describe : Montre les informations décrivant le paramètre
dump : Sauve les paramètres dans un fichier
get : Affiche la valeur d'un paramètre
list : Donne la liste des paramètres
set : Spécifie la valeur d'un paramètre

La suite des commandes suppose que les deux nodes de l'exemple turtlesim sont lancés :

```
ros2 run turtlesim turtlesim_node
ros2 run turtlesim turtle_teleop_key
```

10.6.1 ros2 param list

Il est possible d'avoir la liste des paramètres en tapant la commande :

```
ros2 param list
```

On obtient alors la liste suivante :

```
/teleop_turtle:
  scale_angular
  scale_linear
  use_sim_time
/turtlesim:
  background_b
  background_g
  background_r
  use_sim_time
```

Chaque paramètre est dans le namespace d'un node. On trouve donc deux espaces de noms : `teleop_turtle` et `/turtlesim`.

Les 3 premiers paramètres de `turtlesim` permettent de contrôler la couleur du fond de la fenêtre du node `turtlesim`.

La commande suivante change la valeur du canal rouge de la couleur de fond :

```
ros2 param set /turtlesim background_r 150
```

Le résultat est représenté dans la figure 10.12.

On peut voir la valeur des autres paramètres. Par exemple, pour avoir la valeur du canal vert de la couleur de fond on peut utiliser :

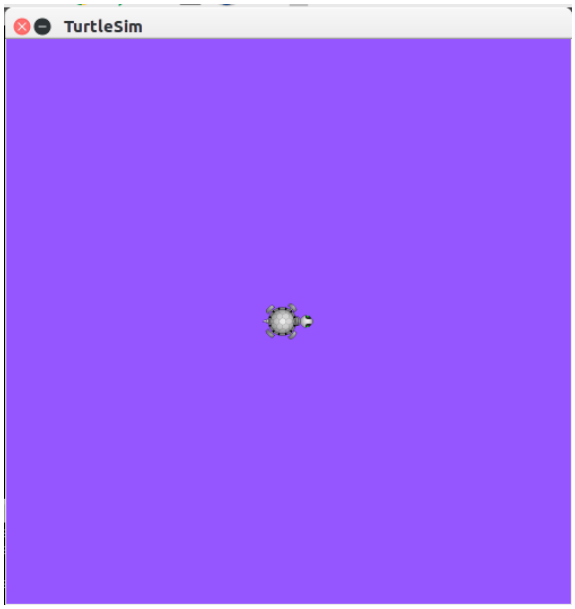


FIGURE 10.12 – Résultat de l'appel **ros2 param set /turtlesim background_r 150**

```
ros2 param set /background_g
```

10.6.2 ros2 param dump

On peut aussi utiliser la commande suivante pour voir toutes les valeurs de paramètres du node `turtlesim` :

```
ros2 param dump /turtlesim > turtlesim.yaml
```

Le résultat est le fichier turtlesim.yaml :

```
turtlesim:
  ros__parameters:
    background_b: 255
    background_g: 86
    background_r: 150
    use_sim_time: false
```

Il est ensuite possible de recharger ces paramètres.

10.6.3 ros2 param load

La structure générale de la command est :

```
ros2 run <package_name> <executable_name>  --ros-args --params-file <file_name>
```

Pour charger le fichier :

```
ros2 run turtlesim turtle_sim_node --ros-args --params-file params.yaml
```

10.7 Actions

Cette section correspond au didacticiel Introduction aux actions

10.7.1 Introduction

Les actions sont l'un des types de communications de ROS-2 et sont utilisées pour des tâches relativement longues. Elles sont constituées de trois parties : un but, un retour d'état, et un résultat.

Les actions sont construites sur les topics et les services. Leurs fonctionnalités sont similaires aux services, excepté que les actions peuvent être annulées. Elles fournissent également un retout constant sur l'état à l'opposé des services qui ne retournent qu'une seule réponse.

Les actions utilisent un modèle client-serveur, similaire au modèle publisher-subscriber. Un node "action client" envoit un but à un node serveur d'action "action server" qui accuse la réception du but et retourne un flux de retour d'état et un résultat.

10.7.2 Préparation

Il faut démarrer les deux nodes de turtlesim : **/turtlesim** et **/teleop_turtle**. Pour cela il faut démarrer un nouveau terminal et démarrer :

```
ros2 run turtlesim turtle_sim_node
```

```
ros2 run turtlesim turtle_teleop_key
```

10.7.3 Utilisation des actions

Quand le node **/teleop_turtle** est lancé, il est possible de voir le message suivant :

```
Use arrow keys to move the turtle.
Use G|B|V|C|D|E|R|T keys to rotate to absolute orientations. 'F' to cancel a rotation.
```

La deuxième ligne correspond aux actions. (La première ligne correspond au topic "cmd_vel" traité dans les didacticiels précédents).

On peut noter qur les lettres **G|B|V|C|D|E|R|T** forment une boîte autour de la lettre **F** sur un clavier US QWERTY. Chaque touche autour de **F** correspond à un orientation pour la tortue de la fenêtre graphique. Par exemple, la lettre **E** fait tourner la tortue vers le coin en haut à gauche.

Il est intéressant de regarder le terminal où le node **/turtlesim** est exécuté. A chaque fois que l'on presse une des touches, un goal est envoyé au serveur d'action (action server) qui fait partir du node **/turtlesim**. Le but est de faire tourner la tortue dans une direction particulière. Un message affichant le résultat du goal s'affiche une fois que la tortue est effectuée :

```
[INFO] [turtlesim]: Rotation goal completed successfully
```

La touche **F** annule le goal en cours d'exécution.

On peut par exemple appuyer sur **C** puis presser la touche **F** avant que la tortue puisse terminer sa rotation. Dans le terminal où le node **/turtlesim** s'exécute, on peut voir le message :

```
[INFO] [turtlesim]: Rotation goal canceled
```

Non seulement il est possible pour le client d'annuler un goal (par une entrée dans teleop) mais le serveur (le node **/turtlesim**) peut également. Quand le service choisit de stopper le goal, on dit qu'il "avorte" le goal.

On peut par exemple appuyer sur **D**, puis la touche **G** avant que la première rotation ne soit terminée. Dans le terminal où le node **/turtlesim** s'exécute on peut voir le message :

```
[WARN] [turtlesim]: Rotation goal received before a previous goal finished. Aborting previous goal
```

Le serveur d'action (action server) choisit d'avorter le premier but car il en reçu un nouveau. Il aurait pu choisir une autre stratégie comme rejecter le nouveau but ou exécuter le second but après que le premier soit finit. On ne peut pas supposer que chaque serveur d'action va choisir d'annuler le but courant quand il en reçoit un nouveau.

10.7.4 ros2 node info

Pour voir les actions du node **/turtlesim**, il faut ouvrir un nouveau terminal et lancer la commande :

```
ros2 node info /turtlesim
```

Cette commande retourne la liste des subscribers, publishers, services, serveurs d'action et clients d'action pour le node **/turtlesim** :

```
/turtlesim
  Subscribers:
    /parameter_events: rcl_interfaces/msg/ParameterEvent
    /turtle1/cmd_vel: geometry_msgs/msg/Twist
  Publishers:
    /parameter_events: rcl_interfaces/msg/ParameterEvent
    /rosout: rcl_interfaces/msg/Log
    /turtle1/color_sensor: turtlesim/msg/Color
    /turtle1/pose: turtlesim/msg/Pose
  Service Servers:
    /clear: std_srvs/srv/Empty
    /kill: turtlesim/srv/Kill
    /reset: std_srvs/srv/Empty
    /spawn: turtlesim/srv/Spawn
    /turtle1/set_pen: turtlesim/srv/SetPen
    /turtle1/teleport_absolute: turtlesim/srv/TeleportAbsolute
    /turtle1/teleport_relative: turtlesim/srv/TeleportRelative
    /turtlesim/describe_parameters: rcl_interfaces/srv/DescribeParameters
    /turtlesim/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
    /turtlesim/get_parameters: rcl_interfaces/srv/GetParameters
    /turtlesim/list_parameters: rcl_interfaces/srv/ListParameters
    /turtlesim/set_parameters: rcl_interfaces/srv/SetParameters
    /turtlesim/set_parameters_atomically: rcl_interfaces/srv/SetParametersAtomically
  Service Clients:

  Action Servers:
    /turtle1/rotate_absolute: turtlesim/action/RotateAbsolute
  Action Clients:
```

On peut noter que l'action **/turtle1/rotate_absolute** pour **/turtlesim** est dans la liste **Action Servers**. Ce qui signifit que **/turtlesim** répond et fournit le retour d'état pour l'action **/turtle1/rotate_absolute**.

Le node **/teleop_turtle** a le nom **/turtle1/rotate_absolute** dans la liste **Action Clients** ce qui signifie qu'il envoit des buts pour cette action.

```
ros2 node info /teleop_turtle
```

ce qui retourne :

```
/teleop_turtle
  Subscribers:
    /parameter_events: rcl_interfaces/msg/ParameterEvent
  Publishers:
    /parameter_events: rcl_interfaces/msg/ParameterEvent
    /rosout: rcl_interfaces/msg/Log
    /turtle1/cmd_vel: geometry_msgs/msg/Twist
  Service Servers:
    /teleop_turtle/describe_parameters: rcl_interfaces/srv/DescribeParameters
    /teleop_turtle/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
    /teleop_turtle/get_parameters: rcl_interfaces/srv/GetParameters
    /teleop_turtle/list_parameters: rcl_interfaces/srv/ListParameters
    /teleop_turtle/set_parameters: rcl_interfaces/srv/SetParameters
    /teleop_turtle/set_parameters_atomically: rcl_interfaces/srv/SetParametersAtomically
  Service Clients:

  Action Servers:

  Action Clients:
    /turtle1/rotate_absolute: turtlesim/action/RotateAbsolute
```

10.7.5 ros2 action list

Pour afficher toutes les actions dans un graphe ROS, il faut exécuter la commande :

```
ros2 action list
```

Ce qui retourne :

```
/turtle1/rotate_absolute
```

C'est la seule action dans le graphe ROS disponible pour le moment. Elle contrôle la rotation de la tortue comme nous l'avons vue précédemment. Il est également possible de savoir qu'il y a un seul action client (qui fait partie de **/teleop_turtle**) et un serveur d'action (qui fait partie de **/turtlesim**) pour cette action en utilisant la commande **ros2 node info <node_name>**.

10.7.5.1 ros2 action list -t

Les actions ont des types, de la même manière que les topics et les services. Pour trouver le type de **/turtle1/rotate_absolute** il faut lancer la commande :

```
ros2 action list -t
```

ce qui retourne :

```
/turtle1/rotate_absolute [turtlesim/action/RotateAbsolute]
```

Dans les crochets à la droite du nom de chaque action (dans ce cas **/turtle1/rotate_absolute**) est le type de l'action : **/turtlesim/action/RotateAbsolute**. Cette information est nécessaire quand il faut exécuter une action à partir de la ligne de commande ou à partir de code.

10.7.6 ros2 action info

Il est possible d'introspecter avec plus de détails l'action **/turtle1/rotate_absolute** avec la commande :

```
ros2 action info /turtle1/rotate_absolute
```

ce qui retourne :

```
Action: /turtle1/rotate_absolute
Action clients: 1
    /teleop_turtle
Action servers: 1
    /turtlesim
```

Ceci nous donne de façon concise les informations obtenues avec la commande **ros2 node info** sur chacun des nodes : Le node **/teleop_turtle** un action client et le node **/turtlesim** est un serveur d'action de l'action **/turtle1/rotate_absolute**.

10.7.7 ros2 interface show

Afin d'exécuter un but d'action (action goal) il faut connaître la structure d'une action. Rappellons qu'il est possible d'identifier le type de l'action **/turtle1/rotate_absolute** lorsque l'on exécute la commande **ros2 action list -t**. Il faut pour cela entrer la commande suivante avec le type de l'action dans le terminal :

```
ros2 interface show turtlesim/action/RotateAbsolute
```

Ce qui retourne :

```
# The desired heading in radians
float32 theta
---
# The angular displacement in radians to the starting position
float32 delta
---
# The remaining rotation in radians
float32 remaining
```

La première section de ce message au-dessus de — est la structure (type de données et nom) de la requête du but. La partie suivante est la structure du résultat. La dernière partie est la structure du retour d'état.

10.7.8 ros2 action send_goal

Il est possible d'envoyer un but d'action (action goal) à partir la ligne de commande avec la syntaxe suivante :

```
ros2 action send_goal <action_name> <action_type> <values>
```

avec **<values>** doit être au format YAML.

En gardant un oeil sur la fenêtre turtlesim, entrer la commande suivante dans un terminal :

```
ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute "{theta: 1.57}"
```

La tortue doit se mettre à tourner, et on doit voir le message suivant dans le terminal :

```
Waiting for an action server to become available...
Sending goal:
   theta: 1.57

Goal accepted with ID: f8db8f44410849eaa93d3feb747dd444

Result:
  delta: -1.568000316619873

Goal finished with status: SUCCEEDED
```

Tous les buts ont un identifiant unique dans le message de retour. Il est possible également de voir le résultat, un champ avec le nom **delta**, qui est le déplacement à la position de départ.

Pour voir le retour de ce but, il suffit d'ajouter **–feedback** à la commande **ros2 action send_goal**.

```
ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute "{theta: -1.57}" --feedback
```

Le terminal va retourner le message :

```
Sending goal:
   theta: -1.57

Goal accepted with ID: e6092c831f994afda92f0086f220da27

Feedback:
  remaining: -3.1268222332000732

Feedback:
  remaining: -3.1108222007751465



Result:
  delta: 3.1200008392333984

Goal finished with status: SUCCEEDED
```

Le retour d'état s'effectue en donnant l'angle en radians qu'il reste à effectuer jusqu'à ce que le but soit atteint.

10.7.9 Résumé

Les actions sont comme des services qui permettent d'exécuter des tâches longues, fournissent un retour régulier, et sont annulables.

Un système robotique utilisent les actions pour de nombreux comportements : la navigation, l'exécution de contrôleurs, la planification de mouvements. Un but d'action peut dire à un robot d'aller vers une position. Tant que le robot navigue vers la position finale, il peut envoyer le long du chemin sa position (i.e. un retour) et finalement un message final lorsqu'il a atteint sa destination.

Turtlesim a un serveur d'action auquel un client d'action peut envoyer des buts pour orienter les tortues. Dans ce didacticiel l' action **/turtle1/rotate_absolute** est analysée pour avoir une meilleure idée de ce que sont les actions et comment elles fonctionnent.

10.8 rqt_console

rqt est une interface d'affichage qui ne fait pas uniquement de la 3D et qui se peuple avec des plugins. Elle permet de construire une interface de contrôle incrémentalement. L'exécutable permettant d'afficher les messages des noeuds de façon centralisé est **rqt_console**.

10.8.1 rqt

Démarrer *rqt* se fait simplement avec :

```
rqt
```

La première fois où cette commande est démarrer la fenêtre peut-être vide. Il suffit de sélectionner dans Plugins > Logging > Console pour obtenir l'équivalent de :

```
ros2 run rqt_console rqt_console
```

10.8.2 rqt_console

```
ros2 run rqt_console rqt_console
```

La commande permet de lancer le système d'affichage des messages d'alertes et d'erreur. Cela crée la fenêtre affichée dans la figure Fig. 10.13. Pour illustrer cet exemple on peut lancer le noeud **turtlesim** avec :

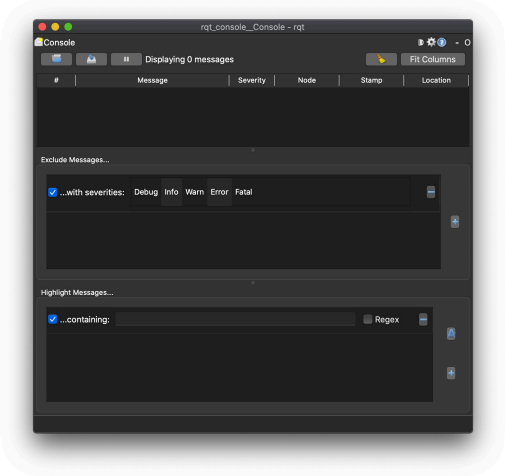


FIGURE 10.13 – Fenêtres correspondant à **rqt_console_console**

```
ros2 run turtlesim turtlesim_node
```

La position de la tortue va alors s'afficher dans la console car il s'agit d'un message de niveau INFO. Il est possible de changer le niveau des messages affichés. En le mettant par exemple à WARN, et en envoyant la commande suivant à **turtlesim** :

```
ros2 topic pub -r 1 /turtle1/cmd_vel geometry_msgs/Twist "{linear: {x: 2000.0, y: 0.0, z: 0.0}, angular: {x: 0.0,
    y: 0.0, z: 2000.0}}"
```

Elle pousse la tortue dans les coins de l'environnement et on peut voir un message affichant un warning dans la fenêtre du node **rqt_console_console** affichée dans la figure.10.14.

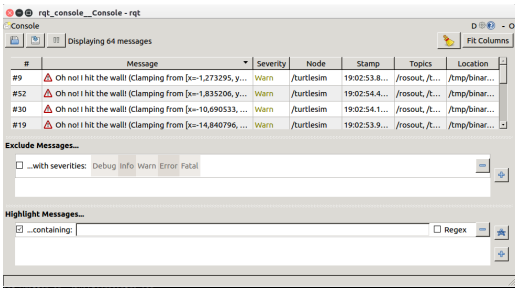


FIGURE 10.14 – Fenêtres correspondant à **rqt_console_console** lorsque la tortue s'écrase dans les murs

10.8.3 Niveaux d'enregistrement

Les niveaux d'enregistrements (logger levels) sont ordonnés par sévérité : Fatal, Error, Warn, Info, Debug.

Fatal : le message indique que le système va s'arrêter pour se protéger de tout préjudice.

Error : le message indique que le système ne va pas être endommager mais qu'il ne va pas fonctionner normalement.

Warn : le message indique une activité inhabituelle ou un résultat non idéal qui peut réprésenter un problème plus profond mais n'impact la fonctionnalité immédiatement.
Info : le message indique des mises à jour d'évènements et de statut qui permettent de visualiser que le système fonctionne correctement.
Debug : le message fournit des informations qui détaille le fonctionnement pas à pas du système.

Le niveau par défaut est **Info**. Seuls les messages d'un niveau de sévérité égal ou supérieur s'afficheront.

Normalement seuls les messages **Debug** seront cachés car ils sont les seuls à avoir un niveau de sévérité inférieurà **Info**. Par exemple, en fixant le niveau par défaut à **Warn**, on ne voit que les messages de sévérité **Warn**, **Error** et **Fatal**.

10.8.3.1 Fixé le niveau d'enregistrement par défaut

Il est possible de spécifier le niveau par défaut d'enregistrement au démarrage du node en utilisant les arguments d'entrée. Les messages de warning de sévérité **Info** ne s'affichent plus dans le terminal. C'est parce que ces messages de sévérité ont un niveau inférieur au niveau par défaut spécifié i.e. **Info**.

```
ros2 run turtlesim turtlesim_node --ros-args --log-level WARN
```

Cependant le mécanisme permettant de changer à la volée les niveaux de logs ne sont pas encore complétement supportés.

Tutoriel associé : Tutorial Using rqt console et roslaunch `https://docs.ros.org/en/humble/Tutorials/Rqt-Console/Using-Rqt-Console.html` ;

10.9 Bag : fichier de données

La commande ros2 *bag* fournit les sous commandes suivantes :

info : Affiche des informations sur le fichier de données à la console
play : Rejoue les données enregistrées dans le fichier de données .bag
record : Enregistre des données ROS dans un fichier de données au format .bag

Il est possible d'enregistrer et de rejouer des flux de données transmis par les topics. Cela s'effectue en utilisant la commande **ros2 bag**. Par exemple la ligne de commande suivante enregistre toutes les données qui passe par les topics à partir du moment où la commande est active (et si des évènements ou données sont génèrés sur les topics).

```
ros2 bag record -a
```

On peut par exemple lancer le node `turtlesim` et le noeud `teleop_key` pour envoyer des commandes.

Par défaut le nom du fichier `rosbag` est constitué de l'année, du mois et du jour et post fixé par `.bag` . Il est possible ensuite de n'enregistrer que certains topics, et de spécifier un nom de fichier. Par exemple la ligne de commande suivante n'enregistre que les topics `/turtle1/cmd_vel` et `/turtle1/pose` dans le fichier `subset.bag` .

```
ros2 bag record -o subset /turtle1/cmd_vel /turtle1/pose
```

Pour rejouer un fichier de données il suffit de faire :

```
ros2 bag play subset.bag
```

10.10 Launch : démarrer une application

10.10.1 Introduction

ros2 launch est une commande qui permet de lancer plusieurs noeuds. **ros2 launch** exécute un fichier python pour lancer plusieurs noeuds. Le fichier python doit se trouver dans un répertoire *launch*.

Dans ROS-2 les fichiers XMLs de fichier de launch ne sont plus proposés. Effectivement utiliser un fichier python permet d'avoir un meilleur contrôle sur la synchronisation du lancement des nodes de calcul.

10.10.2 Un exemple

Cet exemple se base sur l'utilisation de turtlesim. On créée tout d'abord un répertoire launch avec :

```
mkdir launch
```

Il faut créer un fichier turtlesim_mimic_launch.py en entrant la commande suivant :

```
touch launch/turtlesim_mimic_launch.py
```

Il suffit ensuite de copier coller le code python suivant :

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='turtlesim',
            namespace='turtlesim1',
            executable='turtlesim_node',
            name='sim'
        ),
        Node(
            package='turtlesim',
            namespace='turtlesim2',
            executable='turtlesim_node',
            name='sim'
        ),
        Node(
            package='turtlesim',
            executable='mimic',
            name='mimic',
            remappings=[
                ('/input/pose', '/turtlesim1/turtle1/pose'),
                ('/output/cmd_vel', '/turtlesim2/turtle1/cmd_vel'),
            ]
        )
    ])
```

Les deux premières lignes importent les modules python relatifs à la fonctionnalité de launch.

```
from launch import LaunchDescription
from launch_ros.actions import Node
```

La description de la structure de launch est définie dans :

```
def generate_launch_description():
   return LaunchDescription([

   ])
```

Dans la structure *LaunchDepscription* on trouve 3 nodes tous faisant parti du paquet *turtlesim*. Le but étant de lancer deux fenêtres turtlesim et un node qui mime les mouvements d'une tortue vers une autre.

Les deux premières parties lance les deux fenêtres turtlesim.

```
Node(
    package='turtlesim',
    namespace='turtlesim1',
    executable='turtlesim_node',
    name='sim'
),
Node(
    package='turtlesim',
    namespace='turtlesim2',
    executable='turtlesim_node',
    name='sim'
    ),
```

On peut noter que la seule différence entre les deux nodes correspond à leur valeurs d'espace de noms (namespace). Un espace de nom unique permet au système de démarrer les deux fenêtres sans conflit de noms pour les topics ou les nodes.

Les deux tortues recoivent des commandes à travers le même topic et publient leurs posent sur le même topic. Sans l'utilisation d'un namespace unique, il n'y aurait moyen de distinguer les messages pour une tortue ou l'autre.

Le node final vient également du paquet `turtlesim` mais avec un exécutable différent `mimic`.

On peut le vérifier avec **rqt_graph** qui donne la figure Fig.10.15. On trouve également deux topics nommés **/turtlesim1/turtle1/pose** et **/turtlesim2/turtle1/cmd_vel**. Si on exécute :

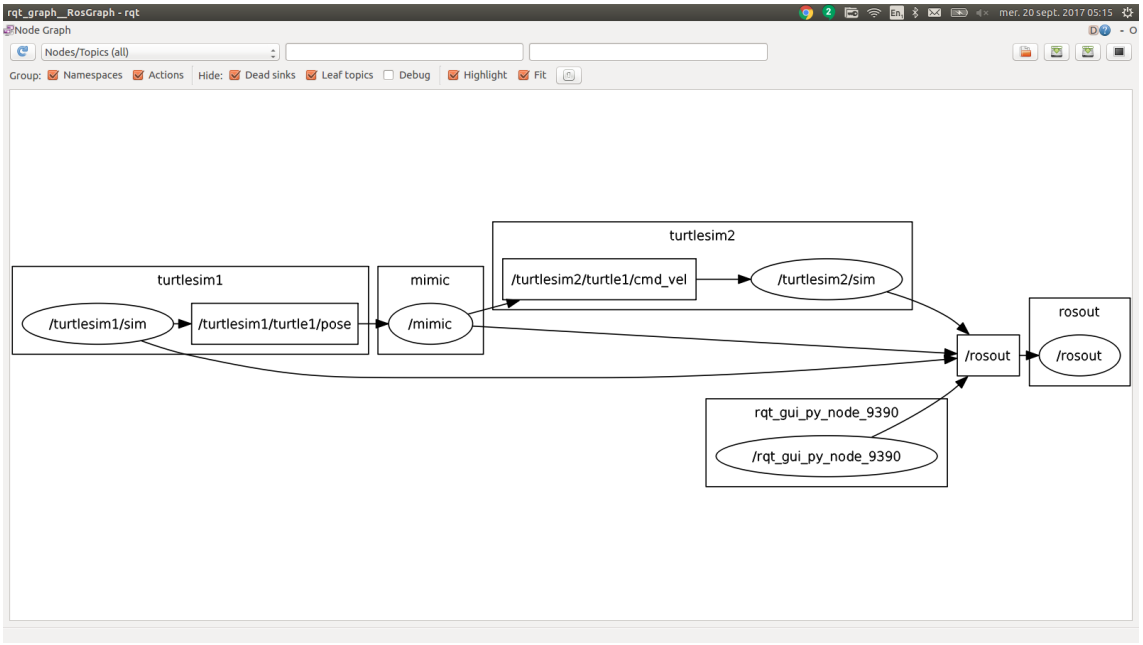


FIGURE 10.15 – Graphe correspondant au fichier turtlemimic.launch

```
ros2 topic list
```

On constate que l'on trouve deux ensembles de topics correspondants aux deux nodes de **turtlesim** et aux deux namespaces **turtlesim1** et **turtlesim2**.

Tutoriel associé : Tutorial Creating a launch file `https://index.ros.org/doc/ros2/Tutorials/Launch-Files/Creating-Launch-Files`

10.10.3 Préparer son package

La suite correspond au tutoriel Launching/monitoring multiple nodes with Launch `https://index.ros.org/doc/ros2/Tutorials/Launch-system/`

10.10.3.1 Paquet python

Pour les paquets python, la structure du paquet doit ressembler à :

```
  src/
    my_package/
      launch/
      setup.py
      setup.cfg
      package.xml
```

Pour que colcon puisse trouver les fichiers launch, il faut informer les outils de paramètrage de python qu'il y a des fichiers de launch dans le champ data_files de setup.

Donc pour le fichier setup.py il faut mettre :

```
import os
from glob import glob
from setuptools import setup

package_name = 'my_package'

setup(
    # Other parameters ...
    data_files=[
        # ... Other data files
        # Mettre tous les fichiers de launch. La ligne la plus importante est ici !
        (os.path.join('share', package_name), glob('launch/*.launch.py'))
    ]
)
```

10.10.3.2 Paquet C++

Si votre paquet est exclusivement un paquet C++, il suffit d'ajuster le fichier CMakeLists.txt en ajoutant :

```
# Install launch files.
install(DIRECTORY
  launch
  DESTINATION share/${PROJECT_NAME}/
)
```

à la fin du fichier mais avant ament_package().

10.10.3.3 Ecrire le fichier de launch

Dans le répertoire launch, il faut créer le fichier launch avec le suffixe `.launch.py`. Par exemple le nom de fichier peut-être `my_script.launch.py`.

Le suffixe `.launch.py` n'est pas spécifiquement obligatoire. Une autre option populaire est d'utiliser `_launch.py` utilisé dans l'exemple précédent. Si vous souhaitez changer le suffixe, il faut s'assurer de changer l'argument `glob()` dans le fichier `setup.py`.

10.11 Création de messages et de services

10.11.1 Introduction

C'est une bonne pratique en général de réutiliser des interfaces qui existent déjà. Mais il peut s'avérer nécessaire de créer vos propres messages ou services.

Les messages sont définis par des structures appellées **msg**, tandis que les services le sont par des les structures **srv**.

— **msg** : msg files sont de simples fichiers textes qui donnent les champs d'un message ROS. Ils sont utilisés pour génèrer le code source des messages dans des languages différents.
— **srv** : un fichier srv décrit un service. Il est composé de deux parties : une requête et une réponse.

Les fichiers `msg` sont stockés dans le répertoire `msg` d'un paquet, et les fichiers `srv` sont stockés dans le répertoire `srv`.

Les fichiers `msg` sont de simples fichiers textes avec un type de champ et un nom de champ par ligne. Les types de champs possibles sont :

— int8, int16, int32, int64 (plus uint *)
— float32, float64
— string
— time, duration
— other msg files
— des tableaux à taille variables ([])
— des tableaux à taille fixes ([C], avec C la taille du tableau)

Il existe un type spécial dans ROS : **Header**, l'entête contient une trace temporelle (horodatage) et des informations sur les repères de réfèrence qui sont utilisés le plus couramment en ROS. Il est fréquent que la première ligne d'un message soit **Header header**.

Voici un exemple qui utilise un entête, une chaïne de caractères, et deux autres messages :

```
Header header
string child_frame_id
geometry_msgs/PoseWithCovariance pose
geometry_msgs/TwistWithCovariance twist
```

srv files sont comme des fichiers msg, à part qu'ils contiennet deux parties : une requête et une réponse. Les deux parties sont séparées par une ligne '—' Voici un exemple de fichier srv :

```
int64 A
int64 B
---
int64 Sum
```

Pour la suite nous allons créer un package ROS-2 dans le workspace `dev_ws` avec la commande suivante :

```
cd ~/dev_ws/src
ros2 pkg create --build-type ament_cmake --license Apache-2.0 cpp_pubsub
```

`tutorial_interfaces` est le nom du nouveau paquet. Il s'agit d'un paquet CMake, car il n'y a pour l'instant pas moyen de générer un fichier `.msg` ou `.srv` à partir d'un paquet Python pur. Il est cependant possible de créer une interface personalisée dans un paquet CMake puis de l'utiliser dans un paquet Python.

10.11.2 Création d'un msg

10.11.2.1 Format du fichier

Créons maintenant un msg dans le paquet `tutorial_interfaces`

```
mkdir msg
echo "int64 num" > msg/Num.msg
```

Le message précèdent ne contient qu'une ligne. Il est possible de faire des fichiers plus compliqués en ajoutant plusieurs élèments (un par ligne) comme ceci :

```
string first_name
string last_name
uint8 age
uint32 score
```

10.11.3 Création d'un srv

10.11.3.1 Format du fichier

Les fichiers srv se trouvent par convention dans le répertoire srv d'un paquet :

```
mkdir srv
```

Dans ce répertoire `turtorial_interfaces/srv` nous allons maintenant créer un fichier `AddThreeInts.srv` avec la requête et la réponse suivantes :

```
int64 a
int64 b
int64 c
---
int64 sum
```

Notons que la requête et la réponse sont séparées par les trois tirets.

10.11.4 CMakeList.txt

Afin de transformer les fichiers `Num.msg` et `AddThreeInts.srv` en fichier C++, Python et autres languages, il faut modifier le fichier `CMakeLists.txt` du paquet.

Il suffit de rajouter les lignes suivantes :

```
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/Num.msg"
  "srv/AddThreeInts.srv"
 )
```

10.11.5 package.xml

Il faut maintenant modifier le fichier *package.xml* pour que les trois lignes suivant aient été décommentées :

```
<build_depend>rosidl_default_generators</build_depend>

<exec_depend>rosidl_default_runtime</exec_depend>

<member_of_group>rosidl_interface_packages</member_of_group>
```

10.11.6 Génération des codes

Afin de générer les codes dans les différents languages que l'on souhaite il faut compiler le paquet en utilisant la commande suivante dans le répertoire `/dev_ws` :

```
colcon build --packages-select tutorial_interfaces
```

Les interfaces sont maintenant découverables pour les autres paquets ROS-2.

10.11.7 Vérifier la création du fichier msg et srv

Dans un nouveau terminal il faut sourcer le fichier de configuration `setup.bash` de l'espace `/dev_ws`.

```
. install/setup.bash
```

Il est maintenant possible d'afficher le message du paquet `tutorial_interfaces` en utilisant la commande suivante :

```
ros2 interface show tutorial_interfaces/msg/Num
```

retourne :

```
int64 num
```

et la commande suivante :

```
ros2 interface show tutorial_interfaces/srv/AddThreeInts
```

retourne :

```
int64 a
int64 b
int64 c
---
int64 sum
```

11. Ecrire des nodes ROS-2

Dans ce chapitre nous allons examiner l'écriture en C++ et en python des *nodes* implémentant les concepts vus dans le chapitre précédent. Les *nodes* est le mot dans la communauté ROS pour désigner un executable connecté au middleware ROS. Nous allons commencer par les *topics* puis examiner l'utilisation des *services*.

11.1 Topics

Dans ce paragraphe nous allons examiner l'écriture d'un node qui émet constamment un message et d'un autre node qui reçoit le message.

11.1.1 Création d'un paquet

Il faut d'abord ouvrir un terminal en s'assurant que l'environnement ROS-2 est bien sourcé de telle sorte que les commandes ROS-2 fonctionnent.

Aller ensuite dans le répertoire `dev_ws` créer précédemment. Il faut également se souvenir que les paquets doivent être créés dans le répertoire `src` et pas à la racine du répertoire.

11.1.1.1 Création d'un paquet C++

Il faut donc aller dans `dev_ws/src` et lancer la commande de création de paquet :

```
ros2 pkg create --build-type ament_cmake --license Apache-2.0 cpp_srvcli --dependencies rclcpp example_interfaces
```

L'argument `--build-type ament_cmake` indique qu'il s'agit d'un paquet C++. L'argument `--license Apache-2.0` indique que la license permet une utilisation et une redistribution commerciale libre. Le nom du paquet est `cpp_srvcli`. Les dépendences des messages sont `rclpp` pour accéder à la librairie client C++, et `example_interfaces` pour le message que nous allons utiliser. Le terminal doit retourner un message confirmant la création du paquet `cpp_srvcli` et de tous les fichiers et répertoires nécessaires.

Il faut naviguer jusqu'à `dev_ws/src/cpp_srvcli/src`. C'est dans ce répertoire que les fichiers sources des exécutables de tous les paquets CMake doivent être présents.

11.1.1.2 Création d'un paquet Python

En allant dans le répertoire dans `dev_ws/src` et lancer la commande de création de paquet :

```
ros2 pkg create --build-type ament_python --license Apache-2.0 py_pubsub
```

Le terminal doit retourner un message confirmant la création du paquet `py_pubsub` et de tous les fichiers et répertoires nécessaires.

11.1.2 Emetteur

Dans l'émetteur nous devons faire les choses suivantes :

1. Initialiser le système ROS.
2. Avertir que le node va publier des messages *std_msgs/String* sur le topic "chatter".
3. Boucler sur la publication de messages sur "chatter" 10 fois par seconde.

11.1.2.1 C++

11.1.2.1.1 Obtenir le code

Il faut tout d'abord créer un répertoire src dans le paquet `cpp_pubsub` :

```
mkdir src
```

Ce répertoire va contenir tous les fichiers sources C++ du paquet.

Il faut ensuite créer le fichier `publisher_member_function.cpp` en faisant :

```
wget -O publisher_member_function.cpp https://raw.githubusercontent.com/ros2/examples/master/rclcpp/topics/
     minimal_publisher/member_function.cpp
```

11.1.2.1.2 Le code lui-même

Le code est le suivant :

```
#include <chrono>
#include <functional>
#include <memory>
#include <string>

#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"

using namespace std::chrono_literals;

/* This example creates a subclass of Node and uses std::bind() to register a
* member function as a callback from the timer. */

class MinimalPublisher : public rclcpp::Node
{
  public:
    MinimalPublisher()
    : Node("minimal_publisher"), count_(0)
    {
      publisher_ = this->create_publisher<std_msgs::msg::String>("topic", 10);
      timer_ = this->create_wall_timer(
      500ms, std::bind(&MinimalPublisher::timer_callback, this));
    }

  private:
    void timer_callback()
    {
      auto message = std_msgs::msg::String();
      message.data = "Hello, world! " + std::to_string(count_++);
      RCLCPP_INFO(this->get_logger(), "Publishing: '%s'", message.data.c_str());
      publisher_->publish(message);
    }
    rclcpp::TimerBase::SharedPtr timer_;
    rclcpp::Publisher<std_msgs::msg::String>::SharedPtr publisher_;
    size_t count_;
};

int main(int argc, char * argv[])
{
  rclcpp::init(argc, argv);
  rclcpp::spin(std::make_shared<MinimalPublisher>());
  rclcpp::shutdown();
  return 0;
}
```

11.1.2.1.3 Explications

L'entête ros.h

```
6 #include "rclcpp/rclcpp.hpp"
```

permet d'inclurer toutes les en-têtes ROS nécessaires d'une manière efficace et compact.

L'entête "std_msgs/msg/string.hpp" :

```
7 #include "std_msgs/msg/string.hpp"
```

permet d'accéder à la déclaration C++ des messages std_msgs/String. Cette entête est générée automatiquement à partir du fichier **std_msgs/msg/String.msg** qui se trouve dans le paquet std_msgs.

```
40   rclcpp::init(argc, argv);
```

initialise ROS. Contrairement à ROS-1 le nom n'est plus spécifié à ce niveau. On démarre ensuite un mécanisme de synchronisation en attente de la fin de la réalisation de la classe `MinimalPublisher`.

```
41   rclcpp::spin(std::make_shared<MinimalPublisher>());
```

Une fois que le processus de la classe est terminé, on indique au système que le Node est arrêté.

```
42   rclcpp::shutdown();
```

Le membre `publisher_` déclare l'émetteur à la ligne 34 :

```
34   rclcpp::Publisher<std_msgs::msg::String>::SharedPtr publisher_;
```

il est ensuite instancié à la ligne 20 :

```
20   publisher_ = this->create_publisher<std_msgs::msg::String>("topic", 10);
```

Le type du publisher est défini par le paramètre de template `std_msgs ::msg ::String`, le nom du topic est `topic`, et la taille de la file d'attente est définie à 10.

Le timer `timer_` déclaré à la ligne 33 :

```
33   rclcpp::TimerBase::SharedPtr timer_;
```

est instancié avec les ligne 21 et 22 :

```
21   timer_ = this->create_wall_timer(
22   500ms, std::bind(&MinimalPublisher::timer_callback, this));
```

Le timer créé appelle la méthode `MinimalPublisher ::timer_callback` toutes les 500 *ms*.

La classe `MinimalPublisher` hérite de la classe `rclcpp ::Node`. Chaque référence à `this` dans le code fait référence à la classe `Node`. La classe est instanciée avec le constructeur de la façon suivante :

```
176 public:
177   MinimalPublisher()
178   : Node("minimal_publisher"), count_(0)
```

Cela permet de nommer le Node `minimal_publisher` et d'initialisé le compteur `count_` à zéro.

La méthode `timer_callback` est celle où les données des messages sont spécifiées et où les messages sont publiés. La macro `RCLCPP_INFO` permet que chaque message publié soit affiché à la console.

11.1.2.1.4 Ajouter les dépendences dans le package.xml

Il faut aller dans le répertoire au-dessus du répertoire `src`, c'est à dire dans `dev_ws/src/cpp_pubsub` où les fichiers `CMakeLists.txt` et `package.xml` se trouvent.

Il faut ouvrir le fichier `package.xml` dans votre éditeur de texte préféré.

Il faut remplir les champs `<description>`, `<maintener>`, et `<license>` :

```
<description>Examples of minimal publisher/subscriber using rclcpp</description>
<maintainer email="you@email.com">Your Name</maintainer>
<license>Apache License 2.0</license>
```

Il faut ensuite ajouter deux nouvelle lignes après la dépendence buildtool d' `ament_cmake` :

```
<depend>rclcpp</depend>
<depend>std_msgs</depend>
```

Ceci déclare la dépendence aux paquets `rclrcpp` et `std_msgs` .

Il faut ensuite s'assurer d'avoir bien sauvegarder le fichier.

11.1.2.1.5 CMakeLists.txt

Il faut ensuite ouvrir le fichier `CMakeLists.txt` . En-dessous de la ligne qui déclare la dépendence à `ament_cmake` `find_package(ament_cmake REQUIRED)` , il faut ajouter :

```
find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)
```

Ensuite, pour obtenir les exécutables et le nommer `talker` pour pouvoir le démarrer en utilisant `ros2 run` :

```
add_executable(talker src/publisher_member_function.cpp)
ament_target_dependencies(talker rclcpp std_msgs)
```

Il faut ensuite ajouter la section d'installation pour que la commande `ros2 run` le trouve :

```
install(TARGETS
  talker
  DESTINATION lib/${PROJECT_NAME})
```

Il est ensuite possible de nettoyer le fichier `CMakeLists.txt` en enlevant les sections et commentaires inutiles de façon à ce qu'il ressemble à cela :

```
cmake_minimum_required(VERSION 3.5)
project(cpp_pubsub)

# Default to C++14
if(NOT CMAKE_CXX_STANDARD)
  set(CMAKE_CXX_STANDARD 14)
endif()

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)

add_executable(talker src/publisher_member_function.cpp)
ament_target_dependencies(talker rclcpp std_msgs)

install(TARGETS
  talker
  DESTINATION lib/${PROJECT_NAME})

ament_package()
```

Il est maintenant possible de construire le paquet, de sourcer l'environnement et de le lancer, mais il faut le node `subscriber` pour voir l'ensemble du système fonctionner.

11.1.2.2 Python

11.1.2.2.1 Récupérer le fichier python

Pour le code python, contrairement à ROS-1 le code python se trouve dans un répertoire qui porte le même nom que le paquet. Il faut aller dans le répertoire `py_pubsub` dans le paquet `py_pubsub` qui se trouve dans le répertoire `dev_ws/src/py_pubsub/py_pubsub` .

Il est possible de télécharger le script python du publisher `publisher_member_function.py` dans le répertoire `py_pubsub` et de le rendre executable :

```
wget https://raw.githubusercontent.com/ros2/examples/master/rclpy/topics/minimal_publisher/
    examples_rclpy_minimal_publisher/publisher_member_function.py
```

Le fichier se trouve maintenant à côté du fichier `__init__.py` .

11.1.2.2.2 Le fichier python

```python
import rclpy
from rclpy.node import Node

from std_msgs.msg import String


class MinimalPublisher(Node):

    def __init__(self):
        super().__init__('minimal_publisher')
        self.publisher_ = self.create_publisher(String, 'topic', 10)
        timer_period = 0.5  # seconds
        self.timer = self.create_timer(timer_period, self.timer_callback)
        self.i = 0

    def timer_callback(self):
        msg = String()
        msg.data = 'Hello World: %d' % self.i
        self.publisher_.publish(msg)
        self.get_logger().info('Publishing: "%s"' % msg.data)
        self.i += 1


def main(args=None):
    rclpy.init(args=args)

    minimal_publisher = MinimalPublisher()

    rclpy.spin(minimal_publisher)

    # Destroy the node explicitly
    # (optional - otherwise it will be done automatically
    # when the garbage collector destroys the node object)
    minimal_publisher.destroy_node()
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

11.1.2.2.3 Explications

La première ligne importe le module python pour interagir avec la partie système de ROS-2. La deuxième ligne importe la classe Node dont va dériver la classe utilisée dans ce script python.

```python
import rclpy
from rclpy.node import Node
```

La ligne suivante importe le module python qui permet d'accéder au type `String` dont le node a besoin pour construire le message à envoyer au topic.

Ces lignes représentent les dépendences du Node. Il ne faut pas oublier que ces dépendences doivent être ajoutées au fichier `package.xml` .

```python
class MinimalPublisher(Node):
```

La classe `MinimalPublisher` hérite de la classe `Node` avec cette ligne.

```python
def __init__(self):
    super().__init__('minimal_publisher')
    self.publisher_ = self.create_publisher(String, 'topic', 10)
    timer_period = 0.5  # seconds
    self.timer = self.create_timer(timer_period, self.timer_callback)
    self.i = 0
```

Dans la suite, le constructeur spécifie le nom du Node à `minimal_publisher`. Puis l'appel à `create_publisher` déclare que le Node va publier des messages de type `String` (importé par le module `std_msgs.msg`) sur le topic `topic` avec une taille de file d'attente de 10. La taille de la file d'attente est un champ de QoS (Qualité de Service) qui spécifie le nombre de messages qui sont stockés si un subscriber ne les reçoit pas assez vite.

Ensuite un timer est créé avec un fonction de rappel toutes les 500 *ms*. `self.i` est un compteur utilisé dans la fonction de rappel.

La méthode `timer_callback` créé un message avec la valeur du compteur ajoutée, et le publie sur la console avec l'appel à `get_logger().info`.

```
16 def timer_callback(self):
17     msg = String()
18     msg.data = 'Hello World: %d' % self.i
19     self.publisher_.publish(msg)
20     self.get_logger().info('Publishing: "%s"' % msg.data)
21     self.i += 1
```

Finalement la fonction `main` est définie :

```
9  def main(args=None):
10     rclpy.init(args=args)
11
12     minimal_publisher = MinimalPublisher()
13
14     rclpy.spin(minimal_publisher)
15
16     # Destroy the node explicitly
17     # (optional - otherwise it will be done automatically
18     # when the garbage collector destroys the node object)
19     minimal_publisher.destroy_node()
20     rclpy.shutdown()
```

La librarie `rclpy` est initialisée, le Node est créé, puis une boucle d'attente d'événements est appelée avec `spin`. Ceci permet au Node d'attendre les appels du timer.

11.1.2.2.4 Ajouter les dépendances

Il faut aller dans le répertoire `dev_ws/src/py_pubsub` où les fichiers `setup.py`, `setup.cfg`, et `package.xml` ont été créés. Il faut ensuite ouvrir le fichier `package.xml` avec un éditeur de texte, et remplir les balises : `<description>`, `<maintainer>` et `<license>`.

```
<description>Examples of minimal publisher/subscriber using rclpy</description>
<maintainer email="you@email.com">Your Name</maintainer>
<license>Apache License 2.0</license>
```

Après la dépendance à l'outil de construction `ament_python` il faut ajouter les dépendances correspondant aux `imports` de python.

```
<exec_depend>rclpy</exec_depend>
<exec_depend>std_msgs</exec_depend>
```

Ceci déclare le besoin des paquets `rclpy` et `std_msgs` quand le code est exécuté.

11.1.2.2.5 Ajouter un point d'entrée

Il faut ouvrir le fichier `setup.py`. Il faut ensuite que les champs `maintainer`, `maintainer_email`, `description` et `license` correspondent aux champs du fichier `package.xml` :

```
maintainer='YourName',
maintainer_email='you@email.com',
description='Examples of minimal publisher/subscriber using rclpy',
license='Apache License 2.0',
```

Il faut ajouter la ligne suivante dans les crochets de `console_scripts` du champ `entry_points` :

```
entry_points={
        'console_scripts': [
                'talker = py_pubsub.publisher_member_function:main',
        ],
},
```

11.1.2.2.6 Vérification de setup.cfg

Le contenu de `setup.cfg` doit être correctement peuplé automatiquement de la façon suivante :

```
[develop]
script-dir=$base/lib/py_pubsub
[install]
install-scripts=$base/lib/py_pubsub
```

Ceci dit simplement que setuptools mets l'exécutable dans `lib`, parce que `ros2 run` les cherchera ici.

11.1.3 Souscripteur

Les étapes du souscripteur peuvent se résumer de la façon suivante :

— Initialiser le système ROS
— Souscrire au topic `topic`
— Spin, attendre que les messages arrivent
— Quand le message arrive la fonction `topic_callback()` est appellée.

11.1.3.1 C++

11.1.3.1.1 Obtenir le code

Il faut retourner dans le répertoire `dev_ws/src/cpp_pubsub/src` pour créer le souscripteur. Pour obtenir le code il suffit d'utiliser la commande suivante :

```
wget -O subscriber_member_function.cpp https://raw.githubusercontent.com/ros2/examples/master/rclcpp/topics/
     minimal_subscriber/member_function.cpp
\end{lstlistinng}


\paragraph{Le code lui-êmme}
\begin{lstlisting}[language=C++]
#include <memory>

#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"
using std::placeholders::_1;

class MinimalSubscriber : public rclcpp::Node
{
  public:
    MinimalSubscriber()
    : Node("minimal_subscriber")
    {
      subscription_ = this->create_subscription<std_msgs::msg::String>(
      "topic", 10, std::bind(&MinimalSubscriber::topic_callback, this, _1));
    }

  private:
    void topic_callback(const std_msgs::msg::String::SharedPtr msg) const
    {
      RCLCPP_INFO(this->get_logger(), "I heard: '%s'", msg->data.c_str());
    }
    rclcpp::Subscription<std_msgs::msg::String>::SharedPtr subscription_;
};

int main(int argc, char * argv[])
{
  rclcpp::init(argc, argv);
  rclcpp::spin(std::make_shared<MinimalSubscriber>());
  rclcpp::shutdown();
  return 0;
}
```

11.1.3.1.2 Explications

Le code du souscripteur est presque identique à celui de l'émetteur. Le node est maintenant appellé `minimal_subscriber` et le constructeur du node utilise la méthode `create_subscription` pour exécuter la méthode de rappel.

Il n'y a pas de minuteur car la méthode de rappel est appelée dés qu'un message arrive.

```
public:
  MinimalSubscriber()
  : Node("minimal_subscriber")
  {
    subscription_ = this->create_subscription<std_msgs::msg::String>(
    "topic", 10, std::bind(&MinimalSubscriber::topic_callback, this, _1));
  }
```

Rappellons qu'il faut que le nom du topic et que le type de message correspondent à la fois dans le code de l'émetteur et du souscripteur pour qu'ils puissent communiquer.

La méthode `topic_callback` reçoit la chaîne de caractères par le message de données via le topic, et l'affiche simplement dans la console grâce à la macro `RCLCPP_INFO`.

La seule déclaration de champ est celle de la souscription.

```
private:
  void topic_callback(const std_msgs::msg::String::SharedPtr msg) const
  {
    RCLCPP_INFO(this->get_logger(), "I heard: '%s'", msg->data.c_str());
  }
  rclcpp::Subscription<std_msgs::msg::String>::SharedPtr subscription_;
```

La fonction `main` est exactement la même, à part qu'il démarre le Node `MinimalSubscriber`. Pour le node émetteur l'appel à la méthode `spin` démarre le minuteur, mais le souscripteur cela signifie simplement se préparer à recevoir des messages lorsqu'ils arrivent.

Puisque ce node a les mêmes dépendences il n'y a rien à ajouter à `package.xml`.

11.1.3.1.3 CMakeLists.txt

Il faut réouvrir le fichier `CMakeLists.txt`, et ajouter l'exécutable et la cible après l'entrée de l'émetteur.

```
add_executable(listener src/subscriber_member_function.cpp)
ament_target_dependencies(listener rclcpp std_msgs)

install(TARGETS
  talker
  listener
  DESTINATION lib/${PROJECT_NAME})
```

11.1.3.2 Python

Il faut retourner dans le répertoire `dev_ws/src/py_pubsub/py_pubsub` pour créer le node émetteur.

11.1.3.2.1 Obtenir le code

```
wget https://raw.githubusercontent.com/ros2/examples/master/rclpy/topics/minimal_subscriber/
     examples_rclpy_minimal_subscriber/subscriber_member_function.py
```

On a donc maintenant les fichiers suivants :

```
__init__.py  publisher_member_function.py  subscriber_member_function.py
```

11.1.3.2.2 Le code lui-même

Ouvrir le fichier `subscriber_member_function.py` donne le code suivant :

```
import rclpy
from rclpy.node import Node

from std_msgs.msg import String

```

```

class MinimalSubscriber(Node):

    def __init__(self):
        super().__init__('minimal_subscriber')
        self.subscription = self.create_subscription(
            String,
            'topic',
            self.listener_callback,
            10)
        self.subscription  # prevent unused variable warning

    def listener_callback(self, msg):
        self.get_logger().info('I heard: "%s"' % msg.data)


def main(args=None):
    rclpy.init(args=args)

    minimal_subscriber = MinimalSubscriber()

    rclpy.spin(minimal_subscriber)

    # Destroy the node explicitly
    # (optional - otherwise it will be done automatically
    # when the garbage collector destroys the node object)
    minimal_subscriber.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

11.1.3.2.3 Explications

Le code du souscripteur est quasiment identique à celui de l'émetteur. Le constructeur créée un souscripteur avec les mêmes arguments que l'émetteur. Rappellons qu'il faut que le nom du topic et que le type de message correspondent à la fois dans le code de l'émetteur et du souscripteur pour qu'ils puissent communiquer.

```
self.subscription = self.create_subscription(
    String,
    'topic',
    self.listener_callback,
    10)
```

Le constructeur du souscripteur et la fonction de rappel ne contiennent pas de timer parce que cela n'est pas utile. La fonction de rappel est appellée dés qu'un message est reçu.

La fonction de rappel simplement affiche un message sur la console, avec la donnée qu'elle a reçue. Rappellons que l'émetteur définit `msg.data = 'Hello World : %d' % self.i`

```
def listener_callback(self, msg):
    self.get_logger().info('I heard: "%s"' % msg.data)
```

La définition de la fonction `main` est presque exactement la même, replaçant la création et l'appel à l'émetteur par le souscripteur.

```
minimal_subscriber = MinimalSubscriber()

rclpy.spin(minimal_subscriber)
```

Comme le node a les mêmes dépendences que l'émetteur, il n'y a rien à rajouter au fichier `package.xml`. Le fichier `setup.cfg` peut rester également non touché.

11.1.3.2.4 Rajouter un point d'entrée

Il faut réouvrir le fichier `setup.py` et ajouter le point d'entrée pour le souscripteur après le point d'entrée de l'émetteur. Le champ `entry_points` qui doit maintenant ressembler à

```
entry_points={
        'console_scripts': [
                'talker = py_pubsub.publisher_member_function:main',
```

```
                'listener = py_pubsub.subscriber_member_function:main',
        ],
},
```

Il faut s'assurer que le fichier est sauvé, et que le système pub/sub doit être prêt à l'utilisation.

11.1.4 Lancer les nodes

11.1.4.1 Dépendences

Cette partie n'est valable que sous Linux.

Les paquets `rclpy`, `std_msgs` sont normalement déjà installés sur le système ROS-2. C'est une bonne pratique de lancer `rosdep` à la racine de l'espace de travail `dev_ws` pour vérifier les dépendences avant de construire le paquet :

```
  rosdep install -i --from-path src --rosdistro <distro> -y
```

11.1.4.2 Compiler

11.1.4.2.1 C++

Il suffit ensuite de lancer la commande suivante dans un terminal :

```
colcon build --packages-select cpp_pubsub
```

11.1.4.2.2 Python

Il suffit ensuite de lancer la commande suivante dans un terminal :

```
colcon build --packages-select py_pubsub
```

11.1.4.3 Sourcer l'environnement

Afin de pouvoir lancer les nodes il suffit de sourcer le fichier suivant :

```
 source ./dev_ws/install/setup.bash
```

11.1.4.4 Lancer les nodes

11.1.4.4.1 C++

Dans un terminal il faut lancer la commande suivante :

```
ros2 run cpp_pubsub talker
```

Le terminal doit afficher les messages suivant toutes les 0.5 s :

```
[INFO] [minimal_publisher]: Publishing: "Hello World: 0"
[INFO] [minimal_publisher]: Publishing: "Hello World: 1"
[INFO] [minimal_publisher]: Publishing: "Hello World: 2"
[INFO] [minimal_publisher]: Publishing: "Hello World: 3"
[INFO] [minimal_publisher]: Publishing: "Hello World: 4"
...
```

Dans un autre terminal il faut lancer la commande :

```
ros2 run cpp_pubsub listener
```

Le souscripteur va afficher les messages suivants :

```
[INFO] [minimal_subscriber]: I heard: "Hello World: 10"
[INFO] [minimal_subscriber]: I heard: "Hello World: 11"
[INFO] [minimal_subscriber]: I heard: "Hello World: 12"
[INFO] [minimal_subscriber]: I heard: "Hello World: 13"
[INFO] [minimal_subscriber]: I heard: "Hello World: 14"
```

Pour arrêter les nodes il suffit de taper `Ctrl+C`.

11.1.4.4.2 Python

Dans un terminal il faut lancer la commande suivante :

```
ros2 run py_pubsub talker
```

Le terminal doit afficher les messages suivant toutes les 0.5 s :

```
[INFO] [minimal_publisher]: Publishing: "Hello World: 0"
[INFO] [minimal_publisher]: Publishing: "Hello World: 1"
[INFO] [minimal_publisher]: Publishing: "Hello World: 2"
[INFO] [minimal_publisher]: Publishing: "Hello World: 3"
[INFO] [minimal_publisher]: Publishing: "Hello World: 4"
...
```

Dans un autre terminal il faut lancer la commande :

```
ros2 run py_pubsub listener
```

Le souscripteur va afficher les messages suivants :

```
[INFO] [minimal_subscriber]: I heard: "Hello World: 10"
[INFO] [minimal_subscriber]: I heard: "Hello World: 11"
[INFO] [minimal_subscriber]: I heard: "Hello World: 12"
[INFO] [minimal_subscriber]: I heard: "Hello World: 13"
[INFO] [minimal_subscriber]: I heard: "Hello World: 14"
```

Pour arrêter les nodes il suffit de taper `Ctrl+C` .

11.2 Services

Dans cette partie nous allons écrire un serveur et un client pour le service **AddTwoInts.srv**. Le serveur va recevoir deux entiers et retourner la somme. Avant de commencer il faut s'assurer que le fichier **AddTwoInts.srv** existe bien dans le répertoire srv.

Il faut d'abord ouvrir un terminal en s'assurant que l'environnement ROS-2 est bien sourcé de telle sorte que les commandes ROS-2 fonctionnent.

Aller ensuite dans le répertoire `dev_ws` créer précédemment. Il faut également se souvenir que les paquets doivent être créés dans le répertoire `src` et pas à la racine du répertoire.

11.2.1 Création d'un paquet

Il faut d'abord ouvrir un terminal en s'assurant que l'environnement ROS-2 est bien sourcé de telle sorte que les commandes ROS-2 fonctionnent.

Aller ensuite dans le répertoire `dev_ws` créer précédemment. Il faut également se souvenir que les paquets doivent être créés dans le répertoire `src` et pas à la racine du répertoire.

11.2.1.1 Création d'un paquet C++

Il faut donc aller dans `dev_ws/src` et lancer la commande de création de paquet :

```
ros2 pkg create --build-type ament_cmake cpp_srvcli --dependencies rclcpp example_interfaces
```

11.2.1.2 Création d'un paquet python

Il faut donc aller dans `dev_ws/src` et lancer la commande de création de paquet :

```
ros2 pkg create --build-type ament_python py_srvcli --dependencies rclpy example_interfaces
```

11.2.1.3 Explications

L'argument `–dependencies` ajoute automatiquement les dépendences correspondantes dans le fichier `package.xml` . `example_interfaces` est le paquet qui inclut le fichier .srv nécessaire pour construire les requêtes et les réponses.

```
int64 a
int64 b
---
int64 sum
```

Les deux premières lignes sont les paramètres de la requête, et en dessous la réponse.

11.2.1.4 Mise à jour de `package.xml`

Parce que la commande précédent indique les dépendences il n'y a pas besoin de les ajouter dans le fichier `package.xml`. Il faut cependant s'assurer d'ajouter la description, l'email du mainteneur et son nom, et la license.

11.2.1.4.1 C++

```
<description>C++ client server tutorial</description>
<maintainer email="you@email.com">Your Name</maintainer>
<license>Apache License 2.0</license>
```

11.2.1.4.2 Python

```
<description>Python client server tutorial</description>
<maintainer email="you@email.com">Your Name</maintainer>
<license>Apache License 2.0</license>
```

11.2.2 Serveur

Dans cette partie, nous allons écrire un serveur et un client pour le service `AddTwoInts.srv`. Le serveur va recevoir deux entiers et retourner la somme.

— La première étape est l'initialisation de la librairie client ROS-2.
— La deuxième étape est de créer le Node `add_two_ints_server`.
— La troisième étape est de créer le service.
— La quatrième étape est d'attendre les requêtes des clients.

11.2.2.1 C++

11.2.2.1.1 Le code lui même

Il faut aller dans le répertoire `dev_ws/src/cpp_srvcli/src` et créer le fichier `add_two_ints_server.cpp`. Il suffit ensuite de copier coller dans le fichier le code suivant :

```
#include "rclcpp/rclcpp.hpp"
#include "example_interfaces/srv/add_two_ints.hpp"

#include <memory>

void add(const std::shared_ptr<example_interfaces::srv::AddTwoInts::Request> request,
          std::shared_ptr<example_interfaces::srv::AddTwoInts::Response>      response)
{
  response->sum = request->a + request->b;
  RCLCPP_INFO(rclcpp::get_logger("rclcpp"), "Incoming request\na: %ld" " b: %ld",
                request->a, request->b);
  RCLCPP_INFO(rclcpp::get_logger("rclcpp"), "sending back response: [%ld]", (long int)response->sum);
}

int main(int argc, char **argv)
{
  rclcpp::init(argc, argv);

  std::shared_ptr<rclcpp::Node> node = rclcpp::Node::make_shared("add_two_ints_server");

  rclcpp::Service<example_interfaces::srv::AddTwoInts>::SharedPtr service =
    node->create_service<example_interfaces::srv::AddTwoInts>("add_two_ints", &add);

  RCLCPP_INFO(rclcpp::get_logger("rclcpp"), "Ready to add two ints.");

  rclcpp::spin(node);
  rclcpp::shutdown();
}
```

Les deux premières lignes d'`include` correspondent aux dépendences du paquet.

La méthode `add` ajoute deux entiers à partir de la requête et fournie la somme en retour, et notifie la console de son statut en utilisant les logs.

```
void add(const std::shared_ptr<example_interfaces::srv::AddTwoInts::Request> request,
         std::shared_ptr<example_interfaces::srv::AddTwoInts::Response>      response)
{
```

```
    response->sum = request->a + request->b;
    RCLCPP_INFO(rclcpp::get_logger("rclcpp"), "Incoming request\na: %ld" " b: %ld",
        request->a, request->b);
    RCLCPP_INFO(rclcpp::get_logger("rclcpp"), "sending back response: [%ld]", (long int)response->sum);
}
```

La fonction `main` accomplit les lignes suivantes :

— Initialisation de la librairie client de ROS-2 :

```
    rclcpp::init(argc, argv);
```

— Créer un node nommé `add_two_ints_server` :

```
    std::shared_ptr<rclcpp::Node> node = rclcpp::Node::make_shared("add_two_ints_server");
```

— Créer un service nommé `add_two_ints` pour ce node et le publie à travers le réseau. La méthode de rappel est `add` :

```
rclcpp::Service<example_interfaces::srv::AddTwoInts>::SharedPtr service =
node->create_service<example_interfaces::srv::AddTwoInts>("add_two_ints", &add);
```

— Affiche un message de log lorsqu'il est prêt :

```
RCLCPP_INFO(rclcpp::get_logger("rclcpp"), "Ready to add two ints.");
```

— Lance le système d'attente de requête :

```
rclcpp::spin(node);
```

11.2.2.1.2 Ajout de l'exécutable

La macro `add_executable` génère un exécutable que l'on peut exécuter avec `ros2 run`. Il faut ajouter le bloc suivant à `CMakeLists.txt` pour créer un exécutable nommé `server` :

```
add_executable(server src/add_two_ints_server.cpp)
ament_target_dependencies(server
rclcpp example_interfaces)
```

Pour que `ros2 run` trouve l'exécutable, il faut ajouter les lignes suivantes à la fin du fichier, juste avant `ament_package()` :

```
install(TARGETS
  server
  DESTINATION lib/${PROJECT_NAME})
```

11.2.2.2 Python

11.2.2.1 Obtenir le code

Dans le répertoire `dev_ws/src/py_srvcli/py_srvcli`, il faut créer un nouveau fichier nommé `service_member_function.py` en copiant le code suivant :

```
from example_interfaces.srv import AddTwoInts

import rclpy
from rclpy.node import Node


class MinimalService(Node):

    def __init__(self):
        super().__init__('minimal_service')
        self.srv = self.create_service(AddTwoInts, 'add_two_ints', self.add_two_ints_callback)

    def add_two_ints_callback(self, request, response):
        response.sum = request.a + request.b
        self.get_logger().info('Incoming request\na: %d b: %d' % (request.a, request.b))

        return response


```

```
def main(args=None):
    rclpy.init(args=args)

    minimal_service = MinimalService()

    rclpy.spin(minimal_service)

    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

11.2.2.2.2 Explications

Le premier `import` permet d'importer les types du module `AddTwoInts` du paquet `example_interfaces`. Le deuxième `import` importe la librairie client Python ROS-2 et spécifiquement la classe Node.

```
from example_interfaces.srv import AddTwoInts

import rclpy
from rclpy.node import Node
```

Le constructeur de la classe `MinimalService` initialise le node avec le nom `minimal_service`. Ensuite il crée un service et définit le type, nom et méthode de rappel.

```
def __init__(self):
    super().__init__('minimal_service')
    self.srv = self.create_service(AddTwoInts, 'add_two_ints', self.add_two_ints_callback)
```

La définition de la méthode de rappel reçoit la requête et les données, effectue la somme et la retourne dans une structure de réponse.

```
def add_two_ints_callback(self, request, response):
    response.sum = request.a + request.b
    self.get_logger().info('Incoming request\na: %d b: %d' % (request.a, request.b))

    return response
```

Finalement la classe principale initialise la librairie client ROS-2, instancie la classe `MinimalService` pour créer le service et lance le node pour gérer les événements.

11.2.2.2.3 Ajouter un point d'entrée

Pour permettre la commande `ros2 run` de lancer le node, il faut ajouter le point d'entrée au fichier `setup.py` (localisé dans le répertoire `dev_ws/src/py_srvcli`).

Il faut ajouter la ligne suivante entre les crochets `'console_scripts' :`

```
'service = py_srvcli.service_member_function:main',
```

11.2.3 Client

11.2.3.1 C++

11.2.3.1.1 Le code

Dans le répertoire `dev_ws/src/cpp_srvcli/src` il faut créer un fichier appellé `add_two_ints_client.cpp` et copier la suite dedans :

```
#include "rclcpp/rclcpp.hpp"
#include "example_interfaces/srv/add_two_ints.hpp"

#include <chrono>
#include <cstdlib>
#include <memory>

using namespace std::chrono_literals;

int main(int argc, char **argv)
```

```
{
  rclcpp::init(argc, argv);

  if (argc != 3) {
      RCLCPP_INFO(rclcpp::get_logger("rclcpp"), "usage: add_two_ints_client X Y");
      return 1;
  }

  std::shared_ptr<rclcpp::Node> node = rclcpp::Node::make_shared("add_two_ints_client");
  rclcpp::Client<example_interfaces::srv::AddTwoInts>::SharedPtr client =
    node->create_client<example_interfaces::srv::AddTwoInts>("add_two_ints");

  auto request = std::make_shared<example_interfaces::srv::AddTwoInts::Request>();
  request->a = atoll(argv[1]);
  request->b = atoll(argv[2]);

  while (!client->wait_for_service(1s)) {
    if (!rclcpp::ok()) {
      RCLCPP_ERROR(rclcpp::get_logger("rclcpp"), "Interrupted while waiting for the service. Exiting.");
      return 0;
    }
    RCLCPP_INFO(rclcpp::get_logger("rclcpp"), "service not available, waiting again...");
  }

  auto result = client->async_send_request(request);
  // Wait for the result.
  if (rclcpp::spin_until_future_complete(node, result) ==
    rclcpp::FutureReturnCode::SUCCESS)
  {
    RCLCPP_INFO(rclcpp::get_logger("rclcpp"), "Sum: %ld", result.get()->sum);
  } else {
    RCLCPP_ERROR(rclcpp::get_logger("rclcpp"), "Failed to call service add_two_ints");
  }

  rclcpp::shutdown();
  return 0;
}
```

11.2.3.1.2 Explications

De façon similaire au node de service, les lignes de code suivantes crée le node et crée le client pour ce node :

```
  std::shared_ptr<rclcpp::Node> node = rclcpp::Node::make_shared("add_two_ints_client");
  rclcpp::Client<example_interfaces::srv::AddTwoInts>::SharedPtr client =
  node->create_client<example_interfaces::srv::AddTwoInts>("add_two_ints");
```

Ensuite, la requête est crée. Sa structure est définie par le fichier `.srv` mentionné précédemment :

```
  auto request = std::make_shared<example_interfaces::srv::AddTwoInts::Request>();
  request->a = atoll(argv[1]);
  request->b = atoll(argv[2]);
```

La boucle `while` donne au client 1 seconde pour chercher les nodes de service dans le réseau. S'il n'en trouve pas alors il va contineur à attendre.

```
  RCLCPP_INFO(rclcpp::get_logger("rclcpp"), "service not available, waiting again...");
```

Alors le client envoit sa requête, et le node est en attente jusqu'à ce qu'il reçoive sa réponse ou échoue.

11.2.3.1.3 Ajouter un exécutable

Il faut retourner dans `CMakeLists.txt` pour ajouter l'exécutable et la cible pour le nouveau node. Après avoir enlever des sections et des commenaires inutilement générés automatiquement, le fichier `CMakeLists.txt` devrait ressembler à ceci :

```
cmake_minimum_required(VERSION 3.5)
project(cpp_srvcli)

find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(example_interfaces REQUIRED)

add_executable(server src/add_two_ints_server.cpp)
ament_target_dependencies(server
```

```
  rclcpp example_interfaces)

add_executable(client src/add_two_ints_client.cpp)
ament_target_dependencies(client
  rclcpp example_interfaces)

install(TARGETS
  server
  client
  DESTINATION lib/${PROJECT_NAME})

ament_package()
```

11.2.3.2 Python

11.2.3.2.1 Le code lui-même

Dans le répertoire `dev_ws/src/py_srvcli/py_srvcli`, il faut créer un nouveau fichier nommé `client_member_function.py` et y copier le fichier suivant :

```python
import sys

from example_interfaces.srv import AddTwoInts
import rclpy
from rclpy.node import Node


class MinimalClientAsync(Node):

    def __init__(self):
        super().__init__('minimal_client_async')
        self.cli = self.create_client(AddTwoInts, 'add_two_ints')
        while not self.cli.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('service not available, waiting again...')
        self.req = AddTwoInts.Request()

    def send_request(self):
        self.req.a = int(sys.argv[1])
        self.req.b = int(sys.argv[2])
        self.future = self.cli.call_async(self.req)


def main(args=None):
    rclpy.init(args=args)

    minimal_client = MinimalClientAsync()
    minimal_client.send_request()

    while rclpy.ok():
        rclpy.spin_once(minimal_client)
        if minimal_client.future.done():
            try:
                response = minimal_client.future.result()
            except Exception as e:
                minimal_client.get_logger().info(
                    'Service call failed %r' % (e,))
            else:
                minimal_client.get_logger().info(
                    'Result of add_two_ints: for %d + %d = %d' %
                    (minimal_client.req.a, minimal_client.req.b, response.sum))
            break

    minimal_client.destroy_node()
    rclpy.shutdown()


if __name__ == '__main__':
main()
```

11.2.3.2.2 Explications du code

Le seul `import` différent pour le client est `import sys`. Le code du node client utilise `sys.argv` pour avoir accés à la ligne de commande afin d'extraire les arguments pour la requête.

La définition du constructeur crée un client avec le même type et nom que le node de service. Le type et nom doivent être les mêmes pour le client et le service pour être capable de communiquer.

La boucle `while` dans le constructeur vérifie si un service avec le même nom et le même type sont disponibles chaque seconde.

En dessous du constructeur se trouve la définition de la requête, suivi de `main`.

11.2.3.2.3 Ajouter un point d'entrée

Comme le node de service, vous devez aussi ajouter un point d'entrée pour être capable d'exécuter le node client.

Le champ `entry_points` du fichier `setup.py` qui doit ressembler à ceci :

```
entry_points={
    'console_scripts': [
        'service = py_srvcli.service_member_function:main',
        'client = py_srvcli.client_member_function:main',
    ],
},
```

11.2.4 Lancer les nodes

11.2.4.1 Dépendences

Cette partie n'est valable que sous Linux.

Les paquets `rclpy`, `std_msgs` sont normalement déjà installés sur le système ROS-2. C'est une bonne pratique de lancer `rosdep` à la racine de l'espace de travail `dev_ws` pour vérifier les dépendences avant de construire le paquet :

```
rosdep install -i --from-path src --rosdistro <distro> -y
```

11.2.4.2 Compiler

11.2.4.2.1 C++

Il suffit ensuite de lancer la commande suivante dans un terminal :

```
colcon build --packages-select cpp_srvcli
```

11.2.4.2.2 Python

Il suffit ensuite de lancer la commande suivante dans un terminal :

```
colcon build --packages-select py_srvcli
```

11.2.4.3 Sourcer l'environnement

Afin de pouvoir lancer les nodes il suffit de sourcer le fichier suivant :

```
source ./dev_ws/install/setup.bash
```

11.2.4.4 Lancer les nodes

11.2.4.4.1 C++

Dans un terminal il faut lancer la commande suivante :

```
ros2 run cpp_srvcli server
```

Le terminal doit retourner le message suivant et ensuite attendre :

```
[INFO] [rclcpp]: Ready to add two ints.
```

Il faut ouvrir un autre terminal, sourcer les fichiers de configuration dans le répertoire `dev_ws`. Il faut ensuite démarrer le node client, suivi par deux entiers séparés par un espace :

```
ros2 run cpp_srvcli client 2 3
```

Si vous choisissez 2 ou 3 par exemple, le client doit recevoir une réponse comme :

```
[INFO] [rclcpp]: Sum: 5
```

En retournant dans le terminal où le node de service fonctionne, les messages de logs indiquent les requêtes et les données reçues, et la réponse retournée.

```
[INFO] [rclcpp]: Incoming request
a: 2 b: 3
[INFO] [rclcpp]: sending back response: [5]
```

Les nodes sont arrêtés avec `Ctrl+C`.

11.2.4.4.2 Python

Dans un terminal il faut lancer la commande suivante :

```
ros2 run py_srvcli service
```

Le node attend la requête d'un client.

Il faut alors ouvrir un autre terminal sourcer les fichiers de configuration dans le répertoire de travail `dev_ws`. Il faut démarrer le node client suivi de deux entiers séparés par un espace :

```
ros2 run py_srvcli client 2 3
```

Si vous choisissez 2 ou 3 par exemple, le client va recevoir une réponse qui ressemble à :

```
[INFO] [minimal_client_async]: Result of add_two_ints: for 2 + 3 = 5
```

Il faut retourner au terminal où le node de service a été lancé. Celui affiche un message correspondant à la requête reçue :

```
[INFO] [minimal_service]: Incoming request
a: 2 b: 3
```

Il faut utiliser `Ctrl+C` pour arrêter les nodes.

V Tutoriaux sur TIAGo

12 TIAGo 143
12.1 Installation

13 Contrôle 145
13.1 Téléopérer la base mobile avec le clavier
13.2 Faire bouger la base à travers des commandes en vitesse
13.3 Contrôleur des trajectoires articulaires
13.4 Bouger des articulations individuellement
13.5 Contrôle de la tête
13.6 Rejouer des mouvements du haut du corps prédéfinis

14 TIAGo Navigation 155
14.1 Créer une carte avec gmapping
14.2 Localisation et planification de mouvement

15 Planification de mouvements 161
15.1 Planifier dans l'espace de configuration
15.2 Planifier dans l'espace cartésien
15.3 Planification avec Octomap
15.4 Démonstration d'une prise et d'un replacement d'un objet

12. TIAGo

12.1 Installation

L'installation des tutoriaux est faite dans l'espace de travail **tiago_tutorials_ws**.

13. Contrôle

13.1 Téléopérer la base mobile avec le clavier

La version originale de ce tutorial est disponible en anglais ici ici.

13.1.1 But

Le but de ce tutorial est de montrer que l'on peut faire bouger un robot de la même façon que les tortues dans leur interface graphique (voir la section 3.1.3 pour ROS-1 et la section **??** pour ROS-2).

Lancer deux consoles et assurez vous que votre environnement est bien configuré (cf Section 12.1).

13.1.2 Lancer la simulation

Cette simulation lance un robot TIAGo version Titanium (c'est avec une main noire) et un environnement de bureau avec des gens :

```
roslaunch tiago_gazebo tiago_gazebo.launch public_sim:=true robot:=titanium world:=simple_office_with_people
```

Pour Noetic avec la VMWare il vaut mieux utiliser le TIAGo version Steel avec la commande suivante :

```
roslaunch tiago_gazebo tiago_gazebo.launch public_sim:=true robot:=steel world:=simple_office_with_people
```

L'environnement chargé par la simulation est représenté dans la figure Fig.13.1.

13.1.3 Faire bouger le robot

Pour faire bouger le robot d'avant en arriére avec les touches, il faut lancer la commande suivante :

```
rosrun key_teleop key_teleop.py
```

La commande s'effectue grâce aux touches du clavier avec les flèches (haut, bas, gauche,droite).

13.2 Faire bouger la base à travers des commandes en vitesse

Ce tutorial est disponible en Anglais ici.

13.2.1 But

Le but de cet tutorial est de faire bouger le robot TIAGo en envoyant des commandes sur un topic.

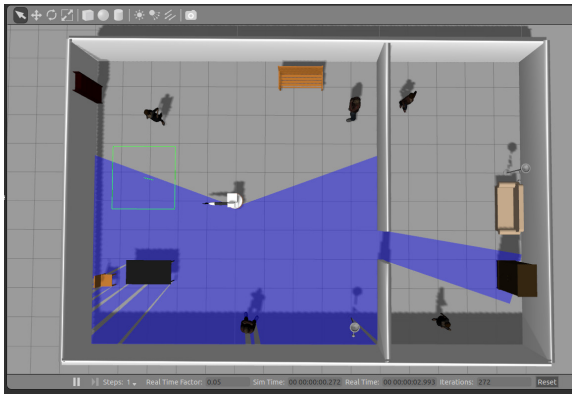


FIGURE 13.1 – TIAGo dans un bureau avec des humains

13.2.2 Lancer la simulation

La simulation proposée ici lance le robot TIAGo en version steel et sans environnement :

```
roslaunch tiago_gazebo tiago_gazebo.launch public_sim:=true robot:=steel world:=empty
```

L'environnement chargé par la simulation est représenté dans la figure Fig.13.2.

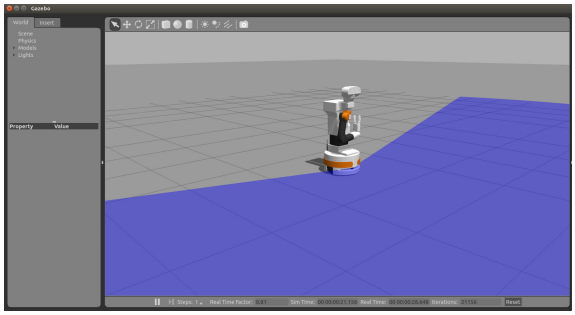


FIGURE 13.2 – Gazebo environnement vide

13.2.3 Faire bouger le robot en avant et en arrière

Pour cela on va utiliser une commande en vitesse publiée sur un topic.

```
rostopic pub /mobile_base_controller/cmd_vel geometry_msgs/Twist "linear:
  x: 0.5
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0" -r 3
```

ou il est possible également de l'écrire dans la forme suivante :

```
rostopic pub /mobile_base_controller/cmd_vel geometry_msgs/Twist -r 3 -- '[0.5,0.0,0.0]' '[0.0, 0.0, 0.0]'
```

L'option *-r 3* permet de répéter la commande toutes les 3 secondes. Les commandes sont limitées dans le temps pour éviter les accidents.

13.3 Contrôleur des trajectoires articulaires

Ce tutoriel est disponible en Anglais ici.

13.3.1 But du tutoriel

Le but est d'utiliser le contrôleurjoint_trajectory_controller de ros_control pour bouger le bras de TIAGo. L'action peut-être utilisée pour exprimer la trajectoire du bras par plusieurs points intermédiaires.

Il y a deux interfaces pour envoyer des trajectories au robot : l'interface des actions, et les topics. Les deux utilisent le message **trajectory_msgs/jointTrajectory** pour spécifier les trajectoires.

Pour le TIAGO les interfaces suivantes sont disponibles : **torso control**, **head control**, **arm control**.

13.3.2 Contrôleur du torse

Le joint prismatic du torse peut être contrôlé avec les interfaces de trajectoires articulaires suivantes :

— /torso_controller/command (trajectory_msgs/jointTrajectory)
Interface de type topic permettant de bouger le torse

— /torso_controller/follow_joint_trajectory (control_msgs/FollowJointTrajectoryAction)
Interface de type action pour bouger le torse

13.3.3 Contrôleur de la tête

Les 2 joints de la tête peuvent être contrôlés en définissant des trajectoires et en utilisant les interfaces suivantes :

— /head_controller/command (trajectory_msgs/JointTrajectory)
Interface de type topic pour bouger la tête.

— /head_controller/follow_joint_trajectory (control_msgs/FollowJointTracjectoryAction)
Interface de type action pour bouger la tête.

13.3.4 Contrôleur du bras

Les trajectoires des 7 joints du bras peuvent être controlés par les interfaces suivantes :

— /arm_controller/command (trajectory_msgs/JointTrajectory)
Interface de type topic pour bouger le bras.

— /arm_controller/follow_joint_trajectory (control_msgs/FollowJointTracjectoryAction)
Interface de type action pour bouger le bras.

13.3.5 Contrôleur du grippeur Hey5

Pour la version titanium du robot TIAGO le control de l'interface de la main est aussi disponbile par les interfaces suivantes :

— /hand_controller/command (trajectory_msgs/JointTrajectory)
Interface de type topic pour bouger les 3 moteurs de la main Hey5.

— /hand_controller/follow_joint_trajectory (control_msgs/FollowJointTracjectoryAction)
Interface de type action pour bouger les 3 moteurs de la main Hey5.

13.3.6 Contrôleur du grippeur

Pour la version steel du robot TIAGO le contrôle de l'interface du gripper est disponible par les interface suivantes :

— /parallel_gripper_controller/command (trajectory_msgs/JointTrajectory)
Interface de type topic pour bouger le joint virtuel du gripper parallèle, i.e. pour contrôler la séparation des doigts du grippeur.

— /parallel_gripper_controller/follow_joint_trajectory (control_msgs/FollowJointTracjectoryAction)
Interface de type action pour contrôler la séparation de doigts du grippeur.

— /gripper_control/command (trajectory_msgs/JointTrajectory)
Interface de type topic pour bouger les deux moteurs du grippeur, i.e. pour chaque doigt individuellement.

13.3.7 Lancer la simulation

Cette simulation lance un robot TIAGo version Steel avec un monde vide :

```
roslaunch tiago_gazebo tiago_gazebo.launch public_sim:=true robot:=steel world:=empty
```

13.3.8 Lancer le node

Si les commandes des moteurs est spécifée dans l'espace articulaire et plus particulièrement dans cet ordre : [arm_1_joint,arm_2_joint,arm_3_joint,arm_4_joint,arm_5_joint,arm_6_joint,arm_7_joint], considérons deux points dans cet espace : $[0.2, 0.0, -1.5, 1.94, -1.57, -0.5, 0.0]$, $[2.5, 0.2, -2.1, 1.9, 1.0, -0.5, 0.0]$.

La commande suivante effectue un mouvement entre les deux positions spécifiées :

```
rosrun tiago_trajectory_controller run_traj_control
```

Le code donnée ci-dessous implémente la séquence suivante :

— Créer un action client pour contrôler le bras.
— Attendre que le serveur de l'action permettant le contrôle du bras soit en marche.
— Générer une trajectoire simple avec deux points intermédiaires (dont le but)
— Envoyer la commande pour démarrer la trajectoire.
— Donner la durée de l'exécution de la trajectoire.

Il faut faire attention à la vitesse spécifiée lors du passage au premier point intermédiaire si elle est à zéro cela risque de provoquer une discontinuité.

```
// C++ standard headers
#include <exception>
#include <string>

// Boost headers
#include <boost/shared_ptr.hpp>

// ROS headers
#include <ros/ros.h>
#include <actionlib/client/simple_action_client.h>
#include <control_msgs/FollowJointTrajectoryAction.h>
#include <ros/topic.h>


// Our Action interface type for moving TIAGo's head, provided as a typedef for convenience
typedef actionlib::SimpleActionClient<control_msgs::FollowJointTrajectoryAction> arm_control_client;
typedef boost::shared_ptr< arm_control_client>  arm_control_client_Ptr;


// Create a ROS action client to move TIAGo's arm
void createArmClient(arm_control_client_Ptr& actionClient)
{
  ROS_INFO("Creating action client to arm controller ...");

  actionClient.reset( new arm_control_client("/arm_controller/follow_joint_trajectory") );

  int iterations = 0, max_iterations = 3;
  // Wait for arm controller action server to come up
  while( !actionClient->waitForServer(ros::Duration(2.0)) && ros::ok() && iterations < max_iterations )
  {
    ROS_DEBUG("Waiting for the arm_controller_action server to come up");
    ++iterations;
  }

  if ( iterations == max_iterations )
    throw std::runtime_error("Error in createArmClient: arm controller action server not available");
}


// Generates a simple trajectory with two waypoints to move TIAGo's arm
void waypoints_arm_goal(control_msgs::FollowJointTrajectoryGoal& goal)
{
  // The joint names, which apply to all waypoints
  goal.trajectory.joint_names.push_back("arm_1_joint");
  goal.trajectory.joint_names.push_back("arm_2_joint");
  goal.trajectory.joint_names.push_back("arm_3_joint");
  goal.trajectory.joint_names.push_back("arm_4_joint");
  goal.trajectory.joint_names.push_back("arm_5_joint");
  goal.trajectory.joint_names.push_back("arm_6_joint");
  goal.trajectory.joint_names.push_back("arm_7_joint");

  // Two waypoints in this goal trajectory
  goal.trajectory.points.resize(2);

  // First trajectory point
  // Positions
  int index = 0;
  goal.trajectory.points[index].positions.resize(7);
  goal.trajectory.points[index].positions[0] = 0.2;
  goal.trajectory.points[index].positions[1] = 0.0;
  goal.trajectory.points[index].positions[2] = -1.5;
```

```
  goal.trajectory.points[index].positions[3] = 1.94;
  goal.trajectory.points[index].positions[4] = -1.57;
  goal.trajectory.points[index].positions[5] = -0.5;
  goal.trajectory.points[index].positions[6] = 0.0;
  // Velocities
  goal.trajectory.points[index].velocities.resize(7);
  for (int j = 0; j < 7; ++j)
  {
    goal.trajectory.points[index].velocities[j] = 1.0;
  }
  // To be reached 2 second after starting along the trajectory
  goal.trajectory.points[index].time_from_start = ros::Duration(2.0);

  // Second trajectory point
  // Positions
  index += 1;
  goal.trajectory.points[index].positions.resize(7);
  goal.trajectory.points[index].positions[0] = 2.5;
  goal.trajectory.points[index].positions[1] = 0.2;
  goal.trajectory.points[index].positions[2] = -2.1;
  goal.trajectory.points[index].positions[3] = 1.9;
  goal.trajectory.points[index].positions[4] = 1.0;
  goal.trajectory.points[index].positions[5] = -0.5;
  goal.trajectory.points[index].positions[6] = 0.0;
  // Velocities
  goal.trajectory.points[index].velocities.resize(7);
  for (int j = 0; j < 7; ++j)
  {
    goal.trajectory.points[index].velocities[j] = 0.0;
  }
  // To be reached 4 seconds after starting along the trajectory
  goal.trajectory.points[index].time_from_start = ros::Duration(4.0);
}


// Entry point
int main(int argc, char** argv)
{
  // Init the ROS node
  ros::init(argc, argv, "run_traj_control");

  ROS_INFO("Starting run_traj_control application ...");

  // Precondition: Valid clock
  ros::NodeHandle nh;
  if (!ros::Time::waitForValid(ros::WallDuration(10.0))) // NOTE: Important when using simulated clock
  {
    ROS_FATAL("Timed-out waiting for valid time.");
    return EXIT_FAILURE;
  }

  // Create an arm controller action client to move the TIAGo's arm
  arm_control_client_Ptr ArmClient;
  createArmClient(ArmClient);

  // Generates the goal for the TIAGo's arm
  control_msgs::FollowJointTrajectoryGoal arm_goal;
  waypoints_arm_goal(arm_goal);

  // Sends the command to start the given trajectory 1s from now
  arm_goal.trajectory.header.stamp = ros::Time::now() + ros::Duration(1.0);
  ArmClient->sendGoal(arm_goal);

  // Wait for trajectory execution
  while(!(ArmClient->getState().isDone()) && ros::ok())
  {
    ros::Duration(4).sleep(); // sleep for four seconds
  }

  return EXIT_SUCCESS;
}
```

13.4 Bouger des articulations individuellement

13.4.1 Lancer la simulation

Cette simulation lance un robot TIAGo version Titanium (notamment avec une main) et un environnement de bureau avec des gens :

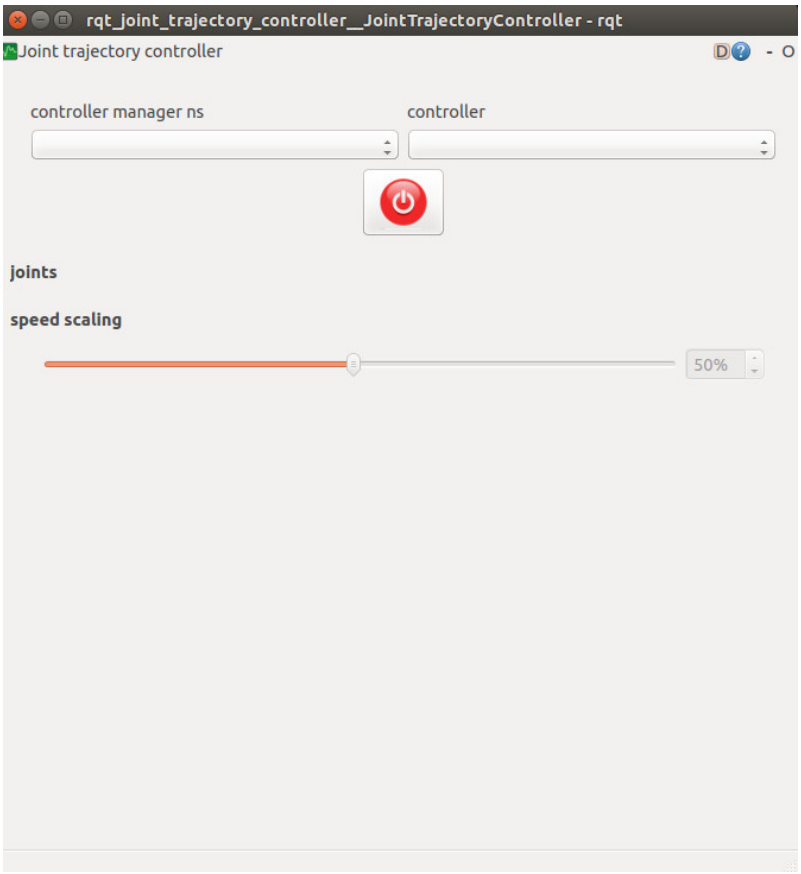
```
roslaunch tiago_gazebo tiago_gazebo.launch public_sim:=true robot:=steel world:=simple_office_with_people
```

13.4.2 Bouger les articulations individuellement

Dans une autre console, lancer la commande suivante :

```
rosrun rqt_joint_trajectory_controller rqt_joint_trajectory_controller
```

et la fenêtre suivante de la Figu.13.4.2 va apparaître. Il faut d'abord sélectionner dans la liste à gauche



le **controller_manager** pour ensuite pouvoir sélectionnner les différents groupes de controleurs qui seront dans la liste à droite (donnée aussi dans le paragraphe 13.3.2). Par exemple, si on sélectionne **arm_controller**, les curseurs de la figure Fig.13.4.2 vont apparaître dans la fenêtre. Il suffit de cliquer sur le boutton rouge pour qu'il devienne vert. Cela signifie que l'interface graphique peut envoyer des commandes sur l'interface topic de **joint_trajectory_controller** correspondant au groupe sélectionné d'articulations.

On peut noter en bas de l'interface, un facteur d'échelle qui sera appliqué aux articulations.

13.4.3 Bouger les articulations à partir de la ligne de commande

Il est possible de baisser la tête de TIAGO avec la commande suivante :

```
rosrun play_motion move_joint head_2_joint -0.6 2.0
```

cela va afficher la ligne suivante :

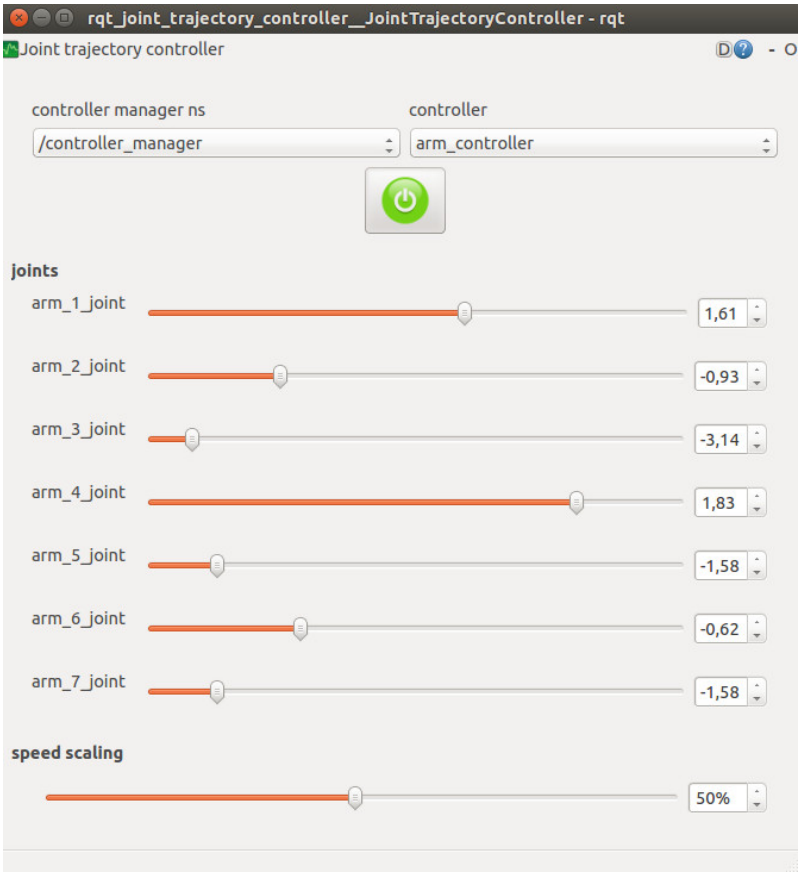
```
Moving joint head_2_joint to position -0.6 in 2.0s
```

13.5 Contrôle de la tête

Ce tutorial est disponible en Anglais ici.

13.5.1 But du tutoriel

Le but de ce tutorial est de montrer un exemple de contrôle de la tête en utilisant l'image de la caméra.



13.5.2 Lancer la simulation

```
roslaunch tiago_gazebo tiago_gazebo.launch robot:=steel public_sim:=true world:=look_to_point
```

Gazebo montre alors TIAGO en face d'une table avec des objets dans la figure Fig.13.5.2.

13.5.3 Lancer l'exemple d'un client d'action en C++

Un exemple de node C++ utilisant un client d'action est disponible dans **tiago_tutorials/look_to_point/src/look_to_point.cp**
De façon à exécuter le node il faut utiliser la commande suivante :

```
rosrun look_to_point look_to_point
```

Le node va souscrire aux topics suivants :

— /xtion/rgb/camera_info
— /xtion/rgb/image_raw

Le premier topic contient les paramètres intrinséques de la caméra et le second l'image rgb de la caméra RGBD de la tête de TIAGO.

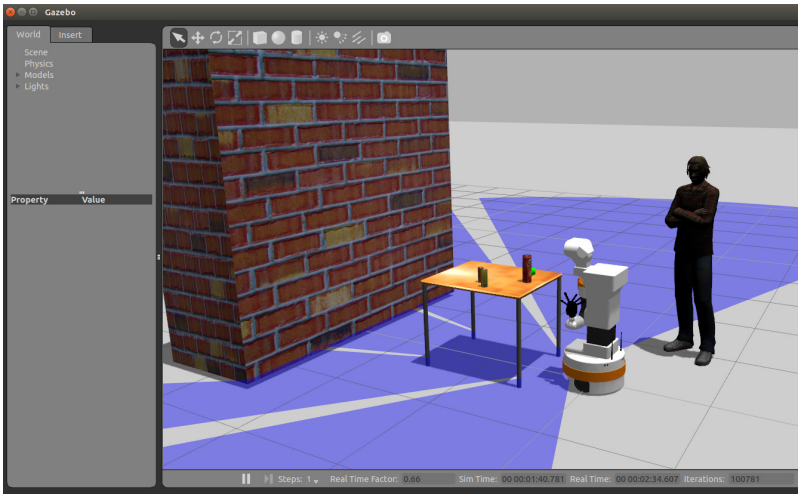
L'application va afficher l'image rgb que l'on voit dans la figure Fig.13.5.3.
Afin de générer un mouvement il suffit de cliquer sur un pixel de l'image.

13.6 Rejouer des mouvements du haut du corps prédéfinis

13.6.1 Lancer la simulation

La simulation est lancée avec la commande suivante :

```
roslaunch tiago_gazebo tiago_gazebo.launch public_sim:=true robot:=steel world:=empty
```





Quand la simulation est démarée, le serveur d'action **play_motion** est automatiquement démarré et les mouvements définis dans **tiago_bringup/config/tiago_motions_titanium.yaml** ont été chargés dans le serveur de paramètres.

De façon à voir le nom des mouvements chargés dans le serveur de paramètres, on peut utiliser l'instruction suivante :

```
rosparam list | grep  "play_motion/motions" | grep "meta/name" | cut -d '/' -f 4
```

On peut notamment y trouver le mouvement **wave**. Les informations permettant de faire ce mouvement sont stockés dans les paramètres suivants :

```
/play_motion/motions/wave/joints
/play_motion/motions/wave/points
```

Dans ce cas le paramètre **joints** spécifie quelless sont les articulations qui sont utilisés dans le mouvement et dns quel ordre :

```
$ rosparam get /play_motion/motions/wave/joints
  [arm_1_joint, arm_2_joint, arm_3_joint, arm_4_joint, arm_5_joint, arm_6_joint, arm_7_joint]
```

La trajectoire articulaire est donnée sous forme de séquence temporelle de points. La définition exacte est donnée à cette adresse `http://wiki.ros.org/joint_trajectory_controller`.

En voici un exemple :

```
$ rosparam get /play_motion/motions/wave/points
- positions: [0.06337464909724033, -0.679638896132783, -3.1087325315620733, 2.0882339360702575,
    -1.1201172410014792, -0.031008601325809293, -2.0744261217334135]
  time_from_start: 0.0
- positions: [0.06335930908588873, -0.7354151774072313, -2.939624246421942, 1.8341256735249563,
    -1.1201355028397157, -0.031008601325809293, -2.0744261217334135]
  time_from_start: 1.0
- positions: [0.06335930908588873, -0.7231278283145929, -2.9385504456273295, 2.2121050027803877,
    -1.1201355028397157, -0.031008601325809293, -2.0744261217334135]
  time_from_start: 2.0
- positions: [0.06335930908588873, -0.7354151774072313, -2.939624246421942, 1.8341256735249563,
    -1.1201355028397157, -0.031008601325809293, -2.0744261217334135]
  time_from_start: 3.0
```

Chaque ligne donne la position à laquelle chaque articulation doit être à un moment donné par rapport au démarrage du mouvement.

13.6.2 Jouer un mouvement via une interface graphique

Comme **play_motion** est un serveur d'action, il est possible d'utiliser l'interface graphique des clients d'action. Pour Melodic la commande est :

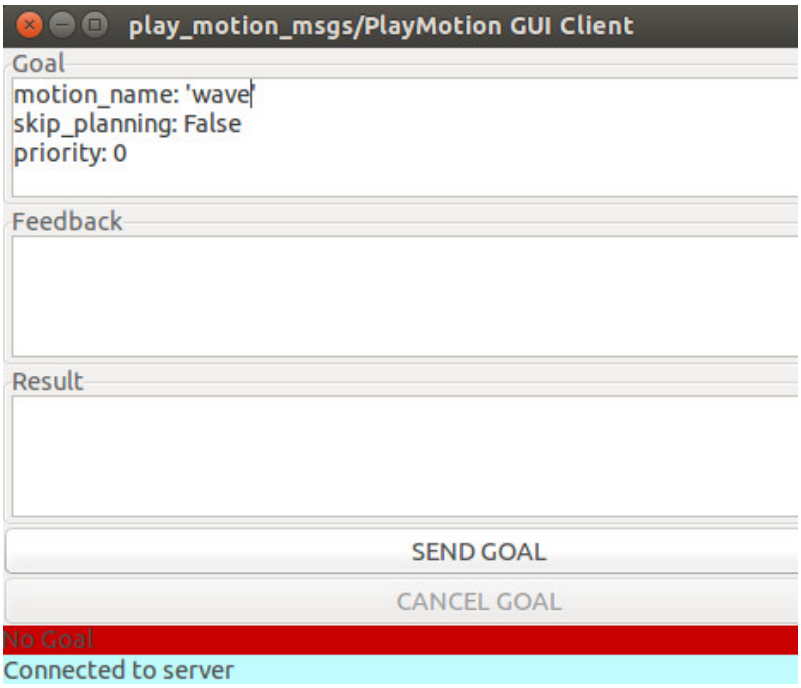
```
rosrun actionlib axclient.py /play_motion
```

Pour Noetic :

```
rosrun actionlib_tools axclient.py /play_motion
```

L'interface est représentée dans la figure Fig.13.6.2. Il faut simplement mettre le nom du mouvement dans la chaîne de caractères à côté de **motion_name**. Le champ **skip_planning** est défaut à False. Il effectue une planification de mouvements de l'état courant à la première pause pour s'assurer qu'il n'y aura pas collision. C'est de la responsabilité de l'utilisateur de s'assurer que le reste du mouvement n'est pas en collision.

En appuyant sur **SEND GOAL** la requête est envoyée au server d'action play_motion et le mouvement s'exécute.



13.6.3 Jouer un mouvement en ligne de commande

Il est possible de le faire avec le client C++ :

```
rosrun play_motion run_motion wave
```

Il ne faut pas oublier le nom du mouvement en argument du node.

Ou le client python :

```
rosrun play_motion run_motion_python_node.py home
```

14. TIAGo Navigation

14.1 Créer une carte avec gmapping

Ce tutorial est disponible en Anglais ici.

14.1.1 But

Ce tutorial montre comment créer une carte de l'environnement avec la simulation publique de TIAGo en utilisant gmapping. Cette carte est requise pour utiliser ensuite le système de localisation AMCL. Celui-ci compare les scans lasers avec la carte pour fournir une estimation fiable de la position du robot dans la carte.

14.1.2 Execution

Dans une première console il faut lancer la simulation suivante :

```
roslaunch tiago_2dnav_gazebo tiago_mapping.launch public_sim:=true
```

La simulation fournie est disponbile dans la figure Fig.14.1

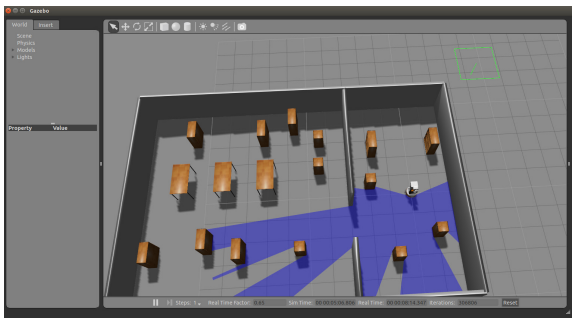


FIGURE 14.1 – TIAGo dans une bibliothèque (Simulation Gazebo).

Rviz apparaît également de façon à pouvoir visualiser la construction de la carte, comme il est possible de voir dans la figure.14.2.

Dans un second terminal, il faut lancer le node pour téléopérer le robot par le clavier.

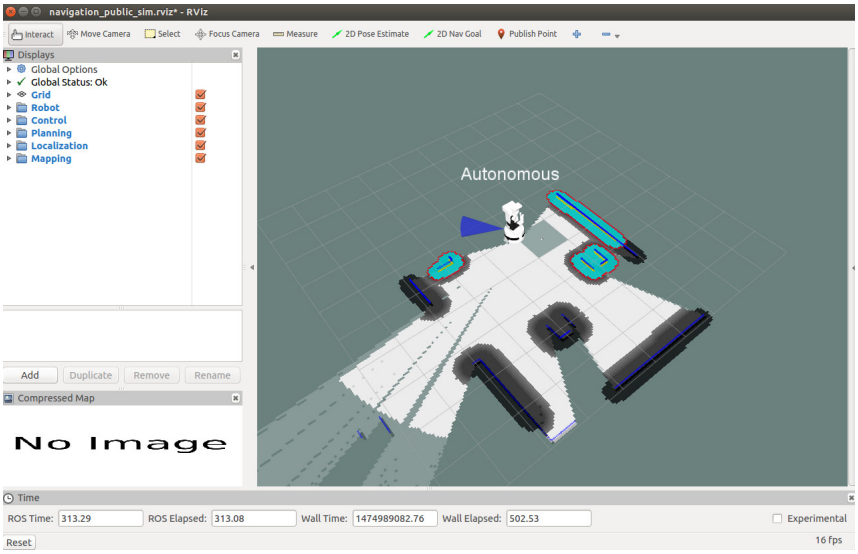


FIGURE 14.2 – Visualisation de la carte construite par tiago dans la simulation Gazebo.

```
rosrun key_teleop key_teleop.py
```

En utilisant les flèches du clavier dans cette console il est possible de guider le robot TIAGo dans le monde. La carte construite apparaît dans rviz. Quand la carte est complétement construite, on trouve l'exemple représenté dans la figure.14.3.

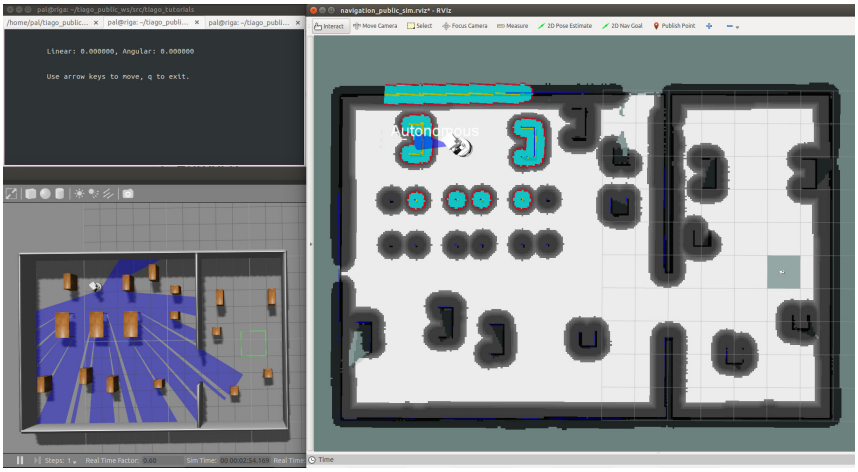


FIGURE 14.3 – Visualisation de la carte complète construite par tiago dans la simulation Gazebo.

En pressant la touche 'q' dans la console key_teleop et sauve la carte de la façon suivante :

```
rosservice call /pal_map_manager/save_map "directory: ''"
```

L'appel au service sauve la carte dans le répertoire suivant :

```
~/.pal/tiago_maps/config
```

Maintenant TIAGO est prêt à effectuer une localisation autonome et de la planification de mouvements en utilisant la carte. La section suivante montre comment il est possible d'effectuer une navigation autonome.

14.2 Localisation et planification de mouvement

Ce tutorial est disponible en Anglais ici.

14.2.1 But

Ce tutorial montre comme il est possible de faire naviguer TIAGo automatiquement lorsque l'on fournit une carte construire à partir de scans laser, et en prenant en compte le laser et la caméra RGBD pour éviter les obstacles.

Dans une première console il est possible de lancer la simulation de TIAGO avec tous les noeuds requis pour lancer la navigation autonome.

```
roslaunch tiago_2dnav_gazebo tiago_navigation.launch public_sim:=true lost:=true
```

La carte créée dans la section 14.1 est affichée dans rviz. L'information suivante est affichée sur la carte :

— **Nuage de particules** : un nuage de petites flèches rouges représentant les particules du filtre d'amcl.
— **Carte globale de coûts** : régions autour des obstacles qui sont utilisées par la planification globale de façon à calculer les chemins pour naviguer d'un point de la carte à un autre sans s'approcher trop prêt des obstacles statiques enregistrés durant la cartographie. Plus de détails sont fournis ici : `http://wiki.ros.org/costmap_2d`.
— **Carte locale des coûts** : similaire à la carte globale, mais plus petite elle bouge avec le robot. Elle est utilisée pour prendre en compte les nouveaux objets qui n'étaient pas dans la carte originale. Cette carte est utilisée par des planificateurs locaux pour éviter les obstacles, à la fois statique et dynamique, tout en essayant de suivre le chemin global calculé par le planificateur global. Plus de détails sont fournis ici : `http://wiki.ros.org/costmap_2d`.
— **Scan laser** : les lignes en bleu représentent les points mesurés avec le laser de la base mobile. Ce scan est utilisé pour ajouter/retirer des obstacles dans les cartes de coûts locales et globales.
— **RGBD scan** :les lignes de couleur magenta représentent la projection sur le sol des points 3D reconstruits par la caméra RGBD de la tête. Ce scan artificiel est utilisé pour ajouter/retirer des obstacles dans les cartes de coûts locales et globales. Ce scan est utile pour obtenir des informations 3D de l'environnement, détecter des obstacles qui sont plus haut ou plus bas que le scan laser.

14.2.2 Localization

Comme on peut le voir dans la figure.14.4, le robot est mal localisé dans sa carte. On peut voir les particules qui indiquent les positions où le robot croit être à partir des mesures qu'il fait actuellement.

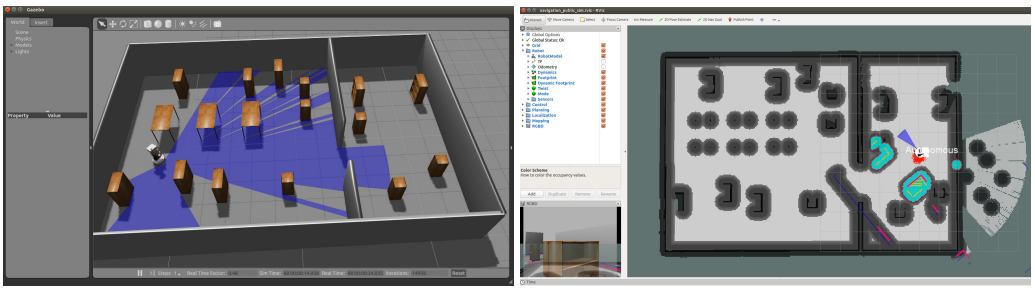


FIGURE 14.4 – Initial pose de TIAGO dans une bibliothèque simulée (à gauche) Localisation du robot dans sa carte (droite)

Pour pouvoir localiser le robot il faut lancer l'instruction suivante dans un second terminal :

```
rosservice call /global_localization "{}"
```

Ceci impose au système amcl de localisation probabiliste de répartir les particules à travers toute la carte comme on peut le voir dans la figure.14.5.

Ensuite une bonne façon de faire converger le filtre particulaire vers la bonne pose est de faire bouger le robot. Une façon sûre de faire cela est de faire tourner le robot sur lui-même. Pour cela il est possible d'utiliser les flèches droite et gauche de **key_teleop** pour le faire :

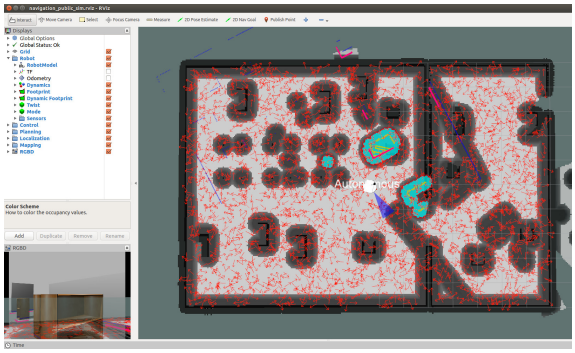


FIGURE 14.5 –

```
rosrun key_teleop key_teleop.py
```

Le résultat est illustré dans la figure.14.6, où les particules invalides disparaissent au fur et à mesure que les scans se retrouvent dans la carte et que la localisation converge éventuellement vers la pose correcte.

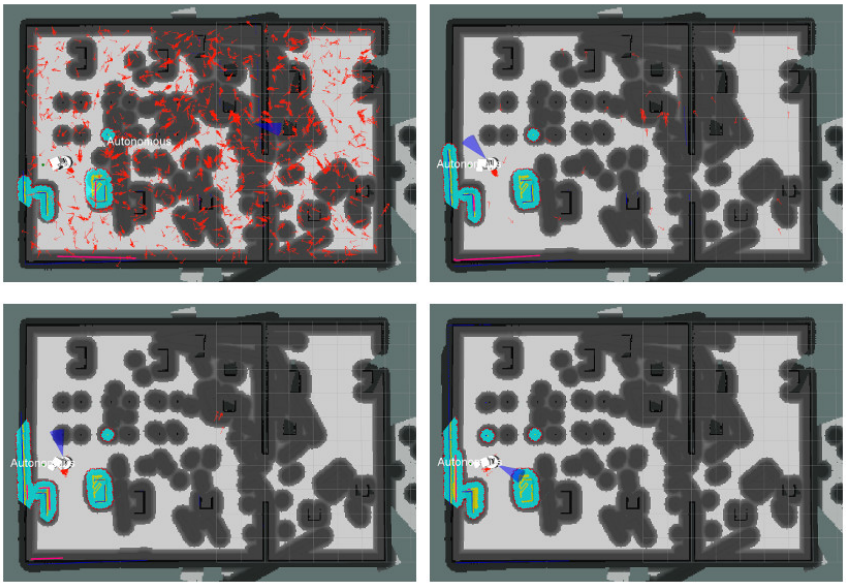


FIGURE 14.6 –

Il est maintenant préférable d'effacer les cartes de coûts car elles contiennent des informations erronées à cause de la mauvaise localisation du robot.

```
rosservice call /move_base/clear_costmaps "{}"
```

La carte de coût ne contient maintenant que les informations statiques et est prête pour la navigation.

14.2.3 Navigation autonome with rviz

Premièrement il faut s'assurer que toutes les nodes **key_teleop** soient tuées. Sinon le robot ne bougera de façon autonome car le node **key_teleop** a une priorité plus élévée.

De façon à envoyer le robot vers une autre position dans la carte il suffit d'utiliser le boutton 2D Nav Goal dans rviz comme indiqué dans la figure Fig.14.8 à gauche.

En cliquant et en maintenant le boutton gauche enfoncé sur le point dans la carte, auquel on veut envoyer le robot, une flèche verte va apparaître. En amenant la souris autour le point de la souris va changer. Dans la figure Fig.14.8 à droite, la flèche reprèsente la position de la cible et l'orientation que le robot doit atteindre.

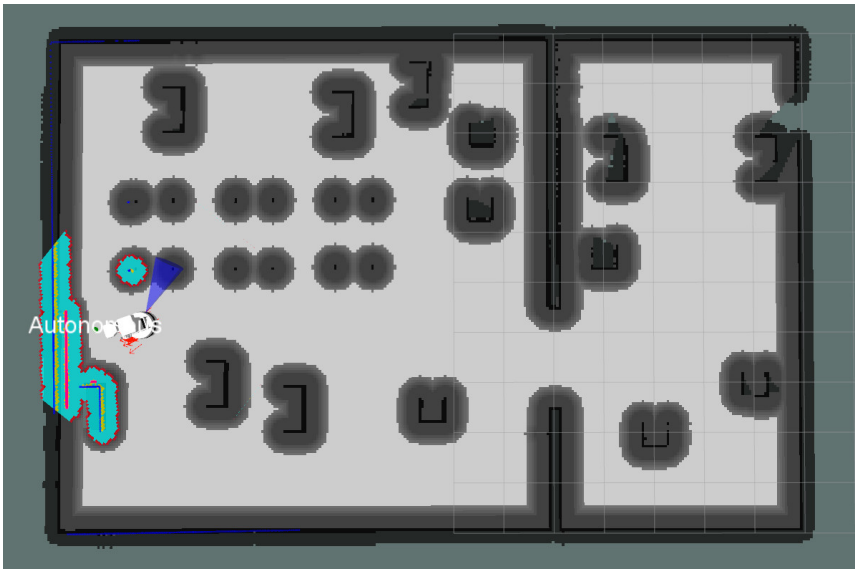


FIGURE 14.7 – Cartes de coûts sans les informations erronées.

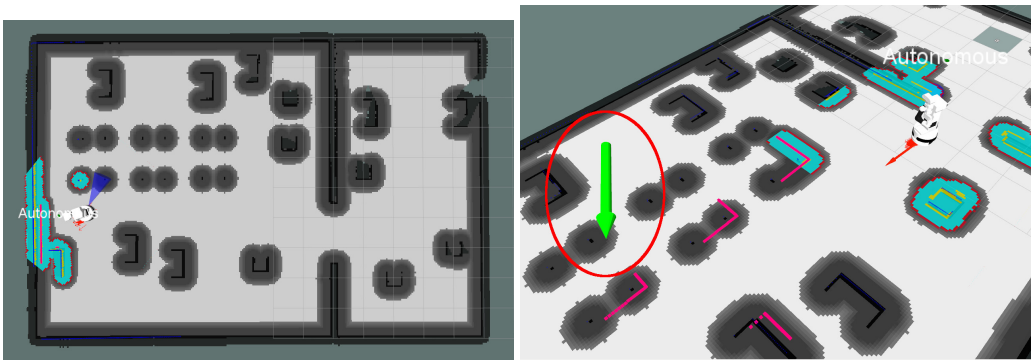


FIGURE 14.8 – Le bouton 2D Nav Goal (à gauche) Le but que TIAGo doit atteindre exprimé par une flèche en position et orientation (à gauche)

Quand le bouton est relâché le planificateur global calcule un chemin qui apparait sous forme d'une ligne bleue qui démarre de la position courante du robot et qui finit avec une flêche rouge représentant la position désirée et l'orientation.

Le robot commence à bouger en suivant la trajectoire, et ensuite va automatiquement changer cette trajectoire s'il se trouver trop prêt d'obstacles non prévus. La figure Fig.14.10 montre différentes vues de rviz et Gazebo durant la navigation vers le but sélectionné.

Finalement le robot s'arrête quand le but est atteint dans la zone de tolérance définie par l'utilisateur.

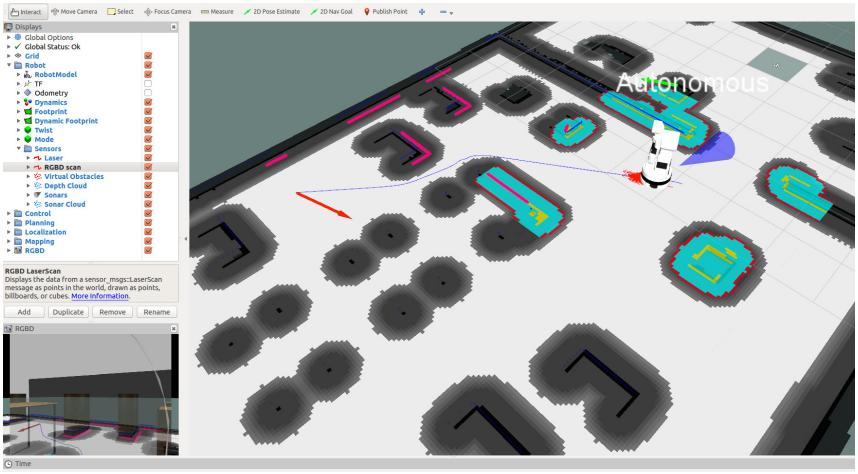


FIGURE 14.9 – Chemin planifié dans une carte de coûts.

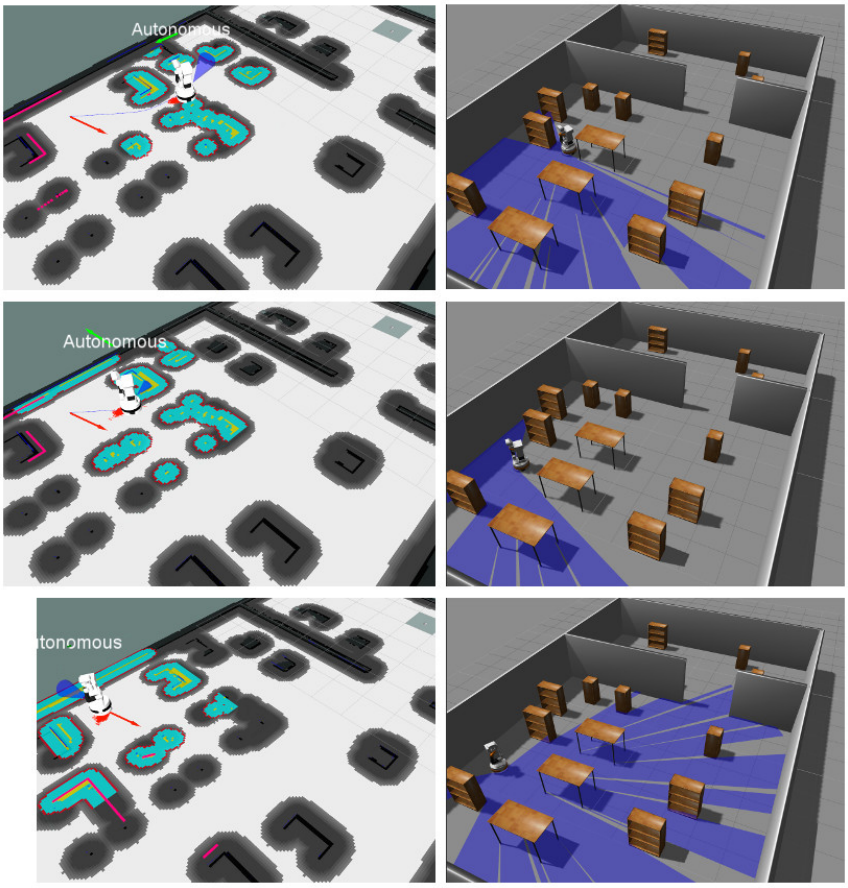


FIGURE 14.10 – Navigation

15. Planification de mouvements

15.1 Planifier dans l'espace de configuration

Ce tutorial est disponible ici en Anglais

15.1.1 But

Ce tutoriel montre comment on peut utiliser MoveIt !pour amener le group d'articulation torse-bras vers la configuration articulaire souhaitée tout en s'assurant que les limites des joints et l'auto-collision sont respectés. L'exemple est donné en C++.

15.1.2 Lancer la simulation

Dans la première console on démarre la simulation de la façon suivante :

```
roslaunch tiago_gazebo tiago_gazebo.launch public_sim:=true robot:=steel
```

Gazebo apparaît avec TIAGo comme illustré dans la figure Fig.15.3.

15.1.3 Lancer les nodes

Ce didacticiel lance un exemple qui amène le robot TIAGO vers la configuration du groupe torse-bras :

```
torso_joint: 0
arm_1_joint: 2.7
arm_2_joint: 0.2
arm_3_joint: -2.1
arm_4_joint: 2.0
arm_5_joint: 1.0
arm_6_joint:-0.8
arm_7_joint: 0
```

On peut noter que si on essait d'atteindre cette cible en bougeant chaque articulation individuellement, il est très probable que le robot se retrouverait en auto-collision. Le node qui va s'occuper de trouver un plan, i.e. une séquence de mouvements, pour atteindre cette configuration articulaire est **plan_arm_torso_fk** disponible dans **tiago_moveit_tutorial**.

Il peut être appelé de la façon suivante :

```
rosrun tiago_moveit_tutorial plan_arm_torso_fk  0 2.7 0.2 -2.1 2.0 1.0 -0.8 0
```

Un exemple de plan exécuté par le node est illustré dans la figure Fig.15.2.

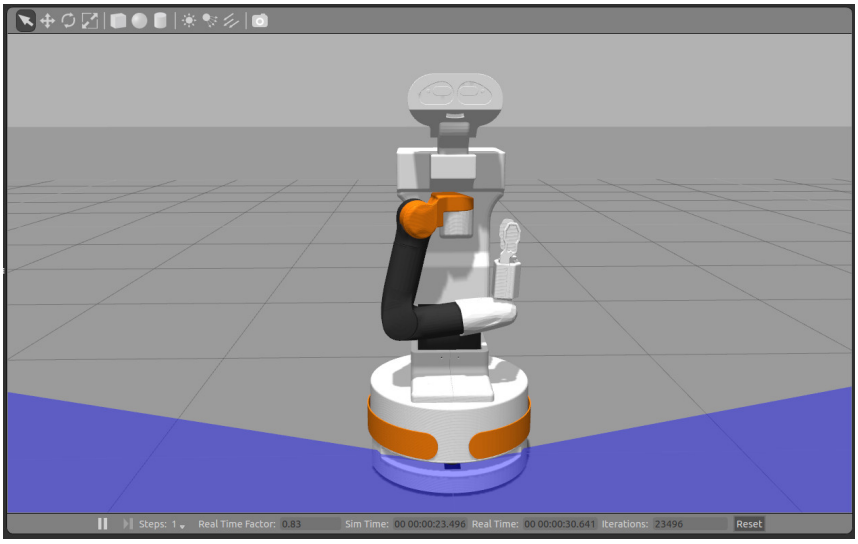


FIGURE 15.1 – TIAGo est prêt pour le planning quand son bras est bien rangé

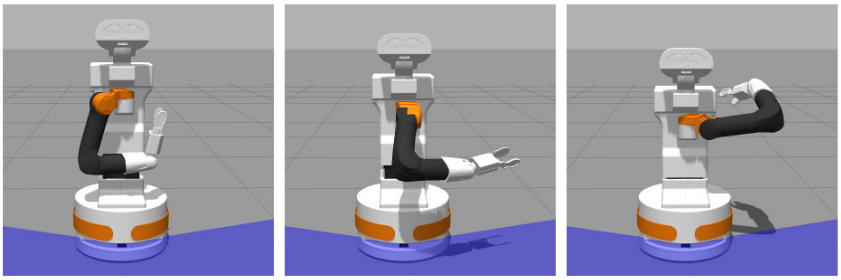


FIGURE 15.2 – Planification de mouvement entre deux configurations articulaires

15.1.4 Inspection du code

Le code pour implémenter un node capable de planifier dans l'espace articulaire est donné dans le code ci-dessous. Les parties clefs du code sont :

— Choisir un groupe de joints
— Choisir un planificateur
— Spécifier l'état initial et l'état final
— Donner du temps pour trouver un plan
— Exécuter le plan trouvé.

```
1  // ROS headers
2  #include <ros/ros.h>
3
4  // MoveIt! headers
5  #include <moveit/move_group_interface/move_group_interface.h>
6
7  // Std C++ headers
8  #include <string>
9  #include <vector>
10 #include <map>
11
12 int main(int argc, char** argv)
13 {
14   ros::init(argc, argv, "plan_arm_torso_fk");
15
16   if ( argc < 9 )
17   {
18     ROS_INFO(" ");
19     ROS_INFO("\tUsage:");
20     ROS_INFO(" ");
```

```
21      ROS_INFO("\trosrun tiago_moveit_tutorial plan_arm_torso_fk torso_lift arm_1 arm_2 arm_3 arm_4 arm_5 arm_6
          arm_7");
22      ROS_INFO(" ");
23      ROS_INFO("\twhere the list of arguments are the target values for the given joints");
24      ROS_INFO(" ");
25      return EXIT_FAILURE;
26    }
27
28    std::map<std::string, double> target_position;
29
30    target_position["torso_lift_joint"] = atof(argv[1]);
31    target_position["arm_1_joint"] = atof(argv[2]);
32    target_position["arm_2_joint"] = atof(argv[3]);
33    target_position["arm_3_joint"] = atof(argv[4]);
34    target_position["arm_4_joint"] = atof(argv[5]);
35    target_position["arm_5_joint"] = atof(argv[6]);
36    target_position["arm_6_joint"] = atof(argv[7]);
37    target_position["arm_7_joint"] = atof(argv[8]);
38
39    ros::NodeHandle nh;
40    ros::AsyncSpinner spinner(1);
41    spinner.start();
42
43    std::vector<std::string> torso_arm_joint_names;
44    //select group of joints
45    moveit::planning_interface::MoveGroup group_arm_torso("arm_torso");
46    //choose your preferred planner
47    group_arm_torso.setPlannerId("SBLkConfigDefault");
48
49    torso_arm_joint_names = group_arm_torso.getJoints();
50
51    group_arm_torso.setStartStateToCurrentState();
52    group_arm_torso.setMaxVelocityScalingFactor(1.0);
53
54    for (unsigned int i = 0; i < torso_arm_joint_names.size(); ++i)
55      if ( target_position.count(torso_arm_joint_names[i]) > 0 )
56      {
57        ROS_INFO_STREAM("\t" << torso_arm_joint_names[i] << " goal position: " << target_position[
            torso_arm_joint_names[i]]);
58        group_arm_torso.setJointValueTarget(torso_arm_joint_names[i], target_position[torso_arm_joint_names[i]]);
59      }
60
61    moveit::planning_interface::MoveGroup::Plan my_plan;
62    group_arm_torso.setPlanningTime(5.0);
63    bool success = group_arm_torso.plan(my_plan);
64
65    if ( !success )
66      throw std::runtime_error("No plan found");
67
68    ROS_INFO_STREAM("Plan found in " << my_plan.planning_time_ << " seconds");
69
70    // Execute the plan
71    ros::Time start = ros::Time::now();
72
73    group_arm_torso.move();
74
75    ROS_INFO_STREAM("Motion duration: " << (ros::Time::now() - start).toSec());
76
77    spinner.stop();
78
79    return EXIT_SUCCESS;
80  }
```

Notons qu'un plan est trouvé et exécuté avec la ligne de code suivante :

```
73    group_arm_torso.move();
```

Les commandes requises sont envoyées aux contrôleurs du bras et du torse à travers les interfaces d'action :

```
1  /arm_controller/follow_joint_trajectory/goal
2  /torso_controller/follow_joint_trajectory/goal
```

15.2 Planifier dans l'espace cartésien

Ce tutorial est disponible ici en Anglais

15.2.1 But

Ce tutoriel montre comment on peut utiliser MoveIt !pour amener le group d'articulation torse-bras vers la position souhaitée dans l'espace Cartésien tout en s'assurant que les limites des joints et l'auto-collision sont respectés. L'exemple est donné en C++.

15.2.2 Lancer la simulation

Dans la première console on démarre la simulation de la façon suivante :

```
roslaunch tiago_gazebo tiago_gazebo.launch public_sim:=true robot:=steel
```

Gazebo apparaît avec TIAGo comme illustré dans la figure Fig.15.3.

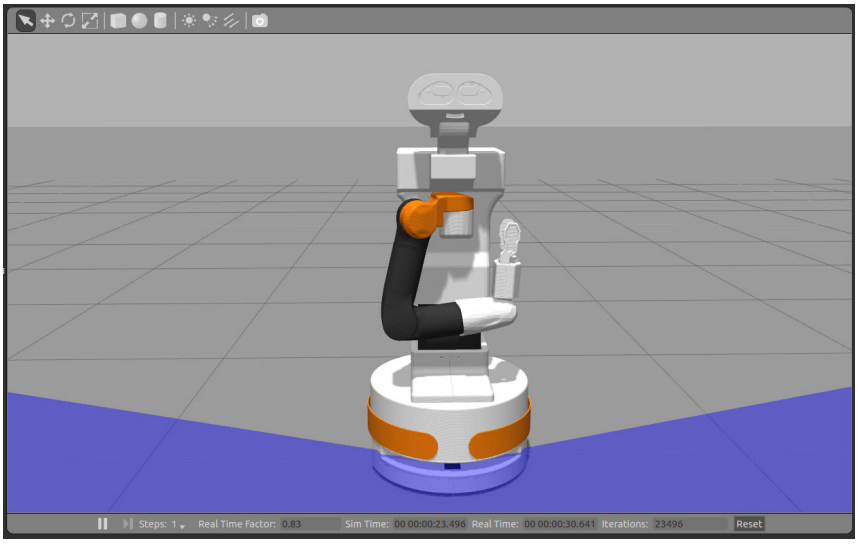


FIGURE 15.3 – TIAGo est prêt pour le planning quand son bras est bien rangé

15.2.3 Lancer les nodes

Ce didacticiel lance un exemple qui va amener le référentiel **arm_tool_link** vers la position cartisienne suivante dans le référentiel **/base_footprint**

```
x: 0.4
y: -0.3
z: 0.26
Roll: -0.011
Pitch: 1.57
Yaw: 0.037
```

qui correspond à la pose montrée dans **rviz** dans la figure Fig.15.4

De façon à atteindre cette position Cartésienne, il est possible d'utiliser le node **plan_arm_torso_ik** inclus dans le paquet **tiago_moveit_tutorial** et peut-être appellé de la façon suivante :

```
rosrun tiago_moveit_tutorial plan_arm_torso_ik 0.4 -0.3 0.26 -0.011 1.57 0.037
```

Un exemple de plan exécuté par le node est illustré dans la figure Fig.15.5. La pose finale de **/arm_tool_link** est celle désirée comme on peut le voir dans la figure Fig.15.6.

15.2.4 Inspection du code

Le code qui implémente le node capable de planifier un plan dans l'espace Cartésien est donné ci-dessous. Notons que les parties clefs de ce code sont :

— Choisir un groupe de joints
— Choisir un planificateur et définir le référentiel de référence (**base_footprint** dans ce cas)
— Spécifier la pose désirée du référentiel **arm_tool_link** encodé comme un geometry_msgs ::PoseStamped

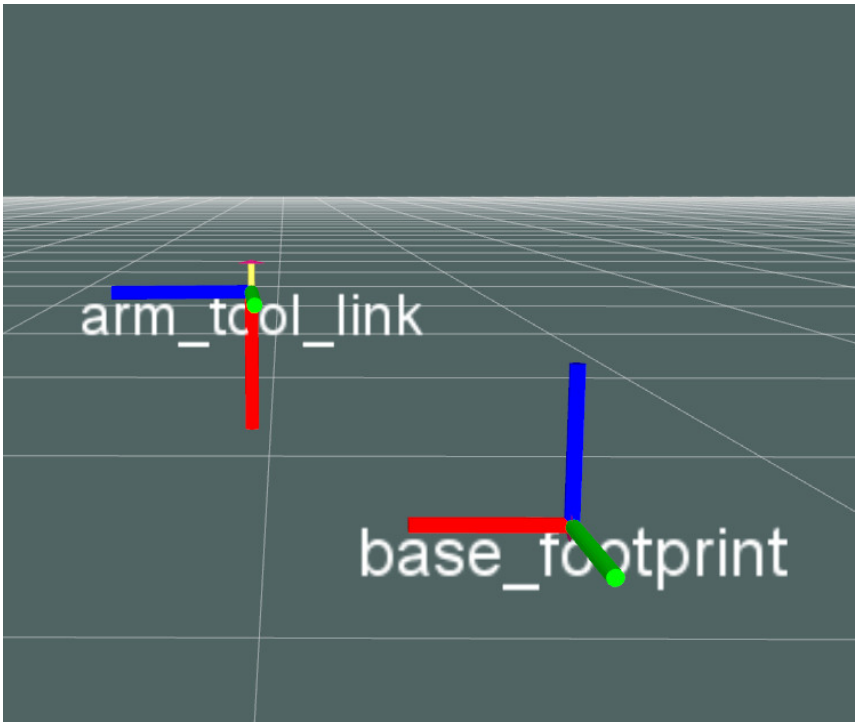


FIGURE 15.4 – Le référentiel **arm_tool_link** exprimé par rapport au référentiel **base_footprint**

— Donner du temps pour trouver un plan
— Exécuter le plan trouvé.

```cpp
// ROS headers
#include <ros/ros.h>

// MoveIt! headers
#include <moveit/move_group_interface/move_group_interface.h>

// Std C++ headers
#include <string>
#include <vector>
#include <map>

int main(int argc, char** argv)
{
  ros::init(argc, argv, "plan_arm_torso_ik");

  if ( argc < 7 )
  {
    ROS_INFO(" ");
    ROS_INFO("\tUsage:");
    ROS_INFO(" ");
    ROS_INFO("\trosrun tiago_moveit_tutorial plan_arm_torso_ik  x y z  r p y");
    ROS_INFO(" ");
    ROS_INFO("\twhere the list of arguments specify the target pose of /arm_tool_link expressed in /base_footprint
      ");
    ROS_INFO(" ");
    return EXIT_FAILURE;
  }

  geometry_msgs::PoseStamped goal_pose;
  goal_pose.header.frame_id = "base_footprint";
  goal_pose.pose.position.x = atof(argv[1]);
  goal_pose.pose.position.y = atof(argv[2]);
  goal_pose.pose.position.z = atof(argv[3]);
  goal_pose.pose.orientation = tf::createQuaternionMsgFromRollPitchYaw(atof(argv[4]), atof(argv[5]), atof(argv[6])
      );

  ros::NodeHandle nh;
  ros::AsyncSpinner spinner(1);
  spinner.start();

```

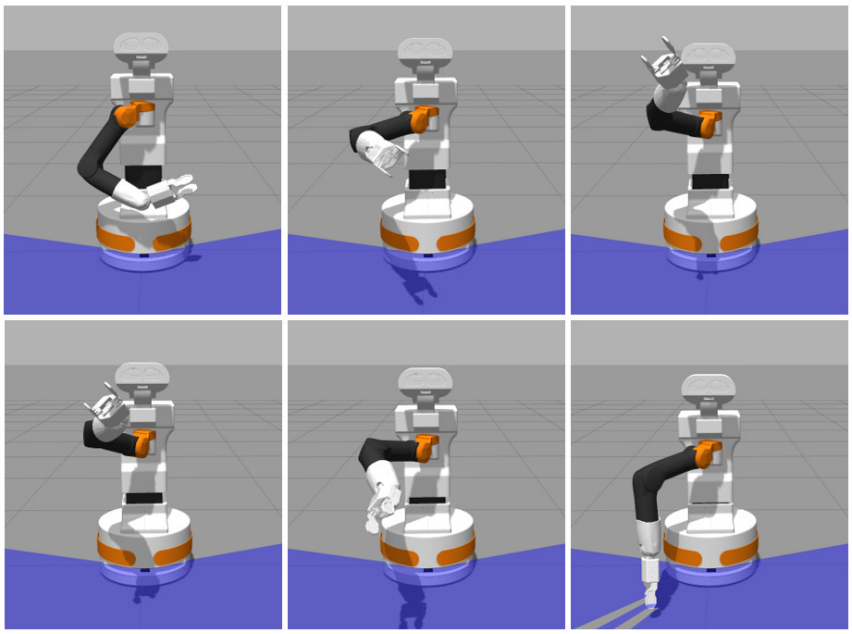


FIGURE 15.5 – La séquence de mouvements pour amener le référentiel **arm_tool_link** vers la position Cartésienne souhaitée.

```
39   std::vector<std::string> torso_arm_joint_names;
40   //select group of joints
41   moveit::planning_interface::MoveGroup group_arm_torso("arm_torso");
42   //choose your preferred planner
43   group_arm_torso.setPlannerId("SBLkConfigDefault");
44   group_arm_torso.setPoseReferenceFrame("base_footprint");
45   group_arm_torso.setPoseTarget(goal_pose);
46
47   ROS_INFO_STREAM("Planning to move " <<
48                   group_arm_torso.getEndEffectorLink() << " to a target pose expressed in " <<
49                   group_arm_torso.getPlanningFrame());
50
51   group_arm_torso.setStartStateToCurrentState();
52   group_arm_torso.setMaxVelocityScalingFactor(1.0);
53
54
55   moveit::planning_interface::MoveGroup::Plan my_plan;
56   //set maximum time to find a plan
57   group_arm_torso.setPlanningTime(5.0);
58   bool success = group_arm_torso.plan(my_plan);
59
60   if ( !success )
61     throw std::runtime_error("No plan found");
62
63   ROS_INFO_STREAM("Plan found in " << my_plan.planning_time_ << " seconds");
64
65   // Execute the plan
66   ros::Time start = ros::Time::now();
67
68   group_arm_torso.move();
69
70   ROS_INFO_STREAM("Motion duration: " << (ros::Time::now() - start).toSec());
71
72   spinner.stop();
73
74   return EXIT_SUCCESS;
75 }
```

Notons qu'un plan est trouvé et exécuté avec la ligne de code suivante :

```
68   group_arm_torso.move();
```

Les commandes requises sont envoyées aux contrôleurs du bras et du torse à travers les interfaces d'action :

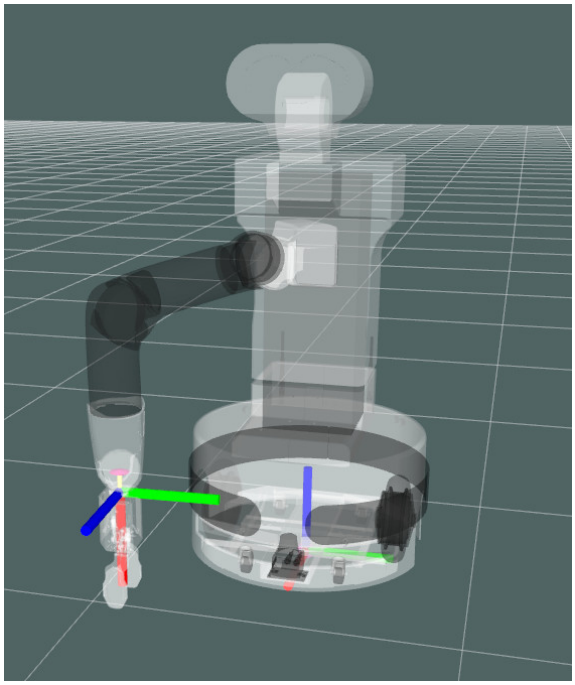


FIGURE 15.6 – La configuration finale du robot TIAGO à la fin du plan.

```
/arm_controller/follow_joint_trajectory/goal
/torso_controller/follow_joint_trajectory/goal
```

15.3 Planification avec Octomap

15.3.1 Goal

Le but de ce didacticiel est de fournir un exemple d'utilisation d'Octomap dans MoveIt !pour planifier des mouvements pour TIAGO. Le système de carte d'occupation de MoveIt ! utilise une structure appelée Octomap pour maintenir une représetation de l'environnement autour du robot. L'environnement simulé comprend une table et quelques objets. Le robot localise l'environnement avec sa caméra et construit l'Octomap. MoveIt !est ensuite utilisé pour planifier une trajectoire articulaire afin d'atteindre une position Cartésienne. MoveIt !utilise Octomap dans la librairie de détection de collision.

15.3.2 Lancer la simulation

Dans une console il faut lancer la simulation qui démarre le robot TIAGO en face d'une table avec des objets dessus :

```
roslaunch tiago_gazebo tiago_gazebo.launch public_sim:=true robot:=steel world:=tutorial_office gzpose:="-x 1.40 -
    y -2.79 -z -0.003 -R 0.0 -P 0.0 -Y 0.0" use_moveit_camera:=true
```

Gazebo va ensuite apparaître avec TIAGo comme illustré dans la figure Fig.15.7. Quand la simulation est lancée, MoveIt !est automatiquement lancé. Le paramètre **camera :=true** va permettre à MoveIt !d'utiliser la caméra de profondeur comme entrée. La configuration d'Octomap définie dans **tiago_moveit_config/launch/tiago_moveit_sensor_manager.launch.xml**, spécifie le référentiel dans lequel la représentation est stockée. Le référentiel utilisé dans cette démonstration est **odom**. Les détails à propos de la configuration d'Octomap dans MoveIt !est disponible ici : `http://docs.ros.org/indigo/api/moveit_tutorials/html/doc/pr2_tutorials/planning/src/doc/perception_configuration.html`.

Il faut attendre que TIAGo range son bras avant de lancer les prochaines étapes.

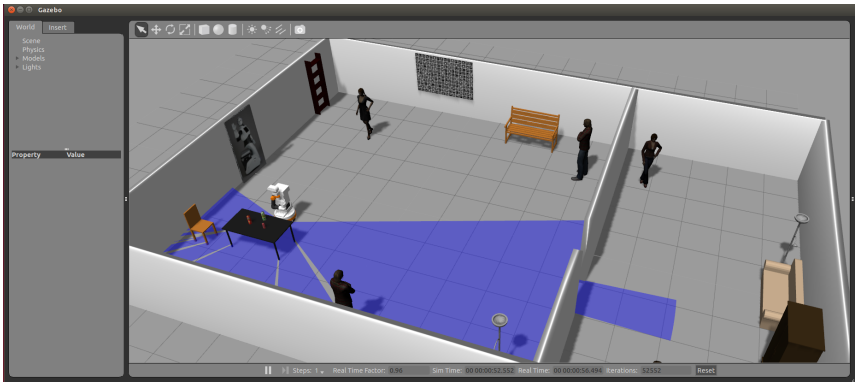


FIGURE 15.7 – Simulation de TIAGo dans Gazebo

15.3.3 Lancer les nodes

Dans un deuxième terminal il faut le node qui extrait des points clefs :

```
roslaunch tiago_moveit_tutorial octomap_tiago.launch
```

cela lance également Rviz de façon à aider la visualisation des différentes étapes de la démonstration (voir la figure Fig.15.8).

Dans une troisième console, il faut lancer un node qui va limiter la fréquence à 2 Hz avec laquelle les nuages de points 3D sont publiés. Le node publie également sur le topic auquel MoveIt !s'attend.

```
rosrun topic_tools throttle messages /xtion/depth_registered/points 2  /throttle_filtering_points/filtered_points
```

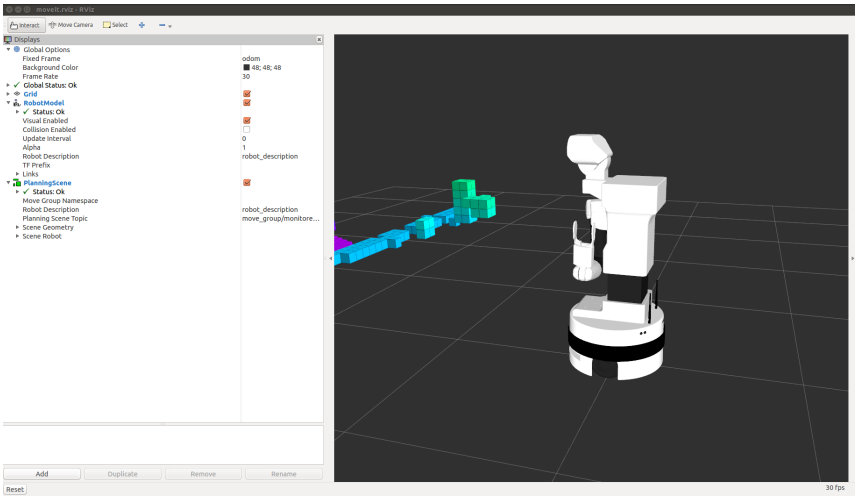


FIGURE 15.8 – Affichage dans Rviz de la carte Octomap reconstruitre par TIAGo

15.3.4 Démarrer la démonstration

Dans une troisième console le mouvement **head_look_around** doit être démarré pour construire une représentation de l'environnement. Quand la simulation est lancée, le serveur d'action **play_motion** est automatiquement lancé (voir la section 13.6). Avant de démarrer le mouvement **head_look_around**, le fichier **tiago_moveit_tutorial/config/tiago_octomap_motions.yaml** doit être chargé dans le serveur de paramètres ros.

```
rosparam load `rospack find tiago_moveit_tutorial`/config/tiago_octomap_motions.yaml
```

Afin de lancer le mouvement, il faut utiliser la ligne de commande suivante :

```
rosrun play_motion run_motion head_look_around
```

Quand le mouvement du node **head_look_around** est terminé, Rviz montre l'octomap construit qui représente l'environnement comme dans la figure Fig.15.9

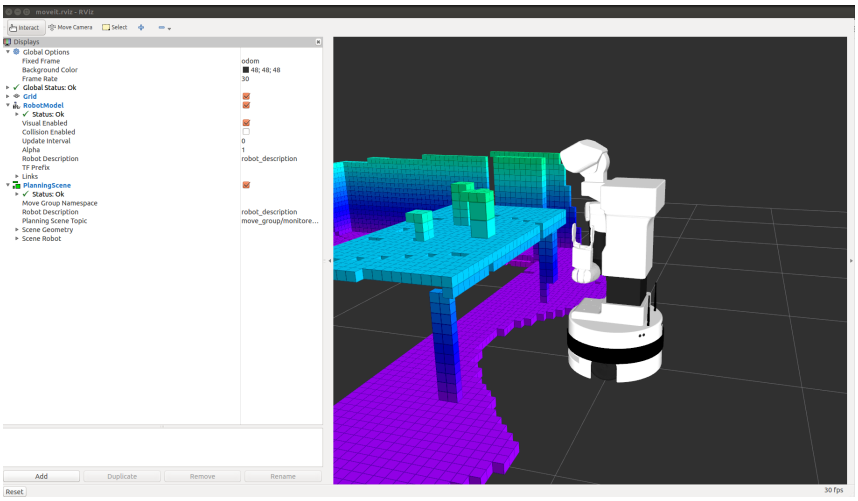


FIGURE 15.9 – Représentation de l'octomap dasn Rviz

Puis, dans la même console, il faut que le robot TIAGo se place dans une pose qui permette de démarrer la planification de mouvement :

```
rosrun play_motion run_motion unfold_arm
```

Il faut attendre que TIAGo deploit son bras. Il est alors possible de procéder aux étapes suivantes.

Quand le node de mouvement **unfold_arm** est terminé, il est possible d'amener le référentiel de l'outil du robot TIAGo à 3 configurations cartésienne par rapport au référentiel **base_footprint**.

Les 3 configurations spatiales cartésiennes avec le référentiel **base_footprint** sont :

```
x: 0.508
y: -0.408
z: 0.832
Roll: 0.005
Pitch: -0.052
Yaw: 0.039
```

```
x: 0.706
y: 0.357
z: 0.839
Roll: 0.007
Pitch: -0.041
Yaw: 0.040
```

```
x: 0.5
y: 0.35
z: 0.38
Roll: 0.003
Pitch: -0.030
Yaw: 0.039
```

Comme dans le tutorial de la section 15.2 il est possible d'utiliser le node **plan_arm_torso_ik** du paquet **tiago_moveit_tutorial**. Il faut cependant attendre que chaque node soit fini avant de lancer le suivant.

```
rosrun tiago_moveit_tutorial plan_arm_torso_ik 0.508 -0.408 0.832 0.005, -0.052, 0.039
```

```
rosrun tiago_moveit_tutorial plan_arm_torso_ik 0.706 0.357 0.839 0.007, -0.041, 0.040
```

```
rosrun tiago_moveit_tutorial plan_arm_torso_ik 0.5 0.35 0.38 0.003, -0.030, 0.039
```

Il arrive que MoveIt !affiche des erreurs internes. Cela peut-être l'échec d'avoir trouver un plan ou l'impossibilité d'exécuter la trajectoire planifiée. Dans ce cas, il faut relancer la ligne de commande qui a généré l'erreur.

Les images dans la figure Fig.15.10 représentent les 3 configurations des commandes précédentes.

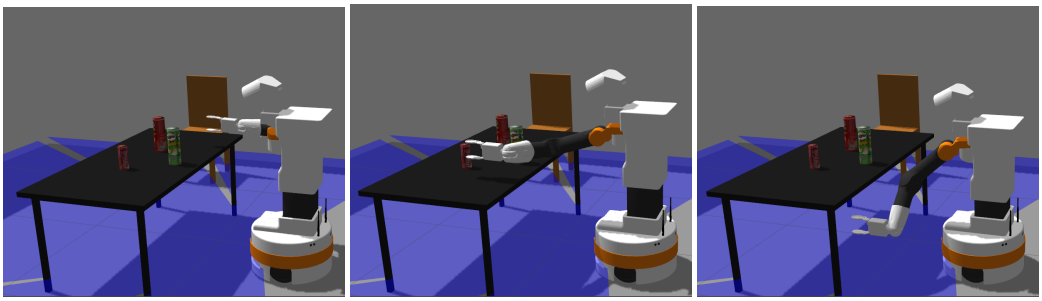


FIGURE 15.10 – Les 3 configurations du robot TIAGo trouvées pour les 3 configurations Cartésiennes demandés

15.3.5 Le plugin Rviz de MoveIt !

MoveIt !vient avec un plugin pour le visualiseur Rviz (voir les détails ici). Il s'agit d'une façon graphique de définir un nouveau but. Ce plugin permet mettre en place des scènes dans lequel le robot va travailler, générer les plans, visualiser le résultat et interagit directement avec un robot visualisé.

L'usage du plugin de MoveIt !avec Octomap dans nos simulations s'effectue de la façon suivante :

```
roslaunch tiago_moveit_config demo_tiago.launch
```

Rviz va apparaître avec TIAGo et les marqueurs interactifs. La figure Fig.15.11 montre le plugin MoveIt !avec Octomap après les mouvements **head_look_around** et **unfold_arm**.

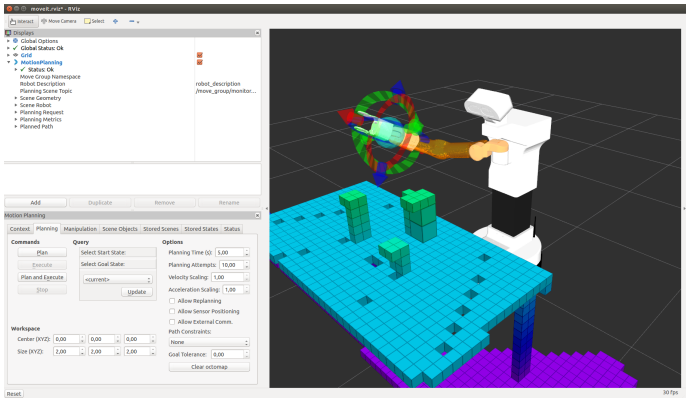


FIGURE 15.11 – TIAGo après les mouvements **head_look_around** et **unfold_arm**

15.4 Démonstration d'une prise et d'un replacement d'un objet

Ce didacticiel est disponible en Anglais ici

15.4.1 But

Le but de ce didacticiel est montrer une exemple de manipulation avec TIAGo. L'environnement simulé comprend une table et une boîte sur laquelle un marqueur ArUco. Le robot localise l'objet grâce à sa caméra RGB et fournit la pose 3D.

Alors, MoveIt !est utilisé de façon à planifier une trajectoire pour attraper un objet, le soulever, et finalement une trajectoire est planifiée pour remettre l'objet dans sa position initiale.

15.4.2 Lancer la simulation

Dans le premier terminal il faut lancer la simulation :

```
roslaunch tiago_pick_demo pick_simulation.launch
```

Gazebo se lance avec TIAGo en face de la table avec l'objet qui a le marqueur ArUco sur le dessus.

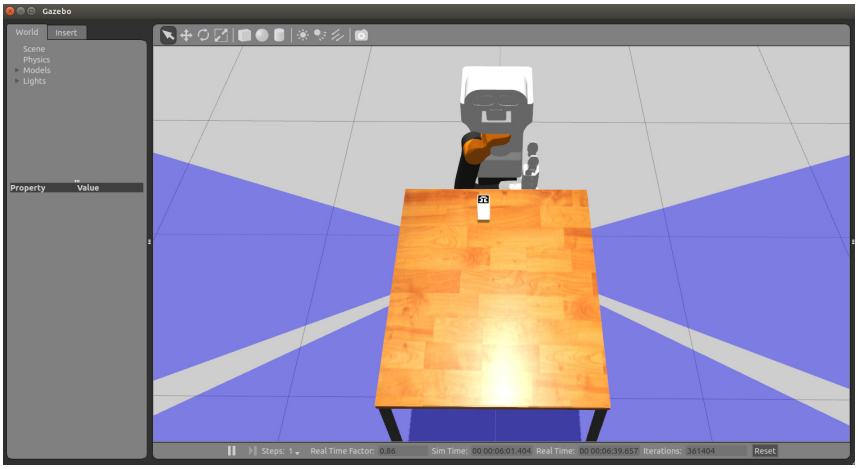


FIGURE 15.12 – Gazebo avec le robot TIAGo en face de la table et l'objet ArUco

Il faut ensuite attendre que TIAGo ait rangé son bras pour procéder aux étapes suivantes.

15.4.3 Lancer les nodes

Dans le second terminal, il faut lancer l'instruction suivante :

```
roslaunch tiago_pick_demo pick_demo.launch
```

ce qui lance les nodes suivants :

- **/aruco_single** le node qui détecte le marqueur ArUco
- **/pick_and_place_server** le node en charge de définir la scène de planification, de faire la requête pour planifier les mouvements avec MoveIt !et de les exécuter.
- **/pick_client** : le node qui prépare le robot pour la détection d'objet et les opérations de pick et place : le robot léve le bras vers une pose sans collision, et ensuite baisse la tête pour regarder la table. Il attend alors que l'objet avec un marqueur soit détecté et que la pose soit reconstruite de façon à envoyer le but pour le serveur d'action **/pick_and_place_server**.
- **/rviz** : de façon à visualiser toutes les étapes impliquées dans la démonstration.

Rviz se lance pour aider l'utilisateur à visualiser les différentes étapes du didacticiel comme on peut le voir dans la figure Fig.15.13.

15.4.4 Démarrer la démonstration

Finalement, dans le troisième terminal, le service suivant sera appellé pour démarrer l'exécution de la démonstration :

```
rosservice call /pick_gui
```

Le service utilise le node **/pick_client** pour faire bouger TIAGo vers une pause permettant de détecter l'objet et de l'attraper. Le torse de TIAGo va se lever et la tête va descendre pour regarder la table. A ce point, le marqueur ArUco sera detecté et la pose estimée est affichée dans rviz.

Une fois que le marqueur est détecté la géométrie de l'objet sera reconstruit grâce à la dimension de la boîte connue à l'avance. Le modèle de l'objet sera ajouté à la scéne de planification de MoveIt !ainsi qu'une grosse boîte en dessous de l'objet de façon à représenter la table.

Une fois que la scène est initialisée MoveIt !planifie plusieurs prises et sélectionne celle qui est la plus appropriée. Les différentes prises calculées sont montrées dans la figure Fig.15.15 sous la forme de flèches rouges qui représentent les poses du référentiel **arm_tool_link** qui permettent de faire une prise.

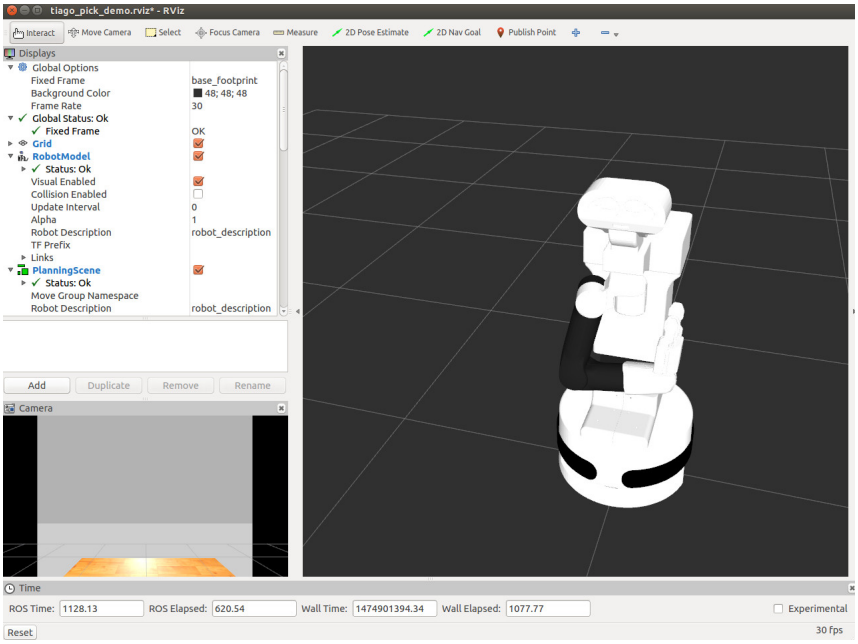


FIGURE 15.13 – Rviz visualisation de la table

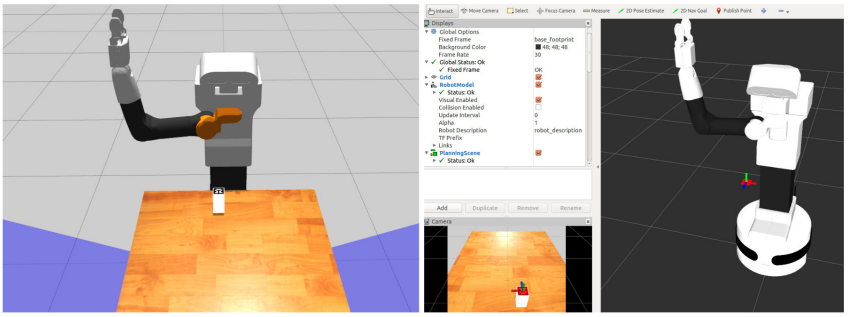


FIGURE 15.14 – Phase de détection

Un exemple d'éxécution est démontrée dans la vidéo disponible ici.

Notons que chaque exécution du plan sélectionné peut être différente à cause des algorithmes de planification utilisant des marches aléatoires. Un exemple d'un tel plan est donné dans la figure Fig.15.16 Le robot commence d'abord par exécuter la trajectoire de prise planifiée controllant l'articulation du torse et les 7 articulatons du bras.

Une fois que l'objet est pris par le robot, le robot le soulève. Ensuite une trajectoire pour reposer l'objet est planifiée pour reposer l'objet à un endroit sensiblement identique à celui où l'objet a été pris. La figure Fig.15.17 illustre une trajectoire planifiée par MoveIt !.

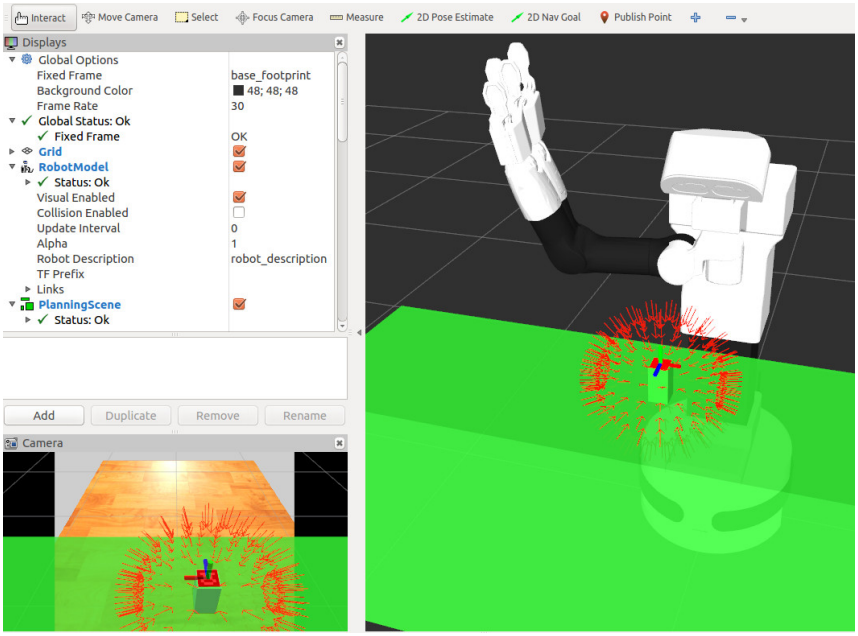


FIGURE 15.15 – Rviz affiche la scène avec laquelle le mouvement est planifié et les affordances possibles.

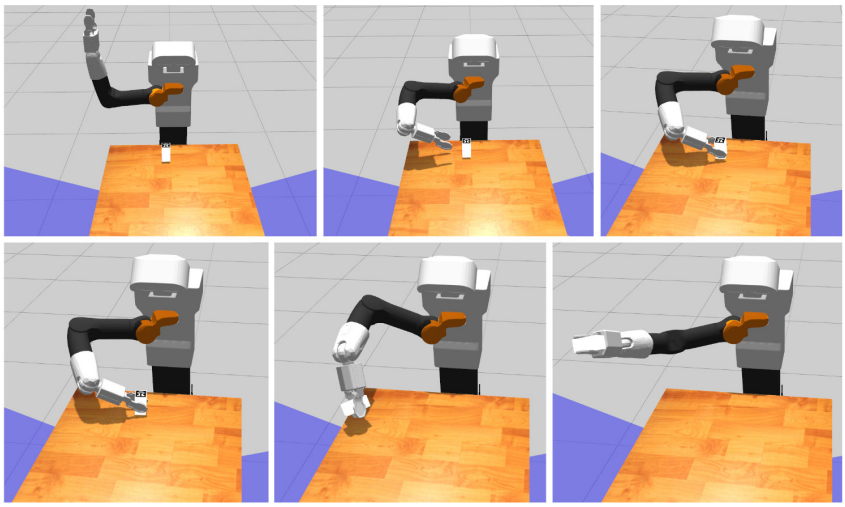


FIGURE 15.16 – Exemple du mouvement de prise par le robot TIAGo planifié par MoveIt !

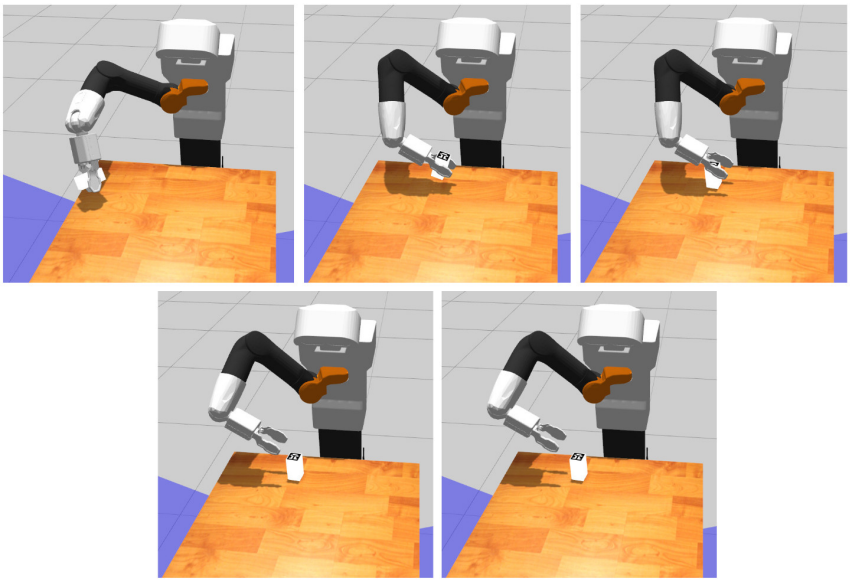


FIGURE 15.17 – Exemple du mouvement pour repose l'objet par le robot TIAGo planifié par MoveIt !

VI Navigation

16 Navigation : Nav2 177
16.1 Introduction
16.2 Concepts de Navigation
16.3 Serveurs de navigation
16.4 Estimation d'état
16.5 Représentation de l'environnement
16.6 REP-105

17 Plugins pour la navigation 185
17.1 Introduction
17.2 Ecrire un nouveau plugin Costmap2D
17.3 Exporter et construire le plugin GradientLayer
17.4 Utiliser le plugin dans Costmap2D
17.5 Lancer le plugin GradientLayer

18 Plugin de planification 191
18.1 Introduction
18.2 Création d'un nouveu plugin de planification de mouvements
18.3 Exporter le plugin de planification
18.4 Passer le nom du plugin par le fichier de paramètres
18.5 Lancer le plugin StraightLine

19 Ecrire un nouveau plugin de contrôleur 195
19.1 Introduction
19.2 Créer un nouveau plugin contrôleur
19.3 Exporter le plugin du contrôleur

16. Navigation : Nav2

16.1 Introduction

Les éléments suivants ont pour source `https://navigation.ros.org/` et en sont pour une large partie la traduction.

Le projet Nav2 est le successeur spirituel de la pile de Navigation ROS. Ce projet chercher à trouver un chemin sûr pour un robot mobile afin d'aller d'un point A à un point B. Il peut être également utiliser dans d'autres applications qui implique de la navigation comme passer par des points de contrôle, faire d'autres types de tâches comme du suivi d'objet, couvrir un espace et plus encore. Ceci implique de la planification dynamique, de calculer des vitesses pour les moteurs, d'éviter des obstacles, et des comportements de recouvrements. Nav2 a un très haut niveau d'intégration et de test : il est utilisé par de nombreuses sociétés dans le monde. Il est possible d'avoir plus d'informations ici : `https://navigation.ros.org/about/index.html#about`.

Nav2 utilise les arbres de comportement (BT pour Behavior Trees en Anglais) pour appeler les serveurs modulaires afin de réaliser une action. Une action peut consister à calculer un chemin, une commande en effort, d'effectuer une action de réparation, ou tout autre tâche liée à la navigation. Chacun est un node séparé qui communique avec l'arbre de comportement (AC-BT) à travers un serveur d'action ROS. Le diagramme 16.1 donne une bonne vue globale de la structure de Nav2.

> **Note 16.1** Il est possible d'avoir des plugins pour les contrôleurs, les planificateurs, et les comportements de récupération avec les plugins de comportement (AC-BT) correspondants. Cela permet de créer des comportements de navigation contextualisés. Une comparaison entre ce projet et la navigation ROS-1 est disponible ici. ▪

Les entrées attendues de Nav2 sont des transformations TF conformément à la REP-105, une carte source si la couche Static Costmap est utilisée, un fichier BT XML, et toutes informations pertinentes issues de capteurs. Il fournira alors des commandes en vitesse valide pour tout robot holonomique ou non holonomiques. Le système supporte tous les types majeurs de robots : holonomique, à roues différentielles, à pattes, et les bases ackermann (voiture). Ils ne sont supportés qu'à travers des formes circulaires et des formes fixées arbitrairement pour de la détection de collison dans SE(2).

Les outils sont les suivants :

— Charge, fournit et sauve des cartes (Map Server)
— Créer des cartes en utilisant des informations fournies par un LiDAR (SLAM Toolbox)
— Localise le robot dans une carte (AMCL)

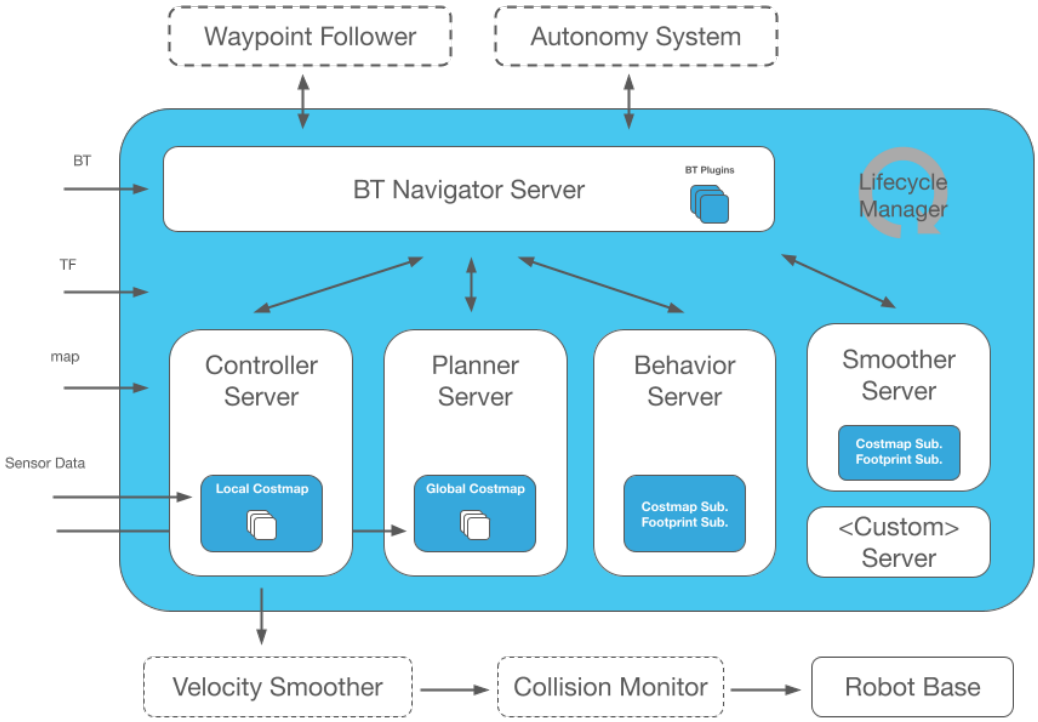


FIGURE 16.1 – Architecture globale de la pile de navigation Nav2

— Planifie un chemin du point A au point B autour des obstacles (Nav2 Planner)
— Contrôle le robot pour qu'il suive le chemin (Nav2 Controller)
— Lisse les chemins planifiés pour les rendre continus et faisables (Nav2 Smoother)
— Convertit les données capteurs dans une carte coût représentant le monde (Nav2 Costmap 2D)
— Construit des comportements de robot complexes en utilisant des arbres de comportements (Nav2 Behavior Tress and BT Navigator)
— Suit des successions de points intermédiaires (Nav2 Waypoint Follower)
— Gére le cycle de vie et les sécurités (chien de garde) des serveurs (Nav2 Lifecycle Manager)
— Des plugins pour permettre de faire votre propre algorithme et comportement (Nav2 Core)

Nav2 fournit également un ensemble de plugins pour démarrer. NavFn calcule le chemin le plus court d'une pose vers une pose cible en utilisant A* ou l'algorithme de Dijkstra. DWB utilise l'algorithme DWA pour calculer une commande en effort pour suivre un chemin, avec plusieurs plugins pour évaluer les trajectoires. Il y a des comportements permettant de résoudre certains problèmes. Ceux-ci incluent : attente, tourner sur place, effacer les cartes de coût, et revenir. Un ensemble de plugins BT sont également fournis. Ils appellent les serveurs pré-cités et calcule les informations nécessaires. Finalement un ensemble de plugins pour rviz est inclus pour interagir avec la pile de navigation et contrôler le cycle de vie des différents serveurs et nodes. Une liste de tous les plugins reportée par les utilisateurs est disponible ici.

16.2 Concepts de Navigation

Cette section est issue du lien disponible ici. Elle est résumée ici puisque de nombreux concepts généraux sont disponibles dans les autres parties du document. Seules les informations spécifiques à la pile de Navigation sont reportées.

16.2.1 Construction de cartes

Les cartes sont construites avec le package

16.2.2 Serveurs d'action

Les serveurs d'actions sont utilisés dans cette pile pour communiquer avec l'arbre de comportement AC-BT de navigation à travers le message d'action **NavigateToPose**. Ils sont aussi utilisés par le AC-BT de navigation pour communiquer avec les autres, plus petits, serveurs d'actions pour calculer les plans, les commandes d'efforts et de récupérations. Chacun a ses propres types d'actions (**.action**) dans le package **nav2_msgs** pour interagir avec les serveurs.

16.2.3 Les nodes avec cycles de vie

Les nodes avec cycles de vie (Lifecycle ou Managed nodes en Anglais) sont nouveaux dans ROS-2. Ces nodes contiennent des transitions de machines à états pour démarrer et arrêter des serveurs ROS-2. Ceci permet d'obtenir des comportements déterministiques au démarrage et à l'arrêt. Cela permet également aux utilisateurs de structurer leur programmes d'une manière raisonnable pour des utilisations commerciales et la résolution de problèmes.

Quand un node démarre, il est dans un état non-configuré **unconfigured**, ne traitant que le constructeur du Node qui ne doit pas contenir aucun mécanisme de mise en place ROS du réseau, ni de lecture de paramètres. A travers le système de launch, ou par le gestionnaire de cycle de vie, les nodes doivent être passés à inactif **inactive** par configuration. Logiquement, il est ensuite possible d'activer le node en faisant la transition à travers l'étape d'activation.

Cet état permet au node de traiter les informations et d'être pleinement configuré pour fonctionner. L'étape de configuration, déclenchée par la méthode **on_configure()**, doit s'occuper de spécifier tous les paramètres, les interfaces réseaux ROS, et pour les systèmes de sécurité toutes les allocations de mémoire. L'étape d'activation, démarre l'appel à la méthode **on_activate()** active les interfaces réseaux ROS et spécifie tous les états nécessaires dans le programme pour démarrer le traitement des informations.

Pour arrêter, les transitions nécessaires sont la désactivation, le nettoyage (par exemple de la mémoire), l'arrêt des processus, pour terminer dans l'état **finalized**. Les interfaces réseaux sont désactivées et le traitement des informations est arrêté, la mémoire est désallouée, et le node termine proprement.

L'approche node avec cycle de vie est utilisée fréquemment à travers le projet et tous les serveurs l'utilise. C'est une bonne pratique recommendée pour tous les systèmes ROS d'utiliser des nodes de cycle de vie quand cela est possible.

Nav2 utilise une encapsulation de LifeCycleNodes : **nav2_utilLifeCycleNode**. Cette encapsulation englobe l'ensemble de la complexité de LifeCycleNode pour des applications typiques. Elle inclue également une connection **bond** pour le gestionnaire de cycle de vie pour s'assurer que lorsqu'un serveur démarre, il reste actif. Lorsqu'un serveur crache, il informe le gestionnaire de cycle de vie et permet au système de se mettre dans un état évitant un échec critique.

16.2.4 Arbre de Comportement - Behavior Trees

Les arbres de comportement sont devenus très populaires dans les tâches robotique communes. Ils correspondent à une structure en arbre de tâches à compléter. Cela permet de créer un cadre plus compréhensible et passant bien à l'échelle pour définir des applications avec plusieurs états et plusieurs étapes. C'est une alternative aux machines d'états finis (FSM) qui peuvent avoir des douzaines d'états et des centaines de transitions. Un exemple est un robot jouant au football. Embarquer la logique d'un jeu de football dans une FSM est difficile et générateur d'erreur. De plus, des choix de modèles pour tirer dans un but de la droite, de la gauche ou du centre peut-être particulièrement compliqué à comprendre. Avec un BT, des primitives de base comme "tirer", "marche", "aller à la balle" peuvent être créer et réutiliser pour plusieurs comportements. Plus d'informations sont disponibles dans ce livre. Il est fortement recommandé de lire les 3 premiers chapitres pour avoir une bonne compréhension de la nomenclature et du l'enchaînement. Cela ne devrait prendre que 30 minutes.

Pour ce projet, la librairie BehaviorTree CPP V3 est utilisée. Des nodes à travers des plugins peuvent être construits dans un arbre dans le **BTNavigator**. Les plugins de node sont chargés dans le BT, et quand le fichier XML de cet arbre est lu, les noms enregistrés sont associés. Il est alors possible d'explorer l'arbre de comportement pour naviguer.

Une des raisons pour laquelle cette librarie est utilisée est sa capacité à charger des sous-arbres. Cela veut dire qu'un arbre de comportement Nav2 peut-être chargé dans un autre BT de haut niveau pour utiliser ce projet comme un plugin de node. Un exemple pourrait être dans un jeu de footbal, où l'arbre de navigation Nav2 serait le node "go to ball" avec une détection de balle comme une partie d'une tâche plus large. De

plus un plugin **NavigateToPoseAction** pour les BT est fournit afin que la pile Nav2 puisse être appelée par une application client à travers l'interaction d'action habituelle.

16.3 Serveurs de navigation

Les planificateurs et les contrôleurs sont au coeur d'une tâche de navigation. Les récupérations sont utilisées pour sortir le robot d'une situation difficile et tenter de gérer plusieurs types de problèmes afin de rendre le système résistant aux fautes. Les lisseurs peuvent être utilisées pour améliorer la qualité du chemin planifié. Dans cette section les concepts généraux associés et leur utilisation dans ce projet sont analysés.

16.3.1 Planificateurs, Contrôleurs, Lisseurs et serveurs de récupération

Quatres types de serveurs d'actions importants existent dans ce projet. Ce sont le planificateur, le serveur de comportements, le lisseur et le contrôleur.

Ces serveurs d'action sont utilisés pour stocker une carte de plugins d'algorithmes pour réaliser diverses tâches. Ils stockent également la représentation de l'environnement utilisés par les plugins d'algorithmes pour calculer leurs sorties.

Le planificateur, le lisseur et le contrôleur sont configurés à l'exécution avec les noms (alias) et les types d'algorithmes à utiliser. Ces types sont les noms de pluginlib qui ont été enregistrés et les noms sont les alias pour les tâches. Un exemple pourrait être le contrôleur DWB utilisé avec le nom **FollowPath**, qui suit un chemin de référence. Dans ce cas, alors tous les paramètres de DWB seront placés dans l'espace de nom e.g. **FollowPath.<param>**.

Ces 3 serveurs exposent alors une interface d'action correspond à leur tâche. Quand le BT arrive au noeud BT correspondant, il va appeler le serveur d'action pour effectuer la tâche. La fonction de traitement (ou callback function) du serveur d'action appelle ensuite l'algorithme désigné par son nom (i.e. **FollowPath**) qui le lie avec un algorithme spécifique. Cela permet à l'utilisateur d'abstraire l'algorithme utilisé dans l'arbre de comportement à des classes d'algorithmes. Par exemple, on peut avoir **N** plugins de contrôleurs pour suivre des chemins, se connecter à un chargeur, éviter des obstacles dynamiques, ou s'interfacer avec un outil. Avoir tous ces plugins dans le même serveur permet à l'utilisateur d'utiliser un seul objet pour la représentation de l'environnement qui est coûteuse à dupliquer.

Pour le serveur de comportements, chacun des comportements contient son propre nom, toutefois chaque plugin expose également son propre serveur d'action. Ceci est réalisé car la large variété des actions de comportement qui peuvent être crées ne peuvent avoir une interface simple à partager. Le serveur de comportement contient également un souscripteur de carte de coût à une carte locale de coût. Il reçoit ainsi des mises à jour en temps-réel du serveur de contrôleurs pour calculer ses tâches. Ceci permet d'éviter d'avoir de multiples instances de la carte de coût qui est chère à dupliquer.

Alternativement comme les nodes BT sont des plugins triviaux appelant une action, de nouveaux nodes BT peuvent être créés pour appeler des serveurs d'actions avec d'autres types d'actions. Il est recommandé d'utiliser les serveurs founirs le plus possible. Si à cause des interfaces des plugins et des interfaces, un nouveau serveur est nécessaire, il peut être intégrer dans le cadre du projet. Le nouveau serveur doit utiliser le nouveau type et le plugin d'interface, de façon similer aux serveurs fournis. Un nouveau plugin de node BT dpot etre créer pour appeler le nouveau serveur d'actions. Cependant il n'y a pas besoin de forker ou de modifier la repository de Nav2 en utilisant de façon intensive les serveurs et les plugins.

16.3.2 Planificateurs

La tâche d'un planificateur est de calculer un plan pour minimizer une fonction objectif. Le chemin peut-être aussi connu comme une route, suivant la nomenclature et de l'algorithme sélectionné. Deux exemples canoniques sont le calcul d'un plan pour atteindre un but (i.e. d'une position courante à un but) ou pour effectuer une couverture complete d'un espace. Le planificateur a accés à une représentation globale de l'environnement et les données capteurs.

Les planificateurs sont écrits pour :

— Calcul de plus court chemin
— Calcul d'un chemin de couverture
— Calcul des chemins le long de chemins connus et éparses

La tâche générale dans Nav2 du planificateur est de calculer un chemin valide, et potentiellement optimal, de la pose courante vers la pose ciblée. Plusieurs classes de plans et de routes existent et sont supportés.

16.3.3 Contrôleurs

Les contrôleurs, également connus comme des planificateurs locaux dans ROS-1, sont les mécanismes qui permettent de suivre un chemin désiré calculé de façon global, ou de réaliser une tâche locale. Les contrôleurs vont avoir accés à une représentation locale de l'environnement pour essayer de calculer des commandes en effort faisables pour la base. Plusieurs contrôleurs vont considérer une fenêtre de temps limité sur laquelle ils calculent un chemin local faisable à chaque itération. Les contrôleurs sont écrits pour :

— Suivre un chemin
— S'interfacer avec une station de chargement en utilisant des détecteurs dans le répère odomètrique
— Rentrer dans un ascenceur
— S'interfacer avec un outil

La tâche générale dans Nav2 pour un contrôleur est de calculer une commande en effort valide pour suivre un plan général. Toutefois, plusieurs classes de contrôleurs et planificateurs locaux existent. Dans le cadre de ce projet, tous les algorithmes de contrôleurs peuvent être des plugins dans le serveur de planification.

16.3.4 Comportements

Les comportements de récupération sont un pilier des systèmes tolérants aux fautes. Le but des comportements de récupérations sont de gérer des situations inconnues et des conditions d'échecs de façon autonome. Les exemples peuvent inclure des erreurs dans le système de perception dont le résultat dans la représentation de l'environnement peut-être de faux obstacles. Effacer la carte de coût peut-être déclenché afin de permettre au robot de bouger.

Un autre exemple de blocage est un robot bloqué par des obstacles dynamiques ou un mauvais contrôle. Revenir en arriére ou tourner sur place, si cela est possible, permet au robot de s'éloigner d'une mauvaise position pour aller dans un espace libre ou le robot peut naviguer correctement.

Finalement, dans le cas d'un échec plus important, un comportement de récupération peut-être d'appeller un opérateur pour demander de l'aide. Ceci peut-être fait via un email, un SMS, Slack, Matrix, etc...

Il est important de noter que le serveur de comportements peut gérer tout type de comportement pour partager l'accés à des ressources chères comme des cartes de coût ou des buffers TFs, pas uniquement des comportements de récupération. Chacun peut avoir leurs propres API.

16.3.5 Lisseurs

Les critères d'optimalité des planificateurs sont souvent insuffisant pour réaliser le chemin dans la réalité. Il est donc nécessaire d'affiner le chemin. Les serveur d'action pour le lissage ont été introduits pour cette raison. Ils sont destinés à réduire la discontinuité des chemins et des rotations, mais également à accroître la distance aux obstacles et des zones à haut risques car les lisseurs ont accés à une représentation global de l'environnement.

L'utilisation d'un lisseur séparé par rapport à un lisseur inclus dans un planificateur est avantageux quand on le combine avec d'autres planificateurs et avec d'autres lisseurs. Ceci est également intéressant si le lissage est réalisé par un contrôle spécifique sur une partie spécifique du chemin.

La tâche générale dans Nav2 d'un lisseur est de recevoir un chemin et de retourner une version améliorée. Il est possible d'utiliser différent chemins d'entrées, des critères d'améliorations, et les méthodes pour les acquérir.

16.3.6 Suiveur de point intermédiaires

Le suivi de points intermédiaires est une fonctionnalité de base d'un système de navigation. Il dit à notre système comment utiliser la navigation pour atteindre de multiples destinations.

Le serveur **nav2_waypoint_follower** contient un programme de suivi d'un point intermédiaire avec une interface de plugin pour les exécuteurs de tâches spécifiques. C'est utile si le robot doit aller à un endroit spécifique et d'effectuer une tâche comme prendre une image, prendre une boîte, ou attendre une entrée de l'utilisateur.

Il existe deux écoles de pensée pour la gestion de flotte de robots : (un robot stupide, un système de gestion de flotte intelligent) ou (un robot intelligent, un système de gestion de flotte stupide).

Dans la première école de pensée, le serveur **nav2_waypoint_follower** est amplement suffisant pour créer une solution sur le robot prête pour un produit. Comme le système autonome / gestionnaire prend

en compte la pose du robot, son niveau de batterie, la tâche courante, et plus quand il assigne des tâches, l'application sur le robot doit simplement s'occuper de la tâche à faire et non de toute la complexité du système pour réaliser la tâche. Dans cette situation, il est préférable de penser à la requête de suivi de point intermédiaire comme une unité de travail (i.e. prendre un objet dans un entrepôt, une boucle de patrouille, une aile) de faire une tâche et puis de retourner au gestionnaire pour la prochaine tâche, et de demander une recharge. Dans cette école de pensée, l'application de suivi de points est juste une étape au-dessus de la navigation et sous l'application de système autonome.

Dans la second école de pensée, le serveur **nav2_waypoint_follower** est un exemple d' application intéressante. On peut le voir comme une preuve de concept. Mais il est nécessaire d'avoir son propre système de suivi de points intermédiaires pour que le robot puisse transporter plus de poids afin de faire une solution robuste. Dans ce cas, l'utilisateur est fortement invité à utiliser le paquet **nav2_behavior_tree** pour créer une application utilisant un arbre de comportement (BT) utilisant la navigation pour réaliser la tâche. Ceci peut inclure des sous-arbres comme vérifier la charge de la batterie pour éventuellement faire revenir le robot à sa station de charge, ou effectuer plus d'une unité de tâche de travail dans une tâche plus complexe. Il est prévu de faire un paquet **nav2_bt_waypoint_follower** qui devrait permettre de faire ces applications plus aisément. Dans cette école de pensée, l'application de suivi de points intermédaires est lié plus intimement au système autonome, et dans beaucoup de cas, est l'autonomie du système.

L'un n'est pas meilleur que l'autre, et le choix dépend très fortement des tâches à effectuer par les robots, du type d'environnement, et des ressources de calcul distribué disponibles. Souvent cette distinction est très claire suivant le cas de figure.

16.4 Estimation d'état

Dans le projet de navigation, il y a 2 transformations majeures qui doivent être fournies selon les standards de la communauté. La transformation de **map** à **odom** est fournie par le système de positionnement (localisation, mapping, SLAM). La transformée de **odom** à **base_line** est elle fournie par un système d'odométrie.

> **Note 16.2** Il n'y aucune obligation à utiliser un LIDAR sur le robot pour utiliser le système de navigation. Il n'y a aucune obligation d'utiliser un système d'évitement de collision basé LIDAR. Il est possible d'utilise la vision par ordinateur ou des capteurs fournissant la profondeur pour l'évitement de collision. La seule requête est suivre les standards décrits dans le paragraphe suivant avec le choix d'implémentation. ■

16.4.1 Standards

REP 105 définit les repères et les conventions requises pour la navigation et le large écosystème de ROS. Les conventions doivent être tout le temps suivies pour utiliser les projets disponibles concernant l'odométrie, la localisation et les projets de SLAM fournis par la communauté.

Pour résumer, REP-105 spécifie qu'il faut au minimum que l'arbre TF doit contenir le chemin complet : **map** -> **odom** -> **base_link** -> **[sensorŕframes]]** pour le robot. TF2 est la librairie de transformations variant dans le temps qui est utilisée dans ROS-2 pour obtenir et représenter les transformations synchronisées dans le temps. C'est le travail du système de positionnement global (GPS, SLAM, Motion Capture) de fournir au minimum la transformation **odom** -> **base_link**. Le reste de la transformation relative à **base_link** doit être statique et définie dans l'URDF du robot.

16.4.2 Localisation globale et SLAM

Le travail du système de positionnement global (GPS, SLAM, Motion Capture) est de fournir au minimum la transformation **map** -> **odom**. Le paquet **amcl** du projet est une technique Adaptative de Localisation Monte-Carlo basée sur une filtre particulaire pour se localiser dans une carte statique. Une boîte à outils de SLAM est l'algorithme de SLAM par défaut pour positionner le robot et construire une carte statique.

Ces méthodes peuvent également produire d'autre sorties des topics pour fournir la position du robot, des cartes, et d'autres méta-données, mais elles doivent fournir cette transformation pour être valide. Des méthodes positionnement multiples peuvent être fusionnées ensemble en utilisant la localisation du robot discutée précédemment.

16.4.3 Odométrie

Le rôle du système d'odométrie est de fournir la transformation **odom -> base_link**. L'odomètrie peut venir de plusieurs sources notamment LIDAR, RADARD, les encodeurs des roues, l'odométrie visuelle (VIO), et les IMUs. Le but de l'odomètrie est de fournir un repère local lisse et continue basé sur le mouvement du robot. Le système de positionnement global va mettre à jour la transformation relative au repère global pour prendre en compte le décalage de l'odomètrie.

Le paquet Robot Localization est typiquement utilisé pour cette fusion. Il va prendre **N** capteurs de différents types et fournir une odométrie continue et lisse à l'arbre des transformations (TF) et sur un topic. Une application mobile typique peuvent avoir l'odométrie à partir des encodeurs des roues, des IMUs, et la vision fusionnée de cette manière.

La sortie lisse peut-être utilisée comme estimation pour faire des mouvements précis et mettre à jour la position du robot précisément entre deux mises à jour de la position global du robot.

16.5 Représentation de l'environnement

La représentation de l'environnement par le robot est l'objet qui est utilisé pour combiner l'information venant de plusieurs sources dans un seul espace. Cet espace est alors utilisé par les contrôleurs, les planificateurs, et les mécanismes de récupération pour calculer les tâches efficacement et de façon sûre.

16.5.1 Les cartes de coût et les couches

La représentation de l'environnement utilisée à l'heure actuelle est une carte de coût. Une carte de coût est une grille régulière de cellules contenant un coût représentant les parties inconnues, libres, occupées ou des coût gonflés. La carte de coût est utilisée pour calculer un plan global ou échantillonné pour calculer des commandes en couple locales.

Plusieurs couches de carte de coût sont implementées comme des plugins pluginlig pour stocker les informations. Ceci inclut des informations des LIDARs, des RADAR, des sonars, de la profondeur, des images, etc... Il peut être nécessaire de traiter les données capteurs avant de les insérer dans la couche du coût, mais cela dépend de la façon dont est exploitée cette carte de coût.

Les couches de cartes de coût peuvent être créées pour détecter et suivre les obstacles dans la scène pour les évitements de collision en utilisant la caméra, et les capteurs de profondeur. De plus, les couches peuvent être créées pour changer algorithmiquement la carte de coût globale en utilisant des règles ou des heuristiques. Finalement, elles doivent être utilisées pour stocker des données en ligne dans les représentations 2D ou 3D pour marquer les obstacles binairement.

16.5.2 Filtres de cartes de coût

Il est possible d'annoter une carte dans un fichier de façon à avoir une action spécifique qui soit exécuté en fonction de la position dans la carte annotée. Les exemples d'utilisation des annotations sont des zones d'exclusion pour éviter de planifier des mouvements à l'intérieur, ou limiter la vitesse. Ces cartes annotées sont appelées "masques de filtre". Ces zones peuvent être, ou ne pas être, différentes de la carte principale en taille, pose, ou échelle. Le but principal de ces filtres de masque est de fournir la possibilité de marquer des zones avec des caractéristiques supplémentaires ou des changements de comportements.

Les filtres de cartes de coût sont une approche basée sur des couches de cartes de coût. Cela permet de réaliser des changements de comportements basé sur des informations spatiales annotées dans des masques/-filtres. Les filtres de carte de coût sont implantés par des plugins de carte de coût. Ces plugins sont appelés des filtres car ils filtrent une carte de coût par des annotations spatiales spécifiées dans un masque/filtre. De façon à filtrer une carte de coût et pour changer le comportement du robot dans les zones annotées, les plugins des filtres lisent les informations qui viennent du masque/filtre. Cette donnée est transformée dans une carte de caractéristique (feature map) dans un espace filtre. Une fois cette carte de caractéristique transformée avec une carte de coût, toute données de capteurs peut mettre à jour la carte de coût sous-jacente et changer le comportement du robot en fonction de l'endroit où il se trouve. Par exemple, les fonctionnalités suivantes peuvent être réalisées grâce à l'utilisation de filtres de cartes de coût :

— Définition de zones où les robots ne vont jamais entrer ou seront en sécurité.
— Des zones de restriction de vitesses en limitant la vitesse maximal du robot.
— Spécification de routes préférées pour des robots évoluant dans des environnements industriels ou des entrepôts.

16.6 REP-105

Cette REP spécifie les conventions de noms à utiliser pour les repères des plateformes mobiles à utiliser pour ROS.

16.6.1 Spécifications

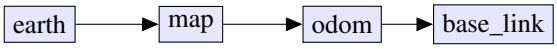
Les noms des repères sont :

— **base_link** : Il s'agit du repère attaché rigidement à la base du robot mobile. Il peut-être attaché à n'importe quel partie de la base qui fournit un point de référence évident. La REP 103 spécifie une direction de préférence. (x : devant, y : gauche, z : haut)

— **odom** : Ce repére de coordonnées est fixe dans le monde. La pose d'une base mobile dans le repère **odom** peut dériver sans limite. Cette dérive rend le repère **odom** inutile comme référence globale à long-terme. Toutefois, la pose d'un robot dans le repère **odom** est continue, ce qui signifie que l'évolution de la base mobile évolue d'une façon continue sans saut ou discontinuité.

— **map** : Ce repére de coordonnées est fixe dans le monde et son axe Z pointe vers le haut. La pose d'une plateforme mobile est exprimée dans le repère **frame** et ne doit pas dériver à travers le temps. Le repère **map** n'est pas continu, ce qui signifie que la pose d'une plateforme mobile dans le repère **map** peut changer abruptement à tout moment.

16.6.2 Relation entre repères



17. Plugins pour la navigation

17.1 Introduction

Ce tutoriel est disponible ici. Il décrit comment écrire son propre plugin de type Costmap2D.

Il est posssible d'avoir une introduction (en Anglais) qui décrit les concepts liés aux couches de Costmap2D et les principes de bases des plugins.

17.1.1 Pré requis

La suite suppose que ROS-2, Gazebo et les paquets Turtlebot 3 sont installés et construits localement. Il faut également que le projet Navigation 2 ait été installé.

17.2 Ecrire un nouveau plugin Costmap2D

A des fins de démonstration, cet exemple va créer un plugin de carte de coût avec des gradients de coûts qui se répétent dans la carte de coût. Le code annoté est disponible dans le dépôt navigation2_tutorials, et plus précisément dans le paquet **nav2_gradient_costmap_plugin**. Il est recommandé de s'y référer lorsque vous faîtes votre propre plugin.

Le plugin de la classe **nav2_gradient_costmap_plugin ::GradientLayer** est hérité de la classe **nav2_costmap_2d ::Layer** :

```
namespace nav2_gradient_costmap_plugin
{

class GradientLayer : public nav2_costmap_2d::Layer
```

La classe de base fournit l'ensemble des méthodes virtuelles de l'API pour travailler avec des couches de cartes de coûts dans un plugin. Ces méthodes sont appelées à l'exécution par **LayeredCostmap**. La liste des méthodes, leurs descriptions et la nécessité d'avoir ces méthodes dans le code d'un plugin sont présentés dans la table 17.1.

Dans l'exemple qui nous intéresse ces méthodes ont la fonctionnalité suivante :

1. **GradientLayer ::onInitialize()** contient les déclarations des paramètres ROS avec leurs valeurs par défaut :

```
declareParameter("enabled", rclcpp::ParameterValue(true));
node_->get_parameter(name_ + "." + "enabled", enabled_);
```

et initialise l'indicateur de recalcul des limites :

```
need_recalculation_ = false;
```

Virtual method	Method description	Requires override
onInitialize()	Cette méthode est appelée à la fin de l'initialisation d'un plugin. Elle est généralement constituée de déclarations de paramètres ROS. C'est ici que toute initialisation requise doit prendre place.	No
updateBounds()	Cette méthode est appelée pour demander au plugin quelle partie de la couche de la carte de coût doit être mise à jour. Il y a 3 paramètres d'entrées à la méthode : la position du robot en 2d et son orientation. et 4 paramètres de sortie : les pointeurs vers les limites de la fenêtre. Ces limites sont utilisées pour des raisons de performances : pour mettre à jour la surface à l'intérieur de la fenêtre où de nouvelles informations sont disponible, évitant ainsi de faire des mises à jour sur l'ensemble de la couche de la carte de coût.	Yes
updateCosts()	Cette méthode est appelée à chaque fois que l'évaluation de la carte de coût est nécessaire. La mise à jour ne s'effectue que dans la région de la fenêtre des limites. Cette méthode a 4 paramètres d'entrée : les limites de la fenêtre, et un paramètre de sortie : la référence vers la carte de coût **master_grid**. La classe **Layer** fournie au plugin une carte de coût interne nommée **costmap_** pour les mises à jour. **master_grid** doit être mise à jour avec les valeurs dans la fenêtre des limites en utilisant une des méthodes de mise à jour suivantes : **updateWithAddition(), updateWithMax(), updateWithOverWrite()** ou **updateWithTrueOverwrite()**.	Yes
matchSize()	La méthode est appelée à chaque fois qu'une taille de carte a changé.	No
onFootprintChanged()	La méthode est appelée à chaque fois que le plan a changé.	No
reset()	La méthode est appelée à chaque que la carte doit être réinitialiser.	Yes

TABLEAU 17.1 – Les méthodes virtuelles du plugin costmap_2d

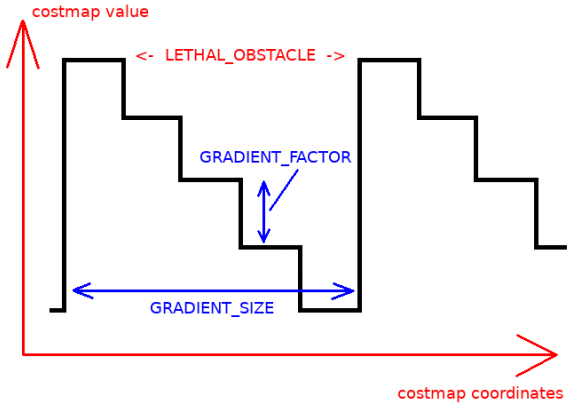


FIGURE 17.1 – Gradient explanations

2. **GradientLayer ::updateBounds()** re-initialise les limites de la fenêtre si **need_recalculation_** est égale à **true**, et sinon effectue une comparaison avec les dernières sauvegardées et prend les plus petites limites. La méthode initiale est une méthode pure virtuelle.
3. **GradientLayer ::updateCosts()** dans cette méthode, le gradient est écrit directement dans la carte de coût résultante **master_grid** sans mélanger les couches précédentes. Ceci est égale à travailler avec le pointeur **costmap_** interne et ensuite appelé la méthode **updateWithTrueOverwrite()**. Voici l'algorithme qui fait le gradient pour la carte de coût générale :

```
int gradient_index;
for (int j = min_j; j < max_j; j++) {
  // Reset gradient_index each time when reaching the end of re-calculated window
  // by OY axis.
  gradient_index = 0;
  for (int i = min_i; i < max_i; i++) {
    int index = master_grid.getIndex(i, j);
    // setting the gradient cost
    unsigned char cost = (LETHAL_OBSTACLE - gradient_index*GRADIENT_FACTOR)%255;
    if (gradient_index <= GRADIENT_SIZE) {
      gradient_index++;
    } else {
      gradient_index = 0;
    }
    master_array[index] = cost;
  }
}

```

où la variable **GRADIENT_SIZE** est la taille de chaque période du gradient dans les cellules de la carte, **GRADIENT_FACTOR** décrémente la valeur de la carte de coût à chaque étape. Ces paramètres sont définis dans le fichier d'en-tête du plugin.
4. **GradientLayer ::onFootprintChanged()** simplement réinitialise la valeur **need_recalculation_**
5. La méthode **GradientLayer ::reset()** ne sert à rien : elle n'est pas utilisée dans cet exemple.

17.3 Exporter et construire le plugin GradientLayer

Le plugin est chargé à l'exécution à travers sa classe mère et est ensuite appelée par le plugin manipulant les modules (pour les cartes de coût 2D par **LayeredCostmap**). Pluginlib ouvre un plugin donné à l'exécution et fournit des méthodes des classes exportées qui peuvent être appelée. Le mécanisme d'exportation des classes spécifie à pluginlib qu'elle classe de base utilisée durant ces appels. Ceci permet d'étendre une application par des plugins sans connaître le code source de l'application ou la recompiler.

Dans l'exemple la classe du plugin **nav2_gradient_costmap_plugin ::GradientLayer** doit être dynamiquement chargée grâce à la classe de base **nav2_costmap_2d ::Layer**. Pour cela le plugin doit être enregistré de la façon suivante :

1. La classe du plugin doit être enregistrée avec un type de base de la classe chargée. Pour cela, il y a une spéciale macro **PLUGINLIB_EXPORT_CLASS**. qui doit être ajoutée à chaque fichier source

qui compose la librairie du plugin :

```
#include "pluginlib/class_list_macros.hpp"
PLUGINLIB_EXPORT_CLASS(nav2_gradient_costmap_plugin::GradientLayer, nav2_costmap_2d::Layer)
```

Cette partie est généralement placée à la fin du fichier source où la classe du plugin est écrite (dans notre exemple **gradient_layer.cpp**. C'est une bonne pratique de placer ces lignes à la fin du fichier mais techniquement, on peut le placer également au démarrage.

2. Les informations du plugin doivent être stockées dans un fichier décrivant le plugin. Pour cela on utilise un fichier XML séparé (dans notre exemple **gradient_plugins.xml** dans le paquet du plugin. Le fichier contient les informations de :
   - **path** : Chemin et nom de la librairie où se trouve le plugin.
   - **name** : Le type du plugin référencé dans le paramètre **plugin_types**.
   - **type** : La classe du plugin avec le namespace tel que donné dans le code source.
   - **basic_class_type** : La classe de base à partir de laquelle la classe du plugin a été dérivé.
   - **description** : La description du plugin sous forme textuelle.

```
  <library path="nav2_gradient_costmap_plugin_core">
  <class name="nav2_gradient_costmap_plugin/GradientLayer" type="nav2_gradient_costmap_plugin::GradientLayer"
     base_class_type="nav2_costmap_2d::Layer">
    <description>This is an example plugin which puts repeating costs gradients to costmap</description>
  </class>
  </library>
```

L'export du plugin s'effectue en incluant la macro **pluginlib_export_plugin_description_file()** dans le fichier **CMakeLists.txt**. Cette fonction installe le fichier de description du plugin dans le répertoire **share** et initialise les index d'ament pour que le fichier XML de description du plugin soit découvrable comme un plugin du type sélectionné :

```
pluginlib_export_plugin_description_file(nav2_costmap_2d gradient_layer.xml)
```

Le fichier de description du plugin doit être également ajouté au fichier **package.xml**. **costmap_2d** est le paquet de définition de l'interface, pour notre cas **Layer**, et requiert un chemin vers le fichier XML :

```
<export>
  <costmap_2d plugin="${prefix}/gradient_layer.xml" />
  ...
</export>
```

Une fois que tout est prêt, il faut mettre le package du plugin dans le répertoire **src** d'un espace de travail ROS-2, construire le package (**colcon build –packages-select nav2_gradient_costmap_plugin –symlink-install**) et sourcer le fichier **setup.bash** quand cela est nécessaire.

Maintenant le plugin est prêt à être utilisé.

17.4 Utiliser le plugin dans Costmap2D

La prochaine étape est de d'indiquer à Costmap2D qu'il y a un nouveau plugin. Pour cela il faut qu'il soit ajouté aux listes **plugin_names** et **plugin_types** du fichier **nav2_params.yaml**, optionnellement pour **local_costmap/global_costmap** de façon à être utilisable à l'exécution des serveurs Controller et Planner. La liste **plugin_names** contient les noms des plugins. Ces noms peuvent être ce que l'on souhaite. **plugin_types** contient les types des plugins listés dans **plugin_names**. Ces types doivent correspondre au champ **name** des classes de plugin specifiée dans le fichier de description XML.

Par exemple :

```
--- a/nav2_bringup/bringup/params/nav2_params.yaml
+++ b/nav2_bringup/bringup/params/nav2_params.yaml
@@ -124,8 +124,8 @@ local_costmap:
       width: 3
       height: 3
       resolution: 0.05
-      plugin_names: ["obstacle_layer", "voxel_layer", "inflation_layer"] # For Eloquent and earlier
-      plugin_types: ["nav2_costmap_2d::ObstacleLayer", "nav2_costmap_2d::VoxelLayer", "nav2_costmap_2d::
     InflationLayer"] # For Eloquent and earlier
+      plugin_names: ["obstacle_layer", "voxel_layer", "gradient_layer"] # For Eloquent and earlier
+      plugin_types: ["nav2_costmap_2d::ObstacleLayer", "nav2_costmap_2d::VoxelLayer", "
    nav2_gradient_costmap_plugin/GradientLayer"] # For Eloquent and earlier
-      plugins: ["obstacle_layer", "voxel_layer", "inflation_layer"] # For Foxy and later
+      plugins: ["obstacle_layer", "voxel_layer", "gradient_layer"] # For Foxy and later
```

```
       robot_radius: 0.22
       inflation_layer:
         cost_scaling_factor: 3.0
@@ -171,8 +171,8 @@ global_costmap:
       robot_base_frame: base_link
       global_frame: map
       use_sim_time: True
-      plugin_names: ["static_layer", "obstacle_layer", "voxel_layer", "inflation_layer"] # For Eloquent and earlier
-      plugin_types: ["nav2_costmap_2d::StaticLayer", "nav2_costmap_2d::ObstacleLayer", "nav2_costmap_2d::VoxelLayer", "nav2_costmap_2d::InflationLayer"] # For Eloquent and earlier
+      plugin_names: ["static_layer", "obstacle_layer", "voxel_layer", "gradient_layer"] # For Eloquent and earlier
+      plugin_types: ["nav2_costmap_2d::StaticLayer", "nav2_costmap_2d::ObstacleLayer", "nav2_costmap_2d::VoxelLayer", "nav2_gradient_costmap_plugin/GradientLayer"] # For Eloquent and earlier
-      plugins: ["static_layer", "obstacle_layer", "voxel_layer", "inflation_layer"] # For Foxy and later
+      plugins: ["static_layer", "obstacle_layer", "voxel_layer", "gradient_layer"] # For Foxy and later
       robot_radius: 0.22
       resolution: 0.05
       obstacle_layer:
```

Le fichier YAML peut aussi contenir la liste des paramètres (s'il y en a) pour chaque plugin, identifié leur noms.

NOTE : Il peut y avoir plusieurs plugins chargés à partir du même type. Pour cela la liste **plugin_names** doit contenir plusieurs noms de plugins différents tandis que **plugin_types** conserver la même valeur.

```
plugin_names: ["obstacle_layer", "gradient_layer_1", "gradient_layer_2"] # For Eloquent and earlier
plugin_types: ["nav2_costmap_2d::ObstacleLayer", "nav2_gradient_costmap_plugin/GradientLayer", "nav2_gradient_costmap_plugin/GradientLayer"] # For Eloquent and earlier
plugins: ["obstacle_layer", "gradient_layer_1", "gradient_layer_2"] # For Foxy and later
```

Dans ce cas chaque plugin sera géré par son propre arbre de paramètres dans un fichier YAML comme :

```
gradient_layer_1 :
  plugin : nav2_gradient_costmap_plugin/GradientLayer # For Foxy and later
  enabled : True
  ...
gradient_layer_2 :
  plugin : nav2_gradient_costmap_plugin/GradientLayer # For Foxy and later
  enabled : False
  ...
```

17.5 Lancer le plugin GradientLayer

Il faut lancer la simulation Turtlebot3 qui permet d'avoir Nav2. La commande est :

```
$ ros2 launch nav2_bringup tb3_simulation_launch.py
```

Il faut ensuite aller sur RViz et clicker sur le boutton "2D Pose Estimate" tout en haut et pointer la position sur la carte. Le robot sera localisé sur une carte et le résultat est représenté dans la figure Fig. 17.2. On peut voir la carte de coût avec le gradient. On peut voir très clairement la mise à jour dynamique de la carte de coût effectué par la méthode **GradientLayer ::updateCosts()** avec ses limites et le chemin global courbé par le gradient.

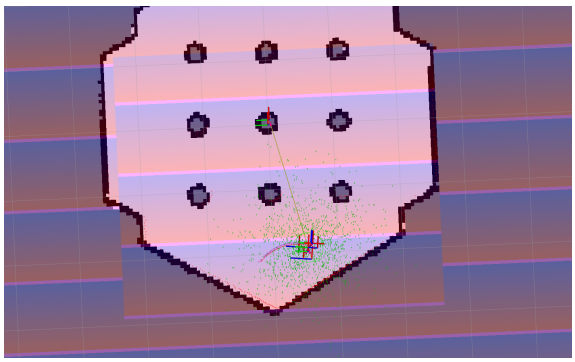


FIGURE 17.2 – Rendu du gradient pour la carte de coût

18. Plugin de planification

18.1 Introduction

Le but de ce didacticiel est de montre comment il est possible d'écrire son propore planificateur.
Il requiet d'avoir installer ROS-2, Nav2, Gazebo et turtlebot3.

18.2 Création d'un nouveu plugin de planification de mouvements

Nous allons créer un simple planificateur de ligne droite. Le code annoté est accessible sur la repository navigation2_tutorials dans le paquet **nav2_straightline_planner**. Ce paquet peut-être considéré comme la référence pour écrire un plugin de planification.

Le plugin exemple que nous considérons ici hérite de la class de base **nav2_core ::GlobalPlanner**. La classe de base fournie 5 méthodes pures virtuelles pour implémenter un plugin de planification. Le plugin sera utilisée par le serveur de planificateurs pour calculer des trajectoires. On peut voir dans la table 19.1 une description détailéee des différentes méthodes virtuelles.

Dans ce didacticiel, les méthodes **StraightLine ::configure()** et **StraightLine ::createPlan()** sont utilisées pour planifier des lignes droites.

Dans les planificateurs, la méthode **configure()** doit initiliaser les variables membres à partir des paramètres ROS et effectuer toutes les initialisations requises.

```
node_ = parent;
tf_ = tf;
name_ = name;
costmap_ = costmap_ros->getCostmap();
global_frame_ = costmap_ros->getGlobalFrameID();

// Parameter initialization
nav2_util::declare_parameter_if_not_declared(node_, name_ + ".interpolation_resolution", rclcpp::ParameterValue
      (0.1));
node_->get_parameter(name_ + ".interpolation_resolution", interpolation_resolution_);
```

Ici, la chaîne de caractères construite avec `name_ + ".interpolation_resolution"` permet d'extraire le paramètre **interpolation_resolution** qui est spécifié pour le planificateur. Navigation2 permet de charger plusieurs plugins et de garder une organisation où chaque plugin est associé à un identifiant ou nom. Maintenant si nous souhaitons retrouver les paramètres pour chaque spécifique plugin, nous utilisons **<mapped_name_of_plugin>.<name_of_parameter** comme cela est fait dans l'extrait de code. Par exemple, notre exemple de planificateur est identifié par le nom "GridBased" et pour retrouver le paramètre **interpolation_resolution** qui est spécifique à "GridBased", la chaîne **GridBased.interpolation_resolution** est

Méthodes virtuelles	Description de la méthode	Ecriture par dessus requise
configure()	La méthode est appelée quand le serveur de planification entre dans l'état on_configure. Idéalement cette méthode doit effectuer la déclaration des paramètres ROS et l'initialisation des variables membres de la classe. Cette méthode prends 4 paramètres en entrée : un pointeur partagé du node parent, le nom du planificateur, le pointeur vers le buffer tf et une pointeur partagé vers la carte de coût.	Oui
activate()	La méthode est appelée quand le serveur de planification entre dans l'état *on_activate*. Idéalement cette méthode doit implémenter les opérations qui sont nécessaires à effectuer avant que le planificateur n'aille dans l'état *activate*.	Oui
deactivate()	La méthode est appelée quand le serveur de contrôleurs entre dans l'état *on_deactivate*. Idéalement cette méthode doit implémenter le opérations qui sont nécessaires avant que le planificateur n'aille dans un état inactif.	Yes
cleanup()	La méthode est appelée quand le serveur de planification va dans l'état *on_cleanup*. Idéalement cette méthode doit nettoyer les ressources qui sont créées pour le contrôleur.	Yes
cretePlan()	Cette méthode est appelée quand le serveur de planification demande un plan global pour spécifier la pose de départ et de fin. Cette méthode retourne un message de type *nav_msgs ::msgs ::Path* qui contient tout le plan. Cette méthode prend 2 variables d'entrées : la pose de départ et la pose d'arrivée.	Yes

TABLEAU 18.1 – Liste des méthodes utilisée dans la classe des plugins de planification

utilisée. Il est possible de l'interpréter de la façon suivante : **GridBased** est utilisé comme un namespace pour les paramètres spécifiques au plugin. Ceci est décrit plus en détails dans la suite concernant le fichier de paramètres.

Dans la méthde **createPlan()**, nous devons créer un chemin de la pose de départ à la pose d'arrivée. La méthode **StraightLine ::createPlan()** est appellé en utilisant la pose de départ et la pose d'arrivée pour résoudre le problème général de planification. Une fois la planification réussie le chemin est converti en un message **nav_msgs ::msg ::Path** et retourne au serveur de planification. Ci-dessous suit l'implémentation de la méthode annotée :

```
nav_msgs::msg::Path global_path;

// Checking if the goal and start state is in the global frame
if (start.header.frame_id != global_frame_) {
  RCLCPP_ERROR(
    node_->get_logger(), "Planner will only except start position from %s frame",
    global_frame_.c_str());
  return global_path;
}

if (goal.header.frame_id != global_frame_) {
  RCLCPP_INFO(
    node_->get_logger(), "Planner will only except goal position from %s frame",
    global_frame_.c_str());
  return global_path;
}

global_path.poses.clear();
global_path.header.stamp = node_->now();
global_path.header.frame_id = global_frame_;
// calculating the number of loops for current value of interpolation_resolution_
int total_number_of_loop = std::hypot(
  goal.pose.position.x - start.pose.position.x,
  goal.pose.position.y - start.pose.position.y) /
  interpolation_resolution_;
double x_increment = (goal.pose.position.x - start.pose.position.x) / total_number_of_loop;
double y_increment = (goal.pose.position.y - start.pose.position.y) / total_number_of_loop;
```

```
28
29 for (int i = 0; i < total_number_of_loop; ++i) {
30   geometry_msgs::msg::PoseStamped pose;
31   pose.pose.position.x = start.pose.position.x + x_increment * i;
32   pose.pose.position.y = start.pose.position.y + y_increment * i;
33   pose.pose.position.z = 0.0;
34   pose.pose.orientation.x = 0.0;
35   pose.pose.orientation.y = 0.0;
36   pose.pose.orientation.z = 0.0;
37   pose.pose.orientation.w = 1.0;
38   pose.header.stamp = node_->now();
39   pose.header.frame_id = global_frame_;
40   global_path.poses.push_back(pose);
41 }
42
43 global_path.poses.push_back(goal);
44
45 return global_path;
```

Le reste des méthodes ne sont pas utilisées mais il est nécessaire de les réécrire. Elles sont donc vides.

18.3 Exporter le plugin de planification

Maintenant que nous avons notre planificateur personnalisé, il faut exporter le plugin de planification pour qu'il soit visible par le server de planification. Les plugins sont chargés à l'exécution et s'ils ne sont pas visibles le serveur ne pourra le charger. Dans ROS-2, exporter et charger les plugins sont gérés par **pluginlib**.

Dans ce didacticiel **nav2_straightline_planner ::StraightLine** est chargé dynamiquement avec la class de base **nav2_core ::GlobalPlanner**.

1. Pour exporter le planificateur, nous devons fournir les deux lignes :

```
1 #include "pluginlib/class_list_macros.hpp"
2 PLUGINLIB_EXPORT_CLASS(nav2_straightline_planner::StraightLine, nav2_core::GlobalPlanner)
3
```

   C'est la macro nommée **PLUGINLIB_EXPORT_CLASS** qui fait tout le travail d'export.
   Par convention, les lignes sont placés à la fin du fichier mais techniquement on peut placer ces lignes également au début du fichier.

2. La prochaine étape est de créer le fichier de description à la racine du répertoire du paquet. Par exemple, on trouve le fichier **global_planner_plugin.xml** dans le paquet du didacticiel. Le fichier contient les informations suivantes :
   - **library path** : Le nom de la librairie et son emplacement.
   - **class name** : Le nom de la classe.
   - **class type** : Type de la classe.
   - **base classe** : Nom de la classe de base.
   - **description** : Description du plugin.

```
1     <library path="nav2_straightline_planner_plugin">
2   <class name="nav2_straightline_planner/StraightLine" type="nav2_straightline_planner::StraightLine"
        base_class_type="nav2_core::GlobalPlanner">
3     <description>This is an example plugin which produces straight path.</description>
4   </class>
5 </library>
6
```

3. La prochaine étape est d'exporter le plugin en utilisant **CMakeLists.txt** en utilisant la fonction cmake **pluginlib_export_plugin_description_file()**. Cette fonction installe un fichier de description du plugin dans le répertoire **share** et rend le plugin accessible au système d'ament.

```
1 pluginlib_export_plugin_description_file(nav2_core global_planner_plugin.xml)
2
```

4. Le fichier de description doit être aussi ajouter au fichier **package.xml**

```
1 <export>
2   <build_type>ament_cmake</build_type>
3   <nav2_core plugin="${prefix}/global_planner_plugin.xml" />
4 </export>
```

5. Il faut ensuite compiler pour que le fichier soit enregistré. Il est ensuite possible d'utilise le plugin.

18.4 Passer le nom du plugin par le fichier de paramètres

Pour permettre l'utilisation du plugin, il faut modifier le fichier **nav2_params.yaml**. La suite décrit les paragraphes à remplacer.

> **Note 18.1** Depuis Galactic, les champs **plugin_names** et **plugin_types** ont été remplacé par un vecteur de chaînes de caractères nommé **plugins**. Les types sont maintenant définis dans l'espace de nom **plugin_name** dans le champ **filed** (e.g. **plugin : MyPlugin ::Plugin**. Les commentaires dans les blocs du code servent de guides. ■

```
planner_server :
ros__parameters :
  planner_plugin_types : ["nav2_navfn_planner/NavfnPlanner"] # For Eloquent and earlier
  planner_plugin_ids : ["GridBased"] # For Eloquent and earlier
  plugins : ["GridBased"] # For Foxy and later
  use_sim_time : True
  GridBased :
    plugin : nav2_navfn_planner/NavfnPlanner # For Foxy and later
    tolerance : 2.0
    use_astar : false
    allow_unknown : true
```

avec

```
planner_server :
ros__parameters :
  planner_plugin_types : ["nav2_straightline_planner/StraightLine"] # For Eloquent and
      earlier
  planner_plugin_ids : ["GridBased"] # For Eloquent and earlier
  plugins : ["GridBased"] # For Foxy and later
  use_sim_time : True
  GridBased :
    plugin : nav2_straightline_planner/StraightLine # For Foxy and later
    interpolation_resolution : 0.1
```

Dans le fragment de code précédent, on peut observer que le planificateur **nav2_straightline_planner/StraightLine** a comme identifiant **GridBased**. On peut constater que pour passer des paramètres spécifiques au plugin on utilise **<plugin_id>.<plugin_specific_parameter>**.

18.5 Lancer le plugin StraightLine

Il est possible de lancer la simulation de Turtlebot3 avec un fichier de paramètres spécifiques :

```
ros2 launch nav2_bringup tb3_simulation_launch.py params_file:=/chemin/vers/votre_fichier_params.yaml
```

On peut ensuite utiliser rviz et cliquer sur le boutton "2D Pose Estimate" en haut de l'interface. Il faut indiquer une position sur la carte. Le robot va se localiser sur la carte et ensuite il faut cliquer sur "Navigation2 goal" puis indiquer un but sur la carte. Une fois que le planificateur fournisse le plan, le robot démarre son mouvement vers le but.

19. Ecrire un nouveau plugin de contrôleur

19.1 Introduction

Ce didacticiel montre comment créer son propre contrôleur.

Dans ce didacticiel nous allons implémenter un algorithm de suivi de chemin basé sur le papier [5]. Il est recommandé de le parcourir.

> **Note 19.1** Ce didacticiel est basé sur une version simplifiée précédente du contrôleur de poursuite pure régulée maintenant dans la pile Nav2. Il est possible de trouver les codes sources correspondant au didacticiel dont le serveur github est ici. ■

19.2 Créer un nouveau plugin contrôleur

Ce didacticiel implémente un contrôleur de poursuite. Le code annoté de ce didacticiel peut être à nouveau trouvé dans le dépôt navigation_tutorials et plus spécifiquement dans le répertoire **nav2_pure_pursuit_controller**. Ce paquet peut être considéré comme une référence pour écrire son propre contrôleur.

La classe **nav2_pure_pursuit_controller ::PurePursuitController** de l'exemple hérite de la classe de base **nav2_core ::Controller**. La classe de base fournit un ensemble de méthodes virtuelles pour implémenter un plugin de contrôleur. Ces méthodes sont appellées à l'exécution par le serveur de contrôleur pour calculer des commandes en vitesse. La liste des méthodes, et leurs descriptions, et leur nécessité sont présentées dans la table 19.1

Dans ce didacticiel, les méthodes suivantes vont être utilisées : **PurePursuitController ::configure**, **PurePursuitController ::setPlan** et **PurePursuitController ::computeVelocityCommands**.

Dans les contrôleurs, la méthode **configure()** doit initialiser les membres de la classe à partir des paramètres ROS et effectuer les initialisations requises.

```
1  void PurePursuitController::configure(
2    const rclcpp_lifecycle::LifecycleNode::WeakPtr & parent,
3    std::string name, const std::shared_ptr<tf2_ros::Buffer> & tf,
4    const std::shared_ptr<nav2_costmap_2d::Costmap2DROS> & costmap_ros)
5  {
6    node_ = parent;
7    auto node = node_.lock();
8
9    costmap_ros_ = costmap_ros;
10   tf_ = tf;
11   plugin_name_ = name;
12   logger_ = node->get_logger();
```

Méthodes virtuelles	Description de la méthode	Ecriture par dessus requise
configure()	La méthode est appelée quand le serveur de contrôleurs entre dans l'état *on_configure*. Idéalement cette méthode doit effectuer la déclaration des paramètres ROS et l'initialisation des variables membres de la classe. Cette méthode prends 4 paramètres en entrée : un pointeur partagé du node parent, le nom du contrôleur, le pointeur vers le buffer tf et une pointeur partagé vers la carte de coût.	Oui
activate()	La méthode est appelée quand le serveur de contrôleurs entre dans l'état on_activate. Idéalement cette méthode doit implémenter les opérations qui sont nécessaires à effectuer avant que le contrôleur n'aille dans l'état *activate*.	Oui
deactivate()	La méthode est appelée quand le serveur de planification entre dans l'état on_deactivate. Idéalement cette méthode doit implémenter le opérations qui sont nécessaires avant que le planificateur n'aille dans un état inactif.	Yes
cleanup()	La méthode est appelée quand le serveur de planification va dns l'état on_cleanup. Idéalement cette méthode doit nettoyer les ressources qui sont créées pour le planificateur.	Yes
setPlan()	Cette méthode est appelée quand le plan global est mis à jour. Idéalement cette méthode effectue des opérations qui transforme le plan et le stocke.	Yes
computeVelocityCommnds()	Cette méthode est appelée quand une nouvelle commande en vitesse est demandée par le serveur du contrôleur pour que le robot suive le plan global. Cette méthode retourne un message **geometry_msgs ::msg ::TwistStamped** qui représente la commande en vitesse pour que le robot conduise. Cette méthode passe 2 références de paramètres qui correspondent à la pose courante du robot et sa vitesse courante	Yes

TABLEAU 19.1 – Liste des méthodes utilisée dans la classe des plugins de planification

```
  clock_ = node->get_clock();

  declare_parameter_if_not_declared(
    node, plugin_name_ + ".desired_linear_vel", rclcpp::ParameterValue(
      0.2));
  declare_parameter_if_not_declared(
    node, plugin_name_ + ".lookahead_dist",
    rclcpp::ParameterValue(0.4));
  declare_parameter_if_not_declared(
    node, plugin_name_ + ".max_angular_vel", rclcpp::ParameterValue(
      1.0));
  declare_parameter_if_not_declared(
    node, plugin_name_ + ".transform_tolerance", rclcpp::ParameterValue(
      0.1));

  node->get_parameter(plugin_name_ + ".desired_linear_vel", desired_linear_vel_);
  node->get_parameter(plugin_name_ + ".lookahead_dist", lookahead_dist_);
  node->get_parameter(plugin_name_ + ".max_angular_vel", max_angular_vel_);
  double transform_tolerance;
  node->get_parameter(plugin_name_ + ".transform_tolerance", transform_tolerance);
  transform_tolerance_ = rclcpp::Duration::from_seconds(transform_tolerance);
}
```

Ici, la chaîne de caractères **plugin_name_ + ."desired_linear_vel** permet de lire le paramètre ROS **desired_linear_vel** qui est spécifique à ce contrôleur. Nav2 permet de charger plusieurs plugins, afin de les identifier chacun d'eux à un identifiant ou un nom. Donc afin de retrouver les paramètres pour un plugin particulier il est possible d'utiliser la convention suivante **<nom_du_plugin>.<nom_du_paramètre>** comme cela est fait dans le fragment de code précédent. Par exemple, le contrôleur de ce didacticiel est nommé **FollowPath**. Afin de retrouver le paramètre **desired_linear_vel** spécifique à **FollowPath** on utilise **FollowPath.desired_linear_vel**. On peut également voir **FollowPath** comme un espace de noms pour des paramètres spécifiques à **FollowPath**. Certains de ces concepts sont discutés lorsque le fichier de paramètres est discuté.

Les arguments qui sont lus sont stockés dans les variables membres de la classe pour qu'ils soient utilisés plus tard si nécessaire.

La méthode **setPlan()** reçoit le chemin global mis à jour que le robot doit suivre. Dans cet exemple, le chemin global reçu est transformé dans le repère du robot et le stocke pour des usages ultérieurs.

```
void PurePursuitController::setPlan(const nav_msgs::msg::Path & path)
{
  // Transform global path into the robot's frame
  global_plan_ = transformGlobalPlan(path);
}
```

Le calcul des vitesses désirées s'effectue dans la méthode **computeVelocityCommands()**. Elle est utilisée pour calculer la commande en vitesse désirée en considérant la vitesse et la pose courantes. Dans le cas d'une poursuite pure, l'algorithme calcule la commande en vitesse pour que le robot essait de suivre le chemin global aussi précisément qe possible. Cet algorithme suppose une vitesse linéaire constante and calcule la vitesse angulaire basée sur la courbure du chemin global.

```
geometry_msgs::msg::TwistStamped PurePursuitController::computeVelocityCommands(
  const geometry_msgs::msg::PoseStamped & pose,
  const geometry_msgs::msg::Twist & velocity)
{
  // Find the first pose which is at a distance greater than the specified lookahed distance
  auto goal_pose = std::find_if(
    global_plan_.poses.begin(), global_plan_.poses.end(),
    [&](const auto & global_plan_pose) {
      return hypot(
        global_plan_pose.pose.position.x,
        global_plan_pose.pose.position.y) >= lookahead_dist_;
    })->pose;

  double linear_vel, angular_vel;

  // If the goal pose is in front of the robot then compute the velocity using the pure pursuit algorithm
  // else rotate with the max angular velocity until the goal pose is in front of the robot
  if (goal_pose.position.x > 0) {

    auto curvature = 2.0 * goal_pose.position.y /
      (goal_pose.position.x * goal_pose.position.x + goal_pose.position.y * goal_pose.position.y);
    linear_vel = desired_linear_vel_;
    angular_vel = desired_linear_vel_ * curvature;
  } else {
    linear_vel = 0.0;
    angular_vel = max_angular_vel_;
```

```
  }

  // Create and publish a TwistStamped message with the desired velocity
  geometry_msgs::msg::TwistStamped cmd_vel;
  cmd_vel.header.frame_id = pose.header.frame_id;
  cmd_vel.header.stamp = clock_->now();
  cmd_vel.twist.linear.x = linear_vel;
  cmd_vel.twist.angular.z = max(
    -1.0 * abs(max_angular_vel_), min(
      angular_vel, abs(
        max_angular_vel_)));

  return cmd_vel;
}
```

Les autres méthodes ne sont pas utilisées, mais il est nécessaire de les surcharger. Elles sont donc vides.

19.3 Exporter le plugin du contrôleur

Une fois le contrôleur prêt, il faut que celui-ci soit exporté pour qu'il soit visible par le serveur de contrôleur. Les plugins sont chargés à l'exécution, et s'ils ne sont pas visibles, le serveur de contrôleur ne sera pas capable de les charger. Avec ROS-2, les plugins exportés et chargés sont gérés par **pluginlib**.

Dans ce didacticiel, la classe **nav2_pure_pursuit_controller ::PurePursuitController** est chargée dynamiquement comme étant de la classe de base **nav2_core ::Controller**.

1. Pour export le contrôleur, nous avons besoin de fournir les deux lignes suivantes :

```
#include "pluginlib/class_list_macros.hpp"
PLUGINLIB_EXPORT_CLASS(nav2_pure_pursuit_controller::PurePursuitController, nav2_core::Controller)

```

   pluginlib est nécessaire pour exporter la classe de plugin. pluginlib fournit la macro **PLUGINLIB_EXPORT_CLASS** qui effectue toutes les tâches d'export.
   Il est conseillé de mettre ces lignes à la fin du fichier source, mais techniquement il est également possible de les mettre au début.
2. La prochaine étape est de créer le fichier de description du plugin dans la racine du répertoire du paquet. Par exemple dans ce didacticiel il s'agit du fichier **pure_pursuit_controller_plugin.xml**. Ce fichier contient les informations suivantes :
   - **library path** : Le nom de la librairie du plugin et sa place sur le disque.
   - **class name** : Nom de la classe.
   - **class type** : Type de la classe.
   - **base class** : Nom de la classe de base.
   - **description** : Description du plugin.

```
<library path="nav2_pure_pursuit_controller">
  <class type="nav2_pure_pursuit_controller::PurePursuitController" base_class_type="nav2_core::Controller">
    <description>
      This is pure pursuit controller
    </description>
  </class>
</library>

```

3. L'étape suivante est d'exporter le plugin en utilisant le fichier **CMakeLists.txt** en utilisant la fonction CMake en utilisant la fonction **pluginlib_export_plugin_description_file()**. Cette fonction installe le fichier de description du plugin dans le répertoire **share** et positionne les index d'amente pour le rendre découvrable.

```
pluginlib_export_plugin_description_file(nav2_core pure_pursuit_controller_plugin.xml)

```

4. Le fichier de description du plugin doit être ajouté à **package.xml**

```
<export>
  <build_type>ament_cmake</build_type>
  <nav2_core plugin="${prefix}/pure_pursuit_controller_plugin.xml" />
</export>

```

5. Il faut ensuite compiler et le plugin va être enregistré.

19.3.1 Passer le nom du plugin à travers le fichier de paramètres

Pour activer le plugin, il faut modifier le fichier **nav2_params.yaml** de la façon suivante :

```
controller_server :
  ros__parameters :
    controller_plugins : ["FollowPath"]

    FollowPath :
      plugin : "nav2_pure_pursuit_controller::PurePursuitController"
      debug_trajectory_details : True
      desired_linear_vel : 0.2
      lookahead_dist : 0.4
      max_angular_vel : 1.0
      transform_tolerance : 1.0
```

Dans l'exemple précédent on peut observer que le contrôleur **nav2_pure_pursuit_controller/PurePursuitController** a l'identifiant **FollowPath**. Pour passer des paramètres spécifiques au plugin on peut installer : **<plugin_id>.<plugin_specific_parameter>**.

19.3.2 Lancer le plugin de contrôleur en poursuite pure

Il faut lancer la simulation Turtlebot3 avec Nav2. La commande pour lancer cette partie est :

```
ros2 launch nav2_bringup tb3_simulation_launch.py params_file:=/path/to/your_params_file.yaml
```

Il faut ensuite aller dans rviz et cliquer sur le boutton "2D Pose Estimate" sur la barre du haut et pointer sur la position du robot dans la carte. Le robot va se localiser. Il faut ensuite cliquer le boutton "Nav2 goal" et cliquer sur l'endroit ou l'on souhaite de que le robot navigue. Ensuite le contrôleur va forcer le robot à suivre le chemin global.

VII ROS2 Control

20 ros2_control : Introduction 203
20.1 Concepts
20.2 Exemples d'interfaces matériel
20.3 Exemple de contrôleur : diff_drive_controller

21 Les démonstrations de ros2_control 215
21.1 Exemple 1 : RRBot
21.2 Exemple 3 : Un robot avec plusieurs interfaces
21.3 Exemple 12 : Le chaînage de contrôleurs

22 ROS-2 control pour les robots de l'ODRI 223
22.1 Objet SystemOdriInterface pour la master board
22.2 Création du fichier URDF
22.3 Lancement du robot

20. ros2_control : Introduction

20.1 Concepts

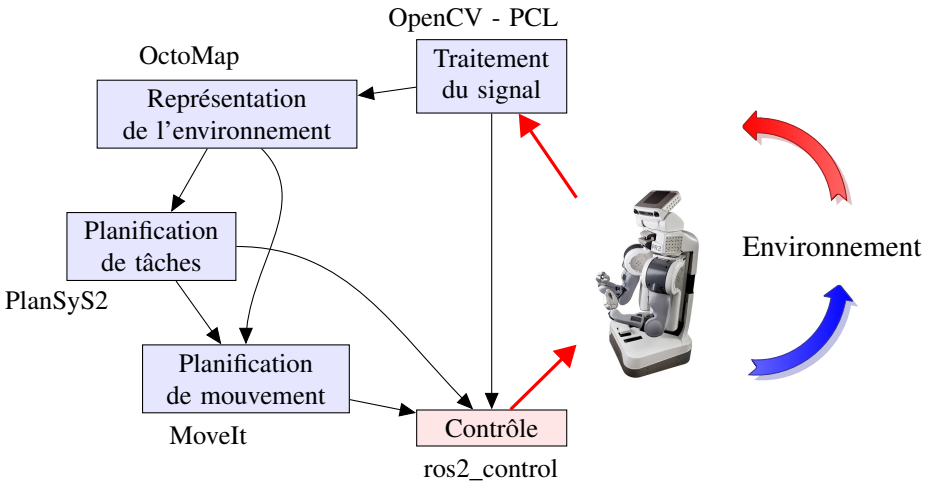


FIGURE 20.1 – Structure d'une application robotique

Dans le cadre d'une application robotique la partie contrôle a un rôle particulier car il s'agit de la boucle action réaction la plus rapide et la plus proche du matériel Fig. 20.1.

D'un point de vue logiciel, idéalement la structure logicielle du contrôle doit permettre d'accéder à la partie matérielle de façon standard et agnostique, et permettre de réutiliser les contrôleurs sur plusieurs robots. La figure Fig. 20.2 synthètise cette interaction avec la boucle d'interaction classique avec la lecture des capteurs *read()*, le calcul des mouvements à effectuer *update()*, et l'écriture des commandes *write()* vers les actionneurs.

Pour cela ros2_control utilise des objets C++ pour présenter le matériel de façon agnostique :

— *ActuatorInterface* : cet objet permet d'envoyer un ordre à un actionneur et peut donc être vu comme un système en *écriture uniquement*. Il peut s'agir d'un actionneur linéaire, d'un moteur pas à pas, d'un moteur à courant alternatif. On accéde à cet objet avec une interface de commande *CommandInterface*.

— *SensorInterface* : cet objet permet de lire les informations d'un capteur il peut être vu comme un système en *lecture uniquement*. Il peut s'agir d'un capteur de position, de vitesse, d'une IMU, d'un

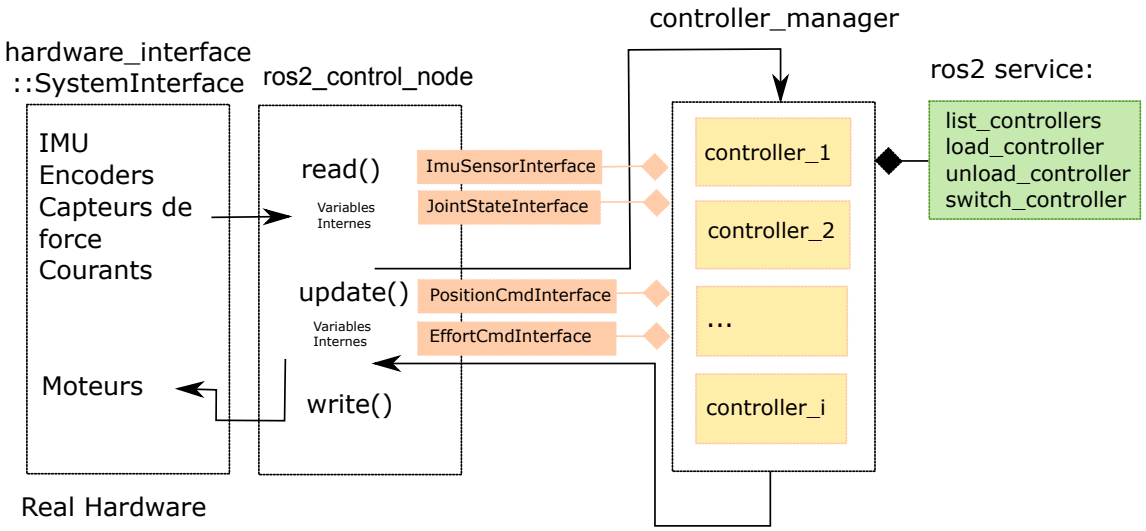


FIGURE 20.2 – Schéma incomplet des interactions des objets ros2_control

capteur de couple. On accéde à cet objet avec une interface d'état *StateInterface*.

— *SystemInterface* : cet objet considére un objet mécatronique plus complexe et permet d'envoyer des ordres à plusieurs actionneurs et de lire les informations des capteurs. Il peut s'agir d'un robot manipulateur à 6 degrés de liberté, d'un robot quadrupède à 12 DLLs, ou d'un robot humanoïde à 32 DLLs. Cet objet est donc constitué d'un vecteur d'interface de commande et un vecteur d'interface d'état.

Ces objets sont gérés par l'objet *RessourceManager* qui charge les plugins C++ appropriés grâce à des balises *ros2_control* incluent dans l'URDF.

Les contrôleurs sont des objets C++ qui lisent l'état d'un système robotique à travers les interfaces des capteurs et en déduisent les valeurs à mettre dans les actionneurs. Il est possible de chaîner ces controleurs les uns avec les autres.

La boucle de perception-action est implementée dans le node ros2_control_node du paquet ROS *controller_manager* dans la repository *ros2_control*.

Voici une version réduite de ce node :

```
auto cm = std::make_shared<controller_manager::ControllerManager>(executor, manager_node_name);

while (rclcpp::ok())
  {
    // calculate measured period
    auto const current_time = cm->now();
    auto const measured_period = current_time - previous_time;
    previous_time = current_time;
    // execute update loop
    cm->read(cm->now(), measured_period);
    cm->update(cm->now(), measured_period);
    cm->write(cm->now(), measured_period);
    // wait until we hit the end of the period
    next_iteration_time += period;
    std::this_thread::sleep_until(next_iteration_time);
  }
```

Listing 20.1 – La boucle perception action

Dans de nombreux robots (par exemple les robots TIAGO et TALOS de la société PAL-Robotics), le noyau LINUX avec le patch RT_PREEMPT fournissent une forme de temps réel (par héritage de priorité) qui permet d'aller jusqu'à des boucles de 2 KHz. Recompiler ce node avec cette option du noyau activée permettra d'obtenir un node fonctionnant en temps réel.

Il est également possible de réimplémenter ce node pour l'adapter à un système d'exploitation temps-réel.

L'ensemble des objets implémentant ros2_control est décrit dans la figure Fig. 20.3.

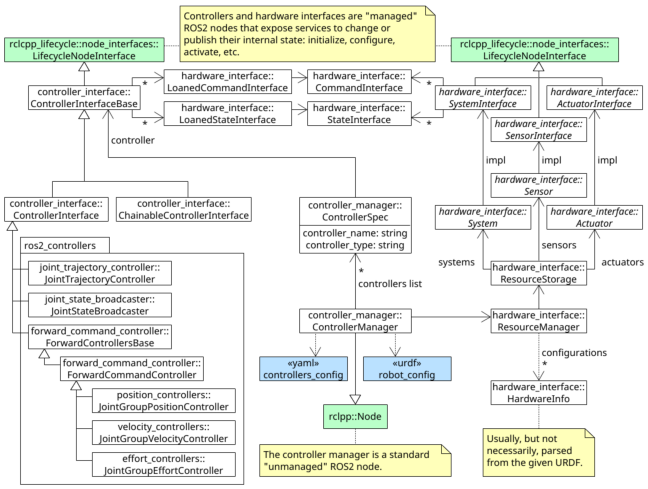


FIGURE 20.3 – Organisation complète des objets de ros2_control

20.1.1 ControllerManager

L'objet central est une instance du *ControllerManager*, ici nommé *cm* que l'on retrouve dans le listing 20.1 de la boucle perception action.

L'objet *cm* lit le topic *robot_description*. Celui-ci contient en plus du modèle cinématique du robot des balises *ros2_control*. Ces balises permettent de charger des plugins qui instancient des objets de classe *System*, *Actuator*, *Sensor*. Ils sont soit des plugins génériques avec des configurations spécifiques au robot, soit des plugins spécifiques au matériel développé.

L'intérêt est ainsi de cacher les spécificités d'un choix de bus industriel : RS485, RS422, EtherCat, CAN etc... en minimisant l'impact sur le logiciel de la structure de contrôle.

Le *ControllerManager* permet ainsi d'abstraire le matériel et les contrôleurs.

La gestion des contrôleurs s'effectue grâce aux services dans l'espace de noms *controller_manager*.

20.1.2 Contrôleurs

Les contrôleurs sont chargés via un appel au service */controller_manager/load_controller*. Il est intéressant d'implémenter des contrôleurs relativement génériques afin de pouvoir les utiliser dans le plus de cas possibles.

Le fichier de configuration spécifie le nom des joints à utiliser et le contrôleur essaient d'acquérir les interfaces des joints. Les interfaces dites de *command* permettent de commander les joints. Les interfaces dites d'état (*state*) permettent de lire les joints.

Pour chaîner deux contrôleurs *A* et *B*, il faut que *A* transmette au moins un *double* à *B*. La classe qui implémente *B* indique donc qu'il posséde une interface de commande mais le nom complet est *nom_instance_du_contrôleur/nom_in*

Il est donc nécessaire de passer l'information à *A* qu'il doit écrire sur l'interface *B*. Cela s'effectue dans le fichier de configuration yaml lors de la construction du graphe de contrôle.

20.2 Exemples d'interfaces matériel

Dans cette section des exemples illustrent des patrons de structure applicatives concernant la gestion du matériel. Le but est de démontrer le gain à passer par ros2_control. Auparavant il est intéressant de voir comment celle-ci est spécifié dans un fichier URDF.

20.2.1 Le champ ros2_control dans le fichier URDF

Les interfaces ros2_control se spécifient par des balises ros2_control. Dans le premier exemple du dépôt de *ros2_control_demos*, le fichier rrbot.urdf.xacro illustre l'organisation recommandée.

En effet, les balises ros2_control sont toutes enregistrées dans un fichier xacro qui est stocké dans un repértoire *ros2_control* du paquet description du robot.

```xml
  <?xml version="1.0"?>
<!-- Revolute-Revolute Manipulator -->
<!--
Copied and modified from ROS1 example -
https://github.com/ros-simulation/gazebo_ros_demos/blob/kinetic-devel/rrbot_description/urdf/rrbot.xacro
-->
<robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="2dof_robot">
  <xacro:arg name="prefix" default="" />

  <!-- Import RRBot macro -->
  <xacro:include filename="$(find ros2_control_demo_description)/rrbot/urdf/rrbot_description.urdf.xacro" />

  <!-- Import Rviz colors -->
  <xacro:include filename="$(find ros2_control_demo_description)/rrbot/urdf/rrbot.materials.xacro" />

  <!-- Import RRBot ros2_control description -->
  <xacro:include filename="$(find ros2_control_demo_example_1)/ros2_control/rrbot.ros2_control.xacro" />

  <!-- Used for fixing robot -->
  <link name="world"/>

  <xacro:rrbot parent="world" prefix="$(arg prefix)">
    <origin xyz="0 0 0" rpy="0 0 0" />
  </xacro:rrbot>

  <xacro:rrbot_ros2_control
    name="RRBot" prefix="$(arg prefix)" />

</robot>
```

Listing 20.2 – Le fichier xacro racine décrivant le modèle URDF du robot RRBot

Le fichier inclut dans le fichier `rrbot.urdf.xacro` à la ligne 17 est donné dans le listing 20.3. La balise *ros2_control_name* spécifie le nom de l'interface *ros2_control* par le paramètre "name" de la balise, et son type par le paramètre "system" à la ligne 6.

L'interface matériel utilisée est décrie dans la balise *hardware* ici à la ligne 7. La librarie a utilisé est donnée par la balise *plugin*. Les paramètres transmis à cette librairie dynamique sont *example_param_hw_start_duration*, *example_param_hw_stop_duration*, *example_param_hw_slowdown* indiqués de la ligne 9 à la ligne 11.

Ensuite l'interaction avec les joints est spécifiée dans chacune des balises *joint* où le nom spécifique d'un joint est donné en paramètre de la balise 14. Pour chacun des joint, il est ensuite possible de spécifier une interface de commande par la balise *command_interface*. La ligne. 15 donne un exemple d'interface de commande en position. Dans le bloc définit par cette balise il est possible de spécifier la valeur maximale et minimale par des paramètres (16-17).

Enfin on peut spécifier une interface d'état pour la lecture d'un capteur ou d'une quantité fournie par un actionneur. La ligne 19 donne l'exemple d'une interface d'état de la position du joint. Pour un moteur électrique cette information est typiquement fournie par un encodeur optique. Si l'actionneur possède un système de transmission comme un entraînement planétaire de marque Harmonic Drive, il est possible d'obtenir une information de position en sortir d'axe avec un capteur magnétique.

```xml
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">

  <xacro:macro name="rrbot_ros2_control" params="name prefix">

    <ros2_control name="${name}" type="system">
      <hardware>
        <plugin>ros2_control_demo_example_1/RRBotSystemPositionOnlyHardware</plugin>
        <param name="example_param_hw_start_duration_sec">0</param>
        <param name="example_param_hw_stop_duration_sec">3.0</param>
        <param name="example_param_hw_slowdown">100</param>
      </hardware>

      <joint name="${prefix}joint1">
        <command_interface name="position">
          <param name="min">-1</param>
          <param name="max">1</param>
        </command_interface>
        <state_interface name="position"/>
      </joint>
      <joint name="${prefix}joint2">
        <command_interface name="position">
          <param name="min">-1</param>
          <param name="max">1</param>
        </command_interface>
        <state_interface name="position"/>
```

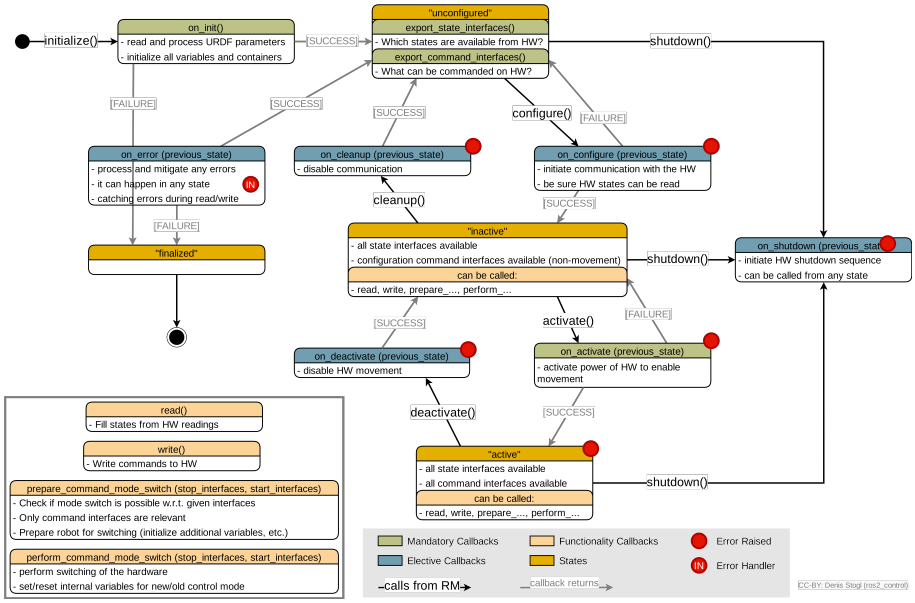


FIGURE 20.4 – Le cycle de vie des interfaces matérielles

```
      </joint>
    </ros2_control>

  </xacro:macro>
</robot>
```

Listing 20.3 – Partie spécifique à ros2_control d'un fichier xacro pour l'URDF du robot RRBot

Regardons la structure de la classe qui implémente le plugin *RRBotSystemPositionOnlyHardware*. Elle reste relativement hypothètique car elle n'illustre par l'interaction avec un matériel réel.

La classe est définie par le fichier rrbot.hpp que l'on retrouve dans les listings 20.4.

D'aprés la définition initiale de la classe SystemInterface, cette classe interagit avec le matériel de la manière suivante :

UNCONFIGURED (on_init, on_cleanup) : Le matériel est initialisé mais les communications n'ont pas démarré et par conséquent aucune interface n'est disponible. L'objet vérifie que la partie ros2_control du fichier URDF du robot est cohérente avec le matériel.

INACTIVE (on_configure, on_deactivate) : Les communications avec la matériel est démarré et elle est configurée. Les états peuvent être lus et les interfaces matérielles de commande qui ne font pas bouger le robot peuvent être spécifiées. Les interfaces matérielles de commande qui font bouger le robot ne doivent pas être disponibles. Les interfaces considérées sont : HW_IF_POSITION, HW_IF_VELOCITY, HW_IF_ACCELERATION, et HW_IF_EFFORT.

FINALIZED (on_shudtdown) : L'interface matériel est prête pour la destruction de l'objet. La mémoire allouée sera détruite.

ACTIVE (on_activate) : Les circuits de puissance sont activés et le matériel peut bouger, i.e. les freins sont désactivés. Les interfaces de commande pour le mouvement sont disponibles et doivent être acceptés. Les interfaces sont : HW_IF_POSITION, HW_IF_VELOCITY, HW_IF_ACCELERATION, et HW_IF_EFFORT.

ACTIVE (read) : Cette méthode lit les valeurs de l'actionneur physique et remplis les paramètres des objets de l'interface d'état utilisés par les contrôleurs.

ACTIVE (write) : Cette méthode écrit les valeurs de commande fournies par les contrôleurs pour qu'ils soient exécutés par les actionneurs.

Ce cycle de vie des interfaces logicielles est disponible dans la figure Fig. 20.4. On retrouve dans les transitions les méthodes réimplémentables (qui doivent être maintenant prefixé par **on_**).

```
#ifndef ROS2_CONTROL_DEMO_EXAMPLE_1__RRBOT_HPP_
#define ROS2_CONTROL_DEMO_EXAMPLE_1__RRBOT_HPP_
```

```
3
4  #include <memory>
5  #include <string>
6  #include <vector>
7
8  #include "hardware_interface/handle.hpp"
9  #include "hardware_interface/hardware_info.hpp"
10 #include "hardware_interface/system_interface.hpp"
11 #include "hardware_interface/types/hardware_interface_return_values.hpp"
12 #include "rclcpp/macros.hpp"
13 #include "rclcpp_lifecycle/node_interfaces/lifecycle_node_interface.hpp"
14 #include "rclcpp_lifecycle/state.hpp"
15 #include "ros2_control_demo_example_1/visibility_control.h"
16
17 namespace ros2_control_demo_example_1
18 {
19 class RRBotSystemPositionOnlyHardware : public hardware_interface::SystemInterface
20 {
21 public:
22   RCLCPP_SHARED_PTR_DEFINITIONS(RRBotSystemPositionOnlyHardware);
23
24   ROS2_CONTROL_DEMO_EXAMPLE_1_PUBLIC
25   hardware_interface::CallbackReturn on_init(
26     const hardware_interface::HardwareInfo & info) override;
27
28   ROS2_CONTROL_DEMO_EXAMPLE_1_PUBLIC
29   hardware_interface::CallbackReturn on_configure(
30     const rclcpp_lifecycle::State & previous_state) override;
31
32   ROS2_CONTROL_DEMO_EXAMPLE_1_PUBLIC
33   std::vector<hardware_interface::StateInterface> export_state_interfaces() override;
34
35   ROS2_CONTROL_DEMO_EXAMPLE_1_PUBLIC
36   std::vector<hardware_interface::CommandInterface> export_command_interfaces() override;
37
38   ROS2_CONTROL_DEMO_EXAMPLE_1_PUBLIC
39   hardware_interface::CallbackReturn on_activate(
40     const rclcpp_lifecycle::State & previous_state) override;
41
42   ROS2_CONTROL_DEMO_EXAMPLE_1_PUBLIC
43   hardware_interface::CallbackReturn on_deactivate(
44     const rclcpp_lifecycle::State & previous_state) override;
45
46   ROS2_CONTROL_DEMO_EXAMPLE_1_PUBLIC
47   hardware_interface::return_type read(
48     const rclcpp::Time & time, const rclcpp::Duration & period) override;
49
50   ROS2_CONTROL_DEMO_EXAMPLE_1_PUBLIC
51   hardware_interface::return_type write(
52     const rclcpp::Time & time, const rclcpp::Duration & period) override;
53
54 private:
55   // Parameters for the RRBot simulation
56   double hw_start_sec_;
57   double hw_stop_sec_;
58   double hw_slowdown_;
59
60   // Store the command for the simulated robot
61   std::vector<double> hw_commands_;
62   std::vector<double> hw_states_;
63 };
64
65 }  // namespace ros2_control_demo_example_1
66
67 #endif  // ROS2_CONTROL_DEMO_EXAMPLE_1__RRBOT_HPP_
```

Listing 20.4 – Définition de la classe pour le plugin RRBotSystemPositionOnlyHardware

L'implémentation de la classe est disponible ici. Seule la méthode *init* donne un exemple intéressant pour lire les paramètres et vérifier si le matériel correspond à la description de l'URDF.

20.2.2 Accés à un micro-contrôleur par Ethernet

Il est courant d'écrire une librairie avec un protocole spécifique pour accéder à un nouveau matériel. Les robots Dual-motors, Bolt, Solo et Sassa (voir Fig. 20.5) développés par le groupe Open Dynamic Robot Initivative utilisent la carte de contrôle de puissance μDriver v2 [9]. Celle-ci est contrôlée par une carte appelée *master_board* [8]. La carte *master_board* utilise un micro contrôleur ESP32 avec une connection Ethernet pour la communication avec un PC comme indiqué dans la figure Fig. 20.6.

Le protocole permettant de communiquer entre le PC et la carte *master_board* est implémenté dans la

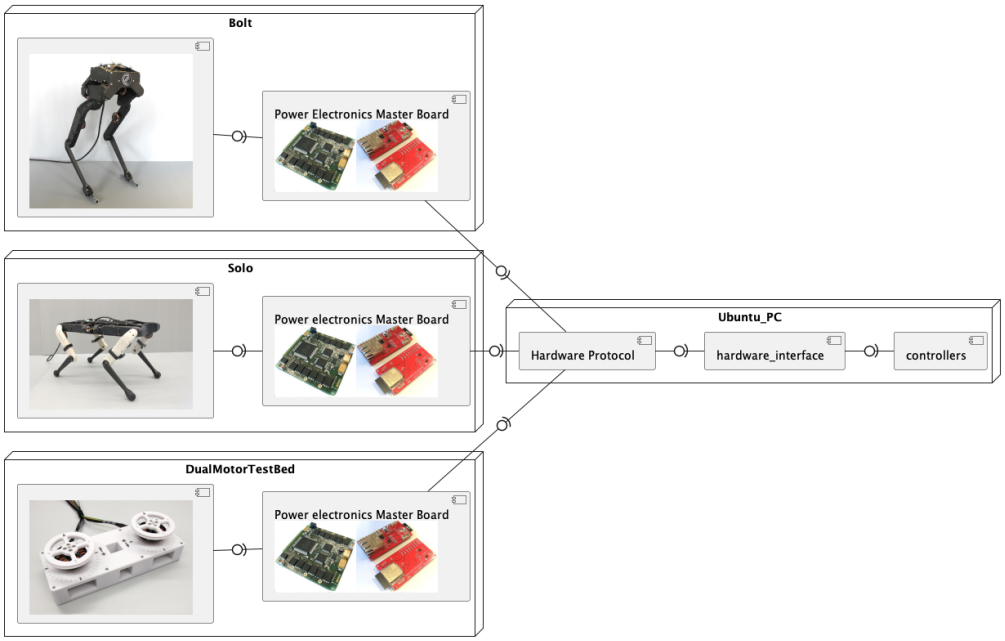


FIGURE 20.5 – Prototypes utilisant la même structure de contrôle de puissance connectée à un micro contrôleur ESP32 et communique avec un PC à travers Ethernet ou Wifi.

librairie *master_board_sdk*. Cette librairie a été encapsulée dans un plugin qui implémente un objet dérivé de la classe *hardware_interface ::SystemInterface*. Le même plugin peut-être alors utilisé dans le fichier URDF des robots dual-motors et bolt.

Cet exemple est détaillé dans le chapitre suivant.

Note 20.1 Il est maintenant possible d'étendre les interfaces localement. Si vous souhaitez rajouter des informations supplémentaires comme la température, des gains supplémentaires pour accéder au matériel c'est possible. Il faut rajouter une constante qui y fait référence et qui sera locale à votre matériel. Un exemple plus étendue est donné dans le chapitre 22. La seule solution avec ros1_control était de faire une autre version de l'objet *HardwareInterface*. ■

20.2.3 Ethercat

Il existe un driver ros2_control pour les bus Ethercat [2]. Ce driver fournit un objet de type *hardware_interface ::SystemInterface* qui permet d'interagir avec le bus Ethercat à travers le master et ainsi atteindre les esclaves spécifiés dans le fichier URDF.

Des explications étendues sont données ici : `https://icube-robotics.github.io/ethercat_driver_ros2`.

Un exemple de configuration est donné dans le listing 20.5. Il est possible de spécifier l'identifiant du module maître avec le paramètre *master_id*, et la fréquence de contrôle c'est à dire d'envoi des données avec le paramètre *control_frequency*.

```
<ros2_control name="mySystem" type="system">
  <hardware>
    <plugin>ethercat_driver/EthercatDriver</plugin>
    <param name="master_id">0</param>
    <param name="control_frequency">100</param>
  </hardware>
</ros2_control>
```

Listing 20.5 – Exemple de configuration Ethercat

Il est également possible de pouvoir spécifier des esclaves Ethercat notamment dans le cadre de *joint*, *sensor*, *gpio*. On a ainsi un exemple de ressource gpio dans le listing 20.6.

```
<gpio name="myGPIO">
```

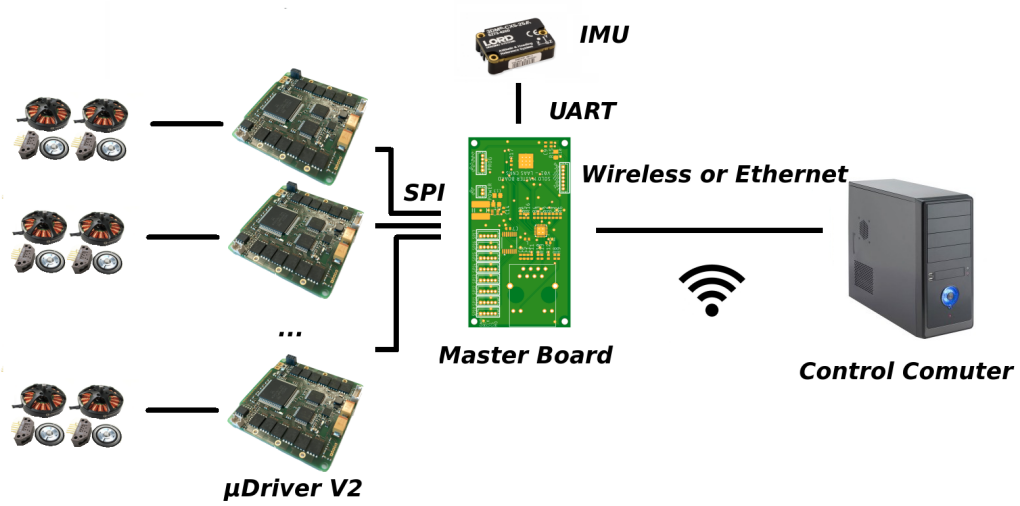


FIGURE 20.6 – Topologie entre les moteurs, la carte μDriver v2, la carte *master_board*, l'IMU, et l'ordinateur de contrôle.

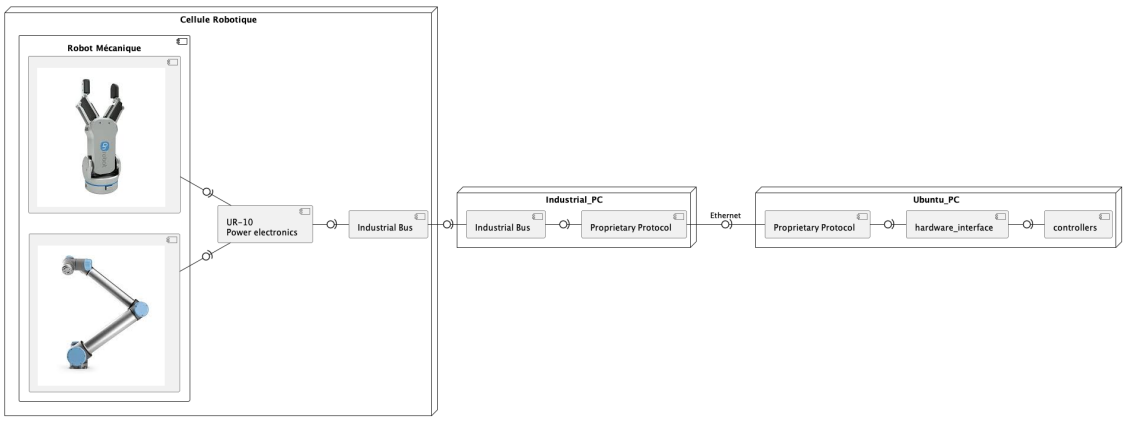


FIGURE 20.7 – Description d'une cellule robotique industrielle avec un UR-10 et un gripper

```
  <command_interface name="dig_output.1"/>
  <command_interface name="dig_output.2"/>
  <state_interface name="ana_input.1"/>
  <state_interface name="ana_input.2"/>
  <ec_module name="EL3104">
    <plugin>ethercat_generic_plugins/GenericEcSlave</plugin>
    <param name="alias">0</param>
    <param name="position">0</param>
    <param name="slave_config">/path/to/EL3104_slave_config.yaml</param>
  </ec_module>
  <ec_module name="EL2008">
    <plugin>ethercat_generic_plugins/GenericEcSlave</plugin>
    <param name="alias">0</param>
    <param name="position">1</param>
    <param name="slave_config">/path/to/EL2008_slave_config.yaml</param>
  </ec_module>
</gpio>
```

Listing 20.6 – Exemple de configuration de modules guidés

20.2.4 Les robots industriels

Les robots industriels offrent dans quelques cas la possibilité d'envoyer des ordres à leur PC de contrôle. C'est généralement réalisé grâce à un protocole réseau que l'on envoit via un liaison ethernet. C'est le cas

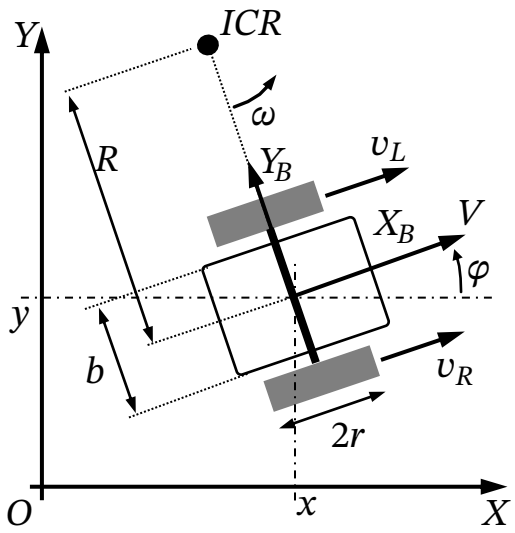


FIGURE 20.8 – Description mathématique d'un robot mobile à roues différentiel. Cette figure est tirée de [12].

pour les Universal Robots, Panda et Staubli. Dans les deux premiers cas, les constructeurs ont implémenté un objet *hardware_interface ::SystemInterface* qui permet d'accéder aux joints et aux capteurs à travers ros2_control.

De la même façon que pour l'exemple RRBot

20.3 Exemple de contrôleur : diff_drive_controller

On considére ici le contrôleur d'un robot mobile différentiel de façon générale, celui-ci est modélisé par la figure Fig. 20.8

with :

— V - la vitesse linéaire du robot mobile,
— ω - la vitesse angulaire par laquelle le robot tourne,
— X et Y - le système de coordonnées globales,
— X_B et Y_B - le système de cooordonnées locales,
— ϕ - l'angle du robot par rapport au système de coordonnées global,
— r - le rayon des roues,
— b - la largueur du véhicule,
— ICR - le centre de rotation instantané,
— v_L, v_R : la vitesse de contact au sol de la roue gauche et droite,
— ω_L, ω_R : la vitesse angulaire du contact au sol de la roue gauche et droite.

20.3.0.1 Cinématique inverse

La vitesse linéaire de contact au sol de la roue gauche v_L et de la roue droite v_R donnent lieu à une rotation du véhicule proportionnelle à la vitesse angulaire ω.

$$\omega(R+b/2) = v_R \tag{20.1}$$
$$\omega(R-b/2) = v_L \tag{20.2}$$

Résoudre les deux équations pour ω et R, où ce dernier est défini comme la distance du centre du robot au centre de rotation instantané, on obtient :

$$\omega = (v_R - v_L)/b \tag{20.3}$$
$$R = b/2.(v_R + v_L)/(v_R - v_L) \tag{20.4}$$

On obtient alors la vitesse linéaire du robot au point central c'est à dire entre les deux roues :

$$V = \omega.R = \frac{v_R + v_L}{2} \tag{20.5}$$

20.3.0.2 Contrôleur différentiel

En général on donne au contrôleur la vitesse instantanée désirée V au point central du robot (c'est à dire entre les deux roues), et sa vitesse angulaire au même point. En utilisant les relations $R = V/\omega$ et $\omega_R = v_R/r$ dans

$$\omega(R + b/2) = v_R \tag{20.6}$$

Le contrôleur du véhicule à roues différentielles s'écrit donc :

$$\omega_R = \frac{V + \omega.b/2}{r} \tag{20.7}$$

$$\omega_L = \frac{V - \omega.b/2}{r} \tag{20.8}$$

La ligne correspondante dans le contrôleur diff_drive_controller est accessible ici.

Le robot TIAGO de la société PAL-Robotics utilise ce contrôleur pour la simulation. Les paramètres du contrôleur sont spécifiés dans les fichiers de configuration de la base mobile pmb2 et se trouvent ici. On retrouve b comme étant *wheel_separation* and r est *wheel_radius*. Les autres termes sont des limites que l'on retrouve dans l'implémentation du contrôleur *DiffDriveController*.

```
mobile_base_controller :
 ros__parameters :
  type : diff_drive_controller/DiffDriveController
  use_stamped_vel : False
  left_wheel_names  : ['wheel_left_joint']
  right_wheel_names  : ['wheel_right_joint']

  # In ROS1 these physical properties were parsed from the URDF
  wheel_radius : 0.0985
  wheel_separation : 0.4044

  wheel_separation_multiplier : 1.0
  left_wheel_radius_multiplier : 1.0
  right_wheel_radius_multiplier : 1.0

  publish_rate : 50.0
  pose_covariance_diagonal  : [0.001, 0.001, 0.001, 0.001, 0.001, 0.01]
  twist_covariance_diagonal : [0.001, 0.001, 0.001, 0.001, 0.001, 0.01]

  # Velocity commands timeout [s]
  cmd_vel_timeout : 0.25

  # Base frame_id
  base_frame_id : base_footprint

  # Publish limited velocity
  publish_limited_velocity : true

  # Velocity and acceleration limits
  # Whenever a min_* is unspecified, default to -max_*
  linear :
   x :
    has_velocity_limits    : true
    max_velocity           : 1.0 # m/s
```

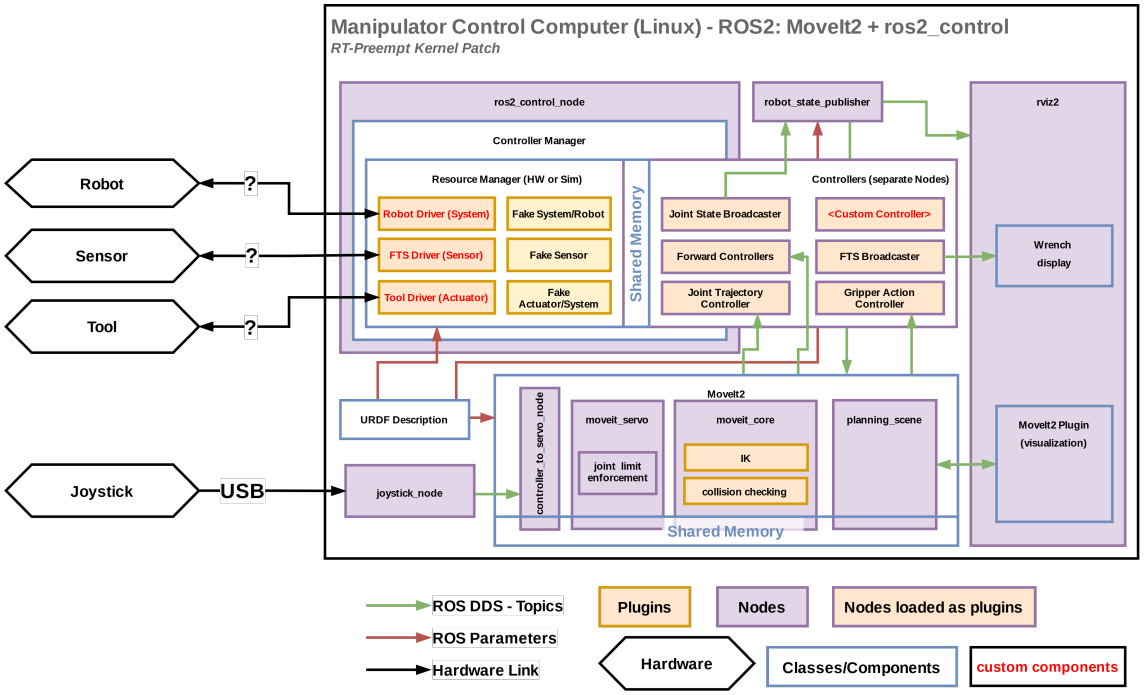


FIGURE 20.9 – Intégration de ros2_control dans MoveIt 2

```
    min_velocity          : -0.2 # m/s
    has_acceleration_limits : true
    max_acceleration       : 0.5 # m/s^2
    has_jerk_limits        : false
    max_jerk               : 5.0 # m/s^3
  angular :
   z :
    has_velocity_limits    : true
    max_velocity           : 1.05 # ž60.0
    has_acceleration_limits : true
    max_acceleration       : 1.05 # ž60.0
    has_jerk_limits        : false
    max_jerk               : 10.47 # ž600.0
```

Listing 20.7 – Fichier de configuration du contrôleur DiffDriveController pour la base mobile pmb2 de PAL-Robotics

20.3.1 Chaînage des contrôleurs

Contrairement à ros1_control il est possible d'enchaîner les contrôleurs les uns à la suite des autres.

20.3.2 Vue générale de ros2_control avec MoveIt 2

La figure Fig.20.3.2 représente l'ensemble des ressources matérielles, les contrôleurs, le node robot_state_publisher et l'interface graphique rviz2.

21. Les démonstrations de ros2_control

Le but de ce chapitre est de parcourir certains exemples du dépôt ros2_control_demos.

21.1 Exemple 1 : RRBot

Le robot RRBot, pour "Revolute-Revolute Manipulator Robot" est un simple bras robotique avec 3 corps, 2 joints qui sera utilisé pour démontrer plusieurs propriétés.

Dans ce premier exemple, le plugin de l'interface matériel ne contient qu'une seule interface.

Les fichiers URDF décrivant le robot RRBot se trouve dans le répertoire `description/urdf`.

21.1.1 Démarrer l'exemple

Pour démarrer l'exemple il faut ouvrir un terminal, sourcer l'espace de travail ros2, et lancer le fichier de launch avec :

```
ros2 launch ros2_control_demo_example_1 rrbot.launch.py
```

Le fichier de launch charge le matériel du robot, les controleurs et ouvre RViz. Dans le terminal de démarrage, on peut voir de nombreuses informations affichées de la part de l'implémentation du matériel qui montre son état interne. Ces affichages ne sont pas là que pour expliquer les différentes étapes et doivent être évitées le plus possible sur un matériel réel.

S'il est possible de voir deux rectangles orange, et un jaune dans RViz, comme illustré dans la figure Fig.21.1.1 tout a démarré proprement.

Il est possible maintenant d'analyser le système avant de lancer des contrôleurs.

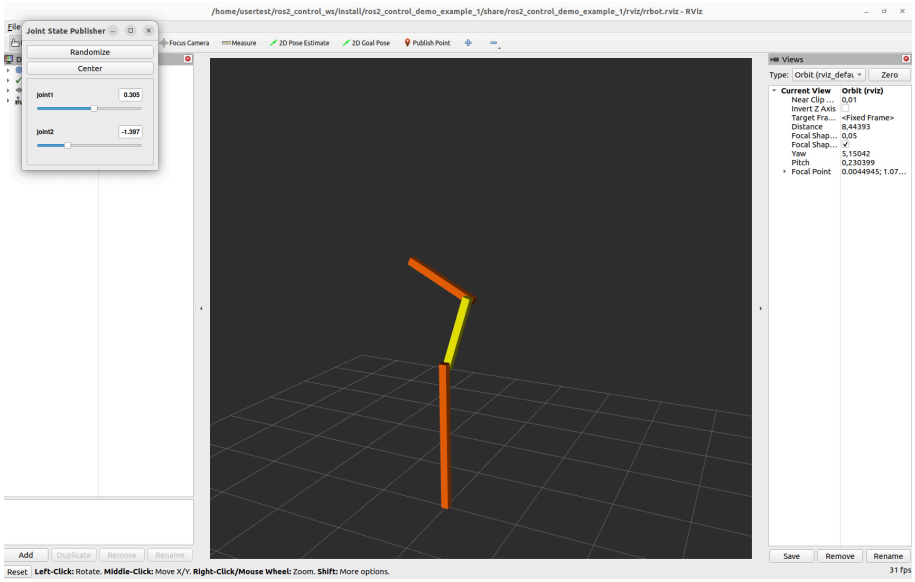
21.1.2 Vérification du matériel

Il est possible de vérifier que le matériel s'est chargé correctement en ouvrant un autre terminal et en lancant :

```
ros2 control list_hardware_interfaces
```

Le résultat est :

```
command interfaces
      joint1/position [available] [claimed]
      joint2/position [available] [claimed]
state interfaces
      joint1/position
      joint2/position
```



.

FIGURE 21.1 – Visualisation du robot RRBot dans RViz.

Le marqueur `[claimed]` sur les interfaces commandes indiquent qu'un contrôleur a accés aux commandes des joints correspondants du robot RRBot.

21.1.3 Vérifier que les contrôleurs fonctionnent

Cela s'effectue avec la commande suivante :

```
ros2 control list_controllers
```

Le résultat devrait être :

```
joint_state_broadcaster[joint_state_broadcaster/JointStateBroadcaster] active
forward_position_controller[forward_command_controller/ForwardCommandController] active
```

Le contrôleur *joint_state_broadcaster* est celui qui publie les positions des joints sur le topic */joint_states*. Il s'agit d'un observateur plus que d'un contrôleur. Il assure la connection avec le middleware.

Le contrôleur *forward_command_controller* est celui qui commande le matériel pour atteindre des positions spécifiques.

21.1.4 Envoyer des commandes

Si le contrôleur *forward_command_controller* est actif, il est possible d'envoyer des commandes suivant deux options :

— Manuellement en utilisant la ligne de commande ros2 qui va publier sur le topic que lit le contrôleur :

```
ros2 topic pub /forward_position_controller/commands std_msgs/msg/Float64MultiArray "data:
- 0.5
- 0.5"
```

— Ou en utilisant le node de démonstration qui envoit trois buts différents par cycle toutes les 5 secondes :

```
ros2 launch ros2_control_demo_example_1 test_forward_position_controller.launch.py
```

Il est possible de voir les valeurs numériques évoluées en affichant le contenu du topic `/joint_states` ou `/dynamic_joint_states` :

```
ros2 topic echo /joint_states
ros2 topic echo /dynamic_joint_states
```

21.1.5 Utilisation d'un autre contrôleur

Il est possible de basculer sur un autre contrôleur nommé *JointTrajectoryController*. Il faut pour cela le charger manuellement avec :

```
ros2 control load_controller joint_trajectory_position_controller
```

qui devrait retourner

```
Successfully loaded controller joint_trajectory_position_controller}.
```

Il est possible de vérifier le status des contrôleurs grâce à :

```
ros2 control list_controllers
```

qui montre que le contrôleur est chargé mais n'est pas configuré.

```
joint_state_broadcaster[joint_state_broadcaster/JointStateBroadcaster] active
forward_position_controller[forward_command_controller/ForwardCommandController] active
joint_trajectory_position_controller[joint_trajectory_controller/JointTrajectoryController] unconfigured
```

Il est possible de configurer le contrôleur en le mettant à `inactive` avec :

```
ros2 control set_controller_state joint_trajectory_position_controller inactive
```

qui donne

```
Successfully configured joint_trajectory_position_controller.
```

Il est maintenant possible de passer d'un contrôleur à l'autre en effectuant l'un ou l'autre commande :

— soit en utilisant deux commandes

```
ros2 control set_controller_state forward_position_controller inactive
ros2 control set_controller_state joint_trajectory_position_controller active

```

— soit en utilisant une commande

```
ros2 control switch_controllers --activate joint_trajectory_position_controller --deactivate
     forward_position_controller

```

On peut alors avoir l'état des contrôleurs grâce à :

```
ros2 control list_controllers
```

qui devrait retourner :

```
joint_state_broadcaster[joint_state_broadcaster/JointStateBroadcaster] active
forward_position_controller[forward_command_controller/ForwardCommandController] inactive
joint_trajectory_position_controller[joint_trajectory_controller/JointTrajectoryController] active
```

Il est alors possible d'envoyer une commande au contrôleur en utilisant un node qui envoit quatre buts toutes les 6 secondes dans une boucle :

```
ros2 launch ros2_control_demo_example_1 test_joint_trajectory_controller.launch.py
```

21.1.6 Pour aller plus loin

— Trouver les fichiers qui stockent les différents buts atteints par le robot RRbot.
— Dans quel paquet se trouve le node qui publie les buts effectués par le contrôleur *joint_trajectory_controller* ?
— Déterminer quels sont les paramètres qui forcent le contrôleur *joint_trajectory_controller* à contrôler le robot en position.

21.2 Exemple 3 : Un robot avec plusieurs interfaces

Cet exemple montre comment il est possible d'utiliser de multiples interfaces pour contrôler un robot.

21.2.1 Démarrage de RRBot

Pour démarrer le robot RRBot il faut ouvrir un terminal, sourcer l'espace de travail ROS-2, et lancer le fichier de launch avec :

```
ros2 launch ros2_control_demo_example_3 rrbot_system_multi_interface.launch.py
```

Il est possible d'utiliser les options suivantes :

— `robot_controller:=forward_position_controller` démarre la démonstration ainsi que le contrôleur en position. Le robot peut-être contrôlé utilisant le contrôleur *forward_position_controller*.

— `robot_controller:=forward_acceleration_controller` démarre la démonstration ainsi que le contrôleur en accélération. Le robot peut-être contrôlé utilisant le contrôleur *forward_acceleration_controller*.

Le fichier de launch charge et démarre le matériel, les contrôleurs et ouvre RViz. Au démarrage du terminal, un grand nombre d'informations vont s'afficher.

21.2.2 Vérification du matériel

Pour vérifier si le matériel s'est chargé correctement il faut ouvrir un terminal et executé la ligne suivante :

```
ros2 control list_hardware_interfaces
```

La sortie devrait ressembler à cela :

```
command interfaces
    joint1/acceleration [available] [unclaimed]
    joint1/position [available] [unclaimed]
    joint1/velocity [available] [claimed]
    joint2/acceleration [available] [unclaimed]
    joint2/position [available] [unclaimed]
    joint2/velocity [available] [claimed]
state interfaces
    joint1/acceleration
    joint1/position
    joint1/velocity
    joint2/acceleration
    joint2/position
    joint2/velocity
```

La mention `[claimed]` pour les interfaces de commande signifie qu'un contrôleur a accés aux commandes du robot RRBot.

21.2.3 Vérification des contrôleurs

On peut vérifier que les contrôleurs fonctionnent avec :

```
ros2 control list_controllers
```

ce qui donne :

```
joint_state_broadcaster[joint_state_broadcaster/JointStateBroadcaster] active
forward_velocity_controller[velocity_controllers/JointGroupVelocityController] active
```

Il est intéressant de voir les différences d'affichage en modifiant les arguments d'entrée décrit précédemment.

21.2.4 Envoi d'une commande au contrôleur

21.2.4.1 Manuellement en utilisant l'interface de commande

— Lors de l'utilisation du contrôleur *forward_position_controller* :

```
ros2 topic pub /forward_position_controller/commands std_msgs/msg/Float64MultiArray "data:
- 0.5
- 0.5"
```

— Lors de l'utilisation du contrôleur *forward_velocity_controller* :

```
ros2 topic pub /forward_velocity_controller/commands std_msgs/msg/Float64MultiArray "data:
- 5
- 5"
```

— Lors de l'utilisation du contrôleur *forward_acceleration_controller* :

```
ros2 topic pub /forward_acceleration_controller/commands std_msgs/msg/Float64MultiArray "data:
- 10
- 10"
```

21.2.4.2 Envoi automatique et cyclique de buts

Il est possible d'exécuter un node qui envoit deux buts chaque 5 secondes dans un cycle quand on utilise le contrôleur *forward_position_controller*.

```
ros2 launch ros2_control_demo_example_3 test_forward_position_controller.launch.py
```

Le robot doit bouger dans l'application RViz, et le terminal affiche les nouvelles valeurs de l'état.

21.2.5 Test des configurations illégales de contrôleur

Afin d'illustrer la configuration illégale d'un contrôleur, il est possible d'utiliser un des arguments du fichier de launch suivant :

— `robot_controller:=forward_illegal1_controller`

— `robot_controller:=forward_illegal2_controller`

Il est possible de voir les messages d'erreurs suivants, parce que l'interface matérielle impose que les joints aient tous la même interface de commande.

```
[ros2_control_node-1] [ERROR] [1676209982.531163501] [resource_manager]: Component 'RRBotSystemMultiInterface' did
      not accept new command resource combination:
[ros2_control_node-1]  Start interfaces:
[ros2_control_node-1] [
[ros2_control_node-1]   joint1/position
[ros2_control_node-1] ]
[ros2_control_node-1] Stop interfaces:
[ros2_control_node-1] [
[ros2_control_node-1] ]
[ros2_control_node-1]
[ros2_control_node-1] [ERROR] [1676209982.531223835] [controller_manager]: Could not switch controllers since
     prepare command mode switch was rejected.
[spawner-4] [ERROR] [1676209982.531717376] [spawner_forward_illegal1_controller]: Failed to activate controller
```

L'utilisation de `ros2 control list_hardware_interfaces` montre qu'aucune interface n'est réclamée.

```
command interfaces
      joint1/acceleration [available] [unclaimed]
      joint1/position [available] [unclaimed]
      joint1/velocity [available] [unclaimed]
      joint2/acceleration [available] [unclaimed]
      joint2/position [available] [unclaimed]
      joint2/velocity [available] [unclaimed]
state interfaces
      joint1/acceleration
      joint1/position
      joint1/velocity
      joint2/acceleration
      joint2/position
      joint2/velocity
```

et `ros control list_controllers` indique que le contrôleur illégal n'est pas chargé.

```
joint_state_broadcaster[joint_state_broadcaster/JointStateBroadcaster] active
forward_illegal1_controller[forward_command_controller/ForwardCommandController] inactive
```

21.3 Exemple 12 : Le chaînage de contrôleurs

Le chaînage de contrôleurs permet d'associer les contrôleurs à la suite les uns des autres. Dans cet exemple, le robot RRBot est à nouveau utilisé. On peut également y trouver un contrôleur chaînable qui possède un vecteur d'interfaces de commandes comme entrée et les transmet sans aucune modification. Ce contrôleur est nommé *passthrough_controller*

21.3.1 Démarrage de l'exemple

Comme pour les autres exemples, il faut ouvrir un terminal, sourcer l'espace de travail ROS2 et lancer le fichier de launch avec :

```
ros2 launch ros2_control_demo_example_12 rrbot.launch.py
```

21.3.2 Vérification des contrôleurs

Pour valider les contrôleurs il suffit de les lister avec :

```
ros2 control list_controllers
```

On peut voir dans la sortie suivante que deux contrôleurs sont disponibles à partir de la même définition :

```
joint_state_broadcaster[joint_state_broadcaster/JointStateBroadcaster] active
joint2_position_controller[passthrough_controller/PassthroughController] active
joint1_position_controller[passthrough_controller/PassthroughController] active
```

21.3.3 Liste des interfaces matérielles

Si l'interface matérielle est bien chargée, en ouvrant un autre terminal et exécuter :

```
ros2 control list_hardware_interfaces
```

Le résultat est donc :

```
command interfaces
      joint1/position [available] [claimed]
      joint1_position_controller/joint1/position [unavailable] [unclaimed]
      joint2/position [available] [claimed]
      joint2_position_controller/joint2/position [unavailable] [unclaimed]
state interfaces
      joint1/position
      joint2/position
```

A ce niveau, les interfaces de référence des contrôleurs sont listées dans les `command_interfaces` quand la commande `ros2 control list_hardware_interfaces` est exécutée.

- Le marqueur `[available]` signifie que les interfaces matériels sont disponibles et prêtes à commander.
- Le marqueur `[claimed]` signifie que le contrôleur a accés à la commande RRBot.
- Le marqueur `[unavailable]` signifie que les interfaces matériels sont indisponibles et ne peuvent être commandés. Par exemple, quand il y a une erreur de lecture ou d'écriture, ces interfaces deviennent inaccessibles.
- Le marqueur `[unclaimed]` signifie que les interfaces de référence de *joint1_position_controller* et *joint2_position_controller* ne sont pas encore en mode chaîné comme les contrôleurs sont actifs.

Note 21.1 Dans le cas de contrôleurs chaînés, les interfaces de commande apparaissent comme `[unavailable]` et `[unclaimed]`, même si les contrôleurs qui exposent leurs interfaces de référence sont actifs, parce que les interfaces de commandes deviennent `[available]` uniquement en mode chaîné, i.e. quand un autre contrôleur utilisent les interfaces de commande.

En mode non chaîné, il est attendu que les contrôleurs chaînés utilisent les références à partir des souscripteurs, par conséquent ils sont marqués comme `unavailable` ■

21.3.4 Lancement de la chaîne complète de contrôleurs

Après avoir ouvert un terminal, et sourcer l'espace de travail ROS-2, il faut exécuter le fichier de launch avec :

```
ros2 launch ros2_control_demo_example_12 launch_chained_controllers.launch.py
```

Ce fichier de launch démarre *position_controller* qui utilise les interfaces de référence à la fois de *joint1_position_controller* et *joint2_position_controller* et les retransmet en une seule. Le contrôleur *forward_position_controller* utilise les interfaces de référence du *position_controller* pour commander les joints du robot RRBot.

Note 21.2 Le second niveau *position_controller* est uniquement présent pour illustrer les concepts. Toutefois un nouveau contrôleur chaînable peut être configuré pour commander directement les interfaces de référence à la fois de *joint1_position_controller* et *joint2_position_controller*. ■

21.3.5 Vérification des contrôleurs

```
ros2 control list_controllers
```

La sortie attendue est :

```
joint_state_broadcaster[joint_state_broadcaster/JointStateBroadcaster] active
joint2_position_controller[passthrough_controller/PassthroughController] active
joint1_position_controller[passthrough_controller/PassthroughController] active
position_controller [passthrough_controller/PassthroughController] active
forward_position_controller[forward_command_controller/ForwardCommandController] active
```

21.3.6 Vérification des interfaces

```
ros2 control list_hardware_interfaces
```

La sortie attendue est :

```
command interfaces
      joint1/position [available] [claimed]
      joint1_position_controller/joint1/position [available] [claimed]
      joint2/position [available] [claimed]
      joint2_position_controller/joint2/position [available] [claimed]
      position_controller/joint1_position_controller/joint1/position [available] [claimed]
      position_controller/joint2_position_controller/joint2/position [available] [claimed]
state interfaces
      joint1/position
      joint2/position
```

A ce stade les interfaces de référence de tous les contrôleurs listés sous `command\_interfaces` doivent être marquées comme `[available]` et `[claimed]`.

21.3.7 Envoi de commandes

```
ros2 topic pub /forward_position_controller/commands std_msgs/msg/Float64MultiArray "data:
- 0.5
- 0.5"
```

Il devrait être possible de voir les états dans le terminal où le fichier de launch a été lancé :

```
[RRBotSystemPositionOnlyHardware]: Got command 0.50000 for joint 0!
[RRBotSystemPositionOnlyHardware]: Got command 0.50000 for joint 1!
```

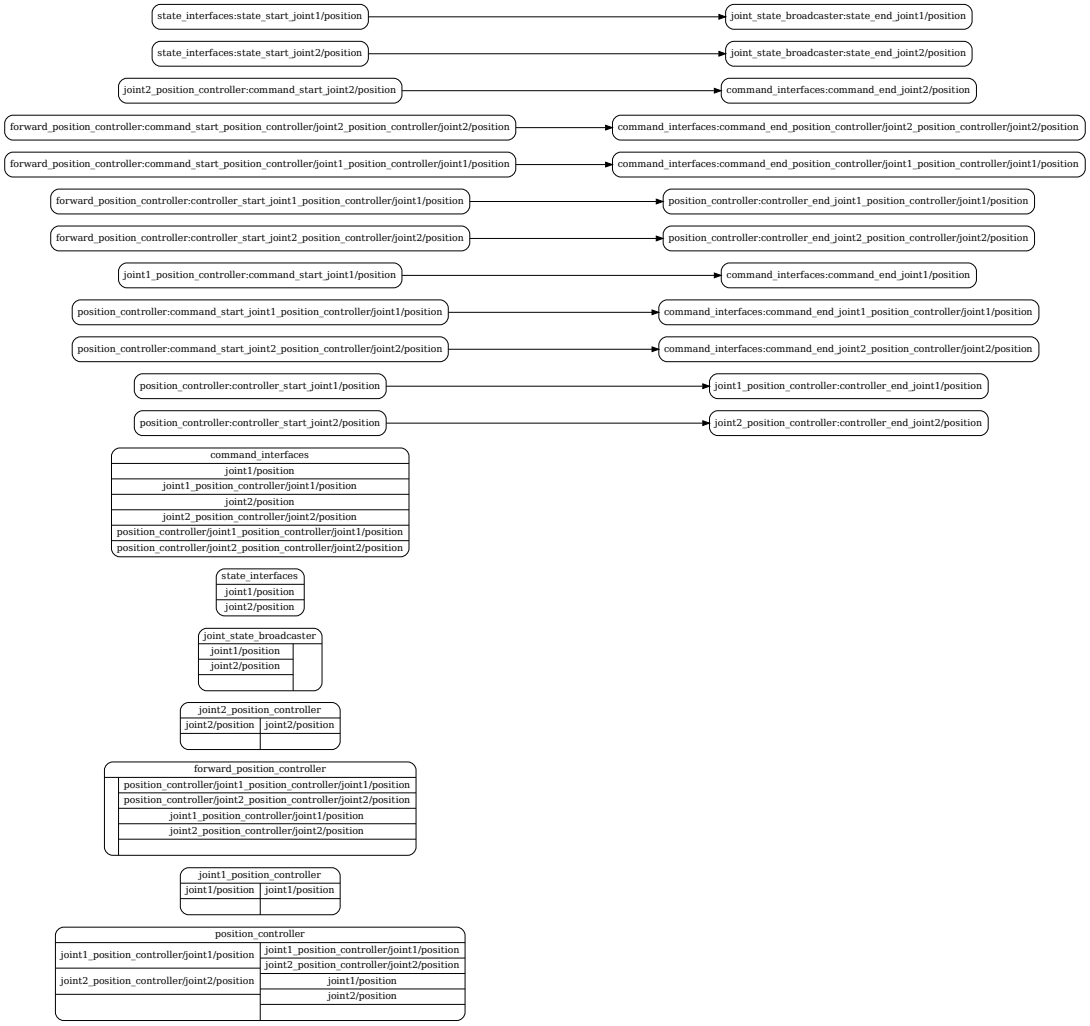


FIGURE 21.2 – Graphe de connection des différentes interfaces en utilisant **ros2 control views_controller_chains**

22. ROS-2 control pour les robots de l'ODRI

L'Open Dynamic Robot Initiative vise à construire des robots légers, simples d'utilisation, et très dynamiques. Le but de ce chapitre est d'étudier le passage d'un code C++ propriétaire à une encapsulation dans ROS-2 control.

La figure Fig.22.1 représente les robots Solo, Bolt, et dual-motors-testbed. C'est ce dernier que nous étudierons pour réaliser le passage d'un robot utilisant un programme propriétaire à une encapsulation dans ROS-2 control. La caractéristique de l'électronique de puissance de la carte μDriver v2 est qu'elle permet de mesurer les courants, et de contrôler celui-ci à 40 KHz. Les gains peuvent être changés à des fréquences de 1 KHz. Ceci permet de mettre en place des structures de contrôle avancées comme des gains de Ricatti.

Il est important de noter que les types d'interface sont limités à la position, velocity, et aux efforts. Il faut donc étendre ces interfaces pour pouvoir prendre en compte les gains du contrôle en courant.

Il est important d'insister sur le fait qu'une fois l'opération réalisée rien n'empêche pas d'utiliser le robot comme il était possible auparavant.

La première étape va consister à implémenter l'objet qui va exposer les commandes des joints et les capteurs. La deuxième étape va consister à créer l'URDF du robot odri_dual_motor_testbed_robot afin de pouvoir charger le plugin correspondant et le fichier de configuration spécifique utilisé pour le robot. La troisième étape sera de mettre en place un contrôleur capable d'utiliser les spécificités du robot et notamment de son électronique de puissance.

22.1 Objet SystemOdriInterface pour la master board

Comme indiqué dans la figure Fig. 20.6, l'accés aux moteurs s'effectue par une carte μDriver v2, une carte master board, une liaison Ethernet ou Wireless pour arriver au PC de contrôle.

Du point de vue logiciel, le protocole de communication Ethernet ou Wifi est implémenté dans la librairie appelée *master_board_sdk* et disponible ici.

Afin de simplifier l'accès au bas niveau et la création de nouvelles structures de contrôle, le consortium ODRI a créer une librairie très simple d'accés qui donne un code particulièrement simple comme afficher dans le listing 22.10.

L'objet robot est créer à partir du fichier de configuration spécifié par la chaîne de caractères CONFIG_SOLO12_YAML à la ligne 18. La position désirée initiale du robot est donnée par un vecteur de réels à la ligne 22, l'étape d'initialisaion étant effectuée à la ligne 27.

La boucle perception action démarre à la ligne 34. La perception des capteurs est réalisée par l'appel à la méthode *ParseSensorData* à la ligne 36, l'appel à *GetPositions* ligne 39 et à *GetVelocities* ligne 40. Les



FIGURE 22.1 – Les 3 robots du consortium ODRI permettant de faire de la commande en direct drive

torques calculés avec le PID sont enregistrés dans la structure du robot à la ligne 47. Et ils sont envoyés à l'électronique de commande à la ligne 52.

```
1  #include <odri_control_interface/calibration.hpp>
2  #include <odri_control_interface/robot.hpp>
3
4  #include <odri_control_interface/utils.hpp>
5
6  using namespace odri_control_interface;
7
8  #include <iostream>
9  #include <stdexcept>
10
11 typedef Eigen::Matrix<double, 12, 1> Vector12d;
12
13 int main()
14 {
15     nice(-20);  // Give the process a high priority.
16
17     // Define the robot from a yaml file.
18     auto robot = RobotFromYamlFile(CONFIG_SOLO12_YAML);
19
20     // Store initial position data.
21     Vector12d des_pos;
22     des_pos << 0.0, 0.7, -1.4, -0.0, 0.7, -1.4, 0.0, -0.7, +1.4, -0.0, -0.7,
23         +1.4;
24
25     // Initialize the communication, session, joints, wait for motors to be ready
26     // and run the joint calibration.
27     robot->Initialize(des_pos);
28
29     // Initialize simple pd controller.
30     Vector12d torques;
31     double kp = 3.;
32     double kd = 0.05;
33     int c = 0;
34     while (!robot->IsTimeout())
35     {
36         robot->ParseSensorData();
37
38         // Run the main controller.
39         auto pos = robot->joints->GetPositions();
```

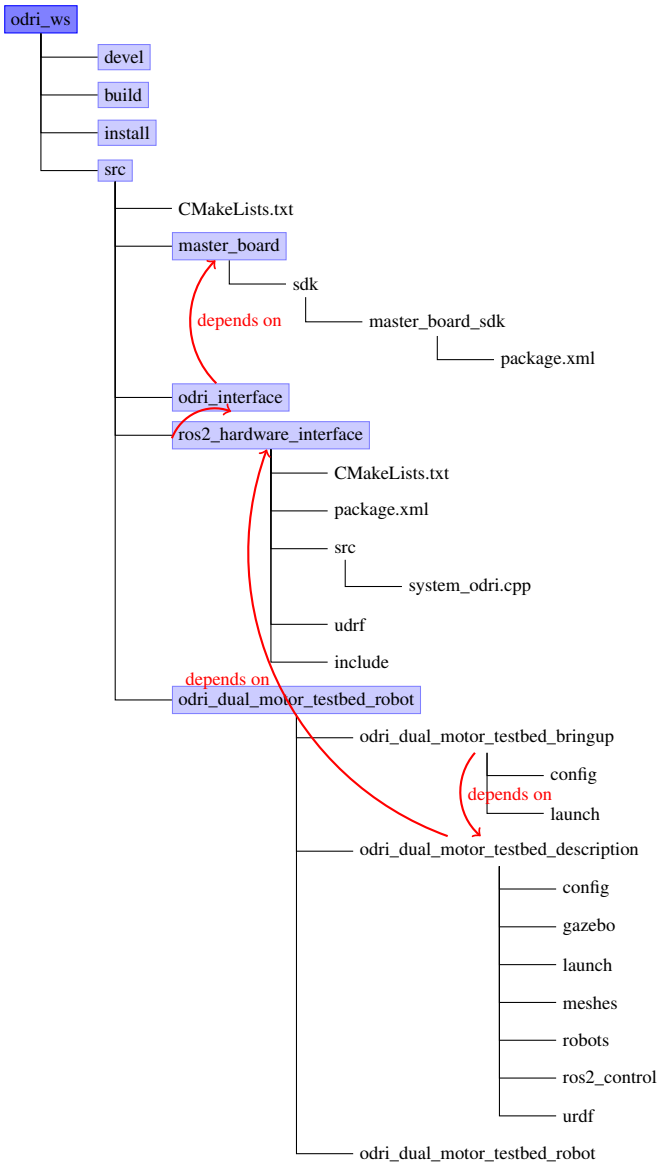


FIGURE 22.2 – Relation entre les différents paquets fournis par l'ODRI et les autres paquets ROS-2.

```
40          auto vel = robot->joints->GetVelocities();
41
42          // Compute PD control on the zero position.
43          for (int i = 0; i < 12; i++)
44          {
45              torques[i] = kp * (des_pos[i] - pos[i]) - kd * vel[i];
46          }
47          robot->joints->SetTorques(torques);
48
49          // Checks if the robot is in error state (that is, if any component
50          // returns an error). If there is an error, the commands to send
51          // are changed to send the safety control.
52          robot->SendCommandAndWaitEndOfCycle(0.001);
53
54          c++;
55          if (c % 1000 == 0)
56          {
57              std::cout << "Joints: ";
58              robot->joints->PrintVector(robot->joints->GetPositions());
59              std::cout << std::endl;
60          }
61      }
62
63      printf("Timeout detected\n");
64
65      return 0;
66  }
```

Listing 22.1 – Code de demo_solo12.cpp : une boucle simple perception action

22.1.1 En-tête de l'interface matérielle

La première étape permettant l'encapsulation de l'interface de commande est de définir des structures de données permettant de gérer les capteurs, et les deux types interfaces gain_kp et gain_kd aux lignes 30 et 31. On y associe les différents modes de contrôle possible avec le matériel 39-45, et les types d'interfaces 33-37,

```
1  namespace ros2_control_odri {
2
3  struct PosVelEffortGains {
4    double position;
5    double velocity;
6    double effort;
7    double Kp;
8    double Kd;
9  };
10
11 struct GyroAccLineEulerQuater {
12   double gyro_x;
13   double gyro_y;
14   double gyro_z;
15   double accelero_x;
16   double accelero_y;
17   double accelero_z;
18   double line_acc_x;
19   double line_acc_y;
20   double line_acc_z;
21   double euler_x;
22   double euler_y;
23   double euler_z;
24   double quater_x;
25   double quater_y;
26   double quater_z;
27   double quater_w;
28 };
29
30 constexpr const auto HW_IF_GAIN_KP = "gain_kp";
31 constexpr const auto HW_IF_GAIN_KD = "gain_kd";
32
33 std::set<std::string> odri_list_of_cmd_inter{"position", "velocity", "effort",
34                                              "gain_kp", "gain_kd"};
35
36 std::set<std::string> odri_list_of_state_inter{"position", "velocity", "effort",
37                                                "gain_kp", "gain_kd"};
38
39 enum control_mode_t {
40   POSITION,
41   VELOCITY,
42   EFFORT,
43   POS_VEL_EFF_GAINS,
```

```
  NO_VALID_MODE
};

}  // namespace ros2_control_odri
```

Listing 22.2 – Structures de SystemOdriHardware

22.1.2 Code de l'objet SystemInterface

Les entêtes nécessaires sont spécifiés au départ du programme. On y retrouve à la fois les entêtes des interfaces matérielles pour ros2-control, les en-têtes des librairies l'accés bas niveau matériel qui sera caché par l'encapsulation.

```
#include <chrono>
#include <cmath>
#include <fstream>
#include <limits>
#include <memory>
#include <string>
#include <system_interface_odri.hpp>
#include <vector>

#include "rcutils/logging_macros.h"

#include "hardware_interface/types/hardware_interface_type_values.hpp"
#include "rclcpp/rclcpp.hpp"

/*Connection to ODRI for read sensors and write commands*/
#include "odri_control_interface/imu.hpp"
#include "odri_control_interface/utils.hpp"

using namespace odri_control_interface;
using namespace Eigen;
using namespace semantic_components;

#include <iostream>
#include <stdexcept>
```

Listing 22.3 – En-têtes pour ros2_hardware_interface_odri

La méthode *read_default_cmd_state_value* lit la valeur d'état ou de commande par défaut du robot à partir d'une chaîne de caractères transmises par les paramètres. L'état comprend les interfaces de position, vitesse, effort et les gains. Elle sont extraites d'un flux de chaînes de caractères et mise dans la structure adéquate grâce à la lambda function définie entre les lignes 45-57.

```
namespace ros2_control_odri {

/* Code issue from demo_odri_actuator_control.cpp (ODRI)*/


hardware_interface::CallbackReturn
SystemOdriHardware::read_default_cmd_state_value(
    std::string &default_joint_cs) {
  // Hardware parameters provides a string
  if (info_.hardware_parameters.find(default_joint_cs) ==
      info_.hardware_parameters.end()) {
    RCLCPP_FATAL(
        rclcpp::get_logger("SystemOdriHardware"),
        "%s not in the parameter list of ros2_control_odri/SystemOdriHardware!",
        default_joint_cs.c_str());
    return CallbackReturn::ERROR;
  }
  std::string str_des_start_pos = info_.hardware_parameters[default_joint_cs];

  typedef std::map<std::string, PosVelEffortGains> map_pveg;
  map_pveg hw_cs;
  if (default_joint_cs == "default_joint_cmd") {
    hw_cs = hw_commands_;
  } else if (default_joint_cs == "default_joint_state") {
    hw_cs = hw_states_;
  } else
    return CallbackReturn::ERROR;

  // Read the parameter through a stream of strings.
  std::istringstream iss_def_cmd_val;
  iss_def_cmd_val.str(str_des_start_pos);

  while (!iss_def_cmd_val.eof()) {
```

```
34      // Find joint name.
35      std::string joint_name;
36      RCLCPP_INFO_STREAM(
37          rclcpp::get_logger("SystemOdriHardware"),
38          " Current value of iss_def_cmd_val:" << iss_def_cmd_val.str().c_str());
39      iss_def_cmd_val >> joint_name;
40
41      // Find the associate joint
42      bool found_joint = false;
43      for (const hardware_interface::ComponentInfo &joint : info_.joints) {
44        if (joint.name == joint_name) {
45          auto handle_dbl_and_msg = [](std::istringstream &iss_def_cmd_val,
46                                       std::string &joint_name, double &adbl,
47                                       std::string &msg) {
48            if (!iss_def_cmd_val.eof())
49              iss_def_cmd_val >> adbl;
50            else {
51              RCLCPP_FATAL(rclcpp::get_logger("SystemOdriHardware"),
52                           "default_joint_cmd '%s' no '%s'.", joint_name.c_str(),
53                           msg.c_str());
54              return CallbackReturn::ERROR;
55            }
56            return CallbackReturn::SUCCESS;
57          };
58
59          std::string amsg("position");
60          if (handle_dbl_and_msg(iss_def_cmd_val, joint_name,
61                                 hw_cs.at(joint_name).position,
62                                 amsg) == CallbackReturn::ERROR)
63            return CallbackReturn::ERROR;
64
65          amsg = "velocity";
66          if (handle_dbl_and_msg(iss_def_cmd_val, joint_name,
67                                 hw_cs.at(joint_name).velocity,
68                                 amsg) == CallbackReturn::ERROR)
69            return CallbackReturn::ERROR;
70
71          amsg = "effort";
72          if (handle_dbl_and_msg(iss_def_cmd_val, joint_name,
73                                 hw_cs.at(joint_name).effort,
74                                 amsg) == CallbackReturn::ERROR)
75            return CallbackReturn::ERROR;
76
77          amsg = "Kp";
78          if (handle_dbl_and_msg(iss_def_cmd_val, joint_name,
79                                 hw_cs.at(joint_name).Kp,
80                                 amsg) == CallbackReturn::ERROR)
81            return CallbackReturn::ERROR;
82
83          amsg = "Kd";
84          if (handle_dbl_and_msg(iss_def_cmd_val, joint_name,
85                                 hw_cs.at(joint_name).Kd,
86                                 amsg) == CallbackReturn::ERROR)
87            return CallbackReturn::ERROR;
88
89          found_joint = true;
90          break;  // Found the joint break the loop
91        }
92      }
93
94      if (!found_joint) {
95        RCLCPP_FATAL(rclcpp::get_logger("SystemOdriHardware"),
96                     "Joint '%s' not found in '%s'.", joint_name.c_str(),
97                     default_joint_cs.c_str());
98        return CallbackReturn::ERROR;
99      }
100   }
101   if (default_joint_cs == "default_joint_cmd")
102     hw_commands_ = hw_cs;
103   else if (default_joint_cs == "default_joint_state")
104     hw_states_ = hw_cs;
105
106   return CallbackReturn::SUCCESS;
107 }
```

Listing 22.4 – Lire les états et les commandes par défaut pour SystemOdriHardware

La méthode permettant de spécifier la position de départ comme pour la ligne 22 dans le fichier initial est fournie par cette méthode.

```
1 /// Reading desired position
2 hardware_interface::CallbackReturn
3 SystemOdriHardware::read_desired_starting_position()
```

```
{
  std::vector<double> vec_des_start_pos;

  if (info_.hardware_parameters.find("desired_starting_position") ==
      info_.hardware_parameters.end()) {
    RCLCPP_FATAL(rclcpp::get_logger("SystemOdriHardware"),
                 "desired_starting_positon not in the parameter list of "
                 "ros2_control_odri/SystemOdriHardware!");
    return CallbackReturn::ERROR;
  }

  // Hardware parameters provides a string
  std::string str_des_start_pos =
      info_.hardware_parameters["desired_starting_position"];

  // Read the parameter through a stream of strings.
  std::istringstream iss_des_start_pos;
  iss_des_start_pos.str(str_des_start_pos);

  RCLCPP_INFO_STREAM(
      rclcpp::get_logger("SystemOdriHardware"),
      " Current desired_start_position size:" << eig_des_start_pos_.size());

  // From istringstream to std::vector
  while (!iss_des_start_pos.eof()) {
    double apos;
    iss_des_start_pos >> apos;
    vec_des_start_pos.push_back(apos);
  }
  eig_des_start_pos_.resize((Eigen::Index)vec_des_start_pos.size());

  RCLCPP_INFO_STREAM(
      rclcpp::get_logger("SystemOdriHardware"),
      " Current desired_start_position size:" << eig_des_start_pos_.size());

  // From std::vector to VectorXd
  int idx_dsp = 0;
  for (auto apos : vec_des_start_pos) eig_des_start_pos_[idx_dsp++] = apos;

  return CallbackReturn::SUCCESS;
}
```

Listing 22.5 – Méthode read_desired_starting_position

La méthode **configure** décrite dans le Listing 22.6 prépare les containeurs des interfaces correspondantes aux capteurs, états et commandes de l'objet en les mettant à la bonne taille.

La méthode vérifie que tous les joints ont une définition dans l'interface ros2_control, et que chacun à 5 interfaces de commandes et d'états.

Elle met des valeurs par défaut qui montrent que le système n'a pas encore lu les paramètres. Ce sera fait dans la méthode **activate**.

Les différents containers sont mis à la bonne taille car dans

```
hardware_interface::CallbackReturn SystemOdriHardware::on_configure(
    const rclcpp_lifecycle::State &) {

  motor_numbers_.resize(info_.joints.size());
  motor_reversed_polarities_.resize(info_.joints.size());
  joint_lower_limits_.resize(info_.joints.size());
  joint_upper_limits_.resize(info_.joints.size());
  position_offsets_.resize(info_.joints.size());

  positions_.resize(info_.joints.size());
  velocities_.resize(info_.joints.size());
  torques_.resize(info_.joints.size());
  gain_KP_.resize(info_.joints.size());
  gain_KD_.resize(info_.joints.size());

  // For each sensor.
  for (const hardware_interface::ComponentInfo &sensor : info_.sensors) {
    imu_states_[sensor.name] = {std::numeric_limits<double>::quiet_NaN(),
                                std::numeric_limits<double>::quiet_NaN(),
                                std::numeric_limits<double>::quiet_NaN(),
                                std::numeric_limits<double>::quiet_NaN(),
                                std::numeric_limits<double>::quiet_NaN(),
                                std::numeric_limits<double>::quiet_NaN(),
                                std::numeric_limits<double>::quiet_NaN(),
                                std::numeric_limits<double>::quiet_NaN(),
                                std::numeric_limits<double>::quiet_NaN(),
                                std::numeric_limits<double>::quiet_NaN(),
                                std::numeric_limits<double>::quiet_NaN(),
```

```
                                std::numeric_limits<double>::quiet_NaN(),
                                std::numeric_limits<double>::quiet_NaN(),
                                std::numeric_limits<double>::quiet_NaN(),
                                std::numeric_limits<double>::quiet_NaN(),
                                std::numeric_limits<double>::quiet_NaN()};
  }
  // For each joint.
  for (const hardware_interface::ComponentInfo &joint : info_.joints) {
    // Initialize state of the joint by default to NaN
    // it allows to see which joints are not properly initialized
    // from the real hardware
    hw_states_[joint.name] = {std::numeric_limits<double>::quiet_NaN(),
                              std::numeric_limits<double>::quiet_NaN(),
                              std::numeric_limits<double>::quiet_NaN(),
                              std::numeric_limits<double>::quiet_NaN(),
                              std::numeric_limits<double>::quiet_NaN()};
    hw_commands_[joint.name] = {std::numeric_limits<double>::quiet_NaN(),
                                std::numeric_limits<double>::quiet_NaN(),
                                std::numeric_limits<double>::quiet_NaN(),
                                std::numeric_limits<double>::quiet_NaN(),
                                std::numeric_limits<double>::quiet_NaN()};
    control_mode_[joint.name] = control_mode_t::NO_VALID_MODE;

    // SystemOdri has exactly 5 doubles for the state and
    // 5 doubles for the command interface on each joint
    if (joint.command_interfaces.size() != odri_list_of_cmd_inter.size()) {
      RCLCPP_FATAL(
          rclcpp::get_logger("SystemOdriHardware"),
          "Joint '%s' has %lu command interfaces found.",  // 5 expected.",
          joint.name.c_str(), joint.command_interfaces.size());
      return CallbackReturn::ERROR;
    }

    // For each command interface of the joint
    for (const auto &a_joint_cmd_inter : joint.command_interfaces) {
      // Check if the command interface is inside the list
      if (odri_list_of_cmd_inter.find(a_joint_cmd_inter.name) ==
          odri_list_of_cmd_inter.end()) {
        // If not then generate an error message
        RCLCPP_FATAL(rclcpp::get_logger("SystemOdriHardware"),
                     "Joint '%s' have %s command interfaces found. One of the "
                     "following values is expected",
                     joint.name.c_str(), a_joint_cmd_inter.name.c_str());
        for (const auto &a_cmd_inter : odri_list_of_cmd_inter) {
          RCLCPP_FATAL(rclcpp::get_logger("SystemOdriHardware"),
                       "'%s' expected.", a_cmd_inter.c_str());
        }
        return CallbackReturn::ERROR;
      }
    }

    // Check if the state interface list is of the right size
    if (joint.state_interfaces.size() != odri_list_of_state_inter.size()) {
      RCLCPP_FATAL(rclcpp::get_logger("SystemOdriHardware"),
                   "Joint '%s' has %lu state interface.",  // 5 expected.",
                   joint.name.c_str(), joint.state_interfaces.size());
      return CallbackReturn::ERROR;
    }

    // For each state interface of the joint
    for (const auto &a_joint_state_inter : joint.state_interfaces) {
      std::string joint_state_inter_name = a_joint_state_inter.name;

      // Check if the state interface is inside the list
      if (odri_list_of_state_inter.find(joint_state_inter_name) ==
          odri_list_of_state_inter.end()) {
        RCLCPP_FATAL(rclcpp::get_logger("SystemOdriHardware"),
                     "Joint '%s' have %s state interface. One of the following "
                     "was expected: ",
                     joint.name.c_str(), a_joint_state_inter.name.c_str());

        for (const auto &a_state_inter : odri_list_of_state_inter) {
          RCLCPP_FATAL(rclcpp::get_logger("SystemOdriHardware"),
                       "'%s' expected.", a_state_inter.c_str());
        }
        return CallbackReturn::ERROR;
      }
    }
  }

  return CallbackReturn::SUCCESS;
}
```

Listing 22.6 – Méthode de configuration de SystemOdriHardware

La méthode la plus importante de cette partie est la méthode *prepare_commande_mode_switch*. Elle permet de changer le mode de contrôle *bas niveau* de la carte μDrive v2 en passant le joint j à l'un des modes suivant :

— position $i = K_p(q_j^* - \hat{q}_j)$
— vitesse $i = K_d(\dot{q}_j^* - \hat{\dot{q}}_j)$,
— effort $i = K_i\tau_j$,
— position-vitesse-effort-gains.

$$i = K_j^i\tau_j^* + K_p(q_j^* - \hat{q}_j) + K_d(\dot{q}_j^* - \hat{\dot{q}}_j)$$

```
void SystemOdriHardware::display_robot_state() {
  for (const hardware_interface::ComponentInfo &joint : info_.joints) {
    std::cout << "joint " << joint.name << " "
              << hw_commands_[joint.name].position << " "
              << hw_commands_[joint.name].velocity << " "
              << hw_commands_[joint.name].effort << " "
              << hw_commands_[joint.name].Kp << " "
              << hw_commands_[joint.name].Kd << std::endl;
  }
  std::cout << " *************************" << std::endl;
}

return_type SystemOdriHardware::prepare_command_mode_switch(
    const std::vector<std::string> &start_interfaces,
    const std::vector<std::string> &stop_interfaces) {
  // Initialize new modes.
  for (const hardware_interface::ComponentInfo &joint : info_.joints) {
    new_modes_[joint.name] = control_mode_t::NO_VALID_MODE;
  }

  /// Check that the key interfaces are coherent
  for (auto &key : start_interfaces) {
    /// For each joint
    for (const hardware_interface::ComponentInfo &joint : info_.joints) {
      if (key == joint.name + "/" + hardware_interface::HW_IF_POSITION) {
        new_modes_[joint.name] = control_mode_t::POSITION;
      }

      if (key == joint.name + "/" + hardware_interface::HW_IF_VELOCITY) {
        new_modes_[joint.name] = control_mode_t::VELOCITY;
      }

      if (key == joint.name + "/" + hardware_interface::HW_IF_EFFORT) {
        new_modes_[joint.name] = control_mode_t::EFFORT;
      }

      if (key == joint.name + "/" + ros2_control_odri::HW_IF_GAIN_KP) {
        new_modes_[joint.name] = control_mode_t::POS_VEL_EFF_GAINS;
      }
      if (key == joint.name + "/" + ros2_control_odri::HW_IF_GAIN_KD) {
        new_modes_[joint.name] = control_mode_t::POS_VEL_EFF_GAINS;
      }
    }
  }
  // Stop motion on all relevant joints that are stopping
  for (std::string key : stop_interfaces) {
    for (const hardware_interface::ComponentInfo &joint : info_.joints) {
      if (key.find(joint.name) != std::string::npos) {
        hw_commands_[joint.name].velocity = 0.0;
        hw_commands_[joint.name].effort = 0.0;
        control_mode_[joint.name] =
            control_mode_t::NO_VALID_MODE;  // Revert to undefined
      }
    }
  }
  // Set the new command modes
  for (const hardware_interface::ComponentInfo &joint : info_.joints) {
    if ((control_mode_[joint.name] == control_mode_t::NO_VALID_MODE) &&
        (new_modes_[joint.name] == control_mode_t::NO_VALID_MODE)) {
      // Something else is using the joint! Abort!
      RCLCPP_ERROR(rclcpp::get_logger("SystemOdriHardware"),
                   "Joint '%s' has no valid control mode %d %d",
                   joint.name.c_str(), control_mode_[joint.name],
                   new_modes_[joint.name]);
```

```
      return return_type::ERROR;
    }
    control_mode_[joint.name] = new_modes_[joint.name];
  }

  std::cout << "in prepare_command_mode_switch" << std::endl;
  display_robot_state();
  return return_type::OK;
}
```

Listing 22.7 – Prepare to switch command

Une étape obligatoire pour les éléments matériels est la publication des interfaces d'état et de commande. Dans notre cas de figure chacun des joints est exposé avec des interfaces de position, vitesse, effort et gains. Il faut également publié les capteurs. La partie la plus génante ici est le fait qu'une interface est pour l'instant limité à un double. Le fait de ne pas vouloir d'utiliser de middleware à ce stade, fait qu'utiliser de façon flexbile un autre type nécessiterait l'utilisation et la création de templates. Ceci a été évité afin de simplifier l'export des interfaces, mais pourrait évoluer dans le futur.

```
std::vector<hardware_interface::StateInterface>
SystemOdriHardware::export_state_interfaces() {
  std::vector<hardware_interface::StateInterface> state_interfaces;
  for (const hardware_interface::ComponentInfo &joint : info_.joints) {
    state_interfaces.emplace_back(hardware_interface::StateInterface(
        joint.name, hardware_interface::HW_IF_POSITION,
        &hw_states_[joint.name].position));
    state_interfaces.emplace_back(hardware_interface::StateInterface(
        joint.name, hardware_interface::HW_IF_VELOCITY,
        &hw_states_[joint.name].velocity));
    state_interfaces.emplace_back(hardware_interface::StateInterface(
        joint.name, hardware_interface::HW_IF_EFFORT,
        &hw_states_[joint.name].effort));
    state_interfaces.emplace_back(hardware_interface::StateInterface(
        joint.name, HW_IF_GAIN_KP, &hw_states_[joint.name].Kp));
    state_interfaces.emplace_back(hardware_interface::StateInterface(
        joint.name, HW_IF_GAIN_KD, &hw_states_[joint.name].Kd));
  }

  for (const hardware_interface::ComponentInfo &sensor : info_.sensors) {
    state_interfaces.emplace_back(hardware_interface::StateInterface(
        sensor.name, "gyroscope_x", &imu_states_[sensor.name].gyro_x));
    state_interfaces.emplace_back(hardware_interface::StateInterface(
        sensor.name, "gyroscope_y", &imu_states_[sensor.name].gyro_y));
    state_interfaces.emplace_back(hardware_interface::StateInterface(
        sensor.name, "gyroscope_z", &imu_states_[sensor.name].gyro_z));
    state_interfaces.emplace_back(hardware_interface::StateInterface(
        sensor.name, "accelerometer_x", &imu_states_[sensor.name].accelero_x));
    state_interfaces.emplace_back(hardware_interface::StateInterface(
        sensor.name, "accelerometer_y", &imu_states_[sensor.name].accelero_y));
    state_interfaces.emplace_back(hardware_interface::StateInterface(
        sensor.name, "accelerometer_z", &imu_states_[sensor.name].accelero_z));
    state_interfaces.emplace_back(hardware_interface::StateInterface(
        sensor.name, "linear_acceleration_x",
        &imu_states_[sensor.name].line_acc_x));
    state_interfaces.emplace_back(hardware_interface::StateInterface(
        sensor.name, "linear_acceleration_y",
        &imu_states_[sensor.name].line_acc_y));
    state_interfaces.emplace_back(hardware_interface::StateInterface(
        sensor.name, "linear_acceleration_z",
        &imu_states_[sensor.name].line_acc_z));
    state_interfaces.emplace_back(hardware_interface::StateInterface(
        sensor.name, "attitude_euler_x", &imu_states_[sensor.name].euler_x));
    state_interfaces.emplace_back(hardware_interface::StateInterface(
        sensor.name, "attitude_euler_y", &imu_states_[sensor.name].euler_y));
    state_interfaces.emplace_back(hardware_interface::StateInterface(
        sensor.name, "attitude_euler_z", &imu_states_[sensor.name].euler_z));
    state_interfaces.emplace_back(
        hardware_interface::StateInterface(sensor.name, "attitude_quaternion_x",
                                           &imu_states_[sensor.name].quater_x));
    state_interfaces.emplace_back(
        hardware_interface::StateInterface(sensor.name, "attitude_quaternion_y",
                                           &imu_states_[sensor.name].quater_y));
    state_interfaces.emplace_back(
        hardware_interface::StateInterface(sensor.name, "attitude_quaternion_z",
                                           &imu_states_[sensor.name].quater_z));
    state_interfaces.emplace_back(
        hardware_interface::StateInterface(sensor.name, "attitude_quaternion_w",
                                           &imu_states_[sensor.name].quater_w));
  }

  return state_interfaces;
```

```
}

std::vector<hardware_interface::CommandInterface>
SystemOdriHardware::export_command_interfaces() {
  std::vector<hardware_interface::CommandInterface> command_interfaces;

  for (const hardware_interface::ComponentInfo &joint : info_.joints) {
    command_interfaces.emplace_back(hardware_interface::CommandInterface(
        joint.name, hardware_interface::HW_IF_POSITION,
        &hw_commands_[joint.name].position));
    command_interfaces.emplace_back(hardware_interface::CommandInterface(
        joint.name, hardware_interface::HW_IF_VELOCITY,
        &hw_commands_[joint.name].velocity));
    command_interfaces.emplace_back(hardware_interface::CommandInterface(
        joint.name, hardware_interface::HW_IF_EFFORT,
        &hw_commands_[joint.name].effort));
    command_interfaces.emplace_back(hardware_interface::CommandInterface(
        joint.name, HW_IF_GAIN_KP, &hw_commands_[joint.name].Kp));
    command_interfaces.emplace_back(hardware_interface::CommandInterface(
        joint.name, HW_IF_GAIN_KD, &hw_commands_[joint.name].Kd));
  }

  return command_interfaces;
}
```

Listing 22.8 – Export state and command interfaces

Dans la phase d'activation, les moteurs sont allumés, les capteurs lus, et le système est mis dans sa pose initiale.

```
hardware_interface::CallbackReturn SystemOdriHardware::on_activate
( const rclcpp_lifecycle::State &)
{
  //// Read Parameters ////
  RCUTILS_LOG_INFO_NAMED(
        "SystemOdriHardware",
        "will be reading file %s",info_.hardware_parameters["odri_config_yaml"].c_str());
  /// Read odri_config_yaml
  // Initialize Robot
  robot_ = RobotFromYamlFile(info_.hardware_parameters["odri_config_yaml"]);

  RCUTILS_LOG_INFO_NAMED(
        "SystemOdriHardware",
        "Number of joints %d",robot_->joints->GetNumberMotors());

  /// Read hardware parameter "desired_starting_position"
  if (read_desired_starting_position() == CallbackReturn::ERROR)
    return CallbackReturn::ERROR;

  /// Read hardware parameter "default_joint_cmd"
  std::string default_joint_cs("default_joint_cmd");
  if (read_default_cmd_state_value(default_joint_cs) == CallbackReturn::ERROR)
    return CallbackReturn::ERROR;

  /// Read hardware parameter "default_joint_state"
  default_joint_cs = "default_joint_state";
  if (read_default_cmd_state_value(default_joint_cs) == CallbackReturn::ERROR)
    return CallbackReturn::ERROR;

  /// Initialize the robot to the desired starting position.
  robot_->Initialize(eig_des_start_pos_);

  /// Build the map from name to index.
  for (const hardware_interface::ComponentInfo &joint : info_.joints) {
    // First set the key
    joint_name_to_array_index_[joint.name] = 0;
  }

  // Then build the index.
  uint idx = 0;
  for (auto it = joint_name_to_array_index_.begin();
       it != joint_name_to_array_index_.end(); ++it) {
    joint_name_to_array_index_[it->first] = (int)(idx++);
  }

  // Update the default values.
  for (const hardware_interface::ComponentInfo &joint : info_.joints) {
    positions_[joint_name_to_array_index_[joint.name]] =
      hw_commands_[joint.name].position;
    velocities_[joint_name_to_array_index_[joint.name]] =
      hw_commands_[joint.name].velocity;
    torques_[joint_name_to_array_index_[joint.name]] =
      hw_commands_[joint.name].effort;
```

```
    gain_KP_[joint_name_to_array_index_[joint.name]] =
      hw_commands_[joint.name].Kp;
    gain_KD_[joint_name_to_array_index_[joint.name]] =
      hw_commands_[joint.name].Kd;
  }

  return CallbackReturn::SUCCESS;
}
hardware_interface::CallbackReturn SystemOdriHardware::on_deactivate
( const rclcpp_lifecycle::State &)
{
  // Stop the MasterBoard
  main_board_ptr_->MasterBoardInterface::Stop();

  return CallbackReturn::SUCCESS;
}
```

Listing 22.9 – Démarrage de l'objet

Les deux méthodes read() and write() font le pont entre les objets manipulés dans ros2_control et la librairie spécifique au bas niveau du robot.

```
hardware_interface::return_type SystemOdriHardware::read
(const rclcpp::Time & /* time */, const rclcpp::Duration & /* period */)
{
  // Data acquisition (with ODRI)
  robot_->ParseSensorData();

  auto sensor_positions = robot_->joints->GetPositions();
  auto sensor_velocities = robot_->joints->GetVelocities();
  auto measured_torques = robot_->joints->GetMeasuredTorques();

  auto imu_gyroscope = robot_->imu->GetGyroscope();
  auto imu_accelero = robot_->imu->GetAccelerometer();
  auto imu_linear_acc = robot_->imu->GetLinearAcceleration();
  auto imu_euler = robot_->imu->GetAttitudeEuler();
  auto imu_quater = robot_->imu->GetAttitudeQuaternion();

  // Assignment of sensor data to ros2_control variables (URDF)
  for (const hardware_interface::ComponentInfo &joint : info_.joints) {
    hw_states_[joint.name].position =
        sensor_positions[joint_name_to_array_index_[joint.name]];
    hw_states_[joint.name].velocity =
        sensor_velocities[joint_name_to_array_index_[joint.name]];
    hw_states_[joint.name].effort =
        measured_torques[joint_name_to_array_index_[joint.name]];
    hw_states_[joint.name].Kp = hw_commands_[joint.name].Kp;
    hw_states_[joint.name].Kd = hw_commands_[joint.name].Kd;
  }

  // Assignment of IMU data (URDF)
  // Modif with for loop possible to optimize the code
  imu_states_["IMU"].gyro_x = imu_gyroscope[0];
  imu_states_["IMU"].gyro_z = imu_gyroscope[1];
  imu_states_["IMU"].gyro_y = imu_gyroscope[2];

  imu_states_["IMU"].accelero_x = imu_accelero[0];
  imu_states_["IMU"].accelero_y = imu_accelero[1];
  imu_states_["IMU"].accelero_z = imu_accelero[2];

  imu_states_["IMU"].line_acc_x = imu_linear_acc[0];
  imu_states_["IMU"].line_acc_y = imu_linear_acc[1];
  imu_states_["IMU"].line_acc_z = imu_linear_acc[2];

  imu_states_["IMU"].euler_x = imu_euler[0];
  imu_states_["IMU"].euler_y = imu_euler[1];
  imu_states_["IMU"].euler_z = imu_euler[2];

  imu_states_["IMU"].quater_x = imu_quater[0];
  imu_states_["IMU"].quater_y = imu_quater[1];
  imu_states_["IMU"].quater_z = imu_quater[2];
  imu_states_["IMU"].quater_w = imu_quater[3];

  return return_type::OK;
}

hardware_interface::return_type SystemOdriHardware::write
(const rclcpp::Time & /* time */, const rclcpp::Duration & /* period */)
{

  for (const hardware_interface::ComponentInfo &joint : info_.joints) {
    if ((control_mode_[joint.name] == control_mode_t::POS_VEL_EFF_GAINS)
        (control_mode_[joint.name] == control_mode_t::POSITION)) {
```

```
62        positions_[joint_name_to_array_index_[joint.name]] =
63            hw_commands_[joint.name].position;
64        velocities_[joint_name_to_array_index_[joint.name]] =
65            hw_commands_[joint.name].velocity;
66        torques_[joint_name_to_array_index_[joint.name]] =
67            hw_commands_[joint.name].effort;
68        gain_KP_[joint_name_to_array_index_[joint.name]] =
69            hw_commands_[joint.name].Kp;
70        gain_KD_[joint_name_to_array_index_[joint.name]] =
71            hw_commands_[joint.name].Kd;
72      }
73    }
74
75    static unsigned int my_perso_counter2 = 0;
76    if (my_perso_counter2 % 1000 == 0) {
77      std::cout << "positions:" << positions_.transpose() << std::endl;
78      std::cout << "velocities:" << velocities_.transpose() << std::endl;
79      std::cout << "torques: " << torques_.transpose() << std::endl;
80      std::cout << "gain_KP: " << gain_KP_.transpose() << std::endl;
81      std::cout << "gain_KD: " << gain_KD_.transpose() << std::endl;
82      std::cout << " " << std::endl;
83    }
84    ++my_perso_counter2;
85
86    robot_->joints->SetDesiredPositions(positions_);
87    robot_->joints->SetDesiredVelocities(velocities_);
88    robot_->joints->SetTorques(torques_);
89    robot_->joints->SetPositionGains(gain_KP_);
90    robot_->joints->SetVelocityGains(gain_KD_);
91
92    robot_->SendCommandAndWaitEndOfCycle(0.00);
93
94    return return_type::OK;
95  }
96
97  }  // namespace ros2_control_odri
98
99  #include "pluginlib/class_list_macros.hpp"
100
101 PLUGINLIB_EXPORT_CLASS(ros2_control_odri::SystemOdriHardware,
102                        hardware_interface::SystemInterface)
```

Listing 22.10 – Lecture des capteurs dans l'objet SystemOdriHardware

22.2 Création du fichier URDF

22.2.1 Source de la description géométrique du robot

Le robot est défini géométriquement dans le répertoire `open_robot_actuator_hardware/mechanics/dual_motor_testbed_v1`, il est disponible ici. Dans le sous-répertoire `stl_files`, on trouve 3 fichiers décrivant la géométrie du robot : `dual_motor_testbed_base.STL`, `dual_motor_testbed_cover.STL`, `hand\_wheel.STL`.

22.2.2 Fichier d'entrée

La partie spécifique à ros2_control est inclue par le fichier `system_dual_motor_testbd.ros2_control.xacro`. Ce fichier se met généralement dans le répertoire `ros2_control` du paquet de description du robot. Le fichier est spécifié à la ligne 11. Cette macro est ensuite instanciée à la ligne 13. On peut voir notamment que le nom transmis à cette macro est **odri_dm_tb_kt**. Ceci aura une importance dans la création du nom des joints des roues.

```
1  <?xml version="1.0"?>
2  <robot  xmlns:xacro="http://ros.org/wiki/xacro"
3          name="odri_dual_motor_testbed" >
4
5    <xacro:arg name="use_sim" default="true"/>
6
7    <!-- Generic simulator_gazebo plugins -->
8    <xacro:include filename="$(find odri_dual_motor_testbed_description)/gazebo/gazebo.urdf.xacro" />
9
10   <!-- ros2_control -->
11   <xacro:include filename="$(find odri_dual_motor_testbed_description)/ros2_control/system_dual_motor_testbed.
       ros2_control.xacro" />
12   <xacro:odri_dual_motor_testbed_ros2_control
13     use_sim="$(arg use_sim)" name="odri_dm_tb_kt"/>
14
```

```
  <xacro:include filename="$(find odri_dual_motor_testbed_description)/urdf/base.urdf.xacro" />
  <xacro:odri_dual_motor_testbed_kin_tree
    name="odri_dm_tb_kt" parent="world" />
</robot>
```

Listing 22.11 – Fichier racine de l'URDF

22.2.3 Description de l'arbre cinématique

Ce fichier décrivant l'arbre cinématique démarre par la base, puis ajoute deux joints pour chacune des roues qui sont inclues via une macro.

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://ros.org/wiki/xacro">

  <xacro:include filename="$(find odri_dual_motor_testbed_description)/urdf/wheel.urdf.xacro" />

  <xacro:macro name="odri_dual_motor_testbed_kin_tree" params="name parent ">

    <!--************************--> 
    <!--   BASE    -->
    <!--************************-->
    <link name="base_footprint">
      <inertial>
        <origin xyz="0.0 0.0 0.0" rpy="0.00000 0.00000 0.00000"/>
        <mass value="0.250"/>
        <inertia ixx="0.002471" ixy="-0.001809" ixz="-0.001202"
                 iyy="0.006132" iyz="0.000494"
                 izz="0.006704"/>
      </inertial>
      <visual>
        <origin xyz="0 0 0" rpy="0 0 0" />
        <geometry>
          <mesh filename="package://odri_dual_motor_testbed_description/meshes/dual_motor_testbed_base_v2.stl" />
        </geometry>
        <material name="white">
          <color rgba="1.0 1.0 1.0 1.0"/>
        </material>

      </visual>
      <collision>
        <origin xyz="0 0 0" rpy="0 0 0" />
        <geometry>
          <mesh filename="package://odri_dual_motor_testbed_description/meshes/dual_motor_testbed_base_v2.stl" />
        </geometry>
      </collision>
    </link>

    <joint name="${name}_1_joint" type="fixed">
      <parent link="base_footprint" />
      <child link="${name}_2_link" />
      <origin xyz="0.0 0.0 0.0" rpy ="0.0 0.0 0.0"/>
    </joint>

    <link name="${name}_2_link">
      <inertial>
        <origin xyz="0.0 0.0 0.0" rpy="0.00000 0.00000 0.00000"/>
        <mass value="1.563428"/>
        <inertia ixx="0.002471" ixy="-0.001809" ixz="-0.001202"
                 iyy="0.006132" iyz="0.000494"
                 izz="0.006704"/>
      </inertial>
      <visual>
        <origin xyz="0 0 0" rpy="0 0 0" />
        <geometry>
          <mesh filename="package://odri_dual_motor_testbed_description/meshes/dual_motor_testbed_cover_v2.stl" />
        </geometry>
        <material name="white">
          <color rgba="1.0 1.0 1.0 1.0"/>
        </material>
      </visual>
      <collision>
        <origin xyz="0 0 0" rpy="0 0 0" />
        <geometry>
          <mesh filename="package://odri_dual_motor_testbed_description/meshes/dual_motor_testbed_cover_v2.stl" />
        </geometry>
      </collision>
    </link>

    <xacro:odri_dual_motor_testbed_wheel name="${name}" parent="${name}_2_link" prefix="left" reflect="1" />

```

```
70    <xacro:odri_dual_motor_testbed_wheel name="${name}" parent="${name}_2_link" prefix="right" reflect="-1" />
71
72  </xacro:macro>
73
74 </robot>
```

Listing 22.12 – Fichier racine de l'URDF

Le nom des deux joints est spécifié par la ligne 39 avec les paramètres *name* et *prefix* de la macro spécifiés à la déclaration en ligne 4. Cette macro est appelée par le fichier décrivant l'arbre cinématique du robot Listing 22.12. Plus spécifiquement la ligne 68 correspond à l'instance de la roue gauche, et la ligne 70 correspond à l'instance de la roue droite. On aura donc les deux noms : **odri_dm_tb_kt_left_joint** et **odri_dm_tb_kt_left_joint** à la ligne 6.

On retrouve ici les fichiers STL utiliser pour la visualisation et le calcul de collision.

```
1  <?xml version="1.0"?>
2  <robot xmlns:xacro="http://ros.org/wiki/xacro">
3
4    <xacro:macro name="odri_dual_motor_testbed_wheel" params="name parent prefix reflect">
5
6      <joint name="${name}_${prefix}_joint" type="revolute">
7        <parent link="${parent}" />
8        <child link="${name}_${prefix}_3_link" />
9        <origin xyz="${reflect * 0.065} 0.0 0.03" rpy ="0.0 0.0 0.0"/>
10       <axis xyz="0 0 1" />
11       <limit effort="30" velocity="1.0" lower="-3.14" upper="3.14" />
12     </joint>
13
14     <link name="${name}_${prefix}_3_link">
15       <inertial>
16         <origin xyz="0.0 0.0 0.0" rpy="0.00000 0.00000 0.00000"/>
17         <mass value="0.050"/>
18         <inertia ixx="0.002471" ixy="-0.001809" ixz="-0.001202"
19                  iyy="0.006132" iyz="0.000494"
20                  izz="0.006704"/>
21       </inertial>
22       <visual>
23         <origin xyz="0 0 0" rpy="0 0 0" />
24         <geometry>
25           <mesh filename="package://odri_dual_motor_testbed_description/meshes/encoder_code_wheel_mount_v2.stl" />
26         </geometry>
27         <material name="white">
28           <color rgba="1.0 1.0 1.0 1.0"/>
29         </material>
30       </visual>
31       <collision>
32         <origin xyz="0 0 0" rpy="0 0 0" />
33         <geometry>
34           <mesh filename="package://odri_dual_motor_testbed_description/meshes/encoder_code_wheel_mount_v2.stl" />
35         </geometry>
36       </collision>
37     </link>
38
39     <joint name="${name}_${prefix}_4_joint" type="fixed">
40       <parent link="${name}_${prefix}_3_link" />
41       <child link="${name}_${prefix}_4_link" />
42       <origin xyz="0.0 0.0 0.01" rpy ="0.0 0.0 0.0"/>
43       <axis xyz="0 0 1" />
44     </joint>
45
46     <link name="${name}_${prefix}_4_link">
47       <inertial>
48         <origin xyz="0.0 0.0 0.0" rpy="0.00000 0.00000 0.00000"/>
49         <mass value="0.050"/>
50         <inertia ixx="0.002471" ixy="-0.001809" ixz="-0.001202"
51                  iyy="0.006132" iyz="0.000494"
52                  izz="0.006704"/>
53       </inertial>
54       <visual>
55         <origin xyz="0 0 0" rpy="0 0 0" />
56         <geometry>
57           <mesh filename="package://odri_dual_motor_testbed_description/meshes/hand_wheel_v2.stl" />
58         </geometry>
59         <material name="white">
60           <color rgba="1.0 1.0 1.0 1.0"/>
61         </material>
62
63       </visual>
64       <collision>
65         <origin xyz="0 0 0" rpy="0 0 0" />
66         <geometry>
```

```
67          <mesh filename="package://odri_dual_motor_testbed_description/meshes/hand_wheel_v2.stl" />
68        </geometry>
69      </collision>
70    </link>
71
72  </xacro:macro>
73 </robot>
```

Listing 22.13 – Fichier racine de l'URDF

22.2.4 Description de la partie ros2_control

Une fois que l'arbre cinématique est décrit on peut maintenant faire le lien avec la partie contrôle de ros2_control.

22.2.4.1 Paramètres de la macro xacro

Les paramètres de la macro spécifiés à la ligne 4 du Listing 22.14 sont les suivants :

- use_sim : Utiliser le simulateur
- use_fake_hardware : Ne pas utiliser le simulateur (gazebo) mais simuler le matériel par simple intégration en vitesse.
- fake_sensor_commands : Ne pas utiliser le simulateur (gazebo) mais simuler les capteurs par simple intégration en vitesse.
- slowdown : Lorque le matériel est simulé sans gazebo, il est ralenti par un facteur spécifié par ce paramètre. Ici la valeur est 2.0 par défaut.
- ethernet_port : Port Ethernet à utiliser pour la connection avec la master_board.

22.2.4.2 Simulation

Lorsque le paramètre **use_sim** est égal à *true* le fichier xacro est construit avec l'utilisation du plugin Gazebo (lignes 8 - 12). Sinon lorsque le paramètre **use_fake_hardware** est égale à *true* le fichier xacro est construit avec l'utilisation du plugin **mock_components/GenericSystem** (ligne 15 -19)

22.2.4.3 Paramètres du plugin SystemOdriInterface

Le plugin décrit dans la section.22.1 est activé lorsqu'aucun des paramètres de simulation n'est activé (ligne 20-27). Afin de retrouver automatiquement l'endroit où le fichier de configuration YAML du robot est stocké il est possible d'utiliser xacro. Ainsi à la ligne 22, il est possible de construire le chemin vers le fichier `odri_dual_motor_test_config.yaml` en utilisant `$(find odri_dual_motor_testbed_description)`. Le résultat est stocké dans la propriété **prop_dual_motor_testbed_config_yaml**, celle-ci est ensuite passée en paramétre au plugin via le paramètre **odri_config_yaml** (ligne 23).

Il est ensuite possible de spécifier la position désirée du robot en utilisant le paramètre **desired_starting_position** (24). La position désirée cependant n'utilise pas toutes les capacités du système, elle est au contraire la version minimale.

Afin de spécifier toutes les variables du système deux paramètres peuvent être utilisés **default_joint_cmd** et **default_joint_state**. Dans ce plugin afin d'alléger la spécification qui est déjà très lourde, ce paramètre est une chaîne de caractères qui suit la convention suivante : $joint_name\ pos\ vel\ eff\ K_p\ K_d$ avec pos la position, vel la vitesse, eff le couple en sortie de l'actionneur, K_p le gain en contrôle de position, K_d le gain en contrôle de vitesse. Pour les deux joints on obtient les lignes 25 et 26.

22.2.4.4 Description des joints

Les noms des joints sont donc comme spécifié auparavant **odri_dm_tb_kt_right_joint** et **odri_dm_tb_kt_left_joint** avec le paramètre **name** de la balise **joint**.

Pour chaque joint il faut donc spécifier les interfaces de commande avec la balise **command_interface_name**. Il est possible de spécifier les valeurs minimales et maximales dans le bloc correspondant. Pour chaque nom d'interface, il faut utiliser les types d'interface déclarés dans dans le fichier hardware_interface_type_values.hpp dans le dépôt ros2_control, ou celles déclarées dans le listing 22.2 à la ligne 30.

```
1 <?xml version="1.0"?>
2 <robot xmlns:xacro="http://www.ros.org/wiki/xacro">
3
4   <xacro:macro name="odri_dual_motor_testbed_ros2_control" params="name use_sim:=^|false use_fake_hardware:=^|true
        fake_sensor_commands:=^|false slowdown:=2.0 ethernet_port:=^|enp6s0f1 true:=^|true false:=^|false">
5
6     <ros2_control name="${name}" type="system">
7
```

```
    <xacro:if value="$(arg use_sim)">
      <hardware>
        <plugin>gazebo_ros2_control/GazeboSystem</plugin>
      </hardware>
    </xacro:if>
    <xacro:unless value="$(arg use_sim)">
      <hardware>
        <xacro:if value="$(arg use_fake_hardware)">
          <plugin>fake_components/GenericSystem</plugin>
          <param name="fake_sensor_commands">${fake_sensor_commands}</param>
          <param name="state_following_offset">0.0</param>
        </xacro:if>
        <xacro:unless value="$(arg use_fake_hardware)">
          <plugin>ros2_control_odri/SystemOdriHardware</plugin>
          <xacro:property name="prop_dual_motor_testbed_config_yaml" value="$(find
    odri_dual_motor_testbed_description)/config/odri_dual_motor_testbed_config.yaml" />
          <param name="odri_config_yaml">${prop_dual_motor_testbed_config_yaml}</param>
          <param name="desired_starting_position">0.0 0.0</param>
          <param name="default_joint_cmd">odri_dm_tb_kt_left_joint 0.0 0.0 0.0 3.0 0.05
    odri_dm_tb_kt_right_joint 0.0 0.0 0.0 3.0 0.05</param>
          <param name="default_joint_state">odri_dm_tb_kt_left_joint 0.0 0.0 0.0 3.0 0.05
    odri_dm_tb_kt_right_joint 0.0 0.0 0.0 3.0 0.05</param>
        </xacro:unless>
      </hardware>
    </xacro:unless>

    <joint name="odri_dm_tb_kt_left_joint">
      <command_interface name="position">
        <param name="min">-0.9</param>
        <param name="max">0.9</param>
      </command_interface>
      <command_interface name="velocity">
        <param name="min">-1</param>
        <param name="max">1</param>
      </command_interface>
      <command_interface name="effort">
        <param name="min">-1</param>
        <param name="max">1</param>
      </command_interface>
      <command_interface name="gain_kp">
        <param name="min">-1000</param>
        <param name="max">1000</param>
      </command_interface>
      <command_interface name="gain_kd">
        <param name="min">-1000</param>
        <param name="max">1000</param>
      </command_interface>
      <state_interface name="position"/>
      <state_interface name="velocity"/>
      <state_interface name="effort"/>
      <state_interface name="gain_kp"/>
      <state_interface name="gain_kd"/>
    </joint>

    <joint name="odri_dm_tb_kt_right_joint">
      <command_interface name="position">
        <param name="min">-0.9</param>
        <param name="max">0.9</param>
      </command_interface>
      <command_interface name="velocity">
        <param name="min">-1</param>
        <param name="max">1</param>
      </command_interface>
      <command_interface name="effort">
        <param name="min">-1</param>
        <param name="max">1</param>
      </command_interface>
      <command_interface name="gain_kp">
        <param name="min">-1000</param>
        <param name="max">1000</param>
      </command_interface>
      <command_interface name="gain_kd">
        <param name="min">-1000</param>
        <param name="max">1000</param>
      </command_interface>
      <state_interface name="position"/>
      <state_interface name="velocity"/>
      <state_interface name="effort"/>
      <state_interface name="gain_kp"/>
      <state_interface name="gain_kd"/>
    </joint>

  </ros2_control>

```

```
  </xacro:macro>

</robot>
```

Listing 22.14 – Code de demo_solo12.cpp

22.3 Lancement du robot

Se connecter sur un utilisateur étant sudo.
Mettre en place la variable d'environnement :

```
export FASTRTPS_DEFAULT_PROFILES_FILE=$HOME/ros2_description_bolt/config/FastRTPS.xml
```

```
<?xml version="1.0" encoding="UTF-8" ?>
    <profiles xmlns="http://www.eprosima.com/XMLSchemas/fastRTPS_Profiles" >
        <transport_descriptors>
            <transport_descriptor>
                <transport_id>CustomUdpTransport</transport_id>
                <type>UDPv4</type>
            </transport_descriptor>
        </transport_descriptors>

        <participant profile_name="participant_profile" is_default_profile="true">
            <rtps>
                <userTransports>
                    <transport_id>CustomUdpTransport</transport_id>
                </userTransports>

                <useBuiltinTransports>false</useBuiltinTransports>
            </rtps>
        </participant>
    </profiles>
```

En effet le lancement de :

```
sudo ls
ros2 launch odri_dual_motor_testbed_bringup odri_dual_motor_testbed.launch.py
```

utilises sudo.
Ensuite pour charger les contrôleurs :

```
ros2 launch odri_dual_motor_testbed_bringup odri_dual_motor_testbed_ctrl.launch.py
```

Ensuite pour publier l'état du robot :

```
ros2 launch odri_dual_motor_testbed_bringup odri_dual_motor_testbed_pub.launch.py
```

Ensuite pour publier l'état du robot :

```
eros2 launch odri_dual_motor_testbed_bringup odri_dual_motor_testbed_rviz.launch.py
```

VIII Divers

23 Docker and ROS 2 243

23.1 Introduction
23.2 Exemples simples
23.3 Exemple de docker avec accélération graphique
23.4 Construire son propre fichier Docker pour ROS
23.5 Exemple d'utilisation pour le déploiement
23.6 Scénarios de déploiements
23.7 Exemple de déploiement
23.8 Connection réseau avec l'hôte

23. Docker and ROS 2

23.1 Introduction

Docker est très utile pour déployer et tester des nodes dans des architectures hétérogènes ou relativement anciennes. Par exemple il est possible de faire interagir un node qui fonctionne avec Foxy et un node qui fonctionne avec humble.

23.2 Exemples simples

Les didacticiels suivants sont tirés de la documentation de ROS-2 : `https://docs.ros.org/en/humble/How-To-Guides/Run-2-nodes-in-single-or-separate-docker-containers.html`.

23.2.1 Lancer deux nodes dans un seul container docker

Il faut récupérer l'image docker ROS ayant l'étiquette humble-desktop.

```
docker pull osrf/ros:humble-desktop
```

Lancer l'image dans un container en mode interactif :

```
$ docker run -it osrf/ros:humble-desktop
root@<container-id>:/#
```

Il est possible d'utiliser la ligne de commande ros2 pour s'aider :

```
root@<container-id>:/# ros2 --help
```

Pour lister tous les paquets installés :

```
root@<container-id>:/# ros2 pkg list
```

(Vous allez voir une liste de paquets.)

Pour lister tous les exécutables :

```
root@<container-id>:/# ros2 pkg executables
```

(Liste d'exécutables)

Pour lancer un exemple minimal de 2 nodes C++, (1 topic subscriber listener, 1 topic publisher talker) du paquet demo_nodes_cpp dans ce container :

```
ros2 run demo_nodes_cpp listener &
ros2 run demo_nodes_cpp talker
```

23.2.2 Lancer deux nodes dans deux dockers séparés

Il faut ouvrir un terminal et lancer l'image dans un container en mode interactic. Ensuite on peut lancer un topic publisher (l'exécutable du paquet demo_nodes_cpp) avec ros2 run :

```
docker run -it --rm osrf/ros:humble-desktop ros2 run demo_nodes_cpp talker
```

Il faut ouvrir un second terminal, lancer l'image dans un container en mode interactif et lancer le topic subscriber (l'exécutable listener du paquet demo_nodes_cpp) avec ros2 run :

```
docker run -it --rm osrf/ros:humble-desktop ros2 run demo_nodes_cpp listener
```

23.2.3 docker compose

A la place de l'invocation par la ligne de commande, il est possible de créer un fichier docker-compose.yml (ici version 2) avec le contenu minimal suivant :

```
version : '2'

services :
  talker :
    image : osrf/ros :humble-desktop
    command : ros2 run demo_nodes_cpp talker
  listener :
    image : osrf/ros :humble-desktop
    command : ros2 run demo_nodes_cpp listener
    depends_on :
      - talker
```

Pour lancer les containers il faut lancer docker compose up dans le même répertoire. Il est possible de fermer les containers avec Ctrl+C.

23.3 Exemple de docker avec accélération graphique

Cette partie est tirée de la page suivante : `http://wiki.ros.org/Robots/TIAGo/Tutorials/Installation/Installing_Tiago_tutorial_docker`

Premièrement, ouvrons un terminal et téléchargeons l'image tiago_tutorials :

```
docker pull palroboticssl/tiago_tutorials:noetic
```

Rocker est nécessaire pour les tutoriaux suivants pour avoir une interface graphique utilisateur avec docker. osrf/rocker, est un outil pour faire fonctionner des images docker avec des supports locaux customisés pour des choses comme les drivers Nvidia et des fichiers ayant un user_id specifique pour le montage de fichiers avec des permissions propres. Il est également possible de trouver plus d'informations sur ce sujet dans les outils pour docker

Pour obtenir rocker avec le support Nvidia :

Il faut d'abord installer le support Nvidia : `https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/install-guide.html#docker` :

```
    distribution=$(. /etc/os-release;echo $ID$VERSION_ID)    && curl -s -L https://nvidia.github.io/nvidia-docker/
      gpgkey | sudo apt-key add -    && curl -s -L https://nvidia.github.io/nvidia-docker/$distribution/nvidia-
      docker.list | sudo tee /etc/apt/sources.list.d/nvidia-docker.list
    sudo apt-get update
    sudo apt-get install -y nvidia-docker2
    sudo systemctl restart docker
```

Il faut ensuite lancer le docker :

```
rocker --home --user --nvidia --x11 --privileged palroboticssl/tiago_tutorials:noetic
```

rocker with intel-integrated graphics support :

```
rocker --home --user --x11 --privileged palroboticssl/tiago_tutorials:noetic --devices /dev/dri/card0
```

Les options **–home** and **–user** sont utilisées pour que l'utilisateur ne soit pas root et que le répertoire utilisateur soit monté.

Si votre machine utilise une version antérieur à la version 0.2.4, il faut retirer l'option **–privileged**.

Une fois que rocker est lancé il est possible de taper

```
terminator -u
```

pour ouvrir un terminal dans le rocker qui permettra d'utiliser plusieurs terminaux dans le même rocker. Il ne faut pas oublier de tous les sourcer.

Pour vérifier que tout est installé proprement il est possible de lancer la simulation de Tiago comme indiqué ici (14.2).

23.4 Construire son propre fichier Docker pour ROS

23.4.1 Créer un fichier Dockerfile pour installer des paquets ROS

Pour construire une image docker ROS et des paquets spécialisés voici un exemple de libraries clients C++ et Python en utilisant la release Debian officielle fournie par apt-get.

```
FROM ros:humble

# install ros package
RUN apt-get update && apt-get install -y \
      ros-${ROS_DISTRO}-demo-nodes-cpp \
      ros-${ROS_DISTRO}-demo-nodes-py && \
    rm -rf /var/lib/apt/lists/*

# launch ros package
CMD ["ros2", "launch", "demo_nodes_cpp", "talker_listener.launch.py"]
```

Note 23.1 Toutes les images ROS incluent un point d'entrée par défaut qui source l'environnement ROS avant d'exécuter la commande configurée, dans ce cas le fichier launch des paquets de démonstration. Il est alors possible de construire et de démarrer l'image de la façon suivante : ■

```
$ docker build -t my/ros:app -f ./Dockerfile-simple  .
$ docker run -it --rm my/ros:app
[INFO] [launch]: process[talker-1]: started with pid [813]
[INFO] [launch]: process[listener-2]: started with pid [814]
[INFO] [talker]: Publishing: 'Hello World: 1'
[INFO] [listener]: I heard: [Hello World: 1]
[INFO] [talker]: Publishing: 'Hello World: 2'
[INFO] [listener]: I heard: [Hello World: 2]
...
```

23.4.2 Créer un fichier Dockerfile pour construire des paquets ROS

Cette partie est disponible ici : `https://hub.docker.com/_/ros/`.

Pour créer ses propres images docker ROS et construire des paquets personnalisés voici un exemple simple qui installe les dépendences de construction, les compile à partir des sources, et installe les objets générés dans une image finale.

```
ARG FROM_IMAGE=ros:humble
ARG OVERLAY_WS=/opt/ros/overlay_ws

# multi-stage for caching
FROM $FROM_IMAGE AS cacher

# clone overlay source
ARG OVERLAY_WS
WORKDIR $OVERLAY_WS/src
RUN echo "\
repositories: \n\
  ros2/demos: \n\
    type: git \n\
    url: https://github.com/ros2/demos.git \n\
    version: ${ROS_DISTRO} \n\
" > ../overlay.repos
RUN vcs import ./ < ../overlay.repos

# copy manifests for caching
```

```
WORKDIR /opt
RUN mkdir -p /tmp/opt && \
    find ./ -name "package.xml" | \
      xargs cp --parents -t /tmp/opt && \
    find ./ -name "COLCON_IGNORE" | \
      xargs cp --parents -t /tmp/opt || true

# multi-stage for building
FROM $FROM_IMAGE AS builder

# install overlay dependencies
ARG OVERLAY_WS
WORKDIR $OVERLAY_WS
COPY --from=cacher /tmp/$OVERLAY_WS/src ./src
RUN . /opt/ros/$ROS_DISTRO/setup.sh && \
    apt-get update && rosdep install -y \
      --from-paths \
        src/ros2/demos/demo_nodes_cpp \
        src/ros2/demos/demo_nodes_py \
      --ignore-src \
    && rm -rf /var/lib/apt/lists/*

# build overlay source
COPY --from=cacher $OVERLAY_WS/src ./src
ARG OVERLAY_MIXINS="release"
RUN . /opt/ros/$ROS_DISTRO/setup.sh && \
    colcon build \
      --packages-select \
        demo_nodes_cpp \
        demo_nodes_py \
      --mixin $OVERLAY_MIXINS

# source entrypoint setup
ENV OVERLAY_WS $OVERLAY_WS
RUN sed --in-place --expression \
      '$isource "$OVERLAY_WS/install/setup.bash"' \
      /ros_entrypoint.sh

# run launch file
CMD ["ros2", "launch", "demo_nodes_cpp", "talker_listener.launch.py"]
```

L'exemple précédent démarre par cloner les dépôts sources en utilisant l'utilitaire vcstool durant l'étape cacher. Il est également possible de copier le code via COPY du contexte build local dans le répertoire source également. Les fichiers manifeste des paquets sont alors mis en cache dans un répertoire temporaire à partir duquel l'étape builder peut installer les dépendences nécessaires avec rosdep. Ceci est fait avant de copier le reste des fichiers sources pour présever l'aspect multiple étapes du cache de construction. Ceci permet aux dépendences de ne pas être réinstallées inutilement ce qui permet de gagner du temps et de la bande passante. L'ensemble est alors construit en utilisant colcon, le point d'entrée est mis à jour pour sourcer l'espace de travail, et la commande par défaut lance la démo.

Note 23.2 `--from-paths` et `--packages-select` sont utilisés ici pour installer uniquement les dépendences des paquets de démo C++ et Python parmi tous ceux qui sont disponible dans le dépot git cloné. Pour installer les dépendences et construire tous les paquets de l'espace de travail source il suffit de spécifier `--from-paths` src/ et d'enlever l' argument `--packages-select` ■

Pour des exemples plus avancés comme le chaînage multiple d'espace de travail pour améliorer le cache d'image docker en utilisant des outils comme ccache pour accélérer la compilation avec colcon, ou utilisant buildkit pour sauver du temps et la bande passante même quand les dépendences changent, les fichiers Dockerfiles dans le dépôt ROS 2 Navigation2 sont d'excellentes resources.

23.5 Exemple d'utilisation pour le déploiement

Cette image docker de ROS fournit une plateforme simplifiée et cohérente pour construire et déployer des applications robotiques. Construire à partir de l'image Ubuntu officielle et des paquets ROS Debian officiels, cela inclut des releases supportées récentes pour un accés rapide par téléchargement. Cela fournit aux roboticiens académiques et industriels une façon simple de développer, de réutiliser et de fournir des logiciels pour des actions autonomes, la planification de tâches, le contrôle, la localisation et la cartographie, les comportements en essaims, ainsi que l'intégration de système en génération.

Développer des systèmes complexes avec des implémentations de pointe de nouveaux algorithmes reste un challenge, parce que la répétabilité et la reproductibilité des logiciels robotiques peut être un dégât

colatéral dans la course à l'innovation. Avec la difficulté supplémentaire que le développement, la mise au point et le déploiement de multiples composants qui utilisents plusieurs discplines, une approche plus collaboratives est préférable. Cependant les difficultés techniques de partager et maintenir une collection de logiciels pour plusieurs robots et plateformes a longtemps été trop important pour de nombreuses structures comme des laboratoires et des PMEs.

Avec les avancées et la standardisation des containers logiciels, les roboticiens ont maintenant accés à des outils avancés pour construire et founir des logiciels. Pour faciliter l'adoption de nouvelles pratiques, le dépôt `https://hub.docker.com/_/ros/` fournit des images standards pour travailler avec ROS.

Pour une liste compléte des architectures supportés et des images de base pour chaque distribution ROS, il est possible de se référer aux REP pour les plateformes cibles soit pour ROS 1 soit pour ROS 2.

23.6 Scénarios de déploiements

Les étiquettes disponibles incluent les distributions supportées avec une hiérarchie des étiquettes basée sur les dépendences les plus standards, choisis pour avoir une configuration simple et un coût très bas :

ros-core : une installation ROS minimale

ros-base : des outils et des librairies de base (avec des étiquettes correspondant au nom de la distribution LTS en dernier)

ros1-bridge : des outils et des librairies pour lancer des systèmes hybrides ROS 1 - ROS 2 et de faire un pont entre eux

De façon à garder ros-core minimal en taille d'image, les outils de développement comme rosdep, colcon et vcstools ne sont pas inclus dans l'étiquette ros-core mais se trouvent dans ros-base.

Le reste des meta-paquets communs tels que desktop sont hébergés dans les dépôts sous le profil Docker de l'OSRF. Ces méta-paquets incluent les dépendences graphiques et comportent de nombreux autres paquets très larges comme X11, X server, etc... De façon à garder les images officielles les plus légéres et sécurisées, les paquets desktops ne sont hébergés qu'avec le profil de l'OSRF.

23.6.1 Volumes

ROS utilise le répertoire /.ros/ pour stocker les logs, les informations de débogage. Si vous souhaitez que ces fichiers persistent au delà du cycle de vie des containers qui les produits, le répertoire /.ros/ doit être monté sur un volume externe sur l'hôte, ou une image dérive peut spécifier des volumes gérés par le moteur Docker. Par défaut, le container fonctionne avec l'utilisateur root, le chemin complet vers ces fichiers est donc /root/.ros.

Par exemple, supposons que l'on souhaite que l'utilisateur ubuntu sur la machine hôte utilise son répertoire utilisateur il est possible de lancer le container avec l'argument suivant :

```
$ docker run -v "/home/ubuntu/.ros/:/root/.ros/" ros
```

23.6.2 Matériel

Certaines applications peuvent requérir l'accés à des appareils pour l'acquisition d'images par des caméras connectées, le contrôle d'une interface humaine, ou des GPUs pour l'accélération matérielle. Ceci peut-être réalisé en utilisant l'argument –device de la commande run pour monter les matériels dans le container, permettant l'accés au matériel.

23.6.3 Réseaux

ROS permet des connections réseaux point à point de processus (potentiellement distribués sur plusieurs machines) qui couplés librement en utilisant l'infrastructure de communication ROS. ROS implémente plusieurs types de communication incluant des communications de type RPC pour les services, de streamer des données typées de façon asynchrone à travers les topics, une combinaison des deux à travers les requêtes/réponses et statut/retour pour les actions, et des paramètres à la volée. Pour se conformer à la meilleure pratique qui consiste à avoir un processus par container, les réseaux Docker peuvent être utilisés pour mettre ensemble plusieurs processus ROS. L'exemple de déploiement ci-dessous fournit des détails.

Une solution alternative consiste à utiliser une paramétrisation du réseau plus permissive. Elle partage toutes les interfaces réseaux de l'hôte avec le container, telle que le driver réseau de l'hôte simplifiant la connectivité avec les participants réseaux extérieurs. Il faut cependant garder à l'esprit que cela enlève la

séparation réseau par espace de noms entre les containers. Cela peut affecter la capacité des participants DDS à communiquer entre les containers.

23.7 Exemple de déploiement

23.7.1 Docker Compose

Dans cet exemple, l'utilisation de docker-compose est illustrée pour lancer une paire de nodes qui publie et soucrivent à un topic dans des containers séparés à travers un réseau logiciel partagé.

Il faut créer le répertoire `~/ros_demos` et ajouter le fichier Dockerfile ci-dessus 23.4.1. Dans le même répertoire, il faut aussi créer le fichier docker-compose.yml suivant pour lancer un publisher C++ et un subscriber Python :

```
version : '3'

services :
  talker :
    build : ./
    command : ros2 run demo_nodes_cpp talker

  listener :
    build : ./
    environment :
      - "PYTHONUNBUFFERED=1"
    command : ros2 run demo_nodes_py listener
```

Pour lancer les nodes ROS, on peut utiliser docker-compose dans le même répertoire. Parce que les containers dérivent du même projet docker compose, ils vont coexister dans le même réseau partagé :

```
$ docker-compose up -d
```

On peut noter qu'un nouveau réseau nommé `ros_demos_default` a été créé, et que l'on peut obtenir plus d'informations avec :

```
$ docker network inspect ros_demos_default
```

Il est possible d'inspecter la sortie des logs de chaque container, pour le node listener on peut faire :

```
$ docker-compose logs listener
```

Finalement, il est possible d'arrêter et d'effacer tous les containers pertinents du même répertoire :

```
$ docker-compose stop
$ docker-compose rm
```

Note 23.3 Le réseau auto-généré, `ros_demos_default`, va persister jusqu'à ce qu'il soit explicité effacer avec `docker-compose down`. ■

23.7.2 ROS 1 Bridge

Pour faciliter la migration vers ROS 2 migration, `ros1_bridge` un paquet ROS 2 qui fournit des communication bidirectionnelles entre ROS 1 et ROS 2. Comme exemple minimal, et en se basant sur le Dockerfile ROS 2 précédent, il est possible de créer son équivalent en ROS 1 et de nommer le Dockerfile de façon appropriée.

```
FROM ros:noetic

# install ros package
RUN apt-get update && apt-get install -y \
      ros-${ROS_DISTRO}-ros-tutorials \
      ros-${ROS_DISTRO}-common-tutorials && \
    rm -rf /var/lib/apt/lists/*

# launch ros package
CMD ["roslaunch", "roscpp_tutorials", "talker_listener.launch"]
```

Le fichier compose suivant lance les nodes pour le talker et le listener tandis qu'il connecte les deux via un pont dynamique. Il est possible de voir les sorties du talker et du listener à travers le topic `/chatter`.

```
version : '3'

services :
  ros1 :
    build :
      context : ./
      dockerfile : ros1.Dockerfile

  ros2 :
    build :
      context : ./
      dockerfile : ros2.Dockerfile

  bridge :
    image : ros :foxy-ros1-bridge
    environment :
      - "ROS_HOSTNAME=bridge"
      - "ROS_MASTER_URI=http://ros1:11311"
    command : ros2 run ros1_bridge dynamic_bridge
```

23.8 Connection réseau avec l'hôte

Cette solution permet d'éviter d'enlever l'isolation entre le container et l'hôte `--net=host`. Pour cela il faut créer une carte ethernet virtuelle. Un argument de cette carte ethernet virtuelle doit être une carte physique du système. Par exemple si vous faites

```
  ...
2: enp0s31f6: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000
  ...
```

Alors il faut écrire :

```
docker network create -d macvlan \
  --subnet=192.168.1.0/24 \
  --gateway=192.168.1.1 \
  --ip-range=192.168.1.64/27 \
  --attachable \
  -o parent=enp0s31f6 \
  macnet
```

```
sudo ip link add macnet-shim link eth0 type macvlan mode bridge
sudo ip addr add 192.168.1.64/27 dev macnet-shim
sudo up link set macnet-shim up
```

IX Appendices

24 Rappels sur le bash 253

24.1 Lien symbolique
24.2 Gestion des variables d'environnement
24.3 Fichier .bashrc

Bibliography 255

Books
Articles
Chapitre de livres
Autres

24. Rappels sur le bash

24.1 Lien symbolique

24.1.1 Voir les liens symboliques

Pour lister le contenu de votre répertoire vous pouvez utiliser la commande suivante :

```
ls
```

La sortie est la suivante :

```
catkin_ws             Music
catkin_ws_prenom_nom  Pictures
Desktop               Public
Documents             Templates
Downloads             Videos
examples.desktop
```

Pour savoir si le répertoire **catkin_ws** est un raccourci vers un autre répertoire vous pouvez utiliser la commande suivante :

```
ls -al catkin_ws
```

Le résultat est alors :

```
lrwxrwxrwx   1 pnom pnom        20 dec.   6 08:43 catkin_ws -> catkin_ws_prenom_nom
```

24.1.2 Créer un lien symbolique

Pour créer un lien de votre répertoire **catkin_ws_prenom_nom** vers **catkin_ws** :

```
ln -s catkin_ws_prenom_nom catkin_ws
```

24.1.3 Enlever un lien symbolique

Pour enlever un lien, il faut faire comme pour enlever un fichier :

```
rm catkin_ws
```

24.2 Gestion des variables d'environnement

La liste des variables d'environnements peut s'obtenir en faisant :

```
env
```

La liste des variables d'environnements peut s'obtenir en cherchant les lignes contenant la chaîne ROS :

```
env | grep ROS
```

Il est possible de changer la valeur de la variable d'environnement **ROS_MASTER_URI** pour pointer vers le robot **turtlebot2** sur le port **11311**.

```
export ROS_MASTER_URI:=http://turtlebot2:11311
```

Les variables d'environnement sont locales à un terminal. Si vous avez ouvert plusieurs terminaux la modification de la valeur ne peut affecter que celui où la modification a été faites.

24.3 Fichier .bashrc

Le fichier **.bashrc** est lu à chaque fois qu'un terminal est lancé. La version qui est lue peut-être différente si le fichier est modifié d'un lancement à l'autre. Il est possible de forcer le fichier **.bashrc** à lire un autre fichier. Par exemple pour lire le fichier **setup.bash** qui se trouve dans votre répertoire **catkin_ws** :

```
source /home/etudiant/catkin_ws/devel/setup.bash
```

Bibliography

Books

[Bro16] Bernard BROGLIATO. *Nonsmooth Mechanics*. Third edition. Springer International Publishing Switzerland, 2016 (cf. page 86).

Articles

[Sta+09] O. STASSE et al. « Strategies for humanoid robots to dynamically walk over large obstacles ». In : *IEEE Transactions on Robotics* 4 (2009), pages 960-967 (cf. page 86).

[Lee+18] J. LEE et al. « DART : Dynamic Animation and Robotics Toolkit ». In : *Journal of Open Source Software* 3.22 (2018) (cf. page 86).

[Set+18] A. SETH et al. « OpenSim : Simulating musculoskeletal dynamics and neuromuscular control to study human and animal movement ». In : *PLOS Computational Biology* (2018) (cf. page 86).

Chapitre de livres

[CHC16] F. CHAUMETTE, S. HUTCHINSON et P. CORKE. « Handbook of Robotics 2nd Edition ». In : Spring Verlag, 2016. Chapitre Chapter 34 : Visual Servoing (cf. page 65).

Autres

[Bed] Maciej BEDNARCZYK. *Ethercat Driver for ROS 2*. URL : `https://github.com/ICube-Robotics/ethercat_driver_ros2` (cf. page 209).

[Goo16] GOOGLE. 2016. URL : `https://developers.google.com/project-tango/` (cf. page 14).

[Inia] Open Robots Robot INITIATIVE. *Carte de contrôle*. URL : `https://github.com/open-dynamic-robot-initiative/master-board` (cf. page 208).

[Inib] Open Robots Robot INITIATIVE. *Carte de puissance*. URL : `https://github.com/open-dynamic-robot-initiative/open_robot_actuator_hardware/tree/master/electronics/micro_driver_electronics` (cf. page 208).

[ros] ROS-CONTROLS. URL : `http://wiki.ros.org/diff_drive_controller` (cf. page 211).